

3X-UI Panel Kullanıcı Kılavuzu

العربية · English · Español · فارسی · Bahasa Indonesia · 日本語 · Português · Русский · Türkçe · Українська · Tiếng Việt · 简体中文 · 繁體中文

3X-UI sürümü: 3.5.0. Bu kılavuz söz konusu sürüme göre hazırlanmıştır ve o sürüm için günceldir. 3.5.0'ın 3.4.2'ye göre değişikliklerinin özeti «[3.5.0'daki Yenilikler](#)» bölümündedir.

3X-UI web paneline (Xray-core yönetimi) ilişkin ayrıntılı Türkçe kılavuz: işlevler, yapılandırma ve işletim; arayüzdeki her alan ve geçiş açıklamalı.

Ad ve etiketler panel arayüzüne karşılık gelir. *inbound* / *outbound* sözcükleri çevrilmez.

İçindekiler

- 3.5.0'daki Yenilikler
- 1. Giriş, Gereksinimler ve Kurulum
 - 1.1. 3X-UI Nedir
 - 1.2. Desteklenen İşletim Sistemleri ve Mimariler
 - 1.3. Kurulum Yöntemleri
 - 1.4. İlk Başlatma ve Varsayılan Kimlik Bilgileri
 - 1.5. Dosya Konumları
 - 1.6. `x-ui` Yönetim Komutu (Betik Menüsü)
 - 1.7. `x-ui` Alt Komutları (Etkileşimsiz Menü)
 - 1.8. SQLite → PostgreSQL Geçişi
- 2. Panele Giriş ve Erişim Güvenliği
 - 2.1. Giriş Formu
 - 2.2. İki Faktörlü Kimlik Doğrulama (2FA / TOTP)
 - 2.3. Giriş Denemelerini Sınırlandırma (login limiter / deneme saldırısına karşı koruma)
 - 2.4. Yönetici Kullanıcı Adı ve Parolasının Değiştirilmesi
 - 2.5. Gizli Yol (URI yolu / webBasePath) ve Panel Portu
 - 2.6. Oturum Ömrü (zaman aşımı)
 - 2.7. LDAP (Senkronizasyon ve Kimlik Doğrulama)
- 3. Genel Bakış / Gösterge Paneli
 - 3.1. Veri Toplama Genel İlkeleri
 - 3.2. İşlemci (CPU)
 - 3.3. Bellek (RAM)
 - 3.4. Takas Alanı (Swap)
 - 3.5. Disk (Storage)
 - 3.6. Sistem Çalışma Süresi (Uptime)
 - 3.7. Sistem Yüğü (Load average)
 - 3.8. Ağ: Hız ve Toplam Trafik
 - 3.9. Sunucu IP Adresleri
 - 3.10. TCP/UDP Bağlantıları
 - 3.11. Xray Durumu ve Süreç Yönetimi
 - 3.12. Panel Güncellemesi (3X-UI)
 - 3.13. Coğrafi Dosya Güncellemesi (GeoIP / GeoSite)
 - 3.14. Veritabanı Yedekleme ve Geri Yükleme
 - 3.15. Ek Arayüz Öğeleri
- 4. Inbounds: oluşturma ve genel parametreler
 - 4.1. Genel form alanları
 - 4.2. Sniffing (Koklama)
 - 4.3. Allocate (Bağlantı noktası dağıtım stratejisi)
 - 4.4. External Proxy (Harici proxy)
 - 4.5. Fallbacks (Fallback'ler)
 - 4.6. Periyodik trafik sınırlama
 - 4.7. inbound JSON (gelişmiş)
 - 4.8. inbound işlemleri: QR / Edit / Reset / Delete ve istatistikler

- 5. Protokoller
 - 5.1. Desteklenen protokol listesi
 - 5.2. Hangi protokoller TLS / REALITY / aktarımı destekler
 - 5.3. VLESS
 - 5.4. VMess
 - 5.5. Trojan
 - 5.6. Shadowsocks
 - 5.7. Dokodemo-door / Tunnel (şeffaf yönlendirici)
 - 5.8. SOCKS / HTTP (mixed protokolü)
 - 5.9. WireGuard (inbound)
 - 5.10. Hysteria (varsayılan olarak v2)
 - 5.11. MTPROTO (Telegram proxy'si)
 - 5.12. Protokol seçimi için hızlı başvuru
- 6. Aktarım (Stream Settings)
 - 6.1. Ağ İletimi Seçimi
 - 6.2. RAW / TCP (tcpSettings)
 - 6.3. mKCP (kcpSettings)
 - 6.4. WebSocket (wsSettings)
 - 6.5. gRPC (grpcSettings)
 - 6.6. HTTPUpgrade (httpupgradeSettings)
 - 6.7. XHTTP / SplitHTTP (xhttpSettings)
 - 6.8. Hysteria Aktarımı (hysteriaSettings)
 - 6.9. İlgili Parametreler
- 7. Bağlantı Güvenliği: TLS,XTLS ve REALITY
 - 7.1. Farklar: TLS vs XTLS vs REALITY
 - 7.2. «Yok» Modu (none)
 - 7.3. TLS Modu
 - 7.4. REALITY Modu
 - 7.5. Yapılandırma İçin Pratik Öneriler
- 8. İstemciler
 - 8.1. İstemci Alanları
 - 8.2. inbound'a Bağlama
 - 8.3. İstemci İşlemleri
 - 8.4. Toplu İşlemler
 - 8.5. Arama, Filtreler ve Sıralama
 - 8.6. Simgeler ve Durumlar
- 9. İstemci Grupları
 - 9.1. İstemci Grubu Nedir ve Ne İşe Yarar
 - 9.2. Grubun İstemciler, inbound'lar, Düğümler ve Protokollerle İlişkisi
 - 9.3. Grup Rehberi ve "Boş" Gruplar
 - 9.4. Grup Alanları ve Sütunları
 - 9.5. Grup Oluşturma
 - 9.6. Grubu Yeniden Adlandırma
 - 9.7. Gruba İstemci Ekleme
 - 9.8. Gruptan İstemci Kaldırma (İstemcileri Silmeden)
 - 9.9. Grup Trafikini Sıfırlama

- 9.10. Grubu Silme ve Gruptaki İstemcileri Silme
- 9.11. "İstemciler" Sayfasıyla İlişki
- 9.12. API Uç Noktaları Özeti
- 9.13. Gruba Göre Trafik
- 10. Abonelikler (Subscription)
 - 10.1. subld nedir ve bağlantı nasıl oluşturulur
 - 10.2. Abonelik sunucusu ayarları
 - 10.3. Çıktı formatları
 - 10.4. Abonelik bilgi sayfası ve QR kodları
 - 10.5. Abonelik sayfası için özel şablonlar
- 11. Xray: yönlendirme, outbounds, DNS ve uzantılar
 - 11.1. Düzenleyici yapısı: sekmeler/modlar
 - 11.2. Ana Ayarlar (General)
 - 11.3. Yönlendirme kuralları (routing)
 - 11.4. Outbounds (giden bağlantılar)
 - 11.5. Yük Dengeleyiciler (Balancers)
 - 11.6. DNS
 - 11.7. Fake DNS
 - 11.8. WireGuard / WARP / NordVPN
 - 11.9. Reverse-proxy ve TUN
 - 11.10. Günlükler ve istatistik (Stats, metrics)
 - 11.11. Kaydetme, yeniden başlatma ve otomatik dönüşümler
 - 11.12. Abonelikten outbound (otomatik güncelleme ile)
 - 11.13. WARP'ta IP rotasyonu
- 12. Düğümler (çoklu panel, master/slave)
 - 12.1. Liste Başındaki Özet
 - 12.2. Düğüm Ekleme ve Düzenleme
 - 12.3. TLS Doğrulama (https düğümleri için)
 - 12.4. Her Düğüm İçin Gösterilenler
 - 12.5. Düğüm Üzerindeki İşlemler
 - 12.6. Metrik Geçmişi
 - 12.7. İnbound'lar ve İstemciler Nasıl Senkronize Edilir
 - 12.8. Düğüm Zincirleri (alt düğümler / geçişli düğümler)
 - 12.9. 3.5.0 Düzeltmeleri (düğüm kararlılığı)
 - 12.10. Düğümler: 3.3.0'daki Yenilikler
- 13. Panel Ayarları
 - 13.1. Panelin Kaydedilmesi ve Yeniden Başlatılması
 - 13.2. Genel Ayarlar (sekme «Panel» / *General*)
 - 13.3. Panele Erişim: IP, Port, Yol, Alan Adı, Sertifika
 - 13.4. Oturum, Panel Proxy'si ve Güvenilir Proxy'ler (sekme «Proxy ve Sunucu» / *Proxy and Server*)
 - 13.5. Telegram Botu (sekme «Telegram Botu» / *Telegram Bot*)
 - 13.6. Tarih ve Saat (sekme «Tarih ve Saat» / *Date and Time*)
 - 13.7. Harici Trafik ve Xray Davranışı (sekme «Harici Trafik» / *External Traffic*)
 - 13.8. Diğer: Xray Yapılandırma Şablonu ve Test URL'si
 - 13.9. Yönetici Hesabı ve API Token'ları
 - 13.10. 3.3.0 Sürümündeki API Değişiklikleri (entegrasyonlar için önemli)

- 14. Telegram Botu
 - 14.1. Botu Etkinleştirme ve Yapılandırma
 - 14.2. Ana Menü ve Düğmeler
 - 14.3. Bot Komutları
 - 14.4. İstemci Yönetimi (Yalnızca Yönetici)
 - 14.5. Bildirimler ve Raporlar
 - 14.6. Yedekleme ve Günlükler
 - 14.7. Çalışma Özellikleri
- 15. Coğrafi Veritabanları (geoip / geosite ve Özel Kaynaklar)
 - 15.1. geoip.dat ve geosite.dat Nedir?
 - 15.2. Standart Geo-Dosyalar ve Güncelleme
 - 15.3. Xray Aracılığıyla Geo-Verilerinin Otomatik Güncellenmesi (Geodata Auto-Update)
 - 15.4. Doğrulama ve Kısıtlamalar
 - 15.5. Panel Başlangıcında Otomatik Kontrol
 - 15.6. Yönlendirme Kurallarında Geo-Veritabanlarının Kullanımı
- 16. İşletim: Yedekler, Günlükler, Güncelleme, CLI
 - 16.1. Veritabanı Yedekleme ve Geri Yükleme
 - 16.2. Günlükleri Görüntüleme
 - 16.3. Xray Günlük Seviyesi ve Yapılandırması
 - 16.4. Xray Yönetimi: Durdurma ve Yeniden Başlatma
 - 16.5. Paneli Yeniden Başlatma ve Güncelleme
 - 16.6. Periyodik Görevler (Cron)
 - 16.7. Konsol Menüsü ve CLI (`x-ui`)
 - 16.8. Paneli Kaldırma
 - 16.9. `x-ui migrateDB` Komutu

3.5.0'daki Yenilikler

3.5.0 sürümü büyük bir sürümdür: MTProto çok istemcili modele geçirildi (mtg-multi motoru, kişisel gizli anahtarlar, kotalar ve ad-tag), yönetilen hostlar grup haline geldi (tek kayıta birden fazla inbound ve adres), PostgreSQL panelindeki geri yükleme artık SQLite yedeklerini kabul ediyor, outbound'lara «Target Strategy», «Real delay» testi ve Egress/Country sütunları eklendi, yük dengeleyici başka bir yük dengeleyiciyi fallback (yedek) olarak kullanabiliyor. Pakette Xray çekirdeği 26.7.11 gelir. Aşağıda, 3.4.2'ye göre değişiklikler kılavuz bölümlerine göre verilmiştir.

Bölüm 1 değişiklikleri – Giriş, Gereksinimler ve Kurulum

- Xray çekirdeği **26.7.11**'e güncellendi. Beraberinde gelen otomatik geçişler: Shadowsocks `none / plain` ve VMess `none / zero` şifreleri çekirdekten kaldırıldı (kayıtlı yapılandırmalar otomatik olarak yeniden yazılır), genel bir adrese giden şifrelenmemiş VLESS/Trojan outbound kaydetme sırasında reddedilir.
- Yeni `x-ui pgclient [sürüm]` komutu ve PostgreSQL menüsünde **10. Install/Upgrade client tools (pg_dump/pg_restore)** ögesi – PostgreSQL istemci araçlarının kurulumu/güncellenmesi.
- Betik düzeltmeleri: RHEL ailesinde PostgreSQL ve fail2ban kurulumu (EPEL), Arch'ta tam `pacman -Syu` çalıştırılmıyor, 32 bit ARM'de doğru Xray ikili dosya adı (`xray-linux-arm32`), IP sertifikası verilmeden önce otomatik algılanan IPv4'ün onaylanması, varsayılan ACME portu seçimindeki yanlış «Your input is invalid» düzeltildi.

Bölüm 2 değişiklikleri – Panele Giriş ve Erişim Güvenliği

- IP limiti: «ölü» bağlantı artık her 10 saniyelik taramada değil, **bir kez** banlanıyor – fail2ban sayaçları artık şişmiyor, `maxretry` değerini yükseltmeye gerek yok.

Bölüm 4 değişiklikleri – Inbounds: oluşturma ve genel parametreler

- inbound listesine **arama** eklendi (açıklamaya, porta ve protokole göre); düğüm açılır listeleri («Şuraya Dağıt», «Düğümler» filtresi) aranabilir hale geldi.

Bölüm 5 değişiklikleri – Protokoller

- MTProto çok istemcili modele geçirildi** (mtg-multi motoru): MTProto kullanıcıları artık kendi gizli anahtarı, kotası, süresi, ad-tag'i ve kişisel `tg://proxy` bağlantısı olan sıradan istemcilerdir. inbound düzeyindeki «Secret» alanı kaldırıldı (mevcut inbound'lar otomatik dönüştürülür), «FakeTLS domain» yeni gizli anahtarlar için varsayılan alan adı oldu. Yeni inbound alanları: **Max connections** (bağlantı sınırı), **Public IPv4/IPv6** (ad-tag middle proxy için). İstemci değişiklikleri, başkalarının Telegram oturumlarını düşürmeden «anında» uygulanır.
- WireGuard: inbound menüsü eksiksiz istemci işlemleri setine kavuştu (Export All URLs, bağlama/bağlantı kesme, gruplar), dışa aktarma **Config** ve **Links** sekmelerine ayrıldı, «**WireGuard izin verilen IP'ler**» alanı **düzenlenebilir oldu** (virgülle ayrılmış birden fazla kayıt), düğümdeki inbound'un istemci yapılandırmasında `Endpoint` artık düğümün adresini gösteriyor.

Bölüm 7 değişiklikleri – Bağlantı Güvenliği: TLS,XTLS ve REALITY

- **Finalmask + REALITY kombinasyonu** kaydetme sırasında **reddediliyor** (ilk bağlantıda Xray-core'un çökmesine yol açıyordu); minClientVer yer tutucusu 26.3.27'ye güncellendi.
- Yeni Finalmask TCP maskesi türü – **XMC (Minecraft)**: akışın Minecraft trafiği gibi maskelenmesi (Hostname, Usernames, otomatik oluşturmalı zorunlu Password).

Bölüm 8 değişiklikleri – İstemciler

- Yeni «**Hız**» sütunu – her istemcinin canlı hızı (↑/↓, ~5 saniyelik kayan ortalama).
- İstemci araması yeniden **Telegram ID** ile bulabiliyor; istemci formunda devre dışı inbound'lar bağlama listesinden gizleniyor; «Bağlantıyı Kes» penceresindeki kopya kayıt birikimi düzeltildi.
- MTProto istemcilerinin kendi alanları var: «**MTProto secret**» (yeniden oluşturmalı) ve «**Ad-tag (sponsörlü kanal)**» (32 hex karakter); kota ve süre artık MTProto'ya gerçekten uygulanıyor.

Bölüm 9 değişiklikleri – İstemci Grupları

- İstemci bilgi penceresinde artık istemcinin **grubu** gösteriliyor.

Bölüm 10 değişiklikleri – Abonelikler (Subscription)

- **Yönetilen hostlar grup haline geldi**: tek kayıt **birden fazla inbound** (çoklu seçim) ve **birden fazla adres** (etiketler; her kaydın kendi :port 'u olabilir; adres otomatik tamamlama; boş – inbound adresi devralınır) kapsıyor. Liste sütunları adres ve inbound çiplerini gösteriyor («+N» ile), işlemler ve API gruplarla çalışıyor (groupId),toplu POST /panel/api/hosts/bulk/add eklendi. Host sıralaması artık geneldir (sıraya, ardından açıklamaya göre).
- **Duyuru** metni (subAnnounce) artık abonelik bilgi sayfasında bir afiş olarak gösteriliyor; özel şablonlarda announce değişkeni kullanılabilir.
- Bilgi sayfası tarayıcıda artık **JSON/Clash bağlantılarıyla** da açılıyor (yalnızca ana bağlantıyla değil).
- Host ayarları **Final Mask** ve **Allow insecure** artık sırasıyla raw bağlantılarda (fm=) ve **Hysteria2** için de (insecure=1 / skip-cert-verify: true) geçerli.
- «Güncelleme aralıkları» (subUpdates) aralığı **0–525600** olarak düzeltildi (önceki 168 arayüz sınırı, 2.x'ten yükseltme sonrasında ayarların kaydedilmesini engelliyordu).
- Yerel **WireGuard istemcileri** artık **Clash ve JSON aboneliklerine giriyor** (önceden yalnızca raw'a).

Bölüm 11 değişiklikleri – Xray: Yönlendirme, outbounds, DNS ve Uzantılar

- Outbound düzenleyicisi: yeni «**Target Strategy**» alanı (AsIs 'ten ForceIPv4 'e 11 değer), «**Real delay**» test modu (tünel kurulumu dahil tam süre; HTTP modu artık «ısınmış» bağlantı üzerinden ölçülüyor), HTTP/Real testinden sonra **Egress** («göz» arkasında çıkış IP'si) ve **Country** (bayrak + ülke, WARP etiketi) sütunları.
- **Yük dengeleyicinin fallback'i başka bir yük dengeleyiciyi gösterebilir**: panel gizli loopback nesnesini (_bl_...) kendisi kurar, döngülere ve kullanılan yük dengeleyicinin silinmesine karşı korur; _bl_ öneki ayrılmıştır.
- «Temel Yönlendirme» sekmesine «**Default Outbound**» seçicisi eklendi – hiçbir kurala girmeyen trafiği hangi outbound'un işleyeceği (seçilen, ilk konuma taşınır).

- Özel IP'lerdeki DNS sunucuları artık `geoip:private` kuralıyla engellenmiyor – panel yönetilen bir izin kuralını kendisi sürdürüyor.
- dial (sockopt) ayarlarındaki Happy Eyeballs artık gerçekten etkinleşiyor; «Try delay» varsayılını 250 ms, açıkça girilen 0 korunuyor.
- outbound abonelik içe aktarması: `?plugin=` /sondaki `/` içeren `ss://` bağlantılarında port doğru ayrıştırılıyor.

Bölüm 12 değişiklikleri – Düğümler (Çok Panelli, master/slave)

- Düzeltme paketi: istemciyi değişiklik yapmadan kaydetmek artık düğüm inbound'larının canlı trafiğini bozmuyor; düğümün Host geçersiz kılmaları ilk kabulde master'a alınıyor; otomatik yenileme yeni bir kota penceresi açıyor; istemcinin master'da silinmesi onu düğümlerde tamamen siliyor; düğümün inbound'ları ilk kabulden önce süpürülüyor; tek bir hatalı inbound düğümün trafik senkronizasyonunu durdurmuyor; port çakışması denetimi kendi düğümüyle sınırlandırıldı.

Bölüm 14 değişiklikleri – Telegram Botu

- Bot komut menüsüne `/usage`, `/inbound`, `/restart` ve yeni yönetici komutu `/clearall` (tüm istemcilerin trafiğini sıfırlama, onaylı) eklendi.
- Çevrimiçi istemci listesi `email – inbound` açıklaması olarak etiketleniyor; yedek ve ban günlüğü mesajları ana bilgisayar adını içeriyor; Telegram ID araması, ayarların biçimlendirmesinden bağımsız çalışıyor.

Bölüm 16 değişiklikleri – İşletim: Yedeklemeler, Günlükler, Güncelleme, CLI

- **PostgreSQL panelinde geri yükleme SQLite dosyalarını kabul ediyor:** normal `.db` yedeği veya geçiş `.dump` dosyası doğrudan PostgreSQL'e aktarılıyor (tek işlemle, Xray durdurulmadan önce denetimlerle). Dosya seçim iletişim kutusu her iki DBMS'te de `.dump`, `.db` kabul ediyor; «Geçiş Dosyasını İndir» yalnızca PostgreSQL panellerinde kaldı.
- Bir `pg_dump` arşivini geri yüklemekten önce panel dökümün okunabilirliğini denetliyor ve sürüm uyumsuzluğunda tam `x-ui pgclient <sürüm>` komutunu öneriyor.
- Başlangıçta otomatik onarımlar: taşan trafik sayaçları sınırlandırılıp kurtarılıyor; inbound portundaki eski UNIQUE kısıtlaması kaldırılıyor (multi-node'u engelliyordu).
- Xray günlükleri: yeni görev, access ve error günlüğü **64 MiB**'ı aştığında her 10 dakikada bir kırıyor; günlük temizleme artık her ikisini de temizliyor.
- Docker: sertifikaların otomatik yenilenmesi onarıldı (crond başlatılıyor, acme.sh durumu bir volume'de saklanıyor).

1. Giriş, Gereksinimler ve Kurulum

1.1. 3X-UI Nedir

3X-UI – [Xray-core](#) sunucuları için açık kaynaklı bir web yönetim panelidir. Panel, tek bir VPS'ten birden fazla düğümden (node) oluşan dağıtık yapılandırmalara kadar geniş bir yelpazede proxy ve VPN protokollerini dağıtmak, yapılandırmak ve izlemek amacıyla çok dilli, birleşik bir web arayüzü sunar.

3X-UI, orijinal X-UI projesinin geliştirilmiş bir çatıdır. Buna kıyasla daha fazla protokol desteği, artırılmış kararlılık, istemci bazında trafik takibi ve pek çok kullanıcı özelliği eklenmiştir.

Temel özellikler:

- **Farklı protokollerde Inbound'lar** – VLESS, VMess, Trojan, Shadowsocks, WireGuard, Hysteria2, HTTP, SOCKS (Mixed), Dokodemo-door / Tunnel, TUN ve **MTProto** (3.3.0 sürümüyle eklenen Telegram proxy'si).
- **Modern taşıma katmanları ve şifreleme** – TCP (Raw), mKCP, WebSocket, gRPC, HTTPUpgrade ve XHTTP; TLS, XTLS ve REALITY ile korunan.
- **Fallback** – Xray'in fallback mekanizması aracılığıyla tek bir portta (örneğin 443 numaralı portta VLESS ve Trojan gibi) birden fazla protokole hizmet verme.
- **İstemci bazında yönetim** – trafik kotaları, son kullanma tarihleri, IP limitleri, "çevrimiçi" durum gösterimi, tek tıkla davet bağlantıları, QR kodları ve abonelikler.
- **Trafik istatistikleri** – her inbound, istemci ve outbound için ayrı ayrı, sıfırlama imkânıyla birlikte.
- **Çok düğümlü (node) destek** – tek bir panelden birden fazla sunucuyu yönetme ve ölçeklendirme.
- **Outbound ve yönlendirme** – WARP, NordVPN, özel yönlendirme kuralları, yük dengeleyiciler, proxy zincirleri.
- **Yerleşik abonelik sunucusu** çoklu çıktı biçimleriyle birlikte.
- **Telegram botu** uzaktan izleme ve yönetim için.
- **REST API** yerleşik Swagger belgeleriyle.
- **Esnek depolama** – SQLite (varsayılan) veya PostgreSQL.
- **13 arayüz dili**, koyu ve açık tema.
- **Fail2ban entegrasyonu** istemci bazında IP limitlerinin uygulanması için.

Önemli: Proje yalnızca kişisel kullanım amacıyla tasarlanmıştır. Yasadışı amaçlarla veya üretim ortamında kullanılması önerilmez.

1.2. Desteklenen İşletim Sistemleri ve Mimariler

İşletim Sistemleri

Kurulum betiği, dağıtımı `/etc/os-release` (veya `/usr/lib/os-release`) dosyasındaki `ID` alanına göre belirler. Resmi olarak desteklenenler:

Ubuntu, Debian, Armbian, Fedora, CentOS, RHEL, AlmaLinux, Rocky Linux, Oracle Linux, Amazon Linux, Virtuozzo, Arch, Manjaro, Parch, openSUSE (Tumbleweed / Leap), Alpine ve Windows.

Alpine tabanlı sistemlerde OpenRC servisi (`rc-service` / `rc-update`) kullanılırken, diğerlerinde systemd kullanılır. CentOS 7 için paketler `yum` aracılığıyla, daha yeni sürümler için ise `dnf` aracılığıyla kurulur. Dağıtım tanınmazsa betik varsayılan olarak `apt-get` paket yöneticisini kullanmayı dener.

İşlemci Mimarileri

Mimari, `uname -m` çıktısına göre belirlenir ve desteklenen değerlerden birine dönüştürülür:

uname -m değeri	3X-UI mimarisi
x86_64 , x64 , amd64	amd64
i*86 , x86	386
armv8* , arm64 , aarch64	arm64
armv7* , arm	armv7
armv6*	armv6
armv5*	armv5
s390x	s390x

Mimari bu listede yer almıyorsa betik "Unsupported CPU architecture!" mesajını gösterir ve kurulumu durdurur.

Temel Bağımlılıklar

Panel kurulmadan önce betik, gerekli temel paket grubunu otomatik olarak kurar (paket adları dağıtıma göre değişebilir): `cron` / `cronie` / `dcron` , `curl` , `tar` , `tzdata` / `timezone` , `socat` , `ca-certificates` , `openssl` .

1.3. Kurulum Yöntemleri

Yöntem 1. Kurulum Betiği (Önerilen)

Kurulum, root kullanıcısı olarak tek bir komutla gerçekleştirilir:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh)
```

Betik, root yetkisi zorunlu kılar: root olmayan bir kullanıcı tarafından çalıştırıldığında "Please run this script with root privilege" mesajı gösterilir ve hatayla sonlanır.

Kurucunun adım adım yaptıkları:

1. İşletim sistemini ve mimariyi belirler.
2. Temel bağımlılıkları kurar.
3. `x-ui-linux-<arch>.tar.gz` sürüm arşivini indirir ve `/usr/local/x-ui` dizinine açar.
4. `x-ui.sh` yönetim betiğini indirir ve `/usr/bin/x-ui` komutu olarak kurar.
5. `/var/log/x-ui` log dizinini oluşturur.
6. İlk yapılandırmayı başlatır: veritabanı seçimi, kimlik bilgisi oluşturma, port seçimi, isteğe bağlı SSL yapılandırması.

7. Otomatik başlatma servisini kurar ve başlatır (Alpine için `systemd` birimi `x-ui.service` veya OpenRC init betiği).

Kurulum sırasında veritabanı seçimi. Kurucu şu seçenekleri sunar:

- 1) SQLite (varsayılan; 500'den az istemci için önerilir) – `/etc/x-ui/x-ui.db` adresinde tek bir dosya, yapılandırma gerektirmez.
- 2) PostgreSQL (çok sayıda istemci veya birden fazla node için önerilir). PostgreSQL yerel olarak kurulabilir (özel bir kullanıcı ve `xui` adında bir veritabanı oluşturulur) ya da mevcut bir sunucuya DSN bağlantı dizesi belirtilebilir. Bağlantı parametreleri, dağıtıma bağlı olarak servis ortam dosyasına (`/etc/default/x-ui` , `/etc/conf.d/x-ui` veya `/etc/sysconfig/x-ui`) `XUI_DB_TYPE` ve `XUI_DB_DSN` değişkenleri biçiminde yazılır.

Örnek: PostgreSQL parametrelerinin servis ortam dosyasına yazılması. PostgreSQL seçilip DSN belirtildikten sonra kurucu ortam dosyasına aşağıdakine benzer satırlar ekler:

```
XUI_DB_TYPE=postgres
XUI_DB_DSN=postgres://xui:S3cretPass@127.0.0.1:5432/xui?sslmode=disable
```

Burada `xui` kullanıcı adı ve veritabanı adıdır; `127.0.0.1:5432` sunucu adresi ve portudur; `sslmode=disable` yerel bağlantı için uygundur (uzak sunucu için genellikle `require` kullanılır).

Belirli bir (eski) sürümün kurulumu. Sürüm etiketi açıkça belirtilebilir; kurucu ilgili sürümü indirir:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/v2.4.0/install.sh)
v2.4.0
```

Bu kurulum için desteklenen en düşük sürüm `v2.3.5` 'tir; daha eski bir sürüm belirtilirse "Please use a newer version (at least v2.3.5)" mesajı görüntülenir.

Geliştirici (dev) derlemesinin kurulumu. Sürüm etiketi yerine `dev-latest` (takma adı `dev`) bağımsız değişkeni de kabul edilir; bu, `main` dalının en son commit'ine dayalı sürekli güncellenen dev derlemesini kurar:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh) dev-
latest
```

Dev derlemesi, her commit için oluşturulan bir ön sürümdür (`dev-latest` etiketi) ve kararlı bir sürüm değildir; bu nedenle minimum sürüm kontrolü uygulanmaz. Çalıştırıldığında "Installing the rolling dev build (tag: dev-latest). This is a per-commit pre-release, not a stable version." uyarısı görüntülenir. Bağımsız değişken belirtilmezse kurucu en son kararlı sürümü kurar. Dev derlemesini kullanmak yalnızca henüz yayımlanmamış düzeltmeleri test etmek için anlamlıdır; normal kullanımda kararlı sürümleri tercih edin.

Yöntem 2. Docker

Varsayılan SQLite veritabanıyla çalıştırmak için:

```
docker compose up -d
```

Yerleşik PostgreSQL servisiyle çalıştırmak için `docker-compose.yml` dosyasındaki `XUI_DB_*` satırlarının yorumunu kaldırın ve profile başlatın:

```
docker compose --profile postgres up -d
```

İmaj, istemcilere uygulanan IP limitlerini zorunlu kılmak için Fail2ban içerir (varsayılan olarak etkin). Fail2ban, ihlal edenleri iptables üzerinden engeller; bu da NET_ADMIN yetkisini gerektirir. `docker-compose.yml` dosyasında bu yetki `cap_add` aracılığıyla zaten tanımlanmıştır. 3.5.0'dan itibaren konteynerde SSL menüsü üzerinden verilen sertifikaların **otomatik yenilenmesi** onarıldı: entrypoint `cron`'u başlatır ve `acme.sh` cron görevini yeniden kaydeder; `docker-compose.yml` ise `acme.sh` durumu için ayrı bir volume aldı – yenileme, konteynerin yeniden oluşturulmasını atlatır (önceden sertifikalar ~90 gün sonra sessizce doluyordu). `docker run` ile manuel başlatmada bu yetkiyi kendiniz eklemeniz gerekir, aksi takdirde engeller yalnızca loglanır ancak uygulanmaz:

Örnek: eksiksiz docker run komutu. Panel portunun yönlendirilmesi, ağ yetkileri ve veritabanı için kalıcı volume ile minimal kullanım:

```
docker run -d \
  --name 3x-ui \
  --restart unless-stopped \
  --cap-add=NET_ADMIN --cap-add=NET_RAW \
  -v $PWD/db:/etc/x-ui \
  -v $PWD/cert:/root/cert \
  -p 2053:2053 \
  ghcr.io/mhsanaei/3x-ui:latest
```

`/etc/x-ui` volume'u, container yeniden başlatmaları arasında `x-ui.db` dosyasını korur; aksi takdirde ayarlar ve hesaplar kaybolur.

```
docker run -d --cap-add=NET_ADMIN --cap-add=NET_RAW ... ghcr.io/mhsanaei/3x-ui
```

Docker'da panel, container'ın ana sürecidir: otomatik başlatma, container içindeki bir servis tarafından değil, container'ın yeniden başlatma politikasıyla (örneğin `restart: unless-stopped`) yönetilir.

1.4. İlk Başlatma ve Varsayılan Kimlik Bilgileri

İlk kurulumda (varsayılan kimlik bilgileri henüz kullanımdayken) kurucu kullanıcı adı, parola, web yolu ve port için **rastgele değerler oluşturur**:

Parametre	Kurulumda nasıl oluşturulur	Not
Kullanıcı adı (Username)	10 karakterlik rastgele dize	otomatik oluşturulur
Parola (Password)	10 karakterlik rastgele dize	otomatik oluşturulur

Parametre	Kurulumda nasıl oluşturulur	Not
Panel web yolu (WebBasePath)	18 karakterlik rastgele dize	panelin kök URL üzerinden keşfedilmesini engeller
Panel portu (Port)	varsayılan olarak 1024–62000 aralığında rastgele port; istenirse manuel olarak belirtilebilir	<code>webPort</code> 'un "fabrika" değeri 2053 'tür, ancak kurucu bunu yazar

Kurulum sonunda betik özet bilgileri görüntüler: kullanıcı adı, parola, port, web yolu, API token'ı ve şu biçimde hazır bir giriş bağlantısı (Access URL):

```
https://<domain-veya-IP>:<port>/<web-yolu>
```

SSL sertifikası yapılandırılmamışsa bağlantı `http://` ile başlar ve betik SSL yapılandırması gerektiğine dair uyarı gösterir (menü maddesi 19).

Kimlik bilgilerinin değiştirilmesi zorunludur. Giriş adı ve parola rastgele oluşturulduğundan, **kurulumdan hemen sonra kaydedilmesi** gerekir. Bunlar, "Reset Username & Password" menü maddesiyle (aşağıya bakın) veya panel ayarlarındaki web arayüzünden her zaman değiştirilebilir. Sıfırlamanın ardından betik şunu hatırlatır: "Please use the new login username and password to access the X-UI panel. Also remember them!".

Kurulumdan sonra yönetim menüsünü açmak için `x-ui` komutu kullanılır (bkz. Bölüm 1.6).

1.5. Dosya Konumları

Yol	Amaç
<code>/usr/local/x-ui/</code>	panel kurulum dizini (<code>x-ui</code> ikili dosyası, <code>x-ui.sh</code> betiği)
<code>/usr/local/x-ui/bin/xray-linux-<arch></code>	Xray-core ikili dosyası (armv5/armv6/armv7'de <code>xray-linux-arm</code> olarak yeniden adlandırılır)
<code>/usr/bin/x-ui</code>	yönetim betiği (<code>x-ui</code> komutu)
<code>/etc/x-ui/x-ui.db</code>	SQLite veritabanı dosyası (varsayılan)
<code>/var/log/x-ui/</code>	panel log dizini
<code>/etc/systemd/system/x-ui.service</code>	servis systemd birimi (Alpine dışı)
<code>/etc/init.d/x-ui</code>	OpenRC init betiği (yalnızca Alpine)
<code>/etc/default/x-ui</code> · <code>/etc/conf.d/x-ui</code> · <code>/etc/sysconfig/x-ui</code>	servis ortam değişkenleri dosyası (yol dağıtımına göre değişir); <code>XUI_DB_TYPE</code> / <code>XUI_DB_DSN</code> buraya yazılır

Veritabanı dizini `XUI_DB_FOLDER` ortam değişkeniyle (varsayılan `/etc/x-ui`), Xray ikili dosyaları dizini ise `XUI_BIN_FOLDER` değişkeniyle (varsayılan olarak panel dizinine göreli `bin`) değiştirilebilir. Veritabanı dosyasının adı `x-ui.db` 'dir.

Örnek: veritabanını ayrı bir diske taşıma. `x-ui.db` dosyasını `/etc/x-ui` yerine örneğin bağlı bir disk olan `/data` 'da saklamak için servis ortam dosyasına değişkeni ekleyin ve paneli yeniden başlatın:

```
echo 'XUI_DB_FOLDER=/data/x-ui' >> /etc/default/x-ui
mkdir -p /data/x-ui
systemctl restart x-ui
```

Veritabanının tam yolu `/data/x-ui/x-ui.db` olur.

Temel Ortam Değişkenleri

Değişken	Amaç	Varsayılan
<code>XUI_DB_TYPE</code>	veritabanı arka ucu: <code>sqlite</code> veya <code>postgres</code>	<code>sqlite</code>
<code>XUI_DB_DSN</code>	PostgreSQL bağlantı dizesi (<code>XUI_DB_TYPE=postgres</code> olduğunda)	–
<code>XUI_DB_FOLDER</code>	SQLite veritabanı dosyasının dizini	<code>/etc/x-ui</code>
<code>XUI_INIT_WEB_BASE_PATH</code>	web panelinin başlangıç URI yolu (yalnızca ilk başlatmada)	<code>/</code>
<code>XUI_DB_MAX_OPEN_CONNS</code>	maksimum açık bağlantı sayısı (PostgreSQL havuzu)	–
<code>XUI_DB_MAX_IDLE_CONNS</code>	maksimum boşta bağlantı sayısı (PostgreSQL havuzu)	–
<code>XUI_ENABLE_FAIL2BAN</code>	Fail2ban aracılığıyla IP limitlerini etkinleştir	<code>true</code>
<code>XUI_LOG_LEVEL</code>	log düzeyi (<code>debug</code> , <code>info</code> , <code>warning</code> , <code>error</code>)	<code>info</code>
<code>XUI_DEBUG</code>	hata ayıklama modu	<code>false</code>

Örnek: ayrıntılı loglama geçici olarak etkinleştirme. Bir sorunu teşhis etmek için log düzeyini `debug` olarak yükseltin ve servisi yeniden başlatın:

```
echo 'XUI_LOG_LEVEL=debug' >> /etc/default/x-ui
systemctl restart x-ui
x-ui log # hata ayıklama logunu görüntüle
```

Teşhis tamamlandıktan sonra, logların şişmemesi için değeri `info` olarak geri alın.

Ortam değişkeni üzerinden web paneli başlangıç yolu. `XUI_INIT_WEB_BASE_PATH` değişkeni, ilk başlatmada web panelinin URI yolunu (`webBasePath`) belirler. Bu, Docker veya systemd üzerinden dağıtımda panel giriş yolunu baştan sabitlemek için kullanışlıdır. Değer otomatik olarak normalleştirilir – baştaki ve sondaki eğik çizgiler gerektiğinde eklenir; boş veya yalnızca boşluklardan oluşan bir değer yok sayılır (bu durumda varsayılan yol `/` uygulanır). Değişken **yalnızca ilk başlatmayı etkiler**: ayarlar zaten oluşturulmuşsa yol, web arayüzünden veya "Reset Web Base Path" menü maddesiyle değiştirilir.

1.6. x-ui Yönetim Komutu (Betik Menüsü)

Kurulumun ardından `x-ui` komutu (root olarak çalıştırılır) "3X-UI Panel Management Script" etkileşimli menüsünü açar. Madde seçimi, numarası girilerek yapılır (0–27 aralığı). Pek çok madde, betikler için doğrudan alt komut olarak da kullanılabilir (bkz. Bölüm 1.7).

Menü tematik bloklara ayrılmıştır.

Kurulum ve Güncelleme

- **1. Install** – paneli kurar (`install.sh` çalıştırır). Kurulmadan önce panelin henüz kurulu olmadığı doğrulanır.
- **2. Update** – tüm x-ui bileşenlerini en son sürüme günceller. Veriler korunur; güncelleme sonrasında panel otomatik olarak yeniden başlatılır. Onay gerektirir.
- **3. Update Menu** – paneli yeniden kurmadan yalnızca yönetim betiğini (`x-ui.sh` / `x-ui` komutu) güncel sürüme günceller.
- **4. Legacy Version** – panelin belirtilen (eski) bir sürümünü kurar. Betik sürüm numarasını sorar (örneğin `2.4.0`) ve ilgili sürümü indirir.
- **5. Uninstall** – paneli **Xray ile birlikte** tamamen kaldırır. Servis durdurulur ve devre dışı bırakılır; `/etc/x-ui/` ve `/usr/local/x-ui/` dizinleri, servis ortam dosyası ve yönetim betiği silinir. Onay gerektirir (varsayılan "hayır").

Kimlik Bilgileri ve Ayarlar

- **6. Reset Username & Password** – panel kullanıcı adı ve parolasını sıfırlar. Kendi değerlerinizi girebilir veya rastgele oluşturulması için boş bırakabilirsiniz (rastgele ad 10 karakter, rastgele parola 18 karakter). Ayrıca 2FA yapılandırıldıysa devre dışı bırakılması önerilir. Sıfırlamanın ardından panel yeniden başlatılır.
- **7. Reset Web Base Path** – panel web yolunu sıfırlar: yeni bir rastgele yol (18 karakter) oluşturulur ve panel yeniden başlatılır. Önceki yol ele geçirildiyse veya unutulduysa kullanılır.
- **8. Reset Settings** – tüm panel ayarlarını varsayılan değerlerine sıfırlar. **Kimlik bilgileri (kullanıcı adı ve parola) ile hesap verileri korunur.** Onay gerektirir; sıfırlamanın ardından panel yeniden başlatılır.
- **9. Change Port** – web paneli portunu değiştirir. Port numarası istenir (1–65535); değişikliğin geçerli olması için yeniden başlatma gerekir.
- **10. View Current Settings** – mevcut ayarları görüntüler (`x-ui setting -show`). Kullanılan veritabanı arka ucunu (DSN'de parola maskelenerek SQLite veya PostgreSQL) ve hazır erişim bağlantısını (Access URL) gösterir. SSL yapılandırılmamışsa IP adresi için Let's Encrypt sertifikası almayı önerir.

Servis Yönetimi

- **11. Start** – panel servisini başlatır. Panel zaten çalışıyorsa tekrar başlatmaya gerek olmadığını belirten bir mesaj görüntülenir.
- **12. Stop** – panel servisini durdurur.
- **13. Restart** – panel servisini yeniden başlatır.
- **14. Restart Xray** – paneli yeniden başlatmadan yalnızca Xray-core çekirdeğini yeniden başlatır (`systemctl reload x-ui` ile; Docker'da panel sürecine `USR1` sinyali gönderilerek).
- **15. Check Status** – servis durumunu kontrol eder (`systemctl status x-ui` veya `rc-service x-ui status`).
- **16. Logs Management** – log yönetimi: hata ayıklama logunu görüntüleme (Debug Log, `journalctl` aracılığıyla) ve Alpine dışı sistemlerde tüm logları temizleme (Clear All logs).

Otomatik Başlatma

- **17. Enable Autostart** – işletim sistemi açılışında panelin otomatik başlatılmasını etkinleştirir (`systemctl enable x-ui` veya `rc-update add`).
- **18. Disable Autostart** – işletim sistemi açılışında otomatik başlatmayı devre dışı bırakır.

Docker'da otomatik başlatma, container'ın yeniden başlatma politikasıyla yönetildiğinden bu maddeler yalnızca ilgili bir ipucu görüntüler.

Güvenlik ve Ağ

- **19. SSL Certificate Management** – acme.sh aracılığıyla SSL sertifika yönetimi: domain için sertifika alma, iptal etme, zorunlu yenileme, mevcut domainleri görüntüleme, panel için sertifika yollarını belirtme ve IP adresi için kısa ömürlü (~6 günlük, otomatik yenilenen) sertifika alma.
- **20. Cloudflare SSL Certificate** – Cloudflare DNS doğrulaması aracılığıyla SSL sertifikası alma.
- **21. IP Limit Management** – istemci bazında IP limiti yönetimi (Fail2ban tabanlı): engellemeleri görüntüleme, kaldırma vb.
- **22. Firewall Management** – güvenlik duvarı yönetimi (port açma/kapatma ve kural görüntüleme).
- **23. SSH Port Forwarding Management** – SSH tüneli aracılığıyla paneli yerel makineden açabilmek için SSH port yönlendirme yapılandırması.

Performans ve Bakım

- **24. Enable BBR** – TCP BBR tıkanıklık kontrolü algoritmasını etkinleştirme/devre dışı bırakma (Enable BBR / Disable BBR seçeneklerini içeren alt menü).
- **25. Update Geo Files** – kaynak seçimiyle geo veritabanlarını günceller (`.dat` dosyaları): Loyalsoldier (`geoip.dat` , `geosite.dat`), chocolate4u (`geoip_IR.dat` , `geosite_IR.dat`), runetfreedom (`geoip_RU.dat` , `geosite_RU.dat`) veya All (tümü aynı anda). Güncellemeden sonra panel yeniden başlatılır.
- **26. Speedtest by Ookla** – Speedtest by Ookla aracılığıyla ağ hızı testi çalıştırır.
- **27. PostgreSQL Management** – yerleşik/bağlı PostgreSQL örneğini yönetir (etkinleştirme ve ilgili işlemler).
- **0. Exit Script** – menüden çıkar.

1.7. x-ui Alt Komutları (Etkileşimsiz Menü)

Betiklerde kullanmak için `x-ui` komutu doğrudan alt komutları destekler (`x-ui` bağımsız değişken olmadan çalıştırıldığında menü açılır):

Komut	İşlem
<code>x-ui</code>	yönetim menüsünü aç
<code>x-ui start</code>	paneli başlat
<code>x-ui stop</code>	paneli durdur
<code>x-ui restart</code>	paneli yeniden başlat
<code>x-ui restart-xray</code>	Xray'i yeniden başlat

Komut	İşlem
<code>x-ui status</code>	mevcut servis durumu
<code>x-ui settings</code>	mevcut ayarlar
<code>x-ui enable</code>	işletim sistemi açılışında otomatik başlatmayı etkinleştir
<code>x-ui disable</code>	otomatik başlatmayı devre dışı bırak
<code>x-ui log</code>	logları görüntüle
<code>x-ui banlog</code>	Fail2ban engelleme loglarını görüntüle
<code>x-ui update</code>	paneli güncelle
<code>x-ui update-all-geofiles</code>	tüm geo dosyalarını güncelle
<code>x-ui migrateDB [file]</code>	<code>.db</code> <code>?</code> <code>.dump</code> dönüşümü (SQLite)
<code>x-ui legacy</code>	eski sürümü kur
<code>x-ui install</code>	paneli kur
<code>x-ui uninstall</code>	paneli kaldır

1.8. SQLite → PostgreSQL Geçişi

Mevcut SQLite kurulumu PostgreSQL'e taşınabilir:

```
x-ui migrate-db --dsn "postgres://xui:password@127.0.0.1:5432/xui?sslmode=disable"
# ardından /etc/default/x-ui dosyasına XUI_DB_TYPE ve XUI_DB_DSN değerlerini girin ve
yeniden başlatın:
systemctl restart x-ui
```

Kaynak SQLite dosyası değiştirilmeden bırakılır – yeni arka ucun çalıştığını doğruladıktan sonra yalnızca elle silin.

Örnek: PostgreSQL'e geçişi doğrulama. Geçişin ardından panelin gerçekten yeni arka uç üzerinde çalıştığını ayar görüntüleme komutuyla doğrulayın – çıktıda PostgreSQL belirtilmiş olmalıdır (DSN'deki parola maskelenir):

```
x-ui settings | grep -i -E 'db|dsn'
```

Panel açılıyorsa ve hesaplar yerindeyse kaynak `x-ui.db` dosyası silinebilir.

2. Panele Giriş ve Erişim Güvenliği

Bu bölüm, 3X-UI paneli yönetici kimlik doğrulamasıyla ilgili her şeyi açıklar: giriş formu, iki faktörlü kimlik doğrulama (TOTP), parola deneme saldırısına karşı koruma, kimlik bilgilerinin değiştirilmesi, gizli yol ve panel portunun değiştirilmesi, oturum ömrü ve LDAP üzerinden senkronizasyon/kimlik doğrulama.

2.1. Giriş Formu

Giriş sayfası, panelin gizli yolunun (`webBasePath`) kökünde sunulur. Kullanıcı zaten oturum açmışsa otomatik olarak `.../panel/` adresine yönlendirilir. Sayfada tema değiştirici, arayüz dili seçimi ve giriş formu bulunur.

Form alanları:

Alan	İpucu/Başlık	Zorunlu	Açıklama
Kullanıcı Adı	«Kullanıcı Adı»	Evet	Yönetici girişi. Boş değer istemci tarafında reddedilir; sunucuda «Kullanıcı adı girin» mesajıyla reddedilir.
Parola	«Parola»	Evet	Yönetici parolası. Boş değer «Parola girin» mesajıyla reddedilir.
2FA Kodu	«2FA Kodu»	Yalnızca 2FA etkinken	Alan yalnızca panelde iki faktörlü kimlik doğrulama etkinleştirilmişse görünür. Kimlik doğrulayıcı uygulamasından alınan 6 haneli kod.

«Giriş Yap» düğmesi formu `POST /login` adresine gönderir.

Davranış ve mesajlar:

- Başarılı girişte «Giriş başarılı» mesajı gösterilir ve `.../panel/` adresine yönlendirme yapılır.
- Hatalı kimlik bilgileri veya yanlış 2FA kodu durumunda sunucu **tek bir** mesaj döndürür: «Geçersiz hesap bilgileri.» (İng.: *Invalid username or password or two-factor code.*). Bu kasıtlıdır – panel neyin yanlış olduğunu (kullanıcı adı, parola veya kod) göstermez; böylece deneme saldırıları kolaylaştırılmaz.
- «2FA Kodu» alanını panel, `POST /getTwoFactorEnable` isteğine göre gösterir veya gizler; bu istek yetkilendirmeden önce mevcut 2FA durumunu döndürür.
- Sunucu oturumu dolmuşsa bir sonraki istekte «Oturum süresi doldu. Lütfen tekrar giriş yapın» mesajı gösterilir ve kullanıcı giriş sayfasına yönlendirilir.

CSRF notu: form gönderilmeden önce istemci bir CSRF belirteci alır (`GET /csrf-token`); `/login` ve `/logout` istekleri CSRF doğrulamasıyla korunur.

Örnek: API üzerinden giriş. 2FA devre dışıyken yalnızca kullanıcı adı ve parola yeterlidir; 2FA etkinken `twoFactorCode` alanı eklenir:

```
# 2FA olmadan
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
-H 'Content-Type: application/x-www-form-urlencoded' \
--data 'username=admin&password=ВашПароль'

# 2FA etkinken – 6 haneli kod eklenir
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
-H 'Content-Type: application/x-www-form-urlencoded' \
--data 'username=admin&password=ВашПароль&twoFactorCode=123456'
```

Başarı durumunda sunucu `Set-Cookie` başlığıyla bir oturum çerezi döndürür – bu çerez `/panel/api/...` adresine yapılan sonraki isteklerde kullanılmalıdır.

2.2. İki Faktörlü Kimlik Doğrulama (2FA / TOTP)

3X-UI'deki 2FA, **TOTP** standardıyla uygulanmıştır ve herhangi bir kimlik doğrulayıcı uygulamasıyla (Google Authenticator, Aegis, FreeOTP vb.) uyumludur. Parametreler sabit olarak tanımlıdır: algoritma **SHA1**, 6 hane, süre **30** saniye, yayıncı (issuer) `3x-ui`, etiket `Administrator`.

Örnek: QR kodunu şifreleyen otpauth URI'si. Kimlik doğrulayıcı uygulama kamerayla tarama yapamazsa, belirteci bu bağlantıyı kullanarak elle ekleyebilirsiniz (`JBSWY3DPEHPK3PXP` yerine kendi Base32 gizlinizi yazın):

```
otpauth://totp/3x-ui:Administrator?secret=JBSWY3DPEHPK3PXP&issuer=3x-
ui&algorithm=SHA1&digits=6&period=30
```

`algorithm=SHA1`, `digits=6`, `period=30` parametreleri panelin sabit değerleriyle örtüşür – bunları değiştirmeye gerek yoktur.

Ayarlar **Ayarlar** → **Hesap** bölümünde, «**İki Faktörlü Kimlik Doğrulama**» sekmesinde yer alır.

Öge	Metin	Açıklama
Geçiş anahtarı	«2FA'yı Etkinleştir»	İki faktörlü kimlik doğrulamayı açar/kapatır.
Açıklama	«Güvenliği artırmak için ek bir kimlik doğrulama katmanı ekler.»	Geçiş anahtarının altındaki ipucu.

2FA Nasıl Etkinleştirilir

Geçiş anahtarı açıldığında panel **yerel olarak yeni bir gizli anahtar üretir** – Base32 kodlamasıyla rastgele bir dize (`A–Z` ve `2–7` alfabeti). «İki faktörlü kimlik doğrulamayı etkinleştir» adlı bir pencere açılır ve adım adım yönergeler sunulur:

1. «**Bu QR kodunu kimlik doğrulayıcı uygulamanızda tarayın ya da QR kodunun yanındaki belirteci kopyalayıp uygulamaya yapıştırın**». QR kodunun altında gizli anahtar metin olarak gösterilir – QR koduna tıklandığında gizli anahtar panoya kopyalanır («Kopyalandı» bildirimi çıkar).
2. «**Uygulamadan kodu girin**» – uygulamanın ürettiği 6 haneli kodu girmeniz gerekir. Kod **tarayıcı tarafında** doğrulanır: panel az önce üretilen gizliyle geçerli TOTP'yi hesaplar ve girilen kodla karşılaştırır. Kod yanlışsa «Geçersiz kod»; alan yalnızca tam olarak 6 rakam kabul eder.

Başarılı onayın ardından gizli anahtar ve etkinleştirme bayrağı kaydedilir. Kaydedildiğinde «İki faktörlü kimlik doğrulama başarıyla kuruldu» mesajı gösterilir.

Önemli: ayarlar bölümündeki değişiklikler **«Kaydet»** düğmesiyle uygulanır; genellikle panel yeniden başlatması gerekir («Değişikliklerin geçerli olması için kaydedin ve paneli yeniden başlatın»). 2FA ilk kez etkinleştirildiğinde sunucu ayrıca **tüm etkin oturumları geçersiz kılar** (oturum açma dönemini artırır); bu nedenle ayar uygulandıktan sonra – artık 2FA koduyla – yeniden giriş yapılması gerekir.

2FA Nasıl Devre Dışı Bırakılır

Geçiş anahtarına tekrar tıklandığında «İki faktörlü kimlik doğrulamayı devre dışı bırak» penceresi açılır ve «İki faktörlü kimlik doğrulamayı devre dışı bırakmak için uygulamadan kodu girin.» ipucu görünür. Doğru kod girildiğinde (3.4.2'den itibaren kod **sunucuda** doğrulanır) bayrak ve gizli anahtar temizlenir; «İki faktörlü kimlik doğrulama başarıyla kaldırıldı» mesajı gösterilir.

Girişte Kod Doğrulama

Giriş sırasında sunucu kayıtlı gizliyi alır ve geçerli TOTP'yi gönderilen 2FA koduyla karşılaştırır. Eşleşmeme, başarısız giriş olarak değerlendirilir; ancak kullanıcıya yine aynı birleşik «Geçersiz hesap bilgileri.» mesajı gösterilir.

Erişim Kurtarma (recovery)

3X-UI'de ayrı bir «kurtarma kodları» mekanizması **yoktur**. Kimlik doğrulayıcı uygulamasına erişim kaybedilirse panel arayüzü üzerinden giriş kurtarılamaz. Tek yol, sunucudaki veritabanında doğrudan 2FA'yı devre dışı bırakmaktır: ayarlar tablosunda `twoFactorEnable` anahtarını `false` olarak sıfırlayın (gerekirse `twoFactorToken` da temizleyin) ve ardından paneli yeniden başlatın. Bu nedenle 2FA etkinleştirilirken gizli anahtarın (Base32 belirteci) güvenli bir yere kaydedilmesi önerilir.

Örnek: sunucuda 2FA'nın acil olarak devre dışı bırakılması. SSH üzerinden sunucuya erişin, paneli durdurun, ayarlar tablosundaki anahtarları sıfırlayın ve paneli yeniden başlatın:

```
x-ui stop
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='false' WHERE
key='twoFactorEnable';"
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='' WHERE key='twoFactorToken';"
x-ui start
```

Bundan sonra giriş yalnızca kullanıcı adı ve parolayla yapılır; 2FA istenirse yeniden kurulabilir.

Kimlik bilgileri değişikliğiyle ilişki: kullanıcı adı/parola değiştirildiğinde (bkz. 2.4) 2FA, eski gizli anahtarın yeni hesapta erişimi engellemesini önlemek amacıyla sunucuda **otomatik olarak devre dışı bırakılır**.

2.3. Giriş Denemelerini Sınırlandırma (login limiter / deneme saldırısına karşı koruma)

Panel, başarısız girişler için yerleşik bir sınırlayıcı içerir (uygulama düzeyinde fail2ban benzeri). Parametreler kod içinde tanımlıdır ve arayüz üzerinden **yapılandırılmaz**:

Parametre	Değer	Amaç
Maksimum başarısız deneme	5	Pencere içinde izin verilen başarısız deneme sayısı.
Sayım penceresi	5 dakika	Başarısız denemelerin biriktirildiği kayan pencere (daha eskiler atılır).
Engelleme süresi (cooldown)	15 dakika	Eşik aşıldıktan sonra anahtarın engellendiği süre.

Nasıl çalışır:

- Engelleme anahtarı «**IP + kullanıcı adı**» çiftinden oluşur (kullanıcı adı küçük harfe dönüştürülür, boşluklar kırpılır). Yani engelleme tüm panele değil, belirli bir adres + kullanıcı adı çiftine uygulanır.
- Her başarısız denemede (yanlış kullanıcı adı/parola veya yanlış 2FA kodu) sayaç artar. **5 dakika** içinde **5** başarısız denemeye ulaşıldığında anahtar **15 dakika** engellenir. Engelleme süresince bu çiftin tüm denemeleri, veriler doğru olsa bile aynı «Geçersiz hesap bilgileri.» mesajıyla anında reddedilir.
- Başarılı giriş sayacı anında sıfırlar** ve bu çiftin engelini kaldırır.
- İstemci IP adresi, güvenirli proxy'ler dikkate alınarak belirlenir (bkz. `trustedProxyCIDRs`): `X-Real-IP` ve `X-Forwarded-For` başlıkları yalnızca istek güvenirli bir adresten geldiyse kabul edilir. Aksi hâlde gerçek bağlantı adresi kullanılır; bu da çıkarılamazsa `unknown` dizesi kullanılır.

Tüm denemeler günlüğe kaydedilir. Başarısızlar için sunucu günlüğüne kullanıcı adı, IP, neden ve engellenmişse `blocked_until` zamanını içeren bir uyarı yazılır. Telegram botu aracılığıyla giriş bildirimi etkinleştirilmişse (`tgNotifyLogin` – «Giriş Bildirimi»), yönetici ek olarak başarılı, başarısız ve engellenmiş denemelerin kullanıcı adı, IP ve zamanını içeren bildirimler alır.

Örnek: Telegram'da giriş bildirimi. `tgNotifyLogin` etkinleştirildiğinde her denemeden sonra yönetici yaklaşık şu içerikteki bir mesaj alır:

```
Уведомление о входе
Пользователь: admin
IP: 203.0.113.45
Время: 2026-06-10 14:32:07
Статус: успешно
```

Engellenen «IP + kullanıcı adı» çifti için durum alanında denemenin sınırlayıcı tarafından reddedildiği belirtilir.

2.4. Yönetici Kullanıcı Adı ve Parolasının Değiştirilmesi

Ayarlar → **Hesap** bölümü, «**Yönetici Kimlik Bilgileri**» sekmesi. Alanlar:

Alan	Metin	Açıklama
Mevcut kullanıcı adı	«Mevcut Kullanıcı Adı»	Geçerli kullanıcı adı. Gerçek kullanıcı adıyla eşleşmezse değişiklik reddedilir.
Mevcut parola	«Mevcut Parola»	Kimliği doğrulamak için geçerli parola.

Alan	Metin	Açıklama
Yeni kullanıcı adı	«Yeni Kullanıcı Adı»	Yeni kullanıcı adı. Boş bırakılamaz.
Yeni parola	«Yeni Parola»	Yeni parola. Boş bırakılamaz.

Değişiklik **«Onayla»** düğmesiyle uygulanır ve `POST /panel/setting/updateUser` adresine gönderilir.

Sunucu mantığı ve mesajları:

- «Mevcut Kullanıcı Adı» gerçekte örtüşmüyorsa veya «Mevcut Parola» yanlışsa – «Yönetici kimlik bilgileri değiştirilirken bir hata oluştu.» mesajı ve «Geçersiz kullanıcı adı veya parola» açıklaması.
- Yeni kullanıcı adı veya yeni parola boşsa – «Yeni kullanıcı adı ve yeni parola doldurulmalıdır» açıklaması.
- Başarı durumunda – «Yönetici kimlik bilgilerinizi başarıyla değiştirdiniz.». Parola bcrypt karması olarak saklanır.

2FA koduyla onay (3.4.2'den itibaren). Panelde iki faktörlü kimlik doğrulama etkinse, kullanıcı adı/parola değişikliği ek olarak kimlik doğrulayıcı uygulamadaki **mevcut kodun girilmesini** gerektirir. **«Kimlik bilgilerini değiştir»** penceresi açılır ve «Yönetici kimlik bilgilerini değiştirmek için uygulamadan kodu girin.» ipucu gösterilir; kod (`twoFactorCode`) sunucuda doğrulanır, kod yanlış veya boşsa değişiklik uygulanmaz. Aynı onay artık 2FA devre dışı bırakılırken de gerekiyor (bkz. 2.2).

Örnek: API üzerinden kimlik bilgisi değiştirme. İstek geçerli bir oturum çerezi (girişte alınır) ve mevcut kullanıcı adı/parolanın onayını gerektirir:

```
curl -X POST https://panel.example.com:2053/мой-секрет/panel/setting/updateUser \
  -b 'session=BAWA_CECCIOHHAY_COOKIE' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'oldUsername=admin&oldPassword=СтарыйПароль&newUsername=root&newPassword=НовыйСложныйПароль'
```

Başarının ardından mevcut oturum geçersiz kılınır – yeni kimlik bilgileriyle yeniden giriş yapılması gerekir.

Kimlik bilgisi değiştirmenin önemli etkileri:

- **Tüm mevcut oturumlar geçersiz kılınır** (kullanıcının `login_epoch` sayacı artırılır); bu nedenle değiştirmenin ardından panel otomatik olarak çıkış yapar ve giriş sayfasına yönlendirir – yeniden giriş yapılması gerekir.
- Değiştirme sırasında **2FA etkinse, otomatik olarak devre dışı bırakılır** (bayrak ve gizli anahtar sıfırlanır). Kullanıcı adı/parola değişikliğinin ardından iki faktörlü kimlik doğrulamanın yeniden kurulması gerekir.

2FA etkinse form gönderilmeden önce «Kimlik bilgilerini değiştir» adlı bir pencere açılır ve «Yönetici kimlik bilgilerini değiştirmek için uygulamadan kodu girin.» ipucu gösterilir – kimlik bilgileri yalnızca geçerli 2FA kodu onaylanarak değiştirilebilir.

2.5. Gizli Yol (URI yolu / webBasePath) ve Panel Portu

Bu parametreler **Ayarlar** → **Panel** bölümünde yer alır ve panelin «görünmezliğini» ile erişilebilirliğini doğrudan etkiler. Kaydedildikten ve panel **yeniden başlatıldıktan** sonra geçerli olurlar.

Alan	Metin	Varsayılan Değer	Açıklama
Panel portu	«Panel Portu» (<code>panelPort</code>), ipucu «Panelin çalıştığı port»	2053	Web arayüzünün TCP portu.
URI yolu	«URI Yolu» (<code>panelUrlPath</code>), ipucu «/' ile başlamalı ve '/' ile bitmeli»	/	Gizli temel yol (<code>webBasePath</code>). Panel yalnızca bu yoldan erişilebilir (örn. <code>/мой-секрет/</code>).
Panel yönetim IP adresi	«Panel Yönetim IP Adresi» (<code>panelListeningIP</code>), ipucu «Herhangi bir IP'den bağlantıya izin vermek için boş bırakın»	boş	Panelin dinlediği adres. Boş = tüm arayüzler.
Panel alan adı	«Panel Alan Adı» (<code>panelListeningDomain</code>), ipucu «Herhangi bir alan adı ve IP'den bağlantıya izin vermek için boş bırakın.»	boş	Alan adına (Host) göre erişim kısıtlaması.
Panel sertifikası genel anahtar yolu	<code>publicKeyPath</code> , ipucu «/' ile başlayan tam yolu girin»	boş	Panel için HTTPS erişiminde kullanılan TLS sertifikası.
Panel sertifikası özel anahtar yolu	<code>privateKeyPath</code> , aynı ipucu	boş	TLS özel anahtarı.

Temel yolun (`webBasePath`) davranışı:

- Değer otomatik olarak normalleştirilir: `/` ile başlamıyorsa başa eklenir; `/` ile bitmiyorsa sona eklenir. Yani gerçekte yol her zaman `/.../` biçimindedir.
- Temel yol panelin kendisine, varlıklara ve oturum çerezine uygulanır (çerez yalnızca bu yol için verilir).

Güvenlik önerileri («Güvenlik Uyarıları» bölümü): yapılandırma «çok açık» olduğunda panel kendi uyarılarını gösterir:

- «Panel düz HTTP üzerinden çalışıyor – üretim ortamı için TLS yapılandırın.»
- «Standart 2053 portu yaygın bilinir – bunu rastgele bir port ile değiştirin.»
- «Varsayılan `/` temel yolu yaygın bilinir – bunu rastgele biriyle değiştirin.»

Başka bir deyişle, üretim sunucusu için **standart dışı bir port, öngörülemeyen bir URI yolu ve TLS sertifikası** ayarlanmalıdır.

Örnek: üretim ortamı için «gizli» panel yapılandırması. Ayarlar → **Panel** bölümünde aşağıdakine benzer değerler ayarlayın:

Alan	Değer
Panel portu	<code>34571</code> (2053 yerine rastgele)
URI yolu	<code>/aXf9Qm2/</code> (öngörülemeyen, <code>/</code> ile başlayıp biten)
Panel sertifikası genel anahtar yolu	<code>/etc/letsencrypt/live/panel.example.com/fullchain.pem</code>

Alan	Değer
Panel sertifikası özel anahtar yolu	/etc/letsencrypt/live/panel.example.com/privkey.pem

Kaydedip yeniden başlattıktan sonra panel yalnızca `https://panel.example.com:34571/aXf9Qm2/` adresinden erişilebilir olur ve güvenlik uyarıları kaybolur.

2.6. Oturum Ömrü (zaman aşımı)

«**Oturum Süresi**» (`sessionMaxAge`) alanı panel/aralık ayarları içinde yer alır.

Alan	Metin	Varsayılan Değer	Birim	Açıklama
Oturum süresi	«Oturum Süresi», ipucu «Sistemdeki oturum süresi (değer: dakika)»	360	dakika	Yönetici oturum çerezinin yaşam süresi.

Davranış:

- Değer **dakika** cinsinden girilir (varsayılan 360 dakika = 6 saat); çerez yapılandırılırken saniyeye çevrilir.
- Değer **0'dan büyükse** oturum çerezine karşılık gelen `MaxAge` atanır. Bu süre dolduğunda çerez geçerliliğini yitirir ve bir sonraki istekte kullanıcı «Oturum süresi doldu. Lütfen tekrar giriş yapın» mesajıyla karşılaşır.
- Oturum ayrıca kimlik bilgileri değiştirildiğinde veya 2FA ilk kez etkinleştirildiğinde (`login_epoch` mekanizmasıyla, bkz. 2.4 ve 2.2) ve açık çıkış yapıldığında (`POST /logout`) erken geçersiz kılınır.
- Oturum çerezi `HttpOnly` olarak işaretlenir; `SameSite=Lax` politikası uygulanır; `Secure` bayrağı panele doğrudan HTTPS erişiminde ayarlanır.

Zaman aşımının yanı sıra ilgili bir bildirim daha vardır: «**Oturum sona erme bildirim gecikmesi**» (`expireTimeDiff` , ipucu «Eşik değerine ulaşmadan önce oturum sona erme bildirimi alın (değer: gün)», varsayılan `0`) – önceden uyarı almayı sağlar.

2.7. LDAP (Senkronizasyon ve Kimlik Doğrulama)

LDAP bölümü iki olanak sunar: (1) yerel parola uymadığında yönetici girişini LDAP üzerinden doğrulamak ve (2) istemcilerin durumunu (VLESS bayrağı etkin/devre dışı) dizinden periyodik olarak senkronize etmek.

Girişte nasıl kullanılır: sunucu önce yerel `bcrypt` parola karmasını kontrol eder. **Eşleşmezse** ve LDAP etkinse panel kullanıcıyı dizinde doğrulamaya çalışır: `Bind DN` tanımlıysa servis `bind` işlemi yapılır, ardından filtre ve öznitelikle kullanıcı kaydı aranır ve bulunan DN altında girilen parolayla `bind` denenir. Başarı, girişe izin verir. (Başarılı LDAP kimlik doğrulamasının ardından 2FA etkinse TOTP kodu yine de doğrulanır.)

Bölüm alanları:

Alan	Metin	Varsayılan Değer	Açıklama
LDAP Senkronizasyonunu Etkinleştir	«LDAP Senkronizasyonunu Etkinleştir» (<code>enable</code>)	false	LDAP entegrasyonunun ana anahtarı.
LDAP sunucusu	«LDAP Sunucusu» (<code>host</code>)	boş	LDAP sunucusunun adresi.

Alan	Metin	Varsayılan Değer	Açıklama
LDAP portu	«LDAP Portu» (port)	389	Port. LDAPS için genellikle 636.
TLS Kullan (LDAPS)	«TLS Kullan (LDAPS)» (useTls)	false	Etkinleştirildiğinde <code>ldaps://</code> şeması kullanılır (varsayılan olarak – sunucu sertifikası doğrulanarak).
TLS sertifikası doğrulamasını atla	«TLS sertifikası doğrulamasını atla» (ldapInsecureSkipVerify)	false	3.4.2'de eklendi. LDAPS'te sunucu sertifikasının doğrulanmasını kapatır – dahili/kendinden imzalı CA'lar için. İpucu: «Güvenli değil – sunucu sertifikası doğrulamasını kapatır. Yalnızca dahili/güvenilmeyen CA'larla kullanın». Geçiş yalnızca «TLS Kullan (LDAPS)» etkinken kullanılabilir.
Bind DN	«Bind DN» (bindDn)	boş	İlk bind/arama için servis hesabının DN'i. Boşsa bind yapılmaz (anonim arama).
Bind parolası	ipuçları: «Yapılandırıldı; mevcut parolayı korumak için boş bırakın.» / «Yapılandırılmadı.» / «Yapılandırıldı – değiştirmek için yeni değer girin»	boş	Bind DN için parola. Ayrıca saklanır; eskisini korumak için alan boş bırakılır.
Base DN	«Base DN» (baseDn)	boş	Aramanın yapıldığı alt ağacın kökü (arama özyinelemeli, tüm alt ağaçta yapılır).
Kullanıcı filtresi	«Kullanıcı Filtresi» (userFilter)	(objectClass=person)	Hesap seçimi için LDAP filtresi. Kimlik doğrulama sırasında kullanıcı adı filtreye kaçış karakterleriyle eklenir.
Kullanıcı özniteliği (username/email)	«Kullanıcı Özniteliği (username/email)» (userAttr)	mail	Kullanıcı adı/istemci tanımlayıcısıyla eşleştirilen öznitelik (örn. mail veya uid).
VLESS bayrağı özniteliği	«VLESS Bayrağı Özniteliği» (vlessField)	vless_enabled	İstemcinin VLESS erişiminin etkin olup olmayacağını belirleyen öznitelik.
Genel bayrak özniteliği (isteğe bağlı)	«Genel Bayrak Özniteliği (isteğe bağlı)» (flagField), ipucu «Tanımlıysa VLESS bayrağını geçersiz kılar – örn. shadowInactive.»	boş	Tanımlıysa vless_enabled yerine kullanılır.

Alan	Metin	Varsayılan Değer	Açıklama
Truthy değerler	«Truthy Değerler» (<code>truthyValues</code>), ipucu «Virgülle ayrılmış; varsayılan: true,1,yes,on»	<code>true,1,yes,on</code>	«Etkin» olarak değerlendirilen bayrak öz niteliği değerlerinin listesi.
Bayrağı ters çevir	«Bayrağı Ters Çevir» (<code>invertFlag</code>), ipucu «Öznitelik «devre dışı» anlamına geldiğinde etkinleştirin (örn. <code>shadowInactive</code>).»	<code>false</code>	Bayrağın anlamını tersine çevirir.
Senkronizasyon programı	«Senkronizasyon Programı» (<code>syncSchedule</code>), ipucu «Cron benzeri dize, örn. <code>@every 1m</code> »	<code>@every 1m</code>	Cron benzeri biçimde senkronizasyon sıklığı.
inbound etiketleri	«inbound Etiketleri» (<code>inboundTags</code>), ipucu «LDAP senkronizasyonunun otomatik istemci oluşturabileceği veya silebileceği inbound'lar.»	boş	Otomatik işlemlere izin verilen inbound'ları kısıtlar. inbound yoksa: «inbound bulunamadı. Önce bir inbound oluşturun.»
Otomatik istemci oluştur	«Otomatik İstemci Oluştur» (<code>autoCreate</code>)	<code>false</code>	Dizinde yeni bir istemci görünürse belirtilen inbound'larda istemci oluşturur.
Otomatik istemci sil	«Otomatik İstemci Sil» (<code>autoDelete</code>)	<code>false</code>	İstemci dizinden kaybolursa siler.
Varsayılan hacim (GB)	«Varsayılan Hacim (GB)» (<code>defaultTotalGb</code>)	<code>0</code>	Otomatik oluşturulan istemciler için trafik limiti (0 = limitsiz).
Varsayılan süre (gün)	«Varsayılan Süre (gün)» (<code>defaultExpiryDays</code>)	<code>0</code>	Otomatik oluşturulan istemciler için geçerlilik süresi (0 = süresiz).
Varsayılan IP limiti	«Varsayılan IP Limiti» (<code>defaultIpLimit</code>)	<code>0</code>	Eş zamanlı IP sayısı sınırı (0 = sınırsız).

Senkronizasyon bayrağı mantığının ayrıntıları: bayrak öz niteliği (`flagField` , varsayılan `vless_enabled`) okunduğunda değer `truthy` değerler listesindeyse «etkin» sayılır; ters çevirme etkinse sonuç tersine döner. Kullanıcı öz niteliği (`userAttr`) eşleştirme anahtarı (email/ad) olarak kullanılır – bu öz niteliğin değeri olmayan kayıtlar atlanır.

Güvenlik: bind parolalarının ve doğrulanan parolaların açık metin olarak iletilmemesi için **TLS (LDAPS)** etkinleştirilmesi önerilir; Bind DN için yalnızca okuma için gerekli minimum haklara sahip bir hesap kullanılmalıdır.

Örnek: tipik LDAP senkronizasyon yapılandırması (Active Directory). Erişim durumunun `userAccountControl` benzeri bir bayrak öz niteliğinde saklandığı ve eşleştirmenin e-postaya göre yapıldığı bir dizin için bölüm alanlarının doldurulması:

Alan	Değer
LDAP sunucusu	ldap.example.com
LDAP portu	636
TLS Kullan (LDAPS)	etkin
Bind DN	CN=svc-3xui,OU=Service,DC=example,DC=com
Base DN	OU=Users,DC=example,DC=com
Kullanıcı filtresi	(objectClass=person)
Kullanıcı özniteliği (username/email)	mail
VLESS bayrağı özniteliği	vless_enabled
Truthy değerler	true,1,yes,on
Senkronizasyon programı	@every 5m

Bu yapılandırmayla panel her 5 dakikada bir `OU=Users` alt ağacını tarar, istemcileri `mail` ile eşleştirir ve `vless_enabled` değerine göre VLESS erişimini açar veya kapatır.

3. Genel Bakış / Gösterge Paneli

Gösterge Paneli (*Overview*) – panelin başlangıç sayfasıdır. Sunucu ve Xray sürecinin durumunu gerçek zamanlı olarak gösterir. Tüm değerler sunucu tarafından gelir. Arka plan zamanlayıcısı her **2 saniyede bir** anlık görüntüyü yeniden oluşturur ve WebSocket aracılığıyla tüm açık sekmelere iletir; dakikada bir birikmiş metrik satırları diske yazılır. `GET /status` HTTP uç noktası, önbelleğe alınmış son anlık görüntüyü döndürür.

Aşağıda sayfadaki her gösterge ve her kontrol öğesi açıklanmaktadır.

Panelin kenar çubuğu menüsünde (ve mobil çekmecede) 3.4.2 sürümünden itibaren «**Belgeler**» düğmesi (kitap simgesi) bulunuyor – <https://docs.sanaei.dev/> adresindeki resmi belgeleri açar. Ayrıca 3.4.2'de genel bir **erişilebilirlik** iyileştirmesi yapıldı: simge düğmeleri ve tıklanabilir öğeler aria etiketleri ve Enter/Space ile etkinleştirme kazandı (ekran okuyucular ve klavye gezintisi için).

3.1. Veri Toplama Genel İlkeleri

- Anlık görüntü `gopsutil` kütüphanesi tarafından toplanır. Belirli bir ölçüm başarısız olursa alan sıfır kalır ve günlüğe bir uyarı yazılır (`get cpu percent failed` , `get uptime failed` vb.) – bu durum gösterge panelinin tamamını çökertmez, yalnızca ilgili kutucuk O/N-A gösterir.
- "Anlık" hızlar (CPU %, ağ, disk I/O) mevcut ve önceki anlık görüntü arasındaki farkın saniye cinsinden aralığa bölünmesiyle hesaplanır. Bu nedenle sayfanın ilk yüklenmesinde, ikinci ölçüm birikmeden önce hız değerleri sıfır olabilir.
- Geçmiş, «Sistem Geçmişi» (*System History*) bölümünde incelenebilir – grafikler aşağıda açıklanan aynı veri satırlarına göre oluşturulur (bkz. madde 3.12).

3.2. İşlemci (CPU)

«İşlemci» (*CPU*) kutucuğu, mevcut işlemci yükünü yüzde olarak ve işlemcinin parametrelerini gösterir.

Gösterge	Açıklama
CPU Yüğü, %	Son aralıktaki meşgul işlemci süresi oranı. Göstergenin ani dalgalanmalardan etkilenmemesi için üstel hareketli ortalama ile yumuşatılır (EMA, katsayı <code>alpha = 0.3</code>). Değer her zaman 0–100 % aralığında tutulur. İlk ölçümde 0 döndürülür (baz noktası başlatması).
Mantıksal İşlemciler	Hyper-Threading dahil mantıksal çekirdek sayısı.
Fiziksel Çekirdekler	Fiziksel çekirdek sayısı.
Frekans	İşlemcinin temel frekansı (MHz). Geç yüklenerek önbelleğe alınır: ilk başarılı ölçüm kaydedilir, yeniden deneme 5 dakikada bir yapılır ve istek 1,5 s zaman aşımıyla sınırlandırılır (bazı sistemlerde frekans sorgusu yavaş yanıt verir).

CPU yükü algoritmik olarak şu şekilde hesaplanır: platforma özgü yerel bir uygulama varsa o kullanılır, yoksa işlemci süresi sayaçlarının deltalarından (`busy / total`) hesaplanır. Guest ve GuestNice süreleri çift sayımı önlemek için hariç tutulur.

3.3. Bellek (RAM)

«Bellek» (*RAM*) kutucuğu kullanılan ve toplam değerleri gösterir. «kullanılan / toplam» ve/veya doluluk yüzdesi şeklinde görüntülenir. Geçmişe yüzde kaydedilir.

3.4. Takas Alanı (Swap)

«Takas Alanı» (*Swap*) kutucuğu kullanılan ve toplam değerleri gösterir. Takas dosyası/bölümü yapılandırılmamışsa (toplam = 0) gösterge sıfırdır; swap yoksa geçmiş satırına 0 yazılır.

3.5. Disk (Storage)

«Disk» (*Storage*) kutucuğu kullanılan ve toplam değerleri gösterir; yalnızca **kök bölüm** / dikkate alınır. «Disk Kullanımı» (*Disk Usage*) geçmişine doluluk yüzdesi yazılır. Aralık boyunca sayaç deltası olarak disk giriş-çıkışı (okuma / yazma, bayt/s) ayrıca toplanır – geçmişin «Disk I/O» sekmesinde görüntülenir.

3.6. Sistem Çalışma Süresi (Uptime)

«Sistem Çalışma Süresi» (*Uptime*) göstergesi. Bu değer **tüm sunucunun** önyüklendiği andan itibaren geçen süredir (saniye cinsinden); panel veya Xray'in çalışma süresi değildir. Xray sürecinin çalışma süresi ayrıca saklanır (bkz. madde 3.9), panel iş parçacıklarının sayısı da («İş Parçacıkları» / *Threads*) gösterilir.

Panel Tarafından Kullanılan Bellek

Panel sürecinin göstergeleriyle birlikte 3X-UI sürecinin kullandığı RAM miktarı görüntülenir. Bu değer, sürecin gerçek RSS değerinden alınır (işletim sisteminin gördüğü şekilde) ve sistem araçlarının gösterdiğiyle örtüşür. Bellek serbest bırakıldıkça değer düşer. Daha önce panel, belleği aşırı gösteren (örneğin tek istemcili boşa bir sunucuda ~300 MB) ve hiçbir zaman azalmayan dahili bir Go sayacı kullanıyordu – bu sorun artık yok. Ek olarak, periyodik bir arka plan işlemi kullanılmayan belleği işletim sistemine iade ederek göstergenin gerçek tüketimi yansıtmasını sağlar.

3.7. Sistem Yüğü (Load average)

«Sistem Yüğü» (*System Load*) bloğu – üç sayıdan oluşan dizi: [Load1, Load5, Load15] . İpucu metni: «Son 1, 5 ve 15 dakikadaki ortalama sistem yüğü» (*System load average for the past 1, 5, and 15 minutes*). Geçmiş grafiğinin adı «Ortalama Sistem Yüğü (1 / 5 / 15 dk)». Geçmiş satırlarına değerler ayrı ayrı yazılır: load1 , load5 , load15 .

Bu standart bir Unix göstergesidir: çalışma kuyruğundaki ortalama süreç sayısını temsil eder. Referans – çekirdek sayısı ile karşılaştırmak: fiziksel çekirdek sayısını sürekli aşan yük, aşırı yüklenmeye işaret eder.

3.8. Ağ: Hız ve Toplam Trafik

Yalnızca **fiziksel arabirimler** dikkate alınır. Sanal ve tünel arabirimleri hariç tutulur: lo / lo0 ve loopback , docker , br- , veth , virbr , tun , tap , wg , tailscale , zt ile başlayan her şey. Değerler kalan tüm arabirimler üzerinden toplanır.

Genel Hız (Overall Speed) – anlık hız, aralık başına sayaç deltası:

Gösterge	Açıklama
Yükleme / gönderme (etiket «Yükleme» / <i>Upload</i>)	Giden hız, bayt/s.
İndirme / alma (etiket «İndir» / <i>Download</i>)	Gelen hız, bayt/s.

Toplam Trafik (Total Data) – sistem başlangıcından bu yana birikmiş sayaçlar:

Gösterge	Açıklama
Gönderildi (etiket «Gönderildi» / <i>Sent</i>)	Toplam gönderilen bayt.
Alındı (etiket «Alındı» / <i>Received</i>)	Toplam alınan bayt.

Ek olarak paket hızları (paket/s) ve toplam paket sayaçları toplanır – geçmişin «Ağ Paketleri» (*Network Packets*) sekmesinde gösterilir. Ağ geçmiş satırları: `netUp` , `netDown` , `pktUp` , `pktDown` .

3.9. Sunucu IP Adresleri

«Sunucu IP Adresleri» (*IP Addresses*) bloğu IPv4 ve IPv6 adreslerini gösterir. Harici adresler üçüncü taraf servisler aracılığıyla belirlenir (`api4.ipify.org` , `ipv4.icanhazip.com` , `v4.api.ipinfo.io/ip` , `ipv4.myexternalip.com/raw` , `4.ident.me` , `check-host.net/ip` – IPv4 için ve IPv6 için de benzer servisler). Liste ilk başarılı yanıt kadar sırayla denenir; her istek için zaman aşımı 3 s'dir.

Özellikler:

- Sonuç, sürecin ömrü boyunca **önbelleğe alınır**: başarıyla belirlenen adres tekrar sorgulanmaz.
- Hiçbir servis yanıt vermezse alanda `N/A` kalır. IPv6 için ilk `N/A` durumunda, IPv6 olmayan ağlarda zaman kaybetmemek amacıyla IPv6 istekleri tamamen devre dışı bırakılır.
- Yanında adres gizleme/gösterme için bir «göz» düğmesi bulunur – ipucu: «Sunucu IP adreslerini gizle veya göster» (*Toggle visibility of the IP*). Bu yalnızca arayüzdeki görsel gizleme işlemidir (örneğin ekran görüntüsü almak için); gerçek adresleri etkilemez.

3.10. TCP/UDP Bağlantıları

«Bağlantı İstatistikleri» (*Connection Stats*) bloğu, sunucudaki etkin TCP ve UDP bağlantılarının toplam sayısını gösterir (sistem genelinde, yalnızca Xray değil). Geçmiş grafiği – «Etkin Bağlantılar (TCP / UDP)» (*Active Connections*), satırlar `tcpCount` , `udpCount` .

3.11. Xray Durumu ve Süreç Yönetimi

«Xray» kartı, Xray-core sürecinin durumunu gösterir ve yönetim imkânı sunar.

Durumlar

Değer	Etiket	Çeviri	Ne zaman ayarlanır
<code>running</code>	«Çalışıyor»	<i>Running</i>	Xray süreci çalışıyor.

Değer	Etiket	Çeviri	Ne zaman ayarlanır
stop	«Durduruldu»	Stopped	Süreç çalışmıyor ve kayıtlı bir başlatma hatası yok.
error	«Hata»	Error	Süreç çalışmıyor ve bir başlatma hatası kaydedildi. Hata metni «Xray çalıştırılırken hata oluştu» (<i>An error occurred while running Xray</i>) başlıklı bir açılır pencerede gösterilir.
—	«Bilinmiyor»	Unknown	Durum henüz alınamamışken görüntülenir.

Durumun yanında **Xray sürümü** gösterilir.

Yönetim Düğmeleri

- **Durdur (Stop).** POST /stopXrayService çağrısı yapar. Başarılı olursa panel WebSocket üzerinden stop durumunu ve «Xray başarıyla durduruldu» (*Xray service has been stopped*) bildirimini yayınlar; hata durumunda — metinle birlikte error durumu. Önemli: panel Xray üzerinden erişilebiliyorsa Xray'i durdurmak panele bağlantıyı kesebilir — panele doğrudan bağlantıda sorun yoktur.
- **Yeniden Başlat (Restart).** POST /restartXrayService çağrısı yapar. İşlemden önce «Xray yeniden başlatılsın mı?» onay mesajı ve «Kaydedilmiş yapılandırmayla xray servisini yeniden başlatır» açıklaması gösterilir. Başarılı olursa — running durumu ve «Xray başarıyla yeniden başlatıldı» (*Xray service has been restarted successfully*) bildirimi. Yeniden başlatma, o an kaydedilmiş yapılandırmayı uygular — ayarları değiştirdikten sonra kullanın.

Not. Bu çatalda gösterge paneline tüm kimlik doğrulama türleri için tam Start / Stop / Restart yönetimi eklenmiştir; orijinal 3x-ui arayüzünde ayrı bir «başlat» düğmesi yoktur — başlatma, yeniden başlatmayla gerçekleştirilir.

Xray Günlüklerini Görüntüleme Düğmesi

Xray kartında bir Xray günlükleri görüntüleme düğmesi (*Logs*) bulunur. Bu düğme yalnızca Xray yapılandırmasında access-log ayarlandığında görünür: yerleşik görüntüleyici bu dosyayı okur, bu nedenle access-log olmadan düğme gizlenir. Düğmenin görünürlüğü ayrı bir accessLogEnable özneliğine bağlıdır ve artık IP sınırına bağımlı değildir — çevrimiçi liste ve IP adresi sınırı, access-log olmadan da çalışmaya devam eder (bkz. madde 8).

Xray Sürümü Seçimi

«Sürüm Seçimi» (*Version*) bölümü, Xray-core'u farklı bir sürüme geçirmenize olanak tanır. Sürüm listesi GET /getXrayVersion üzerinden yüklenir:

- Kaynak — XTLS/Xray-core deposunun GitHub API'si (/releases). İstekler **15 dakika** önbelleğe alınır; GitHub başarısız olursa, seçicinin boş kalmaması için son başarıyla alınan liste döndürülür.
- Listeye yalnızca X.Y.Z biçimindeki ve **26.4.25'ten daha eski olmayan** sürümler dahil edilir.

İpuçları: «Geçmek istediğiniz sürümü seçin» (*Choose the version you want to switch to.*) ve «Önemli: eski sürümler mevcut ayarları desteklemeyebilir» (*Choose carefully, as older versions may not be compatible with current configurations.*) uyarısı.

Geçiş: `POST /installXray/:version`. Senaryo:

Örnek. Belirli bir Xray-core sürümüne geçmek (oturum çerezinin kimlik doğrulamayla önceden alınmış olması gerekir):

```
curl -X POST 'https://panel.example.com:2053/xpanel/installXray/v25.6.8' \
  -b cookie.txt
```

Burada `v25.6.8`, `GET /getXrayVersion` tarafından döndürülen listeden bir etikettir. Sürümün bu listede mevcut olması zorunludur, aksi takdirde panel reddeder. 3.4.2 sürümünden itibaren kurulum için izin verilen minimum Xray sürümü **26.6.27**'ye yükseltildi; 3.5.0 paketinde ise **Xray 26.7.11** çekirdeği gelir. Beraberindeki çekirdek değişiklikleri: Shadowsocks `none / plain` ve VMess `none / zero` şifreleri kaldırıldı (kayıtlı yapılandırmalar otomatik olarak yeniden yazılır: SS – desteklenen bir şifreye, VMess – `auto` 'ya), genel bir adrese giden şifrlenmemiş VLESS/Trojan outbound ise kaydetme sırasında reddedilir – böyle bir yapılandırmayla çekirdek zaten başlamazdı.

1. Seçilen sürüm, güncel sürümler listesinde doğrulanır (yoksa reddedilir).
2. Xray durdurulur.
3. Mevcut işletim sistemi ve mimariye göre `Xray-<os>-<arch>.zip` arşivi GitHub'dan indirilir (amd64/64, arm64-v8a, arm32-v7a/v6/v5, 386/32, s390x desteklenir; Windows için – `xray.exe`). Arşiv ve ikili dosya boyutu 200 MB ile sınırlıdır.
4. İkili dosya atomik olarak değiştirilir (geçici dosya + yeniden adlandırma yoluyla) ve çalıştırılabilir olarak işaretlenir.
5. Xray yeniden başlatılır.

Geçiş öncesinde «Xray Sürümünü Değiştir» (*Do you really want to change the Xray version?*) iletişim kutusu ve «Bu, Xray sürümünü #version# olarak değiştirecek» açıklaması gösterilir. Başarılı olursa – «Xray başarıyla güncellendi» (*Xray updated successfully*) bildirimi.

3.12. Panel Güncellemesi (3X-UI)

Panel güncelleme kontrolü bloğu. Veriler `GET /getPanelUpdateInfo` üzerinden gelir:

Alan	Açıklama
Mevcut panel sürümü	Kurulu panelin sürümü.
En son panel sürümü	GitHub'dan alınan en son 3x-ui sürümü.
Güncelleme mevcut	En son sürümün mevcut sürümden daha yeni olduğunun göstergesi. Güncelleme gerekmiyorsa «Panel güncel» / «Güncellendi» gösterilir.

«Paneli Güncelle» (*Update Panel*) düğmesi `POST /updatePanel` çağrısı başlatır. İpucu: «Bu, 3X-UI'ı en son sürüme güncelleyecek ve panel servisini yeniden başlatacak». Başlatmadan önce «Paneli gerçekten

güncellemek istiyor musunuz?» onay mesajı ve «Bu, 3X-UI'ı #version# sürümüne güncelleyecek ve panel servisini yeniden başlatacak» metni gösterilir.

Özellikler ve sınırlamalar:

- Otomatik güncelleme yalnızca **Linux** üzerinde desteklenir (diğer işletim sistemlerinde hata döndürülür).
- Güncelleme betiği resmi depodan indirilir (raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh , 2 MB sınırı) ve `bash` aracılığıyla, mümkünse `systemd-run` ile izole biçimde çalıştırılır.
- Başarılı başlatmada «Panel güncellemesi başladı» (*Panel update started*) gösterilir; güncelleme kontrolü başarısız olursa – «Panel güncellemesi kontrolü başarısız oldu». Kurulum sırasında «Kurulum devam ediyor. Sayfayı yenilemeyin» uyarısı görüntülenir.

3.13. Coğrafi Dosya Güncellemesi (GeoIP / GeoSite)

Coğrafi veri tabanı güncelleme düğmesi/iletişim kutusu `POST /updateGeofile` (tüm dosyalar) veya `POST /updateGeofile/:fileName` (tek dosya) çağrısı yapar. Güncelleme katı bir ad ve kaynak beyaz listesine göre çalışır:

Dosya	Kaynak
<code>geoip.dat</code> , <code>geosite.dat</code>	<code>Loyalsoldier/v2ray-rules-dat</code> (latest)
<code>geoip_IR.dat</code> , <code>geosite_IR.dat</code>	<code>chocolate4u/Iran-v2ray-rules</code> (latest)
<code>geoip_RU.dat</code> , <code>geosite_RU.dat</code>	<code>runetfreedom/russia-v2ray-rules-dat</code> (latest)

Davranış:

- Dosya adı doğrulanır: `..` , eğik çizgiler, mutlak yollar yasaktır; yalnızca `[a-zA-Z0-9._-]+.dat` formatına izin verilir. Beyaz liste dışındaki dosyalar indirilmez.
- `If-Modified-Since` koşullu isteği kullanılır: kaynak sunucuda dosya değişmemişse (HTTP 304), dosya tekrar indirilmez, yalnızca zaman damgası güncellenir.
- İndirme sonrasında Xray **yeniden başlatılır** (yeni veri tabanlarının yüklenmesi için).

Örnek. Yalnızca Rus coğrafi veri tabanlarını güncellemek, diğer dosyalara dokunmamak:

```
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geoip_RU.dat' -b cookie.txt
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geosite_RU.dat' -b cookie.txt
```

Beyaz listedeki tüm dosyaları aynı anda güncellemek için – dosya adı olmadan `POST /updateGeofile` çağrısı yapın.

- İletişim kutuları: tek dosya için «Coğrafi dosyayı gerçekten güncellemek istiyor musunuz?» ve «Bu, #filename# dosyasını güncelleyecek», «Tümünü Güncelle» düğmesi için ise «Bu, tüm coğrafi dosyaları güncelleyecek». Başarı – «Coğrafi dosyalar başarıyla güncellendi».

3.14. Veritabanı Yedekleme ve Geri Yükleme

«Yedek ve Geri Yükleme» (*Backup & Restore*) bloğu. Davranış kullanılan DBMS'e göre değişir (varsayılan olarak SQLite veya PostgreSQL).

Veritabanı Dışa Aktarma (Yedekleme)

«Veritabanını Dışa Aktar» / «Yedekle» (*Back Up*) düğmesi `GET /getDb` çağrısı yapar. Dosya ek olarak iletilir:

- **SQLite**: önce checkpoint yapılır (WAL temizlenir), ardından `x-ui.db` dosyası indirilir. İpucu: «Mevcut veritabanınızın yedeğini içeren .db dosyasını indirmek için tıklayın...».
- **PostgreSQL**: özel formatta `x-ui.dump` dökümü indirilir (`pg_dump --format=custom --no-owner --no-privileges`). Sunucuda PostgreSQL istemci araçlarının kurulu olması gerekir; aksi halde `pg_dump` 'ın yokluğu hakkında hata alınır.

Veritabanı İçe Aktarma (Geri Yükleme)

«Veritabanını İçe Aktar» / «Geri Yükle» (*Restore*) düğmesi `POST /importDB` üzerinden dosya yükler (form alanı `db`). İpucu: «Veritabanını yedekten geri yüklemek için bir .db dosyası seçip yüklemek için tıklayın...».

SQLite için güvenli senaryo, geri alma desteğiyle:

1. Dosya SQLite formatı açısından doğrulanır ve geçici dosyaya kaydedilir, ardından bütünlük kontrolü yapılır.
2. Xray durdurulur, mevcut veritabanı kapatılır ve `*.backup` olarak yeniden adlandırılır (geri dönüş).
3. Yeni dosya çalışma veritabanının yerine alınır, başlatma ve geçiş gerçekleştirilir. Bir şeyler ters giderse geri dönüş dosyası geri yüklenir.
4. Xray yeniden başlatılır.

PostgreSQL için 3.5.0'dan itibaren «Geri Yükleme» **üç tür dosya** kabul eder (tür, uzantıya göre değil içeriğe göre belirlenir): `pg_dump` arşivi (PGDMP) — `pg_restore --clean --if-exists --single-transaction ...` aracılığıyla uygulanır; **SQLite veritabanı** `.db` (normal yedek) ve **SQLite geçiş** `.dump` dosyası — bunlar denetlenir, gerekirse yeniden derlenir ve `x-ui migrate-db --dsn` ile aynı motor tarafından PostgreSQL'e **tek işlemle (transaction)** aktarılır (hata durumunda PostgreSQL el değmemiş kalır). Dosya seçim iletişim kutusu her iki DBMS'te de `.dump`, `.db` kabul eder; `pg_dump` arşivinin SQLite paneline yüklenmesi anlaşılır bir hata verir. İpucu açıkça uyarır: «Bu, tüm mevcut verilerin yerini alacak».

Mesajlar: «Veritabanı başarıyla içe aktarıldı», «Veritabanı içe aktarılırken hata oluştu», «...veritabanı okunurken», «...veritabanı alınırken».

Geçiş Dosyası (SQLite ve PostgreSQL Arasında)

«Geçiş Dosyasını İndir» (*Download Migration*) düğmesi `GET /getMigration` çağrısı yapar. 3.5.0'dan itibaren yalnızca **PostgreSQL panellerinde** gösterilir ve `x-ui.db` dosyasını — PostgreSQL verilerinden derlenen hazır bir SQLite veritabanını («SQLite'a geri dönüş» yolu) — indirir. SQLite panellerinde düğme gereksiz olduğu için kaldırıldı: normal `.db` yedeği artık zaten doğrudan PostgreSQL paneline geri yüklenebiliyor.

3.15. Ek Arayüz Öğeleri

- **Çevrimiçi istemci göstergesi.** Gösterge paneli `online` (*Online Clients* / «Çevrimiçi İstemciler») satırını tutar – etkin bağlantısı olan istemci sayısı. Xray çalışırken hesaplanır (yoksa 0) ve aynı 2 saniyelik döngüde geçmişe kaydedilir. Grafik – «Çevrimiçi» sekmesi.
- **Sistem Geçmişi (grafikler).** «Grafikler» → «Sistem Geçmişi» düğmesi/bölümü, şu sekmelerle: «Bant Genişliği», «Paketler», «Disk I/O», «Çevrimiçi», «Yük», «Bağlantılar», «Disk Kullanımı». Veriler `GET / history/:metric/:bucket` üzerinden çekilir; izin verilen toplama aralıkları (bucket, sn): **2, 30, 60, 180, 360, 720, 1440, 2880, 10080**, sekmeye en fazla 60 nokta gelir. Sayfadaki aralık seçicisinde şu düğmeler bulunur: **2m, 1h, 3h, 6h, 12h, 24h, 2d, 7d** (sırasıyla `2, 60, 180, 360, 720, 1440, 2880, 10080` bucket'ları). Uzun aralıklarda **2d** ve **7d** için eksen üzerindeki zaman etiketlerine `MM-DD HH:MM` formatında tarih eklenir. Depolama üç katmanlı örnekleme düşürme (rollup) ile organize edilmiştir: taze veriler son **saat** için 2 s adımıyla tutulur, ardından **48 saat** için 1 dk adımı ve **7 gün** için 10 dk adımı düşürülür. Bu nedenle grafikler (CPU, RAM, trafik, paketler, bağlantılar, disk, çevrimiçi, yük) **7 güne kadar** (önceden – 48 saate kadar) incelenebilir; geçmişe ne kadar uzanırsa ayrıntı düzeyi o kadar kaba olur. İzin verilen metrikler: `cpu, mem, swap, netUp, netDown, pktUp, pktDown, diskRead, diskWrite, diskUsage, tcpCount, udpCount, online, load1, load5, load15`. «Son 2 dakika» etiketi bucket = 2'ye karşılık gelir (gerçek zamanlı mod).

Örnek. Son ~2 dakika için CPU yükü satırını almak (bucket = 2 s, en fazla 60 nokta) ve aynı satırı 5 dakikalık toplama ile almak (bucket = 300 s):

```
curl 'https://panel.example.com:2053/xpanel/history/cpu/2' -b cookie.txt
curl 'https://panel.example.com:2053/xpanel/history/cpu/300' -b cookie.txt
```

Metrik, izin verilen herhangi biriyle değiştirilebilir (`mem, netUp, tcpCount, load1` vb.). `2, 30, 60, 180, 360, 720, 1440, 2880, 10080` beyaz listesi dışındaki bucket reddedilir.

- **Xray metrikleri** – Xray'in bellek tüketimi ve çöp toplama verileriyle (`xrAlloc, xrSys, xrHeapObjects, xrNumGC, xrPauseNs`) «Gözlemevi» (giden bağlantı durumları) içeren ayrı bir blok. Yalnızca Xray yapılandırmasında `metrics` bloğu tanımlandığında çalışır (`listen 127.0.0.1:11111`, etiket `metrics_out`); aksi halde «Xray metrikleri uç noktası yapılandırılmamış» gösterilir. Xray metrikleri penceresinde şu düğmelere sahip ayrı bir aralık seçici bulunur: **2m, 1h, 3h, 6h, 12h** (bucket'lar `2, 60, 180, 360, 720`).

Örnek – Xray metrikleri kutucuğunu etkinleştiren blok. Xray ayarları bölümünde aynı anda hem `metrics` (etiketli) hem de bu etiketi dinleyen inbound mevcut olmalıdır:

```
{
  "metrics": {
    "tag": "metrics_out"
  },
  "inbounds": [
    {
      "listen": "127.0.0.1",
      "port": 11111,
      "protocol": "dokodemo-door",
      "settings": { "address": "127.0.0.1" },
      "tag": "metrics_out"
    }
  ]
}
```

127.0.0.1:11111 adresi kasıtlı olarak dışa açılmaz – panel onu yerel olarak sorgular.

- **Koyu tema değiştirici.** Genel menüde/başlıkta bulunur, gösterge panelinin kendisinde değil. Seçenekler: «Tema» (*Theme*) altında «Koyu» ve «Çok Koyu» (*Ultra Dark*). Bu yalnızca görsel bir tasarım ayarıdır, panelin işleyişini etkilemez.
- **Gösterge paneli çevresindeki diğer bağlantılar** (menüden/alt çubuktan): «Günlükler», «Yapılandırma» – Xray'in son JSON yapılandırmasını görüntüleme (`GET /getConfigJson`), «Belgeler».

4. Inbounds: oluşturma ve genel parametreler

«**Gelen Bağlantılar**» (İng. *Inbounds*) bölümü, istemcilerin bağlandığı tüm Xray giriş noktalarının listesidir. Her inbound hem "panel" alanlarını (açıklama, trafik limiti, sıfırlama takvimi) hem de ham JSON yapılandırma bloklarını (`settings` , `streamSettings` , `sniffing`) saklar.

Oluşturma işlemi «**Bağlantı Oluştur**» (*Add Inbound*) düğmesiyle, düzenleme ise «**Bağlantıyı Değiştir**» (*Modify Inbound*) düğmesiyle yapılır. Her iki işlem sırasıyla `POST /add` ve `POST /update/:id` API uç noktalarına gönderilir.

Aşağıda formun belirli bir protokolün ayarlarıyla (istemciler, şifreleme, REALITY/TLS) **ilgili olmayan** ve aktarım/akış ayarlarıyla («**Akış**», «**Güvenlik**» sekmeleri) **ilgili olmayan** tüm alanları açıklanmaktadır — bunlar ayrı bölümlerin konusudur.

4.1. Genel form alanları

Remark (Açıklama)

Parametre	Değer
Alan	<code>remark</code>
Tür	dize
Varsayılan	boş

Inbound'un listede ve iletişim kutusu başlıklarında görüntülenen, insan tarafından okunabilir adıdır («Bağlantı "{remark}" silinsin mi?» vb.). Alan etiketi «**Açıklama**»'dır. Xray'in çalışmasını etkilemez, yalnızca yönetim kolaylığı için gereklidir; dışa aktarılan dosya adlarına ve toplu işlem onaylarına eklendiğinden benzersiz ve anlamlı isimler kullanılması önerilir.

Protocol (Protokol)

Parametre	Değer
Alan	<code>protocol</code>
Etiket	« Protokol »
Doğrulama	<code>required,oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel tun</code>

inbound protokolünün açılır listesi. İzin verilen değerler:

Değer	Açıklama
<code>vmess</code>	
<code>vless</code>	
<code>trojan</code>	

Değer	Açıklama
shadowsocks	
wireguard	
hysteria	Hysteria v2 – streamSettings.version = 2 ile hysteria 'dır, ayrı bir protokol yoktur
http	
mixed	tek portta socks/http
tunnel	
tun	doğrulamayı tarafından kabul edilir, ayrı bir protokol sabiti yoktur

Alan zorunludur (required). Protokol seçimi, hangi istemci ayarı alanlarının ve hangi aktarımın kullanılabileceğini belirler (protokole özgü bölümlere bakınız).

Önemli: kaydedilirken servis streamSettings değerini normalleştirir. Aktarım ayarları yalnızca vmess , vless , trojan , shadowsocks , hysteria protokolleri için bırakılır; diğerleri için (http , mixed , tunnel , wireguard , tun) streamSettings alanı **zorunlu olarak temizlenir**.

tunnel /TPProxy türündeki inbound'larda streamSettings bloğu security anahtarı içermiyorsa (aktarımsız varyant), form streamSettings.security Invalid input doğrulama hatası olmadan açılır ve kaydedilir.

Listen IP (Dinleme IP'si)

Parametre	Değer
Alan	listen
Tür	dize
Varsayılan	boş → Xray tüm IP'lerde dinler (0.0.0.0)

inbound'un bağlantıları kabul ettiği IP adresi. Alan ipucu:

«Tüm IP adreslerini dinlemek için boş bırakın».

Xray yapılandırması oluşturulurken boş değer 0.0.0.0 ile değiştirilir. IP'nin yanı sıra bu alan **Unix soket yolunu** da kabul eder – ipucu:

«TCP bağlantı noktası yerine soket dinlemek için bir Unix soket yolu (örneğin /run/xray/in.sock) veya @ önekli soyut soket adı (örneğin @xray/in.sock) de belirtebilirsiniz; bu durumda bağlantı noktasını 0 olarak ayarlayın».

Böylece alan iki Unix soket biçimini kabul eder: dosya sistemi yolu (/run/xray/in.sock) ve @ önekli soyut soket adı (@xray/in.sock). Her iki durumda da Port değerini 0 olarak ayarlayın.

Bu alan, inbound'u tek bir arayüzle sınırlamak gerektiğinde (örneğin yalnızca Nginx'in arkasında fallback hedefi olarak çalışan inbound için `127.0.0.1`) veya inbound Unix soketinde dinlediğinde değiştirilir.

Örnek. Yalnızca yerel arayüzde dinleyen (Nginx'in arkasındaki tipik fallback hedefi) ve Unix soketinde dinleyen inbound:

```
listen = 127.0.0.1    port = 8443
listen = /run/xray/in.sock    port = 0
```

Port (Bağlantı Noktası)

Parametre	Değer
Alan	port
Etiket	«Bağlantı Noktası»
Doğrulama	gte=0, lte=65535
Varsayılan	– (kullanıcı tarafından belirlenir)

TCP/UDP dinleme bağlantı noktası. 0 ile 65535 arasında değerlere izin verilir. 0 değeri yalnızca Unix soketinde dinlemeyle birlikte kullanılır (yukarıya bakınız).

Kaydedilirken servis bağlantı noktası çakışmasını kontrol eder: iki inbound aynı aktarım (TCP/UDP) için çıkan listen:port değerlerini aynı anda kullanamaz. Aktarım, protokolden ve streamSettings / settings değerinden hesaplanır: örneğin hysteria ve wireguard her zaman UDP kullanır, kcp / quic UDP kullanır, diğerlerin büyük çoğunluğu ise TCP kullanır. Çakışma durumunda kaydetme hatayla reddedilir.

Panel ayrıca **dahili Xray API'sinin ayrılmış bağlantı noktasının** (api etiketi, varsayılan olarak `127.0.0.1` üzerinde `62789`) kullanılmasına izin vermez: dinleme adresi loopback'te bu bağlantı noktasıyla çıkan yerel TCP inbound aynı çakışma hatası ile reddedilir. Gerçek API bağlantı noktası Xray yapılandırma şablonundan okunur (yedek değer `62789`). Düğümlerde (nodes) bu kısıtlama geçerli değildir – bunların kendi Xray'i vardır.

Xray etiketi (Tag , benzersiz), bağlantı noktası ve aktarımdan in-<bağlantı_noktası>-<tcp|udp|tcpudp|any> biçiminde otomatik olarak oluşturulur; bir düğümde dağıtılan inbound için n<nodeId>-öneki eklenir. Çakışma durumunda etiketin sonuna -2 , -3 vb. eklenir. Kullanıcı genellikle etiketi düzenlemez.

Total traffic (Toplam trafik, GB)

Parametre	Değer
Alan	total (bayt cinsinden)
Etiket	«Toplam Kullanım»
Varsayılan	0

inbound'un toplam trafik limiti. Formda değer gigabayt cinsinden girilir, veritabanında bayt cinsinden saklanır. Alan ipucu:

«= Limitsiz. (birim: GB)».

Yani **0 limitsiz anlamına gelir**. Bu, tüm inbound düzeyindeki limittir (tek tek istemcilerin değil); gerçek harcanan trafik **up** (gönderilen) ve **down** (alınan) alanlarında saklanır ve **total** ile karşılaştırılır.

Expiry date / Duration (Bitiş tarihi / süre)

Parametre	Değer
Alan	<code>expiryTime</code> (Unix zaman damgası)
Etiket	« Bitiş Tarihi » (İng. <i>Duration</i>)
Varsayılan	boş / 0

inbound'un geçerlilik süresi. İpucu:

«Sonsuz olması için boş bırakın».

Boş değer (**0**) süresi sınırsız inbound anlamına gelir. Değer Unix zaman damgası olarak saklanır; form hem belirli bir tarih hem de gün cinsinden süre (geçerli andan itibaren göreceli sayım – İng. alan etiketi *Duration*) girmeye olanak tanır.

Enabled (Etkin)

Parametre	Değer
Alan	<code>enable</code>
Etiket	« Etkinleştir » (İng. <i>Enabled</i>)
Varsayılan	oluşturma sırasında belirlenir

inbound'un etkinlik göstergesi. Bu bayrağın listede değiştirilmesi, tam güncelleyin aksine ayrı bir "hafif" `POST / setEnable/:id` uç noktasıyla işlenir – bu, binlerce istemcisi olan bir inbound'da her geçiş tıklamasında tüm `settings` bloğunun (tüm istemcilerin) yeniden serileştirilmemesi için özel olarak yapılmıştır. inbound devre dışı bırakıldığında çalışan Xray'den kaldırılır, etkinleştirildiğinde geri eklenir.

Node / Deploy to (Düğüm / Dağıt)

Parametre	Değer
Alan	<code>nodeId</code>
Etiket	« Şuraya Dağıt », « Yerel Panel »

Parametre	Değer
Varsayılan	boş (yerel panel)

inbound'un fiziksel olarak nerede çalıştığının seçimi: yerel panelde mi yoksa kayıtlı düğümlerden birinde mi. 3.5.0'dan itibaren «Şuraya Dağıt» listesi (inbound listesinin üzerindeki «Düğüm» filtresi gibi) **yazarak arama** destekler; inbound listesinin üzerine ise bir **arama alanı** eklendi (açıklamaya, porta ve protokole göre; düğüm filtresiyle birlikte çalışır). Uygulama detayı: `nodeId = 0`, `nil` olarak normalleştirilir; `0` geçerli bir düğüm id'si değil, form bağlamasının bir eseridir; `nil / 0` yerel panel anlamına gelir. Çevrimdışı bir düğümde inbound kaydedilirken «düğüm yeniden bağlandığında değişiklik senkronize edilecek» bildirimi görünebilir. 3.4.2 sürümünden itibaren bu bildirim yalnızca düğüm gerçekten çevrimdışı veya kapalıyken gösterilir (önceden çevrimiçi bir düğüme kaydederken de görünebiliyordu).

Bağlantı adresi stratejisi (Share address strategy)

Parametre	Değer
Alan	strateji + (isteğe bağlı) özel adres
Etiket	«Bağlantı Adresi Stratejisi» (İng. <i>Share address strategy</i>)
Varsayılan	«inbound dinleme adresi» (<i>Inbound listen</i>)

Açılır liste, bu inbound'un **dışa aktarılan paylaşım bağlantılarına ve QR kodlarına** hangi adresin ekleneceğini belirler. Değerler:

Değer	Etiket	Ne eklenir
node	«Düğüm adresi» (<i>Node address</i>)	inbound'un üzerinde çalıştığı düğümün adresi
listen	«inbound dinleme adresi» (<i>Inbound listen</i>)	inbound'un kendi dinleme adresi
custom	«Özel» (<i>Custom</i>)	«Özel Paylaşım Adresi» (<i>Custom share address</i>) alanından alınan kendi adres

«Özel» seçildiğinde «Özel Paylaşım Adresi» alanı görünür; buraya şema ve bağlantı noktası **olmadan** bir ana bilgisayar adı veya IP girilir (değer doğrulanır). «Düğüm adresi» seçeneği listede yalnızca bu inbound'un üzerinde çalışabileceği etkin bir düğüm varsa gösterilir; aksi takdirde gizlenir ve değer «inbound dinleme adresi»'ne döndürülür.

Bu strateji **yalnızca** doğrudan paylaşım bağlantılarını ve QR kodlarını etkiler. Abonelik çıktısını **etkilemez** — orada adres panelin olağan mantığıyla belirlenir.

4.2. Sniffing (Koklama)

«Koklama» sekmesi, ham JSON olarak saklanan Xray yapılandırmasının `sniffing` bloğunu düzenler. Sniffing, Xray'in yönlendirme amacıyla bağlantı içindeki gerçek alan adını/protokolü "gözetlemesine" olanak tanır.

Alt alan	Etiket	Amaç
<code>enabled</code>	(sekme geçişi)	inbound için koklama işlevini etkinleştirir/devre dışı bırakır
<code>destOverride</code>	–	Hedef adresin yakalandığı protokollerin listesi: <code>http</code> , <code>tls</code> , <code>quic</code> , <code>fakedns</code>
<code>metadataOnly</code>	«Yalnızca meta veri»	Yalnızca bağlantı meta verilerini kullanır, yük okumaz
<code>routeOnly</code>	«Yalnızca yönlendirme»	Koklama sonucunu yalnızca yönlendirme için uygular, hedef adresi değiştirmez
<code>domainsExcluded</code>	«Hariç tutulan alan adları»	Koklamadan hariç tutulan alan adları
(hariç tutulan IP'ler)	«Hariç tutulan IP'ler»	Koklamadan hariç tutulan IP adresleri

- **`destOverride`** – bir dizi koklayıcı: `http` (HTTP Host başlığından alan adını belirler), `tls` (SNI'dan), `quic` (QUIC ClientHello'dan), `fakedns` (FakeDNS havuzuyla eşleştirir). Alan adını belirlemek için genellikle `http` ve `tls` etkinleştirilir.

sniffing bloğu örneği (HTTP ve TLS aracılığıyla alan adını belirle, sonucu yalnızca yönlendirme için kullan, yerel ağa dokunma):

```
{
  "enabled": true,
  "destOverride": ["http", "tls"],
  "routeOnly": true,
  "domainsExcluded": ["courier.push.apple.com"]
}
```

- **`metadataOnly`** – etkinleştirildiğinde Xray ilk paketin içeriğini okumaz ve yalnızca meta verilere dayanır; verilerin "gözetlenemeyeceği" protokolleri bozmamak için kullanışlıdır.
- **`routeOnly`** – koklama sonucu yalnızca yönlendirme kuralları tarafından kullanılır; outbound'daki bağlantı adresi tespit edilen alan adıyla değiştirilmez.

Not: panel `sniffing` değerini opak bir JSON bloğu olarak saklar ve kaydetme sırasında hiçbir şey eklemeyiz – bu onay kutularının varsayılan değerleri istemci uygulama tarafı tarafından oluşturulur. Ham blok, aşağıda açıklanan «inbound JSON» bölümü aracılığıyla düzenlenebilir.

4.3. Allocate (Bağlantı noktası dağıtım stratejisi)

`streamSettings` içindeki `allocate` bloğu, Xray'in dinleme bağlantı noktalarını nasıl dağıttığını yönetir. Bu Xray yapılandırmasının bir parçasıdır; panel bunu `streamSettings` /inbound JSON'unun bir parçası olarak saklar ve iletir. Parametreler (Xray-core terminolojisine göre):

Alt alan	Amaç	Değerler / varsayılan
<code>strategy</code>	Bağlantı noktası tahsis stratejisi	<code>always</code> – her zaman belirtilen bağlantı noktasını dinle (varsayılan); <code>random</code> – aralık içinde dinlenen bağlantı noktalarını periyodik olarak değiştir

Alt alan	Amaç	Değerler / varsayılan
refresh	random kullanılırken bağlantı noktası değiştirme aralığı (dakika)	dakika cinsinden tam sayı (5 önerilir; minimum 2)
concurrency	random kullanılırken aynı anda açık tutulacak bağlantı noktası sayısı	tam sayı (varsayılan 3; bağlantı noktası aralığı genişliğinin üçte birini geçemez)

strategy: always , inbound'u tek bir bağlantı noktasında tutar (standart mod). strategy: random , inbound'un bağlantı noktası aralığında periyodik olarak "zıplaması" gereken engellemeyi önleme senaryoları için gereklidir; bu durumda refresh ve concurrency anlamlıdır. Bu değerleri yalnızca rastgele bağlantı noktası modunu bilinçli olarak kullanırken değiştirin.

streamSettings içindeki **allocate** bloğu örneği (rastgele bağlantı noktası modu: 3 bağlantı noktasını açık tut, her 5 dakikada bir değiştir):

```
{
  "allocate": {
    "strategy": "random",
    "refresh": 5,
    "concurrency": 3
  }
}
```

Bunun çalışması için inbound'un port değeri bir aralık olarak ayarlanmalıdır (örneğin 20000–20100).

4.4. External Proxy (Harici proxy)

«External Proxy» alanı, davet bağlantısı oluşturma ayarlarıyla ilgilidir ve inbound'un streamSettings değerinde saklanır. Gerçek listen:port inbound yerine istemci bağlantılarına eklenen alternatif harici adreslerin listesini tanımlar (ana bilgisayar/bağlantı noktası, gerektiğinde zorunlu TLS – «Zorunlu TLS» ile).

istemcilerin doğrudan sunucuya değil, harici bir proxy/reverse/CDN aracılığıyla bağlanması gerektiğinde kullanılır: bu durumda paylaşılan bağlantılarda bu ön ucun genel adresi belirtilir. Xray'in bağlantı kabul sürecini etkilemez – bu, oluşturulan bağlantıların yalnızca "kozmetik" düzenlemesidir. İlgili form alanları: «Zorunlu TLS», «Fingerprint», her kaydın etiketleri.

4.5. Fallbacks (Fallback'ler)

«Fallback'ler» bölümü, inbound istemcilerinden hiçbirisiyle eşleşmeyen bağlantılar için yeniden yönlendirme kurallarını tanımlar. TLS aktarımındaki (VLESS/Trojan TCP-TLS) ana inbound için kullanılabilir. GET /:id/fallbacks / POST /:id/fallbacks uç noktaları aracılığıyla yönetilir.

Bölüm ipucu:

«Bu inbound'daki bir bağlantı hiçbir istemciyle eşleşmediğinde başka bir yere yönlendirilir. Yönlendirme alanlarının (SNI / ALPN / Path / xver) aktarımından otomatik doldurulması için aşağıdan bir alt inbound seçin ya da seçimi boş bırakıp Nginx gibi harici bir sunucuya yönlendirmek için Dest değerini doğrudan

girin (örneğin 8080 veya 127.0.0.1:8080). Her alt inbound 127.0.0.1 adresinde security=none ile dinlemelidir».

Fallback'ler bölümü yalnızca TLS veya REALITY güvenliğiyle RAW (TCP) üzerindeki VLESS/Trojan inbound için gösterilir. Yeni bir inbound security=none ile başlar, bu nedenle bölüm başta mevcut olmayabilir. Bu durumda (VLESS/Trojan, RAW/TCP, güvenlik henüz yapılandırılmamış), bölüm yerine yerleşik bir ipucu görüntülenir: fallback'ler «**Güvenlik**» sekmesinde TLS veya Reality seçildikten sonra kullanılabilir hale gelecektir.

Fallback satırı alanları

Alan	Varsayılan	Açıklama
(alt inbound)	—	Alt inbound seçimi (etiket « Inbound Seçin »). Seçilirse Name/Alpn/Path/Dest alanları aktarımından otomatik doldurulabilir
Name	boş (= herhangi biri)	Ada (SNI/adı) göre eşleşme koşulu. «herhangi biri» etiketi — « herhangi biri »
Alpn	boş	ALPN'ye göre eşleşme koşulu
Path	boş	Yola göre eşleşme koşulu (alt inbound'un WS/HTTP aktarımları için)
Dest	otomatik	Nereye yönlendirileceği. Yer tutucu « otomatik (listen:alt bağlantı noktası) ». Bağlantı noktası (8080) veya host:port (127.0.0.1:8080) belirtilebilir
Xver	0	PROXY protokol sürümü (« Xver »): 0 — devre dışı, 1 veya 2 — ilgili PROXY protokol sürümü
(sıra)	konuma göre	Kural uygulama sırası; « Yukarı »/« Aşağı » düğmeleriyle ayarlanır

Kaydetme mantığı: ana fallback listesinin tamamı atomik olarak değiştirilir. Seçili alt inbound'u (childId <= 0) ve tanımlanmış Dest değeri olmayan satır **atlanır**. Seçilen alt inbound ana inbound'un id'siyle aynıysa sıfırlanır. Sonuç JSON oluşturulurken: Dest boşsa alt inbound'dan listen:port olarak hesaplanır; 0.0.0.0 / :: / :::0 , 127.0.0.1 ile değiştirilir; boş name / alpn / path alanları çıktı JSON'una dahil edilmez; xver yalnızca 0'dan büyükse eklenir.

Sonuç settings.fallbacks örneği (alpn=h2 olan trafik /ws yolundaki WS hedefine gider, geri kalanı 8080 portundaki yerel Nginx'e gider):

```
{
  "fallbacks": [
    { "alpn": "h2", "path": "/ws", "dest": "127.0.0.1:2001", "xver": 1 },
    { "dest": 8080 }
  ]
}
```

name / alpn / path olmayan son satır, geri kalanı yakalayan "varsayılan" kuraldır.

Fallbacks bölümü düğmeleri ve ipuçları

- «**Fallback Ekle**» – satır ekler; «**Henüz fallback yok**» – boş durum.
- «**Uygun olanları hızlıca ekle**» / «**Tümünü Ekle**» – henüz bağlı olmayan her uygun inbound için bir fallback satırı ekler. Sonuç: «{n} fallback eklendi» veya «Yeni uygun inbound yok».
- «**Alt inbound'dan doldur**» – seçilen alt inbound'un aktarımından yönlendirme alanlarını (SNI/ALPN/Path/xver) yeniden çeker; tamamlandıktan sonra «Alt inbound'dan dolduruldu».
- «**Yönlendirme alanlarını düzenle**» / «**Gelişmiş gizle**» – satırın ayrıntılı alanlarını gösterir/gizler.
- «**Yönlendirme koşulu**» ve «**Varsayılan – geri kalanı yakalar**» etiketleri her satırın tetikleme koşulunu açıklar.

Fallback'ler kaydedildikten sonra sunucu, yeni `settings.fallbacks` değerinin geçerli olması için Xray'i yeniden başlatır.

4.6. Periyodik trafik sıfırlama

«**Trafik Sıfırlama**» bloğu, inbound trafik sayaçlarının zamanlamaya göre otomatik sıfırlanmasını yapılandırır. Açıklama:

«Belirtilen aralıklarla trafik sayacını otomatik olarak sıfırla».

Parametre	Değer
Alan	<code>trafficReset</code>
Doğrulama	<code>omitempty,oneof=never hourly daily weekly monthly</code>
Varsayılan	<code>never</code>
Eşlik eden alan	<code>lastTrafficResetTime</code> – son sıfırlama zaman damgası (etiket « Son Sıfırlama »)

Açılır liste:

Değer	Etiket
<code>never</code>	« Hiçbir zaman »
<code>hourly</code>	« Saatlik »
<code>daily</code>	« Günlük »
<code>weekly</code>	« Haftalık »
<code>monthly</code>	« Aylık »

Her periyot için ilgili zamanlamaya göre çalışan bir cron görevi kaydedilir (`@hourly` , `@daily` , `@weekly` , `@monthly`). Görev, belirtilen `trafficReset` değerine sahip tüm inbound'ları seçer ve her biri için hem inbound'un kendi sayaçlarını (`up=0` , `down=0`) **hem de** tüm istemcilerinin trafiğini sıfırlar. Yani periyodik sıfırlama hem inbound'u hem de istemcilerini etkiler.

Alan değeri örneği. Sayaçların her ayın birinde sıfırlanması için formda «**Aylık**» seçilir; bu şöyle kaydedilir:

```
{ "trafficReset": "monthly" }
```

`never` değeri (varsayılan), otomatik sıfırlamayı tamamen devre dışı bırakır.

4.7. inbound JSON (gelişmiş)

«**inbound JSON Bölümleri**» bölümü, inbound'un ham JSON bloklarına doğrudan erişim sağlar. Açıklama:

«Inbound'un tam JSON'u ve settings, sniffing ve streamSettings için ayrı düzenleyiciler».

Kullanılabilir düzenleyiciler:

Sekme	Etiket	Ne düzenler
Tümü	«Tüm alanların tek bir düzenleyicide bulunduğu tam inbound nesnesi»	tüm Inbound nesnesi
Ayarlar	«Xray settings bloğunun sarmalayıcısı»	<code>settings</code> alanı
Sniffing	«Xray sniffing bloğunun sarmalayıcısı»	<code>sniffing</code> alanı
Stream	«Xray stream bloğunun sarmalayıcısı»	<code>streamSettings</code> alanı

Bu alanlar iç içe JSON nesneleri olarak serileştirilir: boş bloklar `null` olarak döndürülür, geçerli JSON olmayan metin ise verilerin kaybolmaması için dizeye sarılır. Kaydetme sırasında ayrıştırma hataları «**Gelişmiş JSON**» örneğiyle görüntülenir.

«inbound JSON» görüntüleme penceresi ve inbound içe aktarma penceresi, sözdizimi vurgulamalı tam özellikli bir kod düzenleyici kullanır (sıradan metin alanı yerine): yapılandırma görüntüleme, yalnızca okuma modunda vurgulu şekilde; içe aktarma ise düzenlenebilir modda – bu, okuma ve düzenlemeyi kolaylaştırır.

4.8. inbound işlemleri: QR / Edit / Reset / Delete ve istatistikler

Liste ve inbound kartında şu işlemler kullanılabilir («**Menü**» menüsü):

Trafik istatistikleri

inbound için toplu trafik görüntülenir: «**Gönderilen/alınan**» (`up / down` alanları), «**Toplam trafik**», «**Toplam bağlantı**». Kartta ayrıca «**Oluşturuldu**», «**Güncellendi**», «**Bitiş tarihi**» de yer alır.

inbound listesinde her inbound için geçerli trafik hızını (çıkış/indirme) gösteren **Speed** sütunu bulunur; değer, sorgular arası sayaç artışlarından hesaplanır; aynı canlı hız, inbound istatistik penceresinde de gösterilir. Bir sonraki sorgu artış sağlamazsa hız değeri sıfırlanır.

inbound sayfasındaki istemci özetinde durum, «tükenmiş/sona ermiş» önceliğine göre belirlenir: süresi dolmuş veya trafiği tükenmiş (ve otomatik görevin `enable` değerini kaldırdığı) istemciler, gri «**Devre Dışı**» (*Disabled*) durumu yerine «**Tükenmiş/Sona Ermiş**» (*Depleted/Ended*) durumuna dahil edilir ve iki kez sayılmaz. Sınıflandırma, istemci kartının kendisinde gösterilenle örtüşür ve birden fazla inbound'a bağlı istemcileri doğru şekilde hesaba katar.

QR kodu ve bağlantı kopyalama

- «**Ayrıntılar**» – bağlantı ve abonelik bağlantılarını genişletir.
- İstemci QR kodu: ipucu «**Kopyalamak için QR koduna tıklayın**».
- «**Bağlantıyı Kopyala**» (İng. *Copy URL*), «**Bağlantıları Dışa Aktar**». 3.4.2 sürümünden itibaren sayfa eylemi olan «**Tüm bağlantıları dışa aktar**» (GET /panel/api/inbounds/allLinks ; «Tüm inbound bağlantılarını dışa aktar» penceresi, «All-Inbounds» dosyası) bağlantıları abonelik motoru aracılığıyla oluşturur – her istemciye açıklama şablonunu uygular ve yönetilen Host uç noktalarını tercih eder (önceden sabit inbound-email açıklaması kullanılıyordu).

Edit (Düzenle)

«**Bağlantıyı Değiştir**» – düzenleme formunu açar (POST /update/:id). Güncellenirken servis mevcut kaydı yeniden okur, değiştirilen alanları aktarır, gerektiğinde etiketi yeniden oluşturur (eski etiket otomatik oluşturulmuşsa) ve Xray çalışma zamanını senkronize eder. Başarı – «**Bağlantı başarıyla güncellendi**» bildirimi.

Reset Traffic (Trafik Sıfırla)

«**Trafik Sıfırla**» – bu inbound'un up / down sayaçlarını sıfırlar (POST /:id/resetTraffic , up=0 , down=0 olarak ayarlar). Onay:

«"{remark}" trafiği sıfırlansın mı?» / «Bu bağlantının gönderme/alma sayaçlarını 0'a sıfırlar».

inbound trafiğini sıfırlamak, istemci sayaçlarına **dokunmaz** (bunlar için ayrı «İstemci trafiğini sıfırla» işlemleri vardır). Sıfırlamadan sonra Xray yeniden başlatılır. Başarı – «**Gelen trafik sıfırlandı**» bildirimi. Toplu seçenek de mevcuttur – «**Tüm bağlantıların trafiğini sıfırla**» (POST /resetAllTraffics).

Delete (Sil)

«**Bağlantıyı Sil**» (POST /del/:id). Onay:

«"{remark}" bağlantısı silinsin mi?» / «Bağlantı ve tüm istemcileri silinecektir. Bu işlem geri alınamaz».

Silme işlemi, inbound'u çalışan Xray'den kaldırır (gerekirse yeniden başlatarak). Başarı – «**Bağlantı başarıyla silindi**» bildirimi. Toplu silme – POST /bulkDel , öge bazında raporlama ve en fazla bir Xray yeniden başlatması ile.

inbound istemcileriyle diğer işlemler

Menüde ayrıca şunlar bulunur: «**Klonla**» (yeni bağlantı noktasıyla ve boş istemci listesiyle inbound kopyası), «**Tüm İstemcileri Sil**» (POST /:id/delAllClients – tüm istemcileri siler, inbound kendisi korunur), «**Devre dışı istemcileri sil**», «**İstemcileri Bağla/Ayr**», «**İçe Aktar**»/«**Bağlantıları Dışa Aktar**» (POST /import). İstemci işlemlerinin ayrıntıları, istemciler bölümüne aittir.

5. Protokoller

Gelen bağlantı (inbound) oluştururken ilk olarak **Protokol** («Protocol») seçilir. Protokol; Xray-core'un bu inbound için hangi kimlik doğrulama ve trafik şifreleme yöntemini kullanacağını, **settings** içinde hangi alanların doldurulması gerektiğini ve hangi aktarım (network) ile güvenlik türlerinin (TLS / REALITY) kullanılabileceğini belirler.

Protokol alanı inbound oluşturulurken bir kez belirlenir ve **düzenleme sırasında değiştirilemez** (düzenleme formunda açılır liste kilitlidir). Protokolü değiştirmek için yeni bir inbound oluşturulması gerekir.

5.1. Desteklenen protokol listesi

Sunucu, **Protocol** alanı için aşağıdaki değerleri kabul eder:

```
oneof=vless vless trojan shadowsocks wireguard hysteria http mixed tunnel tun mtproto
```

3.3.0 sürümünden itibaren listeye **mtproto** (Telegram proxy) değeri eklenmiştir.

Yapılandırmadaki değer	Amaç	İstemci modeli
vless	Temel proxy protokolü (inbound oluştururken varsayılan)	UUID'li istemciler, flow ve kuantum sonrası şifreleme desteği
vmess	Xray'in klasik proxy protokolü	UUID ve security parametrelili istemciler
trojan	Normal HTTPS trafiğini taklit eden proxy	Parola ile kimlik doğrulayan istemciler
shadowsocks	Shadowsocks proxy'si (SIP022 / 2022-blake3 dahil)	Tek kullanıcı veya çoklu kullanıcı (2022)
wireguard	WireGuard inbound	Peer'lar (istemciler değil)
hysteria	Hysteria inbound (varsayılan olarak sürüm 2)	auth tokenli istemciler
http	Klasik HTTP proxy (forward proxy)	Kullanıcı/parola hesapları, trafik takibi yok
mixed	Birleşik SOCKS + HTTP proxy	Kullanıcı/parola hesapları
tunnel	Şeffaf yönlendirici (xray dokodemo-door)	İstemcisiz
tun	TUN arayüzü (yalnızca mevcut olanların görüntülenmesi)	İstemcisiz
mtproto	Telegram proxy'si (MTPROTO), 3.3.0'da eklendi; Xray değil, ayrı bir mtg işlemi tarafından yönetilir	İstemcisiz (gizli anahtar ile erişim)

tun hakkında not: Bu değer, önceden kaydedilmiş inbound'ların **görüntülenmesi** ve geriye dönük uyumluluk amacıyla listede tutulmaktadır; ancak mevcut sürümde backend tarafından oluşturulması önerilmez – destek kullanımdan kaldırılmış sayılır. Bu türde yeni inbound oluşturmanın bir anlamı yoktur.

Hysteria 2 hakkında not: «hysteria2» adında ayrı bir protokol yoktur. Bu, `streamSettings.version = 2` alanına sahip hysteria protokolüdür. Paylaşım bağlantılarında `hysteria2://` şeması, akış sürümü 2 olduğunda otomatik olarak seçilir.

Tüm protokoller düğümlere (nodes) dağıtımı desteklemez. Yalnızca şunlar düğümlere dağıtılabilir: `vless`, `vmess`, `trojan`, `shadowsocks`, `hysteria`, `wireguard`. `http`, `mixed`, `tunnel`, `tun`, `mtproto` protokolleri yalnızca yerel panel üzerinde çalışır.

5.2. Hangi protokoller TLS / REALITY / aktarımı destekler

Belirli bir güvenlik katmanını veya aktarımı etkinleştirme imkanı, protokole ve seçilen ağa (`streamSettings.network`) bağlıdır:

Özellik	Kullanılabilir protokoller	İzin verilen ağlar (network)
TLS	<code>vmess</code> , <code>vless</code> , <code>trojan</code> , <code>shadowsocks</code> (ayrıca <code>hysteria</code> için her zaman)	<code>tcp</code> , <code>ws</code> , <code>http</code> , <code>grpc</code> , <code>httpupgrade</code> , <code>xhttp</code>
REALITY	<code>vless</code> , <code>trojan</code>	<code>tcp</code> , <code>http</code> , <code>grpc</code> , <code>xhttp</code>
flow (xtls-rprx-vision)	yalnızca <code>vless</code>	yalnızca <code>tcp</code> , <code>security = tls</code> veya <code>reality</code> koşuluyla
Akış / aktarım («Akış» sekmesi)	<code>vmess</code> , <code>vless</code> , <code>trojan</code> , <code>shadowsocks</code> , <code>hysteria</code>	–

`http`, `mixed`, `tunnel`, `tun`, `wireguard` protokollerinde aktarım sekmesi kullanılamaz – bunların Xray akış ayarları yoktur.

5.3. VLESS

Amaç: temel modern proxy protokolü. XTLS-Vision (`flow`), REALITY ve VLESS düzeyinde kuantum sonrası şifrelemeyi (`decryption` / `encryption` alanları) destekler. Yeni inbound'lar için varsayılan olarak kullanılır.

`settings` bloğunun alanları:

Alan	Varsayılan değer	Açıklama
<code>clients</code>	<code>[]</code>	İstemci listesi. Her birinde: <code>id</code> (UUID), <code>email</code> (zorunlu), <code>flow</code> , limitler (<code>limitIp</code> , <code>totalGB</code> , <code>expiryTime</code>), <code>enable</code> , <code>tgId</code> , <code>subId</code> , <code>comment</code> , <code>reset</code>
<code>decryption</code>	<code>none</code>	Sunucu tarafında şifre çözme parametresi. UI etiketi: «Şifre Çözme» (İng. «Decryption»)

Alan	Varsayılan değer	Açıklama
encryption	none	Eşleşen şifreleme parametresi (istemci bağlantısına aktarılır). Etiket: «Şifreleme» (İng. «Encryption»)
fallbacks	[]	Fallback listesi (fallback'ler bölümüne bakın); network = tcp ve security = TLS veya REALITY olduğunda kullanılabilir
testseed	(4 sayı: 900, 500, 900, 256)	«Vision testseed» – XTLS-Vision padding için 4 pozitif tam sayı. Yalnızca xtls-rprx-vision flow'lu istemcilere uygulanır, aksi hâlde yok sayılır

flow (xtls-rprx-vision)

flow , inbound'da değil **istemci** üzerinde ayarlanır ve üç değerden birini alır:

Değer	Anlamı
"" (boş)	XTLS-flow yok (varsayılan)
xtls-rprx-vision	XTLS-Vision – TCP+TLS/REALITY üzerinde VLESS için önerilen mod
xtls-rprx-vision-udp443	Aynı Vision, ancak UDP/443 (QUIC) işleme ile

flow alanı yalnızca tüm koşullar sağlandığında seçilebilir: protokol vless , network = tcp ve security = tls ya da reality . **Vision testseed** alanı da formda yalnızca aynı koşullar altında gösterilir.

XHTTP için istisna: VLESS network = xhttp üzerinde ve VLESS düzeyinde kuantum sonrası kimlik doğrulama (encryption / decryption , vlessenc) etkinleştirildiğinde, güvenlik katmanından bağımsız olarak – REALITY dahil – xtls-rprx-vision flow'u da geçerlidir. Bu durumda panel, paylaşım bağlantılarında ve aboneliklerde (Clash/Mihomo formatı dahil) xtls-rprx-vision 'ı doğru şekilde iletir; istemci Vision yapılandırmasını alır.

Şifre Çözme / Şifreleme (VLESS kuantum sonrası kimlik doğrulama)

decryption ve encryption alanları, VLESS düzeyinde kimlik doğrulamadır (aktarım katmanı TLS/ REALITY'den ayrı). Varsayılan olarak her ikisi de none 'dur. Formda bu alanların altında «**Anahtar Oluşturma**» bloğu yer alır – bir açılır liste modu ve «**Oluştur**» düğmesi (yanında «**Temizle**» düğmesi). Açılır liste altı seçenek içerir: **X25519 (native)**, **X25519 (xorpub)**, **X25519 (random)**, **ML-KEM-768 (native)**, **ML-KEM-768 (xorpub)**, **ML-KEM-768 (random)** – yani iki anahtar türü (klasik X25519 ve kuantum sonrası ML-KEM-768), her biri üç modda:

- **native** – seçilen türün temel anahtar çifti;
- **xorpub** – ortak anahtarın ek işleme türetildiği mod;
- **random** – rastgele bileşenli türev mod.

Listeden istediğiniz modu seçin ve «**Oluştur**» düğmesine tıklayın: panel bu moda ait hazır değer çiftiyle **her iki** alanı da (decryption ve encryption) doldurur. «**Temizle**» düğmesi her iki alanı none olarak sıfırlar.

Bloğun altında **«Seçili: ...»** durum satırı görünür; oluşturulan dizeden hem anahtar türünü (X25519 veya ML-KEM-768) hem de modu (native / xorpub / random) tanıyarak gösterir. Boş alanlar veya `none` değeri «None» olarak görüntülenir.

Teknik olarak düğmeler `GET /panel/api/server/getNewVlessEnc` adresine başvurur (`xray vlessenc` üzerinden anahtar oluşturma) ve **her iki** alanı `mlkem768x25519plus.native.<rtt>.<role>` biçiminde eşleşen değerlerle doldurur (örneğin `decryption = mlkem768x25519plus.native.600s.server-x25519` , `encryption = mlkem768x25519plus.native.0rtt.client-x25519`). `decryption` parametresi sunucuda kalır, `encryption` ise istemci bağlantısına gönderilir.

Önemli: Xray için inbound yapılandırması oluşturulurken panel gereksiz olanı kaldırır: `settings` içinde `encryption` kalırsa (bu istemci tarafına aittir) sunucu yapılandırmasından **çıkarılır**. Sunucuda yalnızca `decryption` kalır.

VLESS ne zaman seçilmeli: REALITY (kendi sertifikası olmadan) veya TLS + XTLS-Vision kombinasyonunda, özellikle kuantum sonrası kimlik doğrulama gerektiğinde, yeni inbound için önerilen varsayılan seçenektir.

Örnek: tek istemcili ve XTLS-Vision'lı VLESS inbound için `settings` bloğu. `flow` alanı istemcidedir, `decryption` sunucuda kalır:

```
{
  "clients": [
    {
      "id": "d342d11e-d424-4583-b36e-524ab1f0afa4",
      "email": "user1",
      "flow": "xtls-rprx-vision",
      "limitIp": 2,
      "totalGB": 0,
      "expiryTime": 0,
      "enable": true
    }
  ],
  "decryption": "none"
}
```

REALITY kombinasyonu için karşılık gelen `streamSettings` bloğu («Transport» sekmesi → Security: REALITY) şöyle görünür:

```
{
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "dest": "www.microsoft.com:443",
    "serverNames": ["www.microsoft.com"],
    "privateKey": "<X25519 özel anahtarı>",
    "shortIds": ["", "6ba85179e30d4fc2"]
  }
}
```

5.4. VMess

Amaç: Xray'in klasik proxy protokolü. UUID ile kimlik doğrulama; istemcide ek olarak yük şifreleme yöntemi (security) yapılandırılır.

settings bloğunun alanları:

Alan	Varsayılan değer	Açıklama
clients	[]	İstemci listesi

Her VMess istemcisinde (ortak email , limit , enable , tgId , subId , comment , reset alanlarına ek olarak):

İstemci alanı	Varsayılan değer	Açıklama
id	–	İstemci UUID'si
security	auto	VMess yük şifreleme yöntemi. İzin verilen değerler: aes-128-gcm , chacha20-poly1305 , auto , none , zero

security değerleri:

- auto – Xray platforma göre şifreleme algoritmasını kendisi seçer (önerilir);
- aes-128-gcm , chacha20-poly1305 – sabit AEAD şifreleri;
- none – yük şifrelemesi yok (yalnızca TLS üzerinde anlamlıdır);
- zero – yük şifrelemesi ve kimlik doğrulaması yok.

Geriye dönük uyumluluk: eski kayıtlarda security: "" bulunabilir – okunurken boş dize auto olarak dönüştürülür. Sunucu yapılandırması oluşturulurken VMess istemcilerindeki security alanı settings 'den **kaldırılır**, zira inbound için bu alan gerekli değildir.

VMess ne zaman seçilmeli: eski istemcilerle veya mevcut yapılandırmalarla uyumluluk için. Yeni dağıtımlarda genellikle VLESS tercih edilir.

5.5. Trojan

Amaç: normal HTTPS trafiğini taklit eden proxy. Parola ile kimlik doğrulama. VLESS gibi fallback'leri ve (network = tcp olduğunda) REALITY/TLS'yi destekler.

settings bloğunun alanları:

Alan	Varsayılan değer	Açıklama
clients	[]	İstemci listesi
fallbacks	[]	Fallback listesi (network = tcp ve TLS/REALITY koşulunda kullanılabilir)

Her Trojan istemcisindeki temel alanlar:

İstemci alanı	Varsayılan değer	Açıklama
password	–	İstemci parolası (zorunlu, en az 1 karakter)
email	–	İstemcinin benzersiz tanımlayıcısı

Diğer istemci alanları ortaktır (limitIp , totalGB , expiryTime , enable , tgId , subId , comment , reset).

Trojan ne zaman seçilmeli: 443 portunda HTTPS görünümü ve istenmeden gelen bağlantılar için bir web sunucusuna (Nginx) yönlendirme (fallback) gerektiğinde.

Örnek: yerel web sunucusuna fallback'li Trojan için settings bloğu. Geçerli parola içermeyen (istenmeden gelen) bağlantılar, 127.0.0.1:8080 dinleyen Nginx'e yönlendirilir:

```
{
  "clients": [
    { "password": "S3cret-Pass-1", "email": "user1" }
  ],
  "fallbacks": [
    { "dest": "127.0.0.1:8080" }
  ]
}
```

Fallback için network = tcp ve Security = TLS veya REALITY gereklidir; aksi hâlde fallbacks alanı kullanılamaz.

5.6. Shadowsocks

Amaç: hafif Shadowsocks proxy'si. Hem eski AEAD şifrelerini hem de yeni SPO22 yöntemlerini (2022-blake3-*) destekler. Tek kullanıcıli veya çok kullanıcıli modda çalışabilir.

settings bloğunun alanları:

Alan	Varsayılan değer	Açıklama
method	2022-blake3-aes-256-gcm	inbound şifreleme yöntemi. UI etiketi: «Şifreleme yöntemi» (İng. «Encryption method»)
password	''	inbound parolası (2022 yöntemleri için seçilen yönteme göre otomatik oluşturulur)
network	tcp,udp	Aktarım. Etiket: «Ağ» (İng. «Network»). Seçenekler: tcp,udp (TCP,UDP), tcp , udp
clients	[]	İstemci listesi
ivCheck	false (kapalı)	«ivCheck» anahtarı — IV yeniden kullanımına karşı koruma

Şifreleme yöntemleri (method)

İzin verilen değerler:

Yöntem	Kategori
aes-256-gcm	Eski AEAD
chacha20-poly1305	Eski AEAD
chacha20-ietf-poly1305	Eski AEAD
xchacha20-ietf-poly1305	Eski AEAD
2022-blake3-aes-128-gcm	SS 2022 (önerilir)
2022-blake3-aes-256-gcm	SS 2022 (varsayılan)
2022-blake3-chacha20-poly1305	SS 2022, tek kullanıcı

Yöntemlere ilişkin panel mantığı:

- **2022 yöntemleri** (2022-blake3-*) «SS 2022» olarak değerlendirilir. 2022-blake3-chacha20-poly1305 yöntemi **tek kullanıcıdır** (çoklu kullanıcı desteklenmez); diğer 2022 yöntemleri birden fazla istemciyi destekler. Parola alanı (yönteme göre anahtar uzunluğunu otomatik ayarlayan oluşturma düğmesiyle birlikte) yalnızca 2022 yöntemleri için formda gösterilir.
- **Eski şifreler** (aes-* , chacha20-*) klasik «tek yöntem + tek parola» şemasıyla çalışır.

Xray başlatılmadan önce normalleştirme: eski şifreler için her istemcinin method değeri inbound'unkiyle eşleşmelidir (aksi hâlde Xray «unsupported cipher method:» hatasıyla çöker). 2022 yöntemleri için ise tam tersi – istemcideki method alanı **boş** olmalıdır (aksi hâlde Xray inbound'u «users must have empty method» hatasıyla reddeder). Panel, yöntem değiştirildiğinde verileri otomatik olarak düzenler.

Anahtar boyutu değiştiğinde istemci anahtarlarının yeniden oluşturulması: Shadowsocks-2022'de şifreleme yöntemi farklı anahtar boyutuna sahip bir yöntemle değiştirildiğinde (örneğin 2022-blake3-aes-256-gcm ile 2022-blake3-aes-128-gcm arasında), panel inbound kaydedilirken istemci PSK'larını yeni uzunluğa göre otomatik olarak yeniden oluşturur. Aksi hâlde eski anahtarlar önceki uzunluklarında kalır ve Xray onları reddeder. Sonuç olarak etkilenen istemcilerin aboneliği yeniden alması gerekir – önceki bağlantılar artık çalışmaz.

Shadowsocks ne zaman seçilmeli: TLS görünümü olmayan basit dağıtımlar için; modern seçim – 2022-blake3-* yöntemleri.

Örnek: 2022-blake3 yöntemi için Shadowsocks settings bloğu (çok kullanıcı mod). inbound'un kendi parolası (gerekli uzunlukta base64 anahtar), her istemcinin kendi parolası vardır; istemcinin method alanı **boş**dur:

```
{
  "method": "2022-blake3-aes-256-gcm",
  "password": "d2hhdGV2ZXItMzItYnl0ZS1iYXNlNjQta2V5LWhlcmU=",
  "network": "tcp,udp",
  "clients": [
    {
      "email": "user1",
      "password": "Y2xpZW50LWtleS0zMilieXRlcy1iYXNlNjQtaGVyZQ==",
      "method": ""
    }
  ]
}
```

Eski şifreler (aes-256-gcm vb.) için tam tersi geçerlidir: inbound için tek parola ve istemcinin `method` değeri inbound'unkiyle eşleşmelidir.

5.7. Dokodemo-door / Tunnel (şeffaf yönlendirici)

Amaç: şeffaf yönlendirici (panelde — dokodemo-door davranışını uygulayan tunnel protokolü). Kimlik doğrulama ve istemci olmaksızın trafiği alır ve belirtilen adres/porta yönlendirir.

`settings` bloğunun alanları:

Alan	Varsayılan değer	Açıklama
<code>rewriteAddress</code>	(yok)	«Adresi yeniden yaz» (İng. «Rewrite address») — trafiğin yönlendirileceği hedef adres
<code>rewritePort</code>	(yok)	«Portu yeniden yaz» (İng. «Rewrite port») — hedef port (0-65535)
<code>allowedNetwork</code>	<code>tcp,udp</code>	«İzin verilen ağ» (İng. «Allowed network»). Seçenekler: <code>tcp,udp</code> , <code>tcp</code> , <code>udp</code>
<code>portMap</code>	<code>{}</code>	«Port eşleme» — port→port haritası (Record<string,string>)
<code>followRedirect</code>	<code>false</code> (kapalı)	«Yönlendirmeyi izle» (İng. «Follow redirect») — ele geçirilen bağlantıdan özgün hedef adresini kullan

Tunnel için «Transport» sekmesi: bu türdeki inbound'da `sockopt` ayarıyla sınırlı «**Transport**» sekmesi kullanılabilir — bu, **TProxy** modu (`sockopt.tproxy` üzerinden şeffaf proxy/redirect) için yeterlidir. Aktarım seçimi açılır listesi (`network`) ve Tunnel için «Security» sekmesi gizlidir; bu tür TLS/REALITY'yi desteklemez.

Ne zaman seçilmeli: dahili servislere şeffaf proxy/port yönlendirme için.

«Portu yeniden yaz» (`rewritePort`) alanı boş bırakılabilir: değerin silinmesi, kaydetme hatasına yol açmak yerine yalnızca değeri inbound ayarlarından çıkarır. (Önceden bu alanın silinmesi `settings.rewritePort` doğrulama hatasına neden olur ve JSON sekmesinden bile kaydetmeyi engellerdi.)

5.8. SOCKS / HTTP (mixed protokolü)

Bu derlemede ayrı bir socks protokolü bulunmaz – SOCKS ve HTTP proxy'si mixed protokolünde (birleşik SOCKS + HTTP) birleştirilmiştir. Ayrıca ayrı bir salt http proxy'si de mevcuttur.

5.8.1. Mixed (SOCKS + HTTP)

settings bloğunun alanları:

Alan	Varsayılan değer	Açıklama
auth	password	«Auth» – kimlik doğrulama modu. Seçenekler: password (kullanıcı adı/parola) veya noauth (kimlik doğrulamasız)
accounts	(isteğe bağlı)	«Hesaplar» – kullanıcı adı/parola çiftlerinin listesi. auth = noauth olduğunda alana yapılandırmaya yazılmaz
udp	false (kapalı)	«UDP» anahtarı – SOCKS üzerinden UDP desteği
ip	127.0.0.1	«UDP IP» – UDP ilişkilendirmeleri için yerel adres. Alan yalnızca udp etkinleştirildiğinde gösterilir

Hesaplar «Ekle» düğmesiyle eklenir; eklendiğinde rastgele kullanıcı adı (8 karakter) ve parola (12 karakter) oluşturulur, bunlar düzenlenebilir.

5.8.2. HTTP (salt proxy)

Amaç: klasik HTTP forward proxy'si. Xray düzeyinde istemcileri «faturalı» olarak takip etmez (e-posta/limit yoktur) – yalnızca hesap listesi bulunur.

settings bloğunun alanları:

Alan	Varsayılan değer	Açıklama
accounts	[]	«Hesaplar» – kullanıcı adı/parola çiftlerinin listesi (her iki alan da zorunludur)
allowTransparent	false (kapalı)	«Şeffaf bağlantılara izin ver» (İng. «Allow transparent») – istekleri özgün Host başlığıyla ilet

SOCKS/HTTP ne zaman seçilmeli: karmaşık gizleme olmaksızın yerel veya hizmet amaçlı proxy erişimi için. Tek bir port üzerinden hem SOCKS hem HTTP istemcilerine hizmet etmesi nedeniyle mixed kullanışlıdır.

5.9. WireGuard (inbound)

Amaç: WireGuard inbound – maskelenen bir proxy değil, bir WireGuard VPN tüneli. Aktarım ve TLS/REALITY bu protokole uygulanamaz.

3.4.2 sürümünden itibaren WireGuard çok istemcili modele geçirildi (VLESS/VMess/Trojan/Shadowsocks/Hysteria gibi). inbound'un kendisi yalnızca sunucu kısmını tutar; peer cihazları artık sıradan **istemciler** olarak eklenir (bkz. bölüm 8) – tünelde otomatik adres atama, abonelik desteği, trafik/süre limitleri ve gruplarla. Eski satır içi «Peer'lar / Peer ekle» listesi inbound formundan kaldırıldı: yeni inbound peer'sız oluşturulur ve her istemciye adres otomatik olarak atanır.

inbound alanları (settings bloğu):

Alan	Varsayılan değer	Açıklama
secretKey	–	Sunucunun özel anahtarı (zorunlu). Yanında oluşturma düğmesi bulunur; genel anahtar (Public Key) ondan otomatik olarak türetilir (salt okunur; ayrıca saklanan pubKey kaldırıldı)
mtu	1420	Arayüz MTU'su
dns	1.1.1.1, 1.0.0.1	3.4.2'de yeni. İstemci .conf 'unun DNS = satırına yazılan DNS
noKernelTun	false (kapalı)	«Çekirdeksiz TUN» – kernel TUN yerine kullanıcı alanı TUN
domainStrategy	(isteğe bağlı)	«Domain Strategy»: ForceIP , ForceIPv4 , ForceIPv4v6 , ForceIPv6 , ForceIPv6v4

İstemcideki WireGuard parametreleri – istemci ekleme/düzenleme penceresindeki «Kimlik bilgileri» sekmesi (bağlı inbound WireGuard olduğunda gösterilir):

Alan	Açıklama
«WireGuard özel anahtarı»	İstemcinin özel anahtarı (düzenlenebilir, «Yeniden oluştur» düğmesi var); girildiğinde ondan genel anahtar otomatik olarak türetilir
«WireGuard genel anahtarı»	Salt okunur; özel anahtardan hesaplanır
«WireGuard paylaşılan anahtarı» (PSK)	İsteğe bağlı ek paylaşılan anahtar
«WireGuard izin verilen IP'ler»	3.5.0'dan itibaren – düzenlenebilir (yer tutucu 10.0.0.2/32 , ipucu «Otomatik atama için boş bırakın; kayıtları virgülle ayırın»). Boşsa adres otomatik atanır; sunucu her kaydı (IP veya CIDR) denetler, tekil adresleri /32 'ye normalleştirir ve bu inbound'un başka bir istemcisi tarafından kullanılan adresi reddeder

Tünel adresi, inbound'un alt ağından (varsayılan 10.0.0.0/24 : sunucu .1 'i alır, istemciler .2 'den başlar) sunucu tarafından otomatik olarak atanır; mevcut istemciler başka bir /24 kullanıyorsa yeni adresler aynı alt ağdan atanır.

Domain Strategy ve Transport sekmesi

Sayıllara ek olarak WireGuard inbound'da **Domain Strategy** alanı bulunur (etki alanı çözümleme stratejisi: ForceIP , ForceIPv4 , ForceIPv4v6 , ForceIPv6 , ForceIPv6v4). Alan isteğe bağlıdır ve yalnızca ayarlandığında yapılandırmaya yazılır.

Workers alanı (workers , çalışan iş parçacığı sayısı) WireGuard formlarından (hem inbound hem outbound) kaldırılmıştır: xray-core v26.6.22 sürümünden itibaren motor bu alanı artık kullanmamakta, bunun yerine wireguard-go'nun dahili mekanizmasına güvenmektedir. Önceden kaydedilmiş yapılandırmalar değişiklik gerektirmeden çalışmaya devam eder — alan ayırıştırma sırasında yok sayılır, geçiş gerekmez.

WireGuard için «**Transport**» sekmesi de mevcuttur — ancak kısıtlı biçimde: yalnızca **sockopt** ve **Finalmask** gizleme ayarları yapılandırılabilir. WireGuard her zaman UDP üzerinden dinlediğinden aktarım seçim açılır listesi (network) gizlidir. Finalmask gürültü kayıtlarında (noise) ayrı bir alan olarak **Rand Range** (0–255 bayt aralığı, doğrulamayla) belirlenir; WireGuard ve Hysteria için gizleme yöntemi olarak **Salamander** da mevcuttur.

inbound işlemleri ve dışa aktarma (3.5.0'dan itibaren)

WireGuard inbound satır menüsü artık eksiksiz «çok istemcili» işlem setini içerir — «**Tüm bağlantıları dışa aktar**», abonelik dışa aktarma, **istemci bağlama/bağlantı kesme**, gruba ekleme ve tüm istemcileri silme (önceden yalnızca dışa aktarma/sıfırlama/klonlama/silme vardı); listede ayrıca istemci sayacı gösterilir. WireGuard için «Tüm bağlantıları dışa aktar» penceresi iki sekmeye ayrılmıştır: **Config** (istemci başına bir tane olmak üzere birleştirilmiş .conf blokları) ve **Links** (kişisel wireguard:// bağlantıları); «Kopyala» ve «İndir» açık olan sekmeye uygulanır. Düğümde barındırılan inbound için istemci .conf /QR içindeki Endpoint artık ana panelin değil, **düğümün adresini** gösterir.

WireGuard istemci yapılandırması ve paylaşımı

WireGuard istemcisinde «Bilgi»/QR pencerelerinde **katlanabilir bir yapılandırma kartı** bulunur: tam .conf ([Interface] — PrivateKey / Address / DNS / MTU ile ve [Peer] — PublicKey / PresharedKey / AllowedIPs = 0.0.0.0/0, ::/0 / Endpoint / PersistentKeepalive ile) ve **Kopyala**, **İndir** ve **QR** düğmeleri. Ayrıca istemci uygulamasının hazır bir .conf 'a çözümlediği wireguard:// wg:// biçiminde bir bağlantı da desteklenir.

WireGuard ne zaman seçilmeli: gizleme yapılan bir proxy değil, tam anlamıyla bir WireGuard VPN tüneli gerektiğinde.

5.10. Hysteria (varsayılan olarak v2)

Amaç: QUIC üzerinde Hysteria inbound. Panel varsayılan olarak sürüm 2 ile çalışır. Her istemci UUID/parola yerine auth tokenıyla kimlik doğrular. Hysteria için TLS her zaman kullanılabilir (5.2'deki özellik tablosuna bakın).

settings bloğunun alanları:

Alan	Varsayılan değer	Açıklama
version	2	Protokol sürümü (en az 1; panel varsayılanı 2)
clients	[]	İstemci listesi

Her istemcideki temel alan – `auth` (token, zorunlu) – ve ortak alanlar (`email` , `limitler` , `enable` , `tgId` , `subId` , `comment` , `reset`).

Ek parametreler `streamSettings.hysteriaSettings` içinde ayarlanır:

Alan	Değer / seçenekler	Açıklama
<code>version</code>	2 olarak sabit (alan kilitli)	«Sürüm» (İng. «Version»)
<code>udpIdleTimeout</code>	(≥1 tam sayı, sn.)	«UDP boşta zaman aşımı (s)» – UDP boşta kalma zaman aşımı
<code>masquerade</code>	varsayılan olarak kapalı	«Masquerade» – «istenmeden gelen» isteklerde normal bir web sunucusunu taklit etme

`masquerade` etkinleştirildiğinde tür (`type`) seçimi yapılabilir:

- `` – varsayılan (404 sayfası);
- `proxy` – ters proxy («Upstream URL», «Host'u yeniden yaz», «TLS doğrulamayı atla» alanları);
- `file` – dizin sunumu («Dizin» alanı, örneğin `/var/www/html`);
- `string` – sabit yanıt («Durum kodu», «Body», «Başlıklar» alanları).

Hysteria ne zaman seçilmeli: QUIC aktarımı ve kararsız/mobil bağlantılarda dayanıklılık gerektiğinde; `masquerade`, giriş noktasının gizliliğini artırır.

5.11. MTProto (Telegram proxy'si)

3.3.0 sürümünde eklendi. Protokol değeri – `mtproto` .

MTProto, Telegram'ın özel proxy protokolüdür. 3X-UI'de bu tür inbound **Xray tarafından değil, panelin kendisinin yönettiği ayrı bir `mtg` işlemi tarafından** sunulur. Panel, etkinleştirilmiş MTProto inbound'larını çalışan `mtg` işlemleriyle periyodik olarak karşılaştırır: eksik olanları başlatır, fazla olanları durdurur ve `mtg` metriklerinden trafik sayaçlarını toplar. Bu nedenle bu tür inbound'da **trafik takibi** normal protokollerdeki gibi çalışır.

3.5.0 sürümünden itibaren MTProto çok istemcili bir protokoldür (motor, `mtg-multi` çatalıyla değiştirildi): tek bir inbound çok sayıda kullanıcıya hizmet verir; bunların her biri kendi FakeTLS gizli anahtarı, trafik kotası, süresi, etkinleştirme anahtarı, isteğe bağlı ad-tag'i ve kişisel `tg://proxy` bağlantısı olan sıradan bir **istemcidir** (bkz. bölüm 8). Önceki «bir inbound = bir gizli anahtar» modeli kaldırıldı: inbound düzeyindeki «Secret» alanı formdan çıkarıldı; panel güncellendiğinde mevcut tekil gizli anahtar otomatik olarak bu inbound'un ilk istemcisine dönüştürülür (elle işlem gerekmez).

Sonuçlar:

- Bu inbound için «**Aktarım**» (**Stream Settings**) **sekmesi geçerli değildir**; istemciler ise artık diğer protokollerdeki gibi yönetilir – bağlama/bağlantı kesme, limitler, gruplar, abonelikler.
- İstemci değişiklikleri (ekleme, silme, gizli anahtar değişimi, etkinleştirme/devre dışı bırakma, ad-tag) `mtg` management API'si üzerinden, diğer kullanıcıların Telegram oturumlarını koparmadan «**anında**» uygulanır;

yalnızca inbound'un yapısal değişiklikleri (adres, domain fronting, bağlantı sınırı, genel IP'ler) süreci yeniden başlatır.

- MTProto inbound yalnızca **ana panelde** çalışır; alt düğümlere (nodes) dağıtılmaz (NodeID tanımlı inbound'lar atlanır).
- MTProto için «**Sniffing**» sekmesi gizlidir – bu protokol Xray değil mtg işlemi tarafından sunulduğundan, sniffing uygulanamaz.

Alanlar. inbound'un settings bölümünde saklanır:

UI'daki alan	Anahtar	Açıklama
Remark	remark	inbound etiketi.
Listen IP	listen	Dinlenecek IP (boş = tüm arayüzler).
Port	port	Proxy portu.
Gizleme alanı (FakeTLS)	settings.fakeTlsDomain	Varsayılan <code>www.cloudflare.com</code> . 3.5.0'dan itibaren – yeni istemcilerin gizli anahtarlarının üretiminde kullanılan varsayılan alan adı (ipucu: «Default FakeTLS domain used to generate a new client's secret. Each client can front its own domain»); her istemci kendi gizli anahtarı aracılığıyla kendi alan adının arkasına gizlenebilir.
Max connections	settings.throttleMaxConnections	3.5.0'da yeni. Tüm kullanıcılar genelinde eş zamanlı bağlantı sınırı, adil paylaşım; <code>0</code> – sınırsız.
Public IPv4 / Public IPv6	settings.publicIpv4 / settings.publicIpv6	3.5.0'da yeni. ad-tag middle proxy için sunucunun genel adresi (yer tutucular <code>1.2.3.4 / 2001:db8::1</code>); boşsa mtg kendisi belirler.

Gizli anahtar formatı (FakeTLS). Gizli anahtar artık **her istemcide** yaşar (istemci formundaki «**MTProto secret**» alanı, yeniden oluşturma düğmesiyle; yanında – isteğe bağlı «**Ad-tag (sponsorlu kanal)**», tam 32 hex karakter). Gizli anahtarın biçimi öncekiyle aynıdır: `ee + 32 hex karakter + gizleme alanının hex kodu` (`ee<hex32><hex(alan adı)>`); yeni bir istemci MTProto inbound'una bağlandığında gizli anahtar, inbound'un varsayılan alan adından otomatik üretilir. İstemcinin trafik kotası ve geçerlilik süresi 3.5.0'dan itibaren (mtg-multi limitleri aracılığıyla) **gerçekten uygulanır**: kotası tükenen veya süresi dolan istemci devre dışı bırakılır, trafik sınırlama ise erişimi hemen geri verir.

Domain-fronting ve gelişmiş mtg seçenekleri

MTProto inbound'da ek mtg işlem parametreleri bulunur. **Domain fronting IP**, **Domain fronting port** ve **Domain fronting PROXY protocol** alanları, mtg 'nin Telegram dışı trafiği nereye göndereceğini (örneğin sahte bir NGINX sitesine) belirler: IP boş bırakılırsa DNS üzerinden FakeTLS alanı kullanılır, varsayılan port `443` 'tür. Ek olarak **Accept PROXY protocol** (dinleyici için), **IP preference** (`prefer-ipv6 / prefer-ipv4 / only-ipv6 / only-ipv4`) ve **Debug logging** seçenekleri de mevcuttur. Her değer yalnızca ayarlandığında `mtg-<id>.toml` dosyasına yazılır.

Telegram trafiğini Xray üzerinden yönlendirme

«Route through Xray» anahtarı (varsayılan olarak kapalı) ve isteğe bağlı **Outbound** alanı, Telegram çıkış trafiğini Xray yönlendiricisine bağlamayı sağlar. Etkinleştirildiğinde panel, Xray yapılandırmasına inbound'un etiketiyle yerel bir SOCKS köprüsü ekler; **mtg** ise Telegram trafiğini bu köprü üzerinden gönderir. Ardından trafik «Routing» sekmesindeki kurallarla eşleştirilebilir ya da **Outbound** alanı aracılığıyla seçili bir outbound veya dengeleyiciye zorla yönlendirilebilir (alan boş bırakılırsa yönlendirme kuralları geçerli olur).

Kullanıcıya nasıl paylaşılır. MTProto inbound için panel bir davet bağlantısı oluşturur:

Örnek: FakeTLS gizli anahtarı ve hazır bağlantı. Gizleme alanı `www.cloudflare.com` ise gizli anahtar `ee + 32` hex karakter + alanın hex kodu olarak oluşturulur, örneğin:

```
secret = ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

Hazır davet bağlantısı (kullanıcıya Telegram'da gönderilir, QR kodla birlikte):

```
tg://proxy?
server=203.0.113.10&port=443&secret=ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

```
tg://proxy?server=<adres>&port=<port>&secret=<gizli anahtar>
```

(eşdeğeri — `https://t.me/proxy?server=...&port=...&secret=...`). 3.5.0'dan itibaren bağlantı **kişiseldir** — belirli istemcinin gizli anahtarından oluşturulur ve `#remark` parçası bağlantıdan kaldırılmıştır (Telegram'ın gevşek ayrıştırıcıları onu gizli anahtarla birleştirip içe aktarmayı bozuyordu); açıklama, bilgi penceresinde ayrı bir etiket olarak gösterilir. Bu bağlantıyı ve QR kodunu Telegram kullanıcısına gönderin — açıldığında proxy uygulamaya anında eklenir. Bağlantı ayrıca abonelik sunucusu üzerinden de sağlanır.

Ne zaman kullanılmalı. Telegram engellemesini aşmanın standart yöntemi; FakeTLS gizlemesi (gizleme alanı), trafiği belirtilen siteye yapılan normal bir ziyarete benzetir.

5.12. Protokol seçimi için hızlı başvuru

- **VLESS** — varsayılan tercih; REALITY veya TLS + XTLS-Vision ile en iyi seçim, kuantum sonrası kimlik doğrulamayı destekler.
- **Trojan** — web sunucusuna fallback'li HTTPS görünümü.
- **VMess** — eski istemcilerle uyumluluk.
- **Shadowsocks** — TLS olmadan basit proxy; modern seçim — 2022-blake3-* yöntemleri.
- **Hysteria** — QUIC, zayıf bağlantılarda dayanıklılık.
- **mixed / http** — hizmet amaçlı SOCKS/HTTP proxy'leri.
- **WireGuard** — tam VPN tüneli.
- **tunnel** — şeffaf port yönlendirme.
- **MTProto** — Telegram engellemesini aşmak için proxy (FakeTLS); ayrı **mtg** işlemi.

6. Aktarım (Stream Settings)

Aktarım (panel arayüzünde «**Aktarım**» alanı, İngilizce *Transmission*), Xray-core'un inbound içindeki verileri nasıl ilettiğini tanımlar: TLS/Reality üzerinde hangi ağ protokolünün kullanıldığını ve trafiğin nasıl çerçeveselendiğini belirtir. Bu parametreler Xray yapılandırmasının `streamSettings` nesnesine kaydedilir ve inbound düzenleyicisindeki aktarım sekmesinden ayarlanır. Şifreleme (TLS / Reality) ayrı bir bölümde ele alınmaktadır; burada yalnızca ağ seçimi ve ağ parametreleri açıklanmaktadır.

6.1. Ağ İletimi Seçimi

Ağ, «**Aktarım**» (`streamSettings.network`) açılır listesinden seçilir. Varsayılan değer `tcp` 'dir (listede **RAW** olarak görünür). Kullanılabilir seçenekler:

Listedeki değer	network alanı	Aktarım
RAW	<code>tcp</code>	Standart TCP (yeni Xray sürümlerinde RAW olarak yeniden adlandırılmıştır), isteğe bağlı HTTP gizlemesiyle
mKCP	<code>kcp</code>	Güvenilir UDP aktarımı mKCP
WebSocket	<code>ws</code>	HTTP(S) üzerinde WebSocket
gRPC	<code>grpc</code>	gRPC tüneli (HTTP/2)
HTTPUpgrade	<code>httpupgrade</code>	HTTP Upgrade
XHTTP	<code>xhttp</code>	XHTTP / SplitHTTP – modern çoğullamalı aktarım

Değer değiştirildiğinde panel, önceki ağın ayarlar bloğunu temizler ve yeni ağın bloğunu şemasındaki varsayılan değerlerle doldurur; bu nedenle alt formun her alanı her zaman anlamlı bir başlangıç değerine sahiptir.

Önemli. Bu panel derlemesinde **HTTP/2 aktarımı (h2) listede yer almamaktadır** – ağ seçenekleri kümesinden çıkarılmıştır; çift yönlü HTTP/2 benzeri tünel için gRPC, modern HTTP maskeli aktarım için XHTTP kullanılmaktadır. **Hysteria** aktarımı (`hysteria`) bu listeden seçilmez: Hysteria protokolüne sabit olarak bağlıdır ve inbound'un kendisi Hysteria protokolüyle oluşturulduğunda otomatik olarak belirir (bkz. madde 6.8).

Aşağıda her ağ ve her alanı ayrı ayrı açıklanmıştır.

6.2. RAW / TCP (`tcpSettings`)

Temel TCP aktarımı. Varsayılan olarak trafik olduğu gibi iletilir; isteğe bağlı olarak sıradan bir HTTP/1.1 alışverişini taklit edecek şekilde gizlenebilir.

Alan	Varsayılan değer	Açıklama
Proxy Protocol (<code>acceptProxyProtocol</code>)	<code>false</code> (kapalı)	Üst akış yük dengeleyici/proxy'den PROXY protokolü başlığını kabul et
HTTP gizlemesi (<code>header.type</code>)	<code>none</code> (kapalı)	Trafiği HTTP/1.1 görünümüne büründürmeyi etkinleştirir

Proxy Protocol

«**Proxy Protocol**» (`acceptProxyProtocol`) düğmesi. Etkinleştirildiğinde Xray, gelen bağlantıda PROXY protokolü başlığı bekler ve gerçek istemci IP adresini bu başlıktan çıkarır. Yalnızca panelin önünde bu başlığı ekleyen bir ters proxy/yük dengeleyici (örneğin `send-proxy` ile HAProxy veya nginx) varsa etkinleştirilir. Varsayılan olarak kapalıdır.

HTTP Gizlemesi (camouflage)

«**HTTP Gizlemesi**» düğmesi. `header` alanını yönetir:

- **Kapalı** → `header.type = "none"` (iletimde `header` alanı tamamen yoktur). Saf TCP.
- **Açık** → `header.type = "http"`. Trafik bir HTTP/1.1 istek ve yanıtı görünümünde çerçevelenir. Etkinleştirildiğinde panel hemen `request` ve `response` alt nesnelerini varsayılan değerlerle doldurur.

HTTP gizlemesi etkinleştirildiğinde taklit istek ve yanıtla ilişkin yapılandırma alanları görünür.

İstek başlığı (`header.request`):

Alan	Anahtar	Varsayılan değer	Açıklama
İstek sürümü	<code>request.version</code>	<code>1.1</code>	İstek başlangıç satırındaki HTTP sürümü
İstek yöntemi	<code>request.method</code>	<code>GET</code>	Taklit edilen isteğin HTTP yöntemi
İstek yolu	<code>request.path</code>	<code>/</code>	Yol(lar). Virgülle ayrılmış değer listesi olarak girilir; iletimde bu bir dize dizisidir. Boş bırakılırsa <code>/</code> atanır
İstek başlıkları	<code>request.headers</code>	<code>{}</code> (boş)	HTTP başlıklarının «Ad/Değer» tablosu. <code>ad</code> → <code>[değerler]</code> eşlemesi olarak saklanır (bir ada birden fazla değer karşılık gelebilir)

Yanıt başlığı (`header.response`):

Alan	Anahtar	Varsayılan değer	Açıklama
Yanıt sürümü	<code>response.version</code>	<code>1.1</code>	Yanıt başlangıç satırındaki HTTP sürümü
Yanıt durumu	<code>response.status</code>	<code>200</code>	Taklit edilen yanıtın HTTP durum kodu
Yanıt nedeni	<code>response.reason</code>	<code>OK</code>	Durum açıklaması (reason-phrase)

Alan	Anahtar	Varsayılan değer	Açıklama
Yanıt başlıkları	<code>response.headers</code>	<code>{}</code> (boş)	Yanıt başlıklarının «Ad/Değer» tablosu (ad → [değerler] eşlemesi)

Başlık alanları satır satır düzenlenir – her satır bir başlık adı (Ad) ve değerini (Değer) tanımlar. Bu parametreler yalnızca trafiğin dışarıdan nasıl görüldüğünü gizlemek için kullanılır; şifrelemeyi etkilemezler. Varsayılan değerler (GET / HTTP/1.1 , 200 OK yanıtı) çoğu senaryo için uygundur – yalnızca belirli bir site/hizmeti taklit etmek gerektiğinde değiştirilmelidir.

HTTP gizlemesiyle RAW için örnek `streamSettings` :

```
{
  "network": "tcp",
  "tcpSettings": {
    "acceptProxyProtocol": false,
    "header": {
      "type": "http",
      "request": {
        "version": "1.1",
        "method": "GET",
        "path": ["/"],
        "headers": {
          "Host": ["www.example.com"]
        }
      },
      "response": {
        "version": "1.1",
        "status": "200",
        "reason": "OK"
      }
    }
  }
}
```

İletimde `path` bir dize dizisidir; her başlık ise bir değer dizisidir (Host → ["www.example.com"]).

6.3. mKCP (`kcpSettings`)

mKCP, UDP üzerinde güvenilir bir aktarımdır. Paket kaybı yaşanan ve gecikmesi yüksek kanallarda kullanışlıdır, ancak ek servis trafiği üretir. Tüm varsayılan değerler xray-core'un önerileriyle örtüşmektedir.

Alan	Anahtar	Varsayılan	İzin verilen	Açıklama
MTU	<code>mtu</code>	1350	576–1460	Maksimum paket boyutu (bayt). Parçalanma sorunlarında azaltılır
TTI (ms)	<code>tti</code>	20	10–100	İletim aralığı (ms). Küçük değer daha düşük gecikme, ancak daha yüksek ek yük

Alan	Anahtar	Varsayılan	İzin verilen	Açıklama
Uplink (MB/s)	<code>uplinkCapacity</code>	5	≥ 0	Tahmini yükleme bant genişliği (MB/s)
Downlink (MB/s)	<code>downlinkCapacity</code>	20	≥ 0	Tahmini indirme bant genişliği (MB/s)
CWND çarpanı	<code>cwndMultiplier</code>	1	≥ 1	Tıkanıklık penceresi (congestion window) çarpanı
Maks. gönderme penceresi	<code>maxSendingWindow</code>	2097152	≥ 0	Maksimum gönderme penceresi boyutu

Alan notları:

- **Uplink / Downlink capacity** mKCP'nin kanalı ne kadar agresif kullandığını belirler. Gerçek kanal genişliğine göre ayarlanmalıdır: fazla yüksek değerler gereksiz trafiğe, fazla düşük değerler ise kanalın yetersiz kullanılmasına yol açar.
- **TTI** «gecikme [?] ek yük» dengesini doğrudan etkiler: küçük değerler gecikmeyi azaltır, ancak servis paketi hacmini artırır.
- **MTU**, tek bir mKCP paketinin boyutunu sınırlar; büyük UDP paketlerinin kesildiği veya kaybolduğu kanallarda azaltmak faydalıdır.

Bu panel derlemesinde mKCP'nin «seed» alanı (mKCP gizleme parolası) ve **başlık türü/gizleme** açılır listesi (`none` , `srtplib` , `utp` , `wechat-video` , `dtls` , `wireguard`) mKCP alt formunda **ayrı alanlar olarak yer almamaktadır** – aktarım katmanı gizlemesi, `mKCP-legacy` modu da dahil olmak üzere ilgili bölümde açıklanan genel «FinalMask» mekanizmasına taşınmıştır. «congestion» parametresi de ayrı bir onay kutusu olarak sunulmamaktadır; tıkanıklık kontrolü `cwndMultiplier` ve `maxSendingWindow` üzerinden yönetilmektedir.

6.4. WebSocket (`wsSettings`)

HTTP(S) üzerinde WebSocket aktarımı. CDN'ler ve ters proxy'ler üzerinden iyi geçer, sıradan web trafiğine benzer.

Alan	Anahtar	Varsayılan	Açıklama
Proxy Protocol	<code>acceptProxyProtocol</code>	<code>false</code>	Üst akış proxy'den PROXY protokolü başlığını kabul et (bkz. madde 6.2)
Host	<code>host</code>	<code>""</code> (boş)	<code>Host</code> HTTP başlığının değeri. CDN/domain fronting üzerinden çalışırken belirtilir
Yol	<code>path</code>	<code>/</code>	WebSocket el sıkışmasının istek satırındaki yol
Heartbeat periyodu	<code>heartbeatPeriod</code>	0	Heartbeat çerçeveleri gönderme aralığı (saniye). 0 heartbeat'i devre dışı bırakır
Başlıklar	<code>headers</code>	<code>{}</code> (boş)	Ek HTTP el sıkışma başlıkları. «Ad → Değer» düz eşlemesi (dizi değil, yalnızca dize değerleri)

Notlar:

- **Yol**, sunucu (inbound) ve istemci tarafında eşleşmelidir. Bu yol genellikle web sunucusu tarafında giriş noktasını gizlemek için kullanılır.
- **Host**, inbound bir CDN'in arkasında bulunuyorsa veya domain fronting kullanılıyorsa belirtilmelidir; aksi takdirde boş bırakılabilir.
- **Heartbeat periyodu**, etkin olmayan oturumları agresif biçimde kesen proxy/CDN'ler üzerinden bağlantıyı «canlı» tutar. Varsayılan olarak (0) heartbeat kapalıdır.
- RAW'dan farklı olarak WebSocket başlık tablosu «düz» ad → değer biçimini kullanır (başlık başına tek değer satırı).

CDN arkasında WebSocket için örnek streamSettings :

```
{
  "network": "ws",
  "wsSettings": {
    "acceptProxyProtocol": false,
    "host": "cdn.example.com",
    "path": "/ray",
    "heartbeatPeriod": 0,
    "headers": {
      "User-Agent": "Mozilla/5.0"
    }
  }
}
```

host ve path değerleri istemci tarafında da eşleşmelidir; RAW'dan farklı olarak burada başlık değeri dizi değil sıradan bir dizedir.

6.5. gRPC (grpcSettings)

Alan sayısı bakımından en «hafif» aktarım. Trafiki gRPC çağrıları içinde tüneller (HTTP/2 üzerinden); gRPC destekleyen CDN'lerle iyi uyum sağlar. Başlık gizlemesi yoktur.

Alan	Anahtar	Varsayılan	Açıklama
Hizmet adı (Service Name)	serviceName	"" (boş)	gRPC hizmet adı (filen tünelin «yolu»). Sunucu ve istemci tarafında eşleşmelidir
Authority	authority	"" (boş)	:authority sözde başlığının değeri (HTTP/2 için Host karşılığı). CDN/domain üzerinden çalışırken belirtilir
Multi Mode	multiMode	false (kapalı)	Tek bağlantı içinde birden fazla paralel gRPC akışının çoğullanmasını etkinleştirir

Notlar:

- **Service**
Name – gRPC kanalının temel tanımlayıcısıdır; her iki tarafta da aynı olmalıdır. Boş değer geçerlidir, ancak gizleme amacıyla genellikle rastgele olmayan bir dize kullanılır.

- **Authority**, HTTP/2 çerçevelerinde gönderilen `:authority` değerini etkiler; öncelikle CDN üzerinden proxy'leme yapılırken gereklidir.
- **Multi Mode**, birden fazla mantıksal akışın tek bir fiziksel bağlantı üzerinden iletilmesine olanak tanır; hem sunucu hem de istemci destekliorsa performansı artırmak için etkinleştirilir.

gRPC için örnek `streamSettings` :

```
{
  "network": "grpc",
  "grpcSettings": {
    "serviceName": "GunService",
    "authority": "grpc.example.com",
    "multiMode": false
  }
}
```

`serviceName` (burada `GunService`) tünelin «yolu» işlevini görür ve sunucu ile istemci tarafında eşleşmelidir.

6.6. HTTPUpgrade (`httpupgradeSettings`)

HTTP Upgrade mekanizmasına dayalı aktarım (WebSocket gibi, ancak WebSocket protokolü olmadan). Proxy'ler ve CDN'ler üzerinden iyi geçer. Alan kümesi WebSocket ile aynıdır, ancak heartbeat periyodu **yoktur**.

Alan	Anahtar	Varsayılan	Açıklama
Proxy Protocol	<code>acceptProxyProtocol</code>	<code>false</code>	Üst akış proxy'den PROXY protokolü başlığını kabul et
Host	<code>host</code>	<code>""</code> (boş)	<code>Host</code> HTTP başlığının değeri
Yol	<code>path</code>	<code>/</code>	<code>Upgrade</code> başlığıyla HTTP isteğinin yolu
Başlıklar	<code>headers</code>	<code>{}</code> (boş)	Ek HTTP başlıkları. Düz <code>ad</code> → <code>değer</code> eşleşmesi (WebSocket gibi)

Host, **Yol** ve **Başlıklar** alanlarının amacı WebSocket (madde 6.4) ile aynıdır. Heartbeat HTTPUpgrade için öngörülmemiştir – bu WebSocket'e özgü bir özelliktir.

6.7. XHTTP / SplitHTTP (`xhttpSettings`)

XHTTP (diğer adıyla SplitHTTP), xray-core'un modern çoğullamalı HTTP aktarımıdır. Yukarı ve aşağı akışları ayrı HTTP isteklerine böler; bu durum CDN'ler ve bağlantı süresi kısıtlaması olan ortamlar için uygundur. Tüm alanlar aynı anda görünmez: bir kısmı seçilen moda (`mode`) ve etkinleştirilen düğmelere bağlı olarak ortaya çıkar.

Temel alanlar (her zaman görünür)

Alan	Anahtar	Varsayılan	Açıklama
Host	host	"" (boş)	Host HTTP başlığının değeri
Yol	path	/	HTTP isteklerinin temel yolu
Mod (Mode)	mode	auto	İletim modu (aşağıya bakın)
Server Max Header Bytes	serverMaxHeaderBytes	0	Sunucuda istek başlıkları boyutu sınırı (bayt). 0 – xray-core varsayılan değeri
Padding Bytes	xPaddingBytes	100-1000	Boyut analizini zorlaştırmak için rastgele «dolgu» (bayt, min-maks biçimi) aralığı
Başlıklar	headers	{ } (boş)	Ek HTTP başlıkları. Düz ad → değer eşlemesi
HTTP yöntemi Uplink	uplinkHTTPMethod	"" (Varsayılan = POST)	Yukarı yön isteklerinin HTTP yöntemi. Seçenekler: boş (varsayılan POST), POST , PUT , GET (sonuncusu yalnızca packet-up modunda kullanılabilir)
Padding Obfs Mode	xPaddingObfsMode	false (kapalı)	Gelişmiş dolgu gizlemesini etkinleştirir ve ek alanları açar (aşağıya bakın)
No SSE Header	noSSEHeader	false (kapalı)	Content-Type: text/event-stream (SSE) başlığını göndermez. Ara düğümlerden geçişi engelliyorsa etkinleştirilir

«Mod» alanı (mode)

Şu değerleri içeren açılır liste:

Değer	Açıklama
auto	Otomatik mod seçimi (varsayılan)
packet-up	Yukarı yön akışı ayrı HTTP isteklerine bölünür (istek başına bir paket)
stream-up	Yukarı yön akışı tek uzun süreli akış isteğiyle iletilir
stream-one	Tek ortak çift yönlü akış isteği

Mod seçimi hangi ek alanların görüneceğini belirler.

Yalnızca mode = packet-up seçildiğinde görünen alanlar:

Alan	Anahtar	Varsayılan	Açıklama
Maks. arabelleğe alınan yükleme	scMaxBufferedPosts	30	Yukarı yön akışında eş zamanlı arabelleğe alınabilecek maksimum POST isteği sayısı
Maks. yükleme boyutu (bayt)	scMaxEachPostBytes	1000000	Tek bir yukarı yön POST isteğinin maksimum boyutu (bayt)

Alan	Anahtar	Varsayılan	Açıklama
Uplink Data Placement	<code>uplinkDataPlacement</code>	"" (Varsayılan = body)	Yukarı yön verilerinin nereye yerleştirileceği: <code>body</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Uplink Data Key	<code>uplinkDataKey</code>	""	Uplink verileri için anahtar/başlık adı. Yalnızca <code>uplinkDataPlacement</code> belirtilmişse ve <code>body</code> değilse görünür

Yalnızca `mode = stream-up` seçildiğinde görünen alan:

Alan	Anahtar	Varsayılan	Açıklama
Stream-Up Server	<code>scStreamUpServerSecs</code>	20-80	Sunucu tarafı akış bağlantısının tutulma süresi aralığı (saniye, min-maks biçimi)

Dolgu gizleme alanları (`xPaddingObfsMode = açık` olduğunda görünür)

Alan	Anahtar	Varsayılan	Açıklama
Padding Key	<code>xPaddingKey</code>	"" (yer tutucu <code>x_padding</code>)	Dolgu için anahtar adı
Padding Header	<code>xPaddingHeader</code>	"" (yer tutucu <code>X-Padding</code>)	Dolgunun iletildiği HTTP başlığının adı
Padding Placement	<code>xPaddingPlacement</code>	"" (Varsayılan = <code>queryInHeader</code>)	Dolgunun nereye yerleştirileceği: <code>queryInHeader</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Padding Method	<code>xPaddingMethod</code>	"" (Varsayılan = <code>repeat-x</code>)	Dolgu oluşturma yöntemi: <code>repeat-x</code> veya <code>tokenish</code>

Oturum ve sıra yerleştirme (her zaman görünür)

Alan	Anahtar	Varsayılan	Açıklama
Session ID Placement	<code>sessionIDPlacement</code>	"" (Varsayılan = <code>path</code>)	Oturum tanımlayıcısının nerede iletileceği: <code>path</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Session ID Key	<code>sessionIDKey</code>	"" (yer tutucu <code>x_session</code>)	Oturum anahtarı adı. Yalnızca <code>sessionIDPlacement</code> belirtilmişse ve <code>path</code> değilse görünür
Session ID Table	<code>sessionIDTable</code>	"" (yer tutucu <code>Base62</code>)	Oturum tanımlayıcısı oluşturmak için karakter kümesi. Otomatik tamamlamalı açılır listeden önceden tanımlanmış bir değer seçilebilir (<code>ALPHABET</code> , <code>Alphabet</code> , <code>BASE36</code> , <code>Base62</code> , <code>HEX</code> , <code>alphabet</code> , <code>base36</code> , <code>hex</code> , <code>number</code>) ya da keyfi bir ASCII dizisi girilebilir. Boş – <code>xray-core</code> varsayılan değeri
Session ID Length	<code>sessionIDLength</code>	"" (boş)	Oluşturulan tanımlayıcıların uzunluğu veya aralığı (örneğin <code>8-16</code>). Yalnızca <code>Session ID Table</code> belirtildiğinde görünür; minimum değer 0'dan büyük olmalıdır

Alan	Anahtar	Varsayılan	Açıklama
Sequence Placement	seqPlacement	"" (Varsayılan = path)	Paket sıra numarasının nerede iletileceği: path , header , cookie , query
Sequence Key	seqKey	"" (yer tutucu x_seq)	Sıra anahtarı adı. Yalnızca seqPlacement belirtilmişse ve path değilse görünür

Oturum alanları xray-core v26.6.22 sürümüyle yeniden adlandırılmıştır: önceki adları **Session Placement / Session Key** (sessionPlacement / sessionKey) iken artık **Session ID Placement / Session ID Key** (sessionIDPlacement / sessionIDKey) oldu; eski ad çekirdek tarafından artık tanınmamaktadır.

Güncellemeden önce kaydedilmiş inbound'lar yeni anahtarlara otomatik olarak taşınır – yeniden kaydetmeye gerek yoktur.

Öneriler:

- Çoğu kurulum için **Mod = auto** bırakıp **Yol/Host** ayarlamak ve CDN üzerinden çalışırken bunları istemciyle uyumlu hâle getirmek yeterlidir.
- Yerleştirme (*Placement / *Key) ve dolgu gizleme alanları yalnızca belirli anti-DPI/CDN senaryolarına yönelik ince ayar için gereklidir; boş bırakıldığında parantez içinde belirtilen xray-core varsayılan değerleri kullanılır.
- İstemci/outbound tarafına ait parametreler (örneğin tekrarlanan POST aralıkları, öbek boyutları) inbound formunda gösterilmez – sunucu dinleyicisi bunları yok sayar. XMUX çoğullayıcısı ise inbound formunda kullanılabilir (aşağıya bakın).
- **Servis varsayılanları artık yazılmaz.** Panel, XHTTP yapılandırmalarına scMaxEachPostBytes ve scMinPostsIntervalMs servis varsayılanlarını artık yazmamaktadır – xray-core'un iç değerleri uygulanır. Bu, daha önce trafiğin engellenmesine neden olan sabit DPI imzasını (scMinPostsIntervalMs=30 değişmezi) ortadan kaldırır. Önceden kaydedilmiş inbound'larda xray-core varsayılanlarıyla örtüşen değerler bağlantı ve aboneliklerde gösterilmez (inbound'ları yeniden kaydetmeye gerek yoktur); elle girilen değerler korunmaktadır.

XHTTP için örnek streamSettings (auto modu):

```
{
  "network": "xhttp",
  "xhttpSettings": {
    "host": "xhttp.example.com",
    "path": "/yourpath",
    "mode": "auto",
    "xPaddingBytes": "100-1000"
  }
}
```

Çoğu kurulum için bu dört alan yeterlidir; oturum/sıra yerleştirme ve dolgu gizleme alanları boş bırakılır – bu durumda xray-core varsayılan değerleri kullanılır.

XMUX (bağlantı çoğullaması)

XMUX (`enableXmux`) düğmesi, paralel istekleri küçük bir fiziksel bağlantı havuzuna dağıtan çoğullama katmanını etkinleştirir. Etkinleştirildiğinde altı yapılandırılabilir alan açılır (`xhttpSettings.xmux` içinde saklanır):

Alan	Anahtar	Varsayılan	Açıklama
Max Concurrency	<code>maxConcurrency</code>	16–32	Bağlantı başına maksimum eş zamanlı istek sayısı (<code>min</code> – <code>maks</code> aralığı)
Max Connections	<code>maxConnections</code>	6	Maksimum fiziksel bağlantı sayısı (0 – sınırsız). 3.4.2 sürümünden itibaren yeni inbound'lar 6 değeriyle oluşturulur; önceden oluşturulanlar kendi değerini korur.
Max Reuse Times	<code>cMaxReuseTimes</code>	"" (boş)	Bağlantının kaç kez yeniden kullanılacağı
Max Request Times	<code>hMaxRequestTimes</code>	600–900	Bağlantı başına maksimum istek sayısı (aralık)
Max Reusable Secs	<code>hMaxReusableSecs</code>	1800–3000	Bağlantının yeniden kullanıma uygun olduğu süre (saniye, aralık)
Keep Alive Period	<code>hKeepAlivePeriod</code>	"" (boş)	Bağlantıyı canlı tutmak için keep-alive periyodu

Önemli. **Max Connections** ve **Max Concurrency** aynı anda belirtilemez – xray-core böyle bir yapılandırmayı reddeder. Varsayılan olarak XMUX etkinleştirildiğinde panel **Max Concurrency** = 16–32 değerini atar; **Max Connections** (0 'dan büyük bir değer) belirtirseniz çakışmayı önlemek için panel **Max Concurrency** varsayılan değerini kaldırır.

6.8. Hysteria Aktarımı (`hysteriaSettings`)

Hysteria aktarımı «Aktarım» listesinden seçilmez: inbound Hysteria protokolüyle oluşturulduğunda otomatik olarak etkinleşir ve diğer protokollerden gizlenir (Hysteria protokolünden çıkıldığında ağ zorla `tcp` 'ye döner). Parametreler:

Alan	Anahtar	Varsayılan	Açıklama
Sürüm	<code>version</code>	2 (sabit, alan kilitli)	Hysteria sürümü. Yalnızca Hysteria 2 desteklenmektedir
UDP idle timeout (s)	<code>udpIdleTimeout</code>	60	UDP oturumu boşta kalma zaman aşımı (saniye). İzin verilen aralık 2–600; xray-core başlatmada bu aralık dışındaki değerleri reddeder
Masquerade	<code>masquerade</code>	kapalı (yok)	Dinleyiciyi yoklama yapılırken HTTP/3 sunucusu gibi göstermeyi etkinleştirir

Masquerade etkinleştirildiğinde tür (type) seçimi ve buna bağlı alanlar görünür:

- **"" – default (404 page)**: standart 404 sayfası döndürülür (ek alan gerekmez).
- **proxy (reverse proxy)**: harici bir siteye ters proxy.
 - **url (Upstream URL)**, yer tutucu `https://www.example.com`) – hedef adres;
 - **rewriteHost (Host'u yeniden yaz)**, varsayılan `false`) – Host başlığını değiştir;
 - **insecure (TLS doğrulamasını atla)**, varsayılan `false`) – üst akımın TLS sertifikasını doğrulama.
- **file (serve directory)**: dizinden dosya sunma.
 - **dir (Dizin)**, yer tutucu `/var/www/html`).
- **string (fixed body)**: sabit HTTP yanıtı.
 - **statusCode (Durum kodu)**, varsayılan `0` , aralık 0–599);
 - **content (Body)** – yanıt gövdesi;
 - **headers (Başlıklar)** – ad → değer eşlemesi.

Masquerade, Hysteria tabanlı inbound'un aktif yoklamalarda sıradan bir HTTP/3 sunucusu gibi görünmesini sağlar; bu da gizliliği artırır. Varsayılan olarak masquerade kapalıdır.

Ters proxy ile örnek hysteriaSettings (masquerade → proxy):

```
{
  "version": 2,
  "udpIdleTimeout": 60,
  "masquerade": {
    "type": "proxy",
    "url": "https://www.example.com",
    "rewriteHost": true,
    "insecure": false
  }
}
```

Bu durumda yoklama yapıldığında dinleyici `https://www.example.com` adresinden yanıt döndürerek sıradan bir HTTP/3 sitesi gibi davranır.

6.9. İlgili Parametreler

Ağ seçimine ek olarak, aynı sekmede belirli bir aktarımdan bağımsız iki genel blok daha bulunmaktadır (ayrıntılar ilgili bölümlerde):

- **External Proxy (externalProxy)** – panel adresinin yerine abonelik bağlantılarına konulan harici adres/port listesi.
- **Sockopt (sockopt)** – düşük seviyeli soket seçenekleri (TCP Fast Open, mark, etki alanı stratejisi, şeffaf proxy vb.).

Real client IP (CDN/röle arkasındaki gerçek IP tespiti)

Bir inbound aracı (Cloudflare gibi CDN, L4 tüneli/rölesi veya başka bir panel) arkasında bulunduğunda Xray, gerçek ziyaretçi yerine aracının adresini görür. Bu adres çevrimiçi istemci listesine girer ve istemci başına IP sınırı bu adresten hesaplanır; böylece her ikisi de proxy arkasında işe yaramaz hâle gelir. Gerçek IP'yi yeniden

kazanmak için inbound formunun **Sockopt** bölümünde `acceptProxyProtocol` ve `trustedXForwardedFor` ayarlarını bir arada yöneten **Real client IP** ön ayar seçici bulunmaktadır:

Ön ayar	Ne yapar	Ne zaman kullanılır
Off / direct	Her iki alanı da temizler.	inbound'a istemciler doğrudan erişiyor
Cloudflare CDN	<code>sockopt.trustedXForwardedFor = ["CF-Connecting-IP"]</code> ayarlar.	Cloudflare CDN (turuncu bulut) arkasında WebSocket / HTTPUpgrade / XHTTP / gRPC
L4 relay / Spectrum (PROXY)	<code>acceptProxyProtocol = true</code> etkinleştirir.	inbound önünde L4 tüneli/rölesi veya Cloudflare Spectrum

Ön ayarlar birbirini dışlar: birini seçmek diğerinin alanını temizler; bu sayede eski `trustedXForwardedFor`, PROXY protokolü ile elde edilen IP'yi geçersiz kılmaz. Ön ayarın altında ham **Proxy Protocol** düğmesi ve **Trusted X-Forwarded-For** listesi görünür olmaya devam eder – ön ayar bunları sizin yerinize doldurur, gerekirse elle düzenlenir. Seçilen ön ayar mevcut aktarımda desteklenmiyorsa (örneğin mKCP'de PROXY protokolü) form bir uyarı gösterir. Bu alanlar yalnızca sunucu tarafına aittir ve **aboneliklerde asla istemcilere gönderilmez**.

Yalnızca birini kullanın. `acceptProxyProtocol` gerçek IP'yi L4 PROXY protokolü başlığından okurken `trustedXForwardedFor` bunu HTTP istek başlığından okur; bunları manuel olarak karıştırmak yalnızca aracı zinciriniz gerektiriyorsa yapılmalıdır.

- **FinalMask** (`finalmask`) – mKCP eski gizleme de dahil olmak üzere aktarım katmanı gizlemesi için genel mekanizma; ağ alt formlarındaki ayrı «seed»/«header type» alanlarının yerini almaktadır.

7. Bağlantı Güvenliği: TLS, XTLS ve REALITY

Transport akışı üzerinden iletim destekleyen her inbound (VMess, VLESS, Trojan, Shadowsocks, Hysteria), düzenleyicide «**Güvenlik**» sekmesine sahiptir. Bu sekmede, taşıma kanalının nasıl şifreleneceği ve gizleneceği yapılandırılır. Radyo düğmeleriyle geçiş yapılabilen üç mod mevcuttur:

Mod	UI'daki Görüntü	Ne Zaman Kullanılabilir
none	Yok	Her zaman (TLS'nin zorunlu olduğu Hysteria hariç)
tls	TLS	tcp , ws , http , grpc , httpupgrade , xhttp ağlarında VMess/VLESS/Trojan/Shadowsocks için; Hysteria için her zaman
reality	Reality	Yalnızca tcp , http , grpc , xhttp ağlarında VLESS/Trojan için

Protokol Hysteria ise **Yok** düğmesi gösterilmez (bunun için TLS zorunludur). **Reality** düğmesi yalnızca izin verilen protokol ve ağ kombinasyonunda görünür (yukarıdaki tabloya bakın).

Mod değiştirildiğinde panel, `streamSettings` bloğunu tamamen yeniden oluşturur: önceki modun `tlsSettings` ve `realitySettings` değerleri silinir ve seçilen mod için varsayılan değerler yerleştirilir. Özellikle **Reality** seçildiğinde panel otomatik olarak şunları yapar: yerleşik popüler alan adları listesinden rastgele bir `target` + `serverNames` (SNI) çifti ekler, rastgele `shortIds` üretir ve X25519 anahtar çifti (`privateKey/publicKey`) için sunucuya istek gönderir.

7.1. Farklar: TLS vs XTLS vs REALITY

- **TLS** – TLS 1.2/1.3 protokolüne dayalı klasik taşıma şifrelemesi. Sunucuda geçerli bir sertifika bulunmalıdır (kendi alan adı + zincir). Trafik normal HTTPS gibi görünür; ancak aktif bir sansürcü için alan adınıza yapılan TLS el sıkışması tanınabilir niteliktedir. SNI bazlı engelleme veya güvenilir sertifikanın yokluğu durumunda bağlantı engellenir ya da hata verir.
- **XTLS (Vision)** – «Güvenlik» listesinde ayrı bir mod değil, TLS **veya** Reality üzerinde çalışan bir *flow* mekanizmasıdır. inbound istemci tarafında **Flow** = `xtls-rprx-vision` (ya da `xtls-rprx-vision-udp443`) alanı aracılığıyla etkinleştirilir. Vision; `security = tls` ya da `security = reality` ile `tcp` ağında VLESS için kullanılabilir. Ayrıca VLESS şifrelemesi (`vlessenc` / ML-KEM) etkin olduğunda `xhttp` taşıması üzerinde VLESS için de **Flow** alanı `xtls-rprx-vision` olarak ayarlanabilir; bu değer `vless://` bağlantısına doğru şekilde eklenir (`flow=xtls-rprx-vision`). Vision, el sıkışmasının ardından yükü doğrudan ileterek «çift şifrelemeyi» (TLS-in-TLS) azaltır; bu sayede aktarım hızlanır ve gizlenme iyileşir. Bu nedenle **VLESS + Reality + Flow xtls-rprx-vision** kombinasyonu önerilen modern yapılandırma olarak kabul edilir.

Flow Vision'ın Otomatik Olarak Geri Yüklenmesi. Bir VLESS/XHTTP-inbound'a şifreleme (ML-KEM, decryption/encryption alanları) istemciler eklendikten sonra etkinleştirilirse, inbound flow için uygun hale gelir. Bu durumda panel, **Flow** alanı boş bırakılmış olan ancak flow'u olması gereken istemcilerde `flow = xtls-rprx-vision` değerini otomatik olarak geri yükler. Önceden bu senaryoda Vision, yapılandırmalardan, paylaşım bağlantılarından ve aboneliklerden sessizce kayboluyordu (özellikle hub inbound'larında belirgin şekilde fark ediliyordu). Herhangi bir manuel işlem gerekmez: düzeltme, inbound

kaydedildiğinde ve panel güncellenirken bir kez otomatik olarak uygulanır. Davranış tutucudur – panel flow uyduramaz ve istemcinin açıkça belirttiği değerin üzerine yazmaz.

- **REALITY** – kendi sertifikası olmayan bir gizleme mekanizmasıdır. Sunucu, gerçek bir üçüncü taraf sitenin (`target / serverNames`) TLS el sıkışmasını «ödünç alır»; bu nedenle gözlemciye yapılan bağlantı o siteye yapılmış gibi görünür ve sertifikaya gerek kalmaz. Kimlik doğrulama, X25519 anahtar çifti ve `shortIds` kümesi üzerine kuruludur. REALITY, SNI gerçek bir harici alana işaret ettiğinden aktif yoklamalara (`active probing`) ve SNI bazlı engellemeye karşı dayanıklıdır. Bunun bedeli, daha sıkı yapılandırma gereksinimleridir (portlu doğru `target` , anahtarların istemciyle senkronizasyonu).

Kısa seçim kuralı:

- Kendi alan adı ve geçerli sertifikası varsa, basit HTTPS görünümü yeterliyse → **TLS** (mümkünse Vision ile);
- alan adı/sertifika yoksa ya da DPI'dan maksimum gizlilik gerekiyorsa → **REALITY** (VLESS/TCP için Vision ile).

7.2. «Yok» Modu (`none`)

Taşıma, TLS sarmalayıcı olmadan iletilir: `tlsSettings` ve `realitySettings` blokları `streamSettings` 'ten çıkarılır. Bu modun ek alanı yoktur. Şu durumlarda uygundur:

- inbound yalnızca `127.0.0.1` üzerinde dinliyor ve fallback hedefi olarak kullanılıyor (panel kuralına göre fallback için alt inbound, `127.0.0.1` üzerinde `security=none` ile dinlemelidir);
- şifreleme/gizleme harici bir katman tarafından sağlanıyor (örneğin panel önündeki Nginx ters proxy'si);
- iç ağda kendi şifrelemesine sahip bir protokol (Shadowsocks) kullanılıyor.

Dışarıdan erişilebilen inbound'lar için «Yok» modu önerilmez.

Örnek: tcp ağında TLS için `streamSettings` bloğu (VLESS/Trojan/VMess). **TLS** modu seçilip SNI ve sertifika yolları doldurulduktan sonraki görünüm:

```
"streamSettings": {
  "network": "tcp",
  "security": "tls",
  "tlsSettings": {
    "serverName": "vpn.example.com",
    "minVersion": "1.2",
    "maxVersion": "1.3",
    "alpn": ["h2", "http/1.1"],
    "settings": { "fingerprint": "chrome" },
    "certificates": [
      {
        "certificateFile": "/root/cert/vpn.example.com.crt",
        "keyFile": "/root/cert/vpn.example.com.key",
        "ocspStapling": 3600,
        "usage": "encipherment"
      }
    ]
  }
}
```

7.3. TLS Modu

`tlsSettings` bloğunun alanları. Varsayılan değerler panel şemasından alınmıştır.

Temel Parametreler

Alan (Etiket)	Varsayılan Değer	Açıklama
SNI (<code>serverName</code>)	<code>""</code> (boş)	Server Name Indication – TLS el sıkışmasında sunulan alan adı. Sertifikanın alan adıyla eşleşmelidir. İngilizce yer tutucu: «Server Name Indication».
Cipher Suites (<code>cipherSuites</code>)	<code>""</code> → Otomatik	İzin verilen şifre takımlarının listesi. Varsayılan olarak boşdur – seçim Xray/Go'ya bırakılır (Otomatik seçeneği). Yalnızca şifreleri açıkça kısıtlamak gerektiğinde değiştirin.
Min/Maks Sürüm (<code>minMaxVersion</code>)	min = <code>1.2</code> , maks = <code>1.3</code>	Minimum ve maksimum TLS sürümleri. Kullanılabilir değerler: <code>1.0</code> , <code>1.1</code> , <code>1.2</code> , <code>1.3</code> . <code>1.2</code> – <code>1.3</code> olarak bırakmanız önerilir; minimumu 1.0/1.1'e düşürmek önerilmez (eski, güvensiz sürümler).
uTLS (<code>settings.fingerprint</code>)	<code>chrome</code> (formda – None = <code>""</code> seçeneği mevcuttur)	İstemci merhaba mesajının taklit edilen TLS parmak izi (uTLS fingerprint); el sıkışmanın popüler bir tarayıcıya benzetilmesi için. Aşağıdaki listeye bakın. TLS'de listenin ilk maddesi – taklidi devre dışı bırakan None (<code>""</code>)'dur.
ALPN (<code>alpn</code>)	<code>["h2", "http/1.1"]</code>	TLS'de müzakere edilen uygulama katmanı protokollerinin listesi (çoklu seçim). Geçerli değerler: <code>h3</code> , <code>h2</code> , <code>http/1.1</code> . Varsayılan olarak <code>h2</code> ve <code>http/1.1</code> sunulur.

uTLS fingerprint için olası değerler (TLS ve REALITY için aynıdır): `chrome` , `firefox` , `safari` , `ios` , `android` , `edge` , `360` , `qq` , `random` , `randomized` , `randomizednoalpn` , `unsafe` . TLS formunda ek olarak boş **None** seçeneği mevcuttur (parmak izi taklidi uygulanmaz).

Cipher Suites için kullanılabilir değerler (TLS 1.3 ve ECDHE takımları): `TLS_AES_128_GCM_SHA256` , `TLS_AES_256_GCM_SHA384` , `TLS_CHACHA20_POLY1305_SHA256` , `TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA` , `TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA` , `TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA` , `TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA` , `TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256` , `TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384` , `TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256` , `TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384` , `TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256` , `TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256` .

TLS Geçiş Düğmeleri

Geçiş Düğmesi	Varsayılan	Açıklama
Bilinmeyen SNI'yi Reddet (<code>rejectUnknownSni</code>)	kapalı (<code>false</code>)	Etkinleştirildiğinde, istemcinin sunduğu SNI sertifikadaki adla eşleşmiyorsa sunucu bağlantıyı keser. Gizliliği artırır (sunucu «yabancı» isteklere yanıt vermez), ancak istemcide SNI'nin tam eşleşmesini gerektirir.

Geçiş Düğmesi	Varsayılan	Açıklama
System Root'u Devre Dışı Bırak (<code>disableSystemRoot</code>)	kapalı (<code>false</code>)	Güvenilir kök sertifikalar için sistem depolama alanının kullanımını devre dışı bırakır.
Oturum Sürdürme (<code>enableSessionResumption</code>)	kapalı (<code>false</code>)	TLS oturum sürdürmeyi etkinleştirir (session resumption / session tickets).

Ek TLS Parametreleri (vcn, eğriler, anahtar günlüğü, ECH Sockopt)

Temel TLS ayarlarının altında ek alanlar mevcuttur.

Alan (Etiket)	Varsayılan	Açıklama
Verify Peer Cert By Name (<code>settings.verifyPeerCertByName</code>)	""	İstemcinin sunucu sertifikasını SNI yerine belirli adlara göre doğrulaması için kullanılan adlar (virgülle ayrılmış). Bu, Xray'den 2026-06-01 sonrasında kaldırılan <code>allowInsecure</code> alanının modern yerine geçer. Yalnızca panel için geçerlidir: xray sunucu yapılandırmasına yazılmaz, ancak istemcinin bunu uygulayabilmesi için davet bağlantılarına ve aboneliklere (<code>vcn=...</code>) eklenir. Yer tutucu: <code>example.com</code> .
Curve Preferences (<code>curvePreferences</code>)	""	TLS anahtar değişim eğrilerinin tercih sırasına göre kısıtlanması ve sıralanması (örneğin <code>X25519MLKEM768</code> , <code>X25519</code>). Boşsa Xray-core varsayılanları kullanılır.
Master Key Log (<code>masterKeyLog</code>)	""	TLS master key'lerinin <code>SSLKEYLOGFILE</code> formatında kaydedileceği yol (hata ayıklama sırasında Wireshark'ta trafiği çözmek için). Yer tutucu: <code>/path/to/sslkeylog.txt</code> . Prodüksiyonda boş bırakın – bu dosya tüm trafiğin çözülmesine izin verir.
ECH Sockopt (<code>echSockopt</code>)	kapalı	Xray'in ECH config list'i sorguladığı bağlantı için socket parametrelerini içeren geçiş düğmesi. Etkinleştirildiğinde şunlar kullanılabilir: Dialer Proxy (<code>dialerProxy</code> – isteği etikete göre belirli bir outbound üzerinden yönlendir), Domain Strategy (<code>domainStrategy</code>), TCP Fast Open (<code>tcpFastOpen</code>), Multipath TCP (<code>tcpMptcp</code>). Gereksinim yoksa kapalı bırakın.

`verifyPeerCertByName`, `curvePreferences`, `masterKeyLog` ve `echSockopt` alanları `tlsSettings` 'in üst düzeyinde yer alır ve yapılandırma kaydedildiğinde panel alanlarının kırılmasından etkilenmez.

Sertifikalar

SSL Sertifikası bölümü (UI'da «SSL Sertifikası» başlığıyla) liste şeklinde sunulur: **+** düğmesiyle yeni sertifika kaydı eklenir, **- Sil** düğmesiyle kaldırılır (silme düğmesi yalnızca birden fazla kayıt olduğunda etkin olur). TLS etkinleştirildiğinde varsayılan olarak bir boş kayıt oluşturulur.

Her kayıt için giriş modu geçiş düğmesi (`useFile`):

- **Sertifika Yolu** (`useFile = true` , varsayılan) – sunucudaki dosya yolları belirtilir:
 - **Genel Anahtar** (`certificateFile`) – sertifika dosyasının yolu (`.crt` / `.pem`);
 - **Özel Anahtar** (`keyFile`) – özel anahtar dosyasının yolu (`.key`).
- **Sertifika İçeriği** (`useFile = false`) – içerik doğrudan alanlara yapıştırılır (çok satırlı metin alanları):
 - **Genel Anahtar** (`certificate`) – sertifikanın PEM içeriği;
 - **Özel Anahtar** (`key`) – anahtarın PEM içeriği.

«Sertifika Yolu» modunun alanlarının altında iki düğme mevcuttur:

- **Panel Sertifikasını Ayarla** – alanlara panelin kendi SSL sertifikasının yollarını ekler. Merkezi paneldeki inbound için panelin sertifikası (`POST /panel/setting/all` → `webCertFile` / `webKeyFile`) alınır; bir node'a atanmış inbound için ise node'un kendi sertifikası (`GET /panel/api/nodes/webCert/{nodeId}`) kullanılır; çünkü merkezi panelin yolları node'da mevcut değildir. Sertifika yapılandırılmamışsa uyarı görüntülenir: «Panel için sertifika yapılandırılmamış. Önce Ayarlar'dan bir tane kurun.» (panelin kendi sertifikası «Ayarlar → Güvenlik» bölümünde yapılandırılır).
- **Temizle** – her iki yolu da siler.

Her sertifika kaydının ek alanları:

Alan	Varsayılan	Açıklama
OCSF Stapling (<code>ocspStapling</code>)	<code>0</code> (kapalı)	OCSF stapling yenileme aralığı (saniye cinsinden, minimum <code>0</code>). Yeni inbound'larda varsayılan olarak kapalıdır (<code>0</code>); bu, OCSF'den vazgeçen Let's Encrypt gibi sertifikalar için xray loglarındaki hataları önler. Yalnızca stapling'i destekleyen sertifikalar için etkinleştirin.
Tek Seferlik Yükleme (<code>oneTimeLoading</code>)	kapalı (<code>false</code>)	Etkinleştirildiğinde, sertifika diskten yalnızca başlangıçta okunur ve dosya değiştiğinde yeniden okunmaz.
Kullanım Seçeneği (<code>usage</code>)	<code>encipherment</code>	Sertifikanın kullanım amacı. İzin verilen değerler: <code>encipherment</code> (şifreleme – normal sunucu sertifikası), <code>verify</code> (doğrulama), <code>issue</code> (yayınlama – sunucu sertifikaları imzalar/yayınlar).
Build Chain (<code>buildChain</code>)	kapalı (<code>false</code>)	Yalnızca <code>usage = issue</code> olduğunda görünür. Sertifika zincirini oluşturur.

inbound düzenleyicisinde ayrı bir kendinden imzalı sertifika düğmesi yoktur: panel, inbound için anında kendinden imzalı sertifika oluşturamaz. Sertifika ya yol/içerik olarak belirtilir ya da «Panel Sertifikasını Ayarla» düğmesiyle panel ayarlarından getirilir. Panelin kendi SSL sertifikasının alınması/oluşturulması (dosya yükleme ve alan adına bağlama dahil) **Ayarlar** → **Güvenlik** bölümünde gerçekleştirilir; burada inbound'lar için ACME/Let's Encrypt uç noktası bulunmamaktadır.

ECH ve Sertifika Sabitleme (Gelişmiş TLS Alanları)

Alan	Varsayılan	Açıklama
ECH key (<code>echServerKeys</code>)	""	Encrypted Client Hello için sunucu anahtarları.

Alan	Varsayılan	Açıklama
ECH config (settings.echConfigList)	""	ECH config list (istemci tarafı, bağlantıya eklenir).
Eş sertifikasının SHA-256'sı (settings.pinnedPeerCertSha256)	[]	Eş sertifikasının SHA-256 özet değerleri (hex dizeler, virgülle ayrılmış). Kelimesi kelimesine ipucu: «Eş sertifikasının SHA-256 özetleri altıgenlik dize olarak (örn. e8e2d3...), virgülle ayrılmış. Yalnızca panel için – xray sunucu yapılandırmasına yazılmaz, ancak istemcilerin sertifikayı sabitleyebilmesi için davet bağlantılarına dahil edilir.»

Düğmeler: **Eş sertifikasının SHA-256'sı** alanının yanında iki otomatik doldurma düğmesi mevcuttur:

- **Fill from this inbound's certificate** (kalkan simgesi) – bu inbound'un kendi sertifikasının SHA-256 özetini ekler (getCertHash uç noktası üzerinden yerel olarak alınır).
- **Fetch the hash by pinging the SNI (xray tls ping)** (indirme simgesi) – belirtilen SNI'ye TLS bağlantısı kurarak canlı sunucunun sertifika özetini alır (sunucuda getRemoteCertHash çağrılır). **SNI** (serverName) alanı dolu olmalıdır – aksi hâlde «Set the SNI (serverName) first to ping the remote certificate.» ipucu görüntülenir.

Alınan özetler alana eklenir (virgülle ayrılarak) ve istemcinin sertifikayı sabitleyebilmesi için davet bağlantılarına dahil edilir.

- **Yeni ECH Sertifikası Al** – geçerli SNI için sunucudan yeni bir ECH çifti ister (POST /panel/api/server/getNewEchCert ,sunucuda xray tls ech --serverName <SNI> çalıştırılır); **ECH key** ve **ECH config** alanlarını doldurur.
- **Temizle** – her iki ECH alanını da sıfırlar.

7.4. REALITY Modu

realitySettings bloğunun alanları. REALITY, SSL sertifikası kullanmaz: bunun yerine harici bir alan adından ödünç alınan TLS el sıkışması ve X25519 anahtar çifti kullanılır.

Gizleme Parametreleri

Alan (Etiket)	Varsayılan Değer	Açıklama
Göster (show)	kapalı (false)	REALITY'nin Xray loglarına hata ayıklama çıktısı. Genellikle kapalı bırakılır.
Xver (xver)	0	Arka uca iletilen PROXY protokolü sürümü (0 – kapalı). Minimum 0 .
uTLS (settings.fingerprint)	chrome	Taklit edilen TLS parmak izi (TLS ile aynı liste, ancak boş None seçeneği yoktur).

Alan (Etiket)	Varsayılan Değer	Açıklama
Hedef (target)	"" (3.4.2'den itibaren REALITY etkinleştirildiğinde boş kalır)	Zorunlu alan. REALITY'nin TLS el sıkışmasını ödünç aldığı gerçek alan adı. Kelimesi kelimesine ipucu: «Zorunlu. Port içermelidir (örneğin example.com:443). Port belirtilmezse Xray-core başlamaz.» Panel doğrulaması, portun varlığını ve geçerliliğini kontrol eder; aksi hâlde «REALITY Hedefi zorunludur» / «REALITY Hedefi port içermelidir...» / «REALITY Hedefinde geçersiz port belirtilmiş» hataları görüntülenir. Alanın yanında «Tara» (mevcut hedefi «canlı» kontrol et) ve «Hedef bul» (REALITY hedef tarayıcısını aç) düğmeleri bulunur; aşağıya bakın.
SNI (serverNames)	[] (hedefle birlikte eklenir)	İzin verilen SNI'lerin listesi (etiket olarak çoklu giriş). Hedef alanındaki alan adıyla eşleşmelidir. Hedef başarıyla tarandığında SNI, onun sertifikasına göre doldurulur.
Maks. Zaman Farkı (ms) (maxTimediff)	0	İstemci ve sunucu saatleri arasındaki maksimum izin verilen sapma (milisaniye, 0 – sınırsız). Minimum 0 .
Min. İstemci Sürümü (minClientVer)	""	Minimum Xray istemci sürümü (3.5.0'dan itibaren yer tutucu – 26.3.27). Boşsa sınırsız.
Maks. İstemci Sürümü (maxClientVer)	""	Maksimum Xray istemci sürümü. Boşsa sınırsız.
Short IDs (shortIds)	[] (etkinleştirildiğinde üretilir)	İstemcileri ayırt eden kısa tanımlayıcıların (hex) listesi. Etiket olarak çoklu giriş; güncelleme düğmesi rastgele bir küme üretir.
SpiderX (settings.spiderX)	/	Dış siteye erişimi taklit ederken kullanılan «örümcek» yolu (REALITY'nin istemci tarafı). Davet bağlantısına eklenir.

3.4.2 sürümünden itibaren **Hedef (target)** ve **SNI (serverNames)**, REALITY etkinleştirildiğinde artık otomatik olarak doldurulmaz – her iki alan da boş kalır ve hedef, canlı tarayıcıyla seçilir (aşağıya bakın). TLS 1.3 ve HTTP/2 desteği olan, kendi sunucunuzun arkasında bulunmayan büyük, kararlı bir üçüncü taraf HTTPS sitesi seçin.

REALITY hedefi bulma (canlı tarayıcı)

3.4.2 sürümünden itibaren eski «rastgele hedef» düğmesi (statik yerleşik listeyle) **Hedef** alanının yanındaki iki eylemle değiştirildi:

- «Tara» – mevcut hedefi «canlı» kontrol eder: bir TLS bağlantısı kurar ve hedef uygunsa onun sertifikasından SNI'yi ekler. Bildirimler: «Hedef uygun – hedef ve SNI dolduruldu.» / «Hedef erişilebilir ancak REALITY için uygun değil.» / «REALITY hedefi taranamadı.».
- «Hedef bul» – «REALITY hedef tarayıcısı» penceresini açar: bir alan adı, **IP veya CIDR aralığı** belirtebilir (bu durumda panel, canlı hostların sertifikalarına göre hedefleri bulur) ya da alanı boş bırakabilirsiniz (yerleşik adaylar kontrol edilir). Sonuçlar sıralanır; sütunlar – «Durum»/«Uygun», «Anahtar değişimi», «Sertifika», TLS , ALPN , «Gecikme». «Kullan» düğmesiyle seçilen satır inbound alanlarına eklenir.

Bir hedef, sunucu **TLS 1.3, HTTP/2 (ALPN h2), X25519/X25519MLKEM768** anahtar değişimi üzerinde anlaşır ve **güvenilir** (joker olmayan) bir sertifika sunarsa **uygun** sayılır. Tarama, panel sunucusu tarafında yürütülür (`POST /panel/api/server/scanRealityTarget` ve `.../scanRealityTargets`).

Örnek: **tcp** ağında **REALITY** için **streamSettings** bloğu (VLESS). Sertifikaya gerek yoktur – bunun yerine ödünç alınan alan adı ve X25519 anahtar çifti kullanılır:

```
"streamSettings": {
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "show": false,
    "xver": 0,
    "dest": "www.nvidia.com:443",
    "serverNames": ["www.nvidia.com"],
    "privateKey": "YOUR_X25519_PRIVATE_KEY",
    "shortIds": ["", "6ba85179e30d4fc2"],
    "settings": {
      "publicKey": "YOUR_X25519_PUBLIC_KEY",
      "fingerprint": "chrome",
      "spiderX": "/"
    }
  }
}
```

Burada panelin **Hedef** (`target`) alanı, nihai Xray yapılandırmasındaki `dest` alanına karşılık gelir. REALITY-inbound, `dest` anahtarıyla (panelin eski sürümleri, API veya harici araçlar aracılığıyla) oluşturulduysa panel ayarıştırma sırasında `target` boşken `dest` → `target` dönüşümünü normalleştirir; böylece bu tür inbound doğru şekilde yüklenir, **Hedef** alanı boş kalmaz ve yeniden kaydetme çalışan REALITY'yi bozmaz.

REALITY Anahtarları (X25519)

Alan	Varsayılan	Açıklama
Genel Anahtar (<code>settings.publicKey</code>)	""	X25519 genel anahtarı (istemci bunu kendi yapılandırmasına/bağlantısına ekler).
Özel Anahtar (<code>privateKey</code>)	""	X25519 özel anahtarı (yalnızca sunucuda saklanır).

Anahtarların altındaki düğmeler:

- **Yeni Sertifika Al** – sunucudan yeni anahtar çifti ister (`GET /panel/api/server/getNewX25519Cert` ; sunucuda `xray x25519` çalıştırılır), **Özel Anahtar** ve **Genel Anahtar** alanlarını doldurur. REALITY modu ilk kez etkinleştirildiğinde bu çift otomatik olarak üretilir.

Örnek: **API aracılığıyla X25519 anahtar çifti alma** (form dışında, örneğin bir betik için). İstek, özel ve genel anahtarları döndürür:

```
curl -s -b cookie.txt https://your-panel:2053/panel/api/server/getNewX25519Cert
# Yanıt:
# {"success":true,"obj":{"privateKey":"...","publicKey":"..."}}
```

cookie.txt – POST /login aracılığıyla giriş yapıldıktan sonra alınan oturum cookie dosyası.

- **Temizle** – her iki anahtar da sıfırlar.

Kuantum Sonrası İmza ML-DSA-65 (mldsa65)

REALITY için ek (isteğe bağlı) kuantum sonrası kimlik doğrulama katmanı:

Alan	Varsayılan	Açıklama
mldsa65 Seed (mldsa65Seed)	""	ML-DSA-65 anahtar seed değeri (sunucu tarafı).
mldsa65 Verify (settings.mldsa65Verify)	""	Doğrulama değeri (istemci tarafı, bağlantıya eklenir).

Düğmeler:

- **Yeni Seed Al** – yeni bir çift ister (GET /panel/api/server/getNewmldsa65 ; sunucuda xray mldsa65 çalıştırılır), **mldsa65 Seed** ve **mldsa65 Verify** alanlarını doldurur.
- **Temizle** – her iki alanı da sıfırlar.

Fallback Hız Sınırı ve REALITY Anahtar Günlüğü

REALITY ayarlarında fallback trafiği için hız sınırı mevcuttur – bu, aktif yoklamacıların sunucuyu ödünç alınan alana ücretsiz bir kanal olarak kullanmasını engeller. Ayar iki yön için ayrı ayrı yapılandırılır: **Limit Fallback Upload** ve **Limit Fallback Download** (limitFallbackUpload / limitFallbackDownload); her biri aynı alan kümesine sahiptir:

Alan (Etiket)	Varsayılan	Açıklama
After Bytes (afterBytes)	0	Sınırlama başlamadan önce tam hızda geçirilecek bayt miktarı. 0 – ilk bayttan itibaren sınırla.
Bytes Per Sec (bytesPerSec)	0	Eşik sonrasında fallback trafiği için saniyedeki hız üst sınırı (bayt). 0 – sınırsız (bu yönü devre dışı bırakır).
Burst Bytes Per Sec (burstBytesPerSec)	0	Sabit hızın üzerinde kısa süreli ani artışlar için token-bucket boyutu. Bytes Per Sec değerinden küçükse o değere yükseltilir.

Aynı yerde **Master Key Log** (masterKeyLog) alanı da mevcuttur – Wireshark'ta hata ayıklama için SSLKEYLOGFILE formatında TLS master key'lerinin kaydedileceği yol; prodüksiyonda boş bırakın.

7.5. Yapılandırma İçin Pratik Öneriler

1. **VLESS + Reality (önerilir):** tcp ağında bir VLESS-inbound oluşturun, «Güvenlik» sekmesinde **Reality** seçin – panel rastgele target /SNI, shortIds oluşturacak ve X25519 anahtarlarını üretecektir. Gerekirse kendi anahtar çiftiniz için «Yeni Sertifika Al» düğmesine basın. VLESS istemcileri için **Flow** = xtls-rprx-vision (XTLS Vision) etkinleştirin – bu maksimum performans ve gizlilik sağlar.

Örnek: VLESS + Reality + Vision için nihai istemci bağlantısı. Panelin bu tür bir inbound için oluşturduğu davet bağlantısının görünümü (anahtar/ID değerleri örnektir):

```
vless://uuid-клиента@1.2.3.4:443?
type=tcp&security=reality&pbk=ПУБЛИЧНЫЙ_КЛЮЧ&fp=chrome&sni=www.nvidia.com&sid=6ba85179e3
0d4fc2&spx=%2F&flow=xtls-rprx-vision#my-reality
```

Burada **pbk** – X25519 genel anahtarı, **sni** – **Hedef** alanından ödünç alınan alan adı, **sid** – **Short IDs**'den biri, **flow=xtls-rprx-vision** – etkin XTLS Vision. 2. **Kendi alan adıyla TLS: TLS** seçin, **SNI** alanına alan adını girin, sertifikayı ekleyin (dosya yolları veya içerik olarak) ya da alan adı ve sertifika «Ayarlar → Güvenlik» bölümünde zaten yapılandırılmışsa «Panel Sertifikasını Ayarla» düğmesine basın. Normal bir tarayıcıyı taklit etmek için **Min/Maks Sürüm** = 1.2 – 1.3 ve **uTLS** = chrome olarak bırakın. 3. Dışarıya açık inbound'lar için **Yok** modunu bırakmayın – yalnızca yerel fallback hedefleri (127.0.0.1) veya TLS'nin harici bir proxy tarafından sağlandığı durumlarda kullanın. 4. Arayüzden ipucu: gelişmiş alanların çoğunda «Varsayılan ayarların olduğu gibi bırakılması önerilir» mesajı görünür – sonuçları anlamadan değiştirmeyin.

8. İstemciler

İstemci, bir VPN kullanıcı hesabıdır: bir veya birden fazla inbound'a bağlı, kendi trafik kotası, geçerlilik süresi ve eş zamanlı bağlantı sınırı olan bir kimlik bilgisi kümesi (UUID veya parola). Bu fork'ta istemci bağımsız bir varlıktır (`clients` tablosu): aynı istemci, ortak UUID/parola ve ortak trafik sayacını koruyarak aynı anda birden fazla inbound'a bağlanabilir. **İstemciler** bölümü, paneldeki tüm kullanıcı hesaplarını inbound'dan bağımsız olarak; arama, filtreler, sıralama ve toplu işlemlerle birlikte gösterir.

8.1. İstemci Alanları

Aşağıda **İstemci Ekle** / **İstemciyi Düzenle** düzenleyicisinin her alanı açıklanmaktadır.

İstemci formu iki sekmeye bölünmüştür: **Temel** (email, inbound bağlantısı, limitler, süre, grup, yorum, ters etiket) ve **Kimlik Bilgileri** (UUID/parola/auth, Flow, VMess Security). Alan etiketlerinde kota **Trafik Limiti (GB)** olarak, süreler ise **Süre (gün)** ve **Otomatik Yenileme (gün)** olarak belirtilmiştir; **Trafik Limiti (GB)** ve **IP Limiti** alanlarında 0 değerinin "sınırsız" anlamına geldiğini açıklayan ipuçları bulunmaktadır. Mevcut bir istemci düzenlenirken rastgele email oluşturma düğmesi gizlenir; IP günlüğü düğmesi ise doğrudan **IP Limiti** alanının yanına taşınır ve kayıtlı adres sayısını gösterir.

Alan	JSON Anahtarı	Varsayılan	Açıklama
Email	<code>email</code>	– (zorunlu)	İstemcinin benzersiz tanımlayıcısı
UUID	<code>id</code>	otomatik oluşturulur	VMess/VLESS için tanımlayıcı
Parola	<code>password</code>	otomatik oluşturulur	Trojan/Shadowsocks için parola
Yetkilendirme	<code>auth</code>	otomatik oluşturulur	Hysteria için parola
Flow	<code>flow</code>	boş	Flow control (XTLS), yalnızca VLESS
VMess Security	<code>security</code>	<code>auto</code>	VMess şifreleme yöntemi
IP Limiti	<code>limitIp</code>	0 (limitsiz)	Eş zamanlı maksimum IP sayısı
Toplam Gönderildi/Alındı (GB)	<code>totalGB</code>	0 (limitsiz)	Trafik kotası
Geçerlilik Süresi	<code>expiryTime</code>	0 (süresiz)	Sona erme tarihi
Otomatik Yenileme	<code>reset</code>	0 (kapalı)	Trafik sıfırlama periyodu, gün
Telegram Kullanıcı ID	<code>tgId</code>	0 (yok)	Sayısal Telegram ID
Abonelik ID	<code>subId</code>	otomatik oluşturulur	Abonelik tanımlayıcısı
Grup	<code>group</code>	boş	Mantıksal gruplama etiketi
Yorum	<code>comment</code>	boş	İsteğe bağlı not
Etkin	<code>enable</code>	<code>true</code>	Hesabın aktif olup olmadığı

Email (Tanımlayıcı)

Email alanı, istemcinin temel ve zorunlu tanımlayıcısıdır. Adına rağmen mutlaka bir e-posta adresi olmak zorunda değildir: herhangi bir metin etiketi (kullanıcı adı, numara) geçerlidir. Değer panel genelinde **benzersiz** olmalıdır; kullanımda olan bir email ile ikinci bir istemci oluşturma girişimi (`email already in use`) reddedilir; ancak `subId` de aynıysa bu durum aynı istemcinin bağlanması olarak değerlendirilir.

Email **boş bırakılamaz** (`client email is required`) ve **boşluk**, `/`, `\` veya **kontrol karakterleri içeremez** ("Email boşluk, '/', '\ veya kontrol karakterleri içeremez"). Email; trafik istatistiklerinde, IP günlüğünde, çevrimiçi listesinde ve işlem adlarında kullanılır – sonradan değiştirilmesi önerilmez.

UUID / Parola / Yetkilendirme (Kimlik Bilgileri)

Belirli kimlik bilgisi alanı, istemcinin bağlandığı inbound protokolüne bağlıdır. Alanlar boş bırakıldığında değerler otomatik atanır:

- **UUID** (`id` alanı) – **VMess** ve **VLESS** protokolleri için. Belirtilmezse rastgele UUID v4 oluşturulur.
- **Parola** (`password` alanı) – **Trojan** ve **Shadowsocks** için. Trojan için varsayılan olarak tireler olmadan UUID oluşturulur. Shadowsocks için inbound şifreleme yöntemine bağlı olarak Base64 biçiminde gerekli uzunlukta bir anahtar oluşturulur: `2022-blake3-aes-128-gcm` için 16 bayt, `2022-blake3-aes-256-gcm` ve `2022-blake3-chacha20-poly1305` için 32 bayt; diğer yöntemler için tireler olmadan UUID. Manuel olarak girilen anahtar `2022-blake3` yöntemine uymuyorsa, oluşturulan bir anahtarla değiştirilir.
- **Yetkilendirme** (`auth` alanı) – **Hysteria** için parola. Varsayılan olarak tireler olmadan UUID.

Bir istemci farklı protokollere sahip birden fazla inbound'a bağlanabileceğinden, istemci kaydında aynı anda UUID, parola ve auth bulunabilir – her protokol kendi alanını kullanır.

Örnek: istemci kimlik bilgilerinin farklı inbound'lardaki `settings` içinde görünümü. Aynı istemci VLESS inbound'da `id` ile, Trojan'da `password` ile, Shadowsocks'ta `password` (Base64 anahtarı) ile tanımlanır:

```
// VLESS inbound'un settings.clients bölümünden bir parça
{ "id": "b831381d-6324-4d53-ad4f-8cda48b30811", "email": "user-a", "flow": "xtls-rprx-vision" }

// aynı istemci Trojan inbound'da
{ "password": "b831381d63244d53ad4f8cda48b30811", "email": "user-a" }

// aynı istemci Shadowsocks inbound'da (yöntem: 2022-blake3-aes-256-gcm)
{ "password": "GPY0aA3f7C00az53eaQ8eqMfRDjmBl0h7v1u3+Z+pHk=", "email": "user-a" }
```

Flow

Flow (`flow` alanı) – XTLS akış kontrolü. Yalnızca **VLESS** için ve inbound'un XTLS Vision için yapılandırıldığı durumlarda geçerlidir: security olarak **tls** veya **reality** ayarlı **TCP** taşıması üzerinde VLESS. Geçerli değer `xtls-rprx-vision` 'dır (ayrıca tarihsel `xtls-rprx-vision-udp443`); boş değer flow olmadığı anlamına gelir.

inbound XTLS flow'u desteklemiyorsa, ayarlanan flow istemci kaydedilirken **sessizce sıfırlanır**: birden fazla inbound'a bağlı aynı istemci için flow yalnızca uygun olduğu yerlerde uygulanır. Yalnızca VLESS-Vision'ı kasıtlı olarak kullanıyorsanız değiştirin.

VMess Security

VMess Security (`security` alanı) – VMess için yük şifreleme yöntemi. Varsayılan değer `auto` 'dur (Xray şifreyi kendisi seçer). Geçerli değerler VMess için standart olanlardır: `auto` , `aes-128-gcm` , `chacha20-poly1305` , `none` , `zero` . Diğer protokoller için bu alan kullanılmaz.

IP Limiti (Eş Zamanlı Bağlantılar)

IP Limiti (`limitIp` alanı) – istemcinin aynı anda bağlanabileceği maksimum **farklı IP adresi** sayısı. Varsayılan değer `0` 'dır, bu da **sınır olmadığı** anlamına gelir. Pozitif bir değer belirlendiğinde panel, istemcinin aktif IP'lerini izler ve limit aşıldığında hesabı arka plan görevi aracılığıyla devre dışı bırakır. (**3.3.1** sürümünden itibaren IP sayımı, Xray çekirdeğinin online-stats API'si üzerinden yapılır ve erişim günlüğü **gerektirmez**; daha eski çekirdek sürümlerinde panel, etkinleştirilmiş olması gereken erişim günlüğünü okumaya geri döner.) Tek bir aboneliğin çok sayıda cihazla paylaşılmasını engellemek için kullanın: örneğin, `2` – iki cihaza izin verir.

IP Limiti **Fail2ban** aracılığıyla uygulanır; bu nedenle **IP Limiti** alanı yalnızca Fail2ban kurulu ve çalışır durumda olduğunda aktiftir (panel, durumu `GET /panel/api/server/fail2banStatus` üzerinden kontrol eder). Fail2ban kurulu değilse, istemci düzenleyicisinin (ve toplu ekleme formunun) alanı devre dışı bırakılır; üzerine gelindiğinde Fail2ban'ı x-ui bash menüsünden kurma önerisini içeren bir ipucu gösterilir ("Fail2ban is not installed, so the IP limit cannot be enforced. Install Fail2ban from the x-ui bash menu to enable this option."); Windows'ta Fail2ban'ın kullanılamaz olduğu belirtilir ("Fail2ban is not available on Windows, so the IP limit cannot be enforced."), sunucuda özellik devre dışıysa ise "The IP limit feature is disabled on this server." mesajı görünür. Panel güncellendiğinde, Fail2ban olmayan sunuculardaki istemcilerin kaydedilmiş IP limiti, zaten uygulanamadığı için tek seferlik bir taşıma ile sıfırlanır.

Örnek değerler. `limitIp: 0` – sınır yok; `limitIp: 1` – aynı anda yalnızca bir cihaz; `limitIp: 3` – üç farklı IP'ye kadar. Dördüncü aktif IP'de arka plan görevi **IP Limitini Sıfırla** işlemini gerçekleştirene kadar istemciyi devre dışı bırakır (`enable = false`).

İlgili işlemler: **IP Günlüğü**, istemcinin kayıtlı IP'lerinin listesini gösterir; her kayıt IP'nin yanı sıra son erişim zamanını ve bağlantının kaydedildiği düğüm etiketini (`@ düğüm_adı`) içerir – çok panelli bir yapılandırmada istemcinin hangi düğüm üzerinden bağlandığı görülür. **IP Limitini Sıfırla**, birikmiş IP günlüğünü temizleyerek istemcinin kayıtların doğal süresinin dolmasını beklemeden yeniden bağlanabilmesini sağlar.

Toplam Gönderildi/Alındı (GB) – Trafik Kotası

Toplam Gönderildi/Alındı (GB) (`totalGB` alanı) – toplam trafik kotası (gönderme + alma). Varsayılan değer `0` 'dır – **limitsiz** anlamına gelir. Kotaya ulaşıldığında (`up + down >= total`) istemci **tükenmiş** (depleted) sayılır ve devre dışı bırakılır. Arayüzde genellikle gigabayt cinsinden girilir; veritabanında bayt olarak saklanır.

İstemci listesinde **Trafik** sütunu, renkli bir kullanım çubuğu gösterir: tüketilen trafik miktarı, limit etiketi (limitsizse ∞ işareti) ve üzerine gelindiğinde gönderilen/alınan trafik ile kalan miktarı ayrıntılı olarak gösteren bir ipucu. Aynı kompakt gösterge, telefondaki istemci kartlarında da görüntülenir.

Geçerlilik Süresi (Expiry)

Geçerlilik Süresi (`expiryTime` alanı), hesabın sona erme anını belirler. Alan üç moda sahiptir:

- **Süresiz** – `0`. İstemci zaman açısından hiçbir zaman sona ermez.
- **Belirli bir tarih** – pozitif Unix zaman damgası (milisaniye cinsinden). Ulaşıldığında (`expiryTime <= şimdi`) istemci süresi dolmuş (expired) sayılır ve devre dışı bırakılır. Arayüzde genellikle tarih seçimi veya gün cinsinden süre (**Süre**, birim – **Gün**) ile belirlenir.
- **İlk kullanımdan sonra başla** – süreyi kodlayan **negatif** değer. İstemci hiç bayt aktarımı yapmadığı sürece süre negatif kalır ("ertelenmiş başlangıç"). İlk trafik istatistiği tiki ile panel bunu mutlak bir tarihe dönüştürür: `şimdi + |süre|`. Bu, müşterinin ne zaman etkinleşeceğini önceden bilmeden "ilk bağlantıdan itibaren 30 gün" gibi bir abonelik satmayı mümkün kılar. Dönüşüm, bağlı tüm inbound'ların aynı süreyi almasını sağlamak amacıyla email başına bir kez gerçekleştirilir.

Süre kodlama örneği. Sabit tarih: 1 Mart 2026, 00:00 UTC → `expiryTime: 1772323200000` (milisaniye cinsinden pozitif zaman damgası). "İlk bağlantıdan itibaren 30 gün" → `expiryTime: -2592000000` (negatif değer, $30 \times 24 \times 60 \times 60 \times 1000$); ilk trafik baytında panel bunu `şimdi + 2592000000` ile değiştirir. Süresiz → `expiryTime: 0`.

Otomatik Yenileme (İstemci Trafik Sıfırlama Periyodu)

Otomatik Yenileme (`reset` alanı) – günlük otomatik yenileme/sıfırlama periyodudur. İpucu: "Sona ermeden sonra otomatik yenileme. (0 = devre dışı) (birim: gün)".

- `0` – otomatik yenileme **devre dışı** (varsayılan değer). Süre dolduğunda istemci yalnızca tükenmiş hale gelir.
- `> 0` – arka plan görevi süre dolduğunda trafik sayaçlarını **sıfırlar** (`up = down = 0`), **geçerlilik süresini** `reset` gün ileri kaydırır (gerekirse yeni süre gelecekte olana kadar birden fazla periyot boyunca) ve gerekirse istemciyi yeniden **etkinleştirir**. Bu, periyodik aboneliği (örneğin aylık) gerçekleştirir. Otomatik yenileme **düğüm sunucularındaki inbound'lara** (`node_id IS NOT NULL`) uygulanmaz.

Önemli bir sonuç: `reset > 0` olan istemciler, toplu silme işlemlerindeki "tükenmiş" kavramından **çıkarılır** – trafik/süreleri beklendiği üzere otomatik yenileme ile sıfırlanır ve hesabı silme adayı yapmaz.

Telegram Kullanıcı ID

Telegram Kullanıcı ID (`tgId` alanı) – panelin yerleşik Telegram botuna (bildirimler, kendi istatistiklerini görüntüleme) bağlanmak için kullanıcının sayısal Telegram tanımlayıcısı. İpucu: "Sayısal Telegram kullanıcı ID'si (0 = yok)". Varsayılan değer `0` – bağlantı yok. Bu alana göre filtreleme yapılabilir (**Var / Yok**).

Abonelik ID (subId)

Abonelik ID (`subId` alanı) – istemcinin **aboneliğe** (subscription) dahil edildiği tanımlayıcı. Aynı `subId` 'ye sahip tüm istemciler tek bir abonelik bağlantısı üzerinden sunulur. Oluşturma sırasında alan boş bırakılırsa panel otomatik olarak rastgele bir `subId` (UUID) **oluşturur**. Değer, farklı email'e sahip istemciler arasında **benzersiz** olmalıdır (`subId already in use`) ve email ile aynı karakter kısıtlamalarına tabidir ("Abonelik ID'si boşluk, '/', '' veya kontrol karakterleri içeremez").

`subId` olmadan istemci için abonelik bağlantısına erişilemez ("Bu istemcinin subId'si yok, paylaşım bağlantısı kullanılamaz.").

Links Sekmesi (Harici Bağlantılar ve Abonelikler)

Temel ve **Kimlik Bilgileri** sekmelerinin yanı sıra, istemci düzenleyicisinde üçüncü bir **Links** sekmesi bulunur (ipucu: "Add third-party share links and remote subscription URLs to include in this client's subscription."). Bu sekmede **Add External Link** düğmesiyle üçüncü taraf paylaşım bağlantıları (`vless://` , `vmess://` , `trojan://` , `ss://` , `hysteria2://` , `wireguard://`), **Add External Subscription** düğmesiyle ise uzak abonelik URL'leri (örneğin `https://provider.example/sub/...`) eklenir.

Bunların tamamı, bu istemcinin abonelik çıktısına (raw, JSON ve Clash formatları) eklenir: bağlantılar olduğu gibi eklenir, uzak abonelikler ise panel tarafından periyodik olarak indirilir (önbellekleme ve kısa zaman aşımıyla) ve yapılandırmaları kendi yapılandırmalarla birleştirilir. Böylece tek bir istemci abonelik bağlantısında kendi sunucularınızla birlikte harici yapılandırmalar da sunulabilir.

Grup

Grup (`group` alanı) – ilgili istemcileri bir araya getirmek için mantıksal etiket. İpucu: "İlgili istemcileri gruplamak için mantıksal etiket (örn. ekip, müşteri, bölge). Araç çubuğundan filtrelenebilir.", yer tutucu – "örn. customer-a". Alan isteğe bağlıdır (varsayılan olarak boş). Gruba göre liste filtrelenebilir ve toplu işlemler gerçekleştirilebilir; bir istemcinin etiketini temizlemek için **Gruptan Çıkar** eylemi kullanılır.

Grup, tek istemci düzenleyicisinde de kaldırılabilir: **Grup** alanını temizleyip kaydederseniz etiket doğru şekilde kaldırılır ve istemci eski grubunda görüntülenmez.

Yorum

Yorum (`comment` alanı) – yönetici için isteğe bağlı metin notu (varsayılan olarak boş). İçerik aramaya dahil edilir ve filtrelemeye açıktır (**Var / Yok** yorum).

Etkin

Etkin (`enable` alanı) – hesap etkinlik bayrağı. Varsayılan olarak **etkin** (`true`); oluşturulurken bayrak iletilmese bile panel zorla `true` atar. Devre dışı bir istemci (`enable = false`) bağlanamaz ve özet istatistiklerde **devre dışı** (deactive) kategorisine girer. Panel; kota dolduran, süresi dolan veya IP limitini aşan istemcileri kendisi devre dışı bırakır.

Yalnızca Okunabilir Alanlar

İstemci kartında ayrıca hizmet alanları da görüntülenir: **Oluşturulma** (`created_at`) ve **Güncellenme** (`updated_at`) – oluşturma ve son değişiklik zaman damgaları, otomatik olarak doldurulur ve düzenlenemez. **Ters Etiket** (`reverse`) alanı – basit VLESS ters proxy için isteğe bağlı Reverse tag ("İsteğe bağlı Reverse tag").

8.2. inbound'a Bağlama

Her istemcinin en az bir inbound'a bağlı olması gerekir – oluşturma sırasında en az biri zorunludur (`at least one inbound is required`). Düzenleyicide bu alan, **Seçili Girişler** olarak belirtilir ve **Bir veya birden fazla giriş seçin** ipucuna sahiptir.

- **Bağla** – istemciyi seçilen inbound'lara ekler (aynı UUID/parola ve ortak trafik). Mevcut bağlamalar korunur.
- **Bağlantıyı Kes** – istemciyi seçilen inbound'lardan kaldırır. İstemci kaydının kendisi korunur (tamamen silmek için **Sil** kullanın). İstemcinin bağlı olmadığı çiftler sessizce atlanır.

Birden fazla inbound'a bağlı bir istemci kaydedilirken, belirli protokol/taşıma ile uyumsuz alanlar (örneğin VLESS-Vision dışında Flow) her inbound için otomatik olarak geçerli değerlere dönüştürülür.

inbound seçim listesinin üzerinde (istemci formunda, istemcileri toplu ekleme sırasında ve toplu bağlama/bağlantıyı kesme pencerelerinde) **Tümünü Seç** ve **Temizle** düğmeleri bulunur. Bu listelerde her inbound, varsa kendi açıklamasıyla (remark), yoksa inbound etiketi ile gösterilir.

8.3. İstemci İşlemleri

Tek bir istemci için (**İstemci Bilgisi** kartı veya **Eylemler** bağlam menüsü üzerinden) şunlar kullanılabilir:

Bilgi, QR Kodu ve Bağlantı Görüntüleme

- **İstemci Bilgisi** – tüm alanları, kullanılan/kalan trafik (**Kalan**), geçerlilik süresi ve bağlı inbound'ları gösteren kart. 3.5.0'dan itibaren kart, grup tanımlandığında istemcinin **grubunu** da gösterir («Yorum» satırının üzerinde, etiketli bir «Grup» satırı).

API üzerinden istemci sorgusu (`GET /panel/api/clients/get/:email`), `client` ve `inboundIds` alanlarının yanı sıra `usedTraffic` değerini de döndürür – gerçekte tüketilen trafik (gönderilen + alınan, düğüm verileri dahil); bu da tüketimi `totalGB` kotasıyla karşılaştırmayı kolaylaştırır.

- **QR Kodu ve Bağlantı** – istemci uygulamasına aktarmak için istemci yapılandırma bağlantısı. Desteklenen protokole sahip tüm bağlı inbound'lara göre oluşturulur (`GET /links/:email`). Uygun bağlantı yoksa: "Paylaşılacak bağlantı yok – önce istemciyi desteklenen protokole sahip bir girişe bağlayın".
- **Abonelik Bağlantısı** – `subId` 'ye göre abonelik URL'si (`GET /subLinks/:subId`). Yalnızca istemcinin `subId` 'si varsa ve abonelik servisi **Panel Ayarları** → **Abonelik** bölümünde etkinleştirilmişse kullanılabilir (aksi takdirde "Abonelik servisi devre dışı."). Ek olarak **JSON abonelik URL'si** de sunulur.

Trafik Sıfırlama

Trafik Sıfırla (`POST /resetTraffic/:email`), belirli bir istemcinin gönderme/alma sayaçlarını (`up` , `down`) sıfırlar. Kota (`totalGB`) ve geçerlilik süresi **etkilenmez** – yalnızca kullanılan miktar sıfırlanır. Bildirim: "Trafik sıfırlandı". İstemci hiçbir inbound'a bağlı değilse: "Önce bu istemciyi bir girişe bağlayın".

Trafik Sıfırla düğmesine, istemci düzenleme formundan da – **İptal /**

Kaydet düğmelerinin yanında, alt panelde – erişilebilir (sıfırlamadan önce onay istenir). İstemci trafik tükenmesi nedeniyle devre dışı bırakılmışsa, sıfırlama (tek veya toplu) otomatik olarak istemciyi yeniden

etkinleştirir (`enable = true`) ve bu değişikliği anında düğüm sunucularına iletir — ana panelde ve düğümlerde istemciyi manuel olarak yeniden etkinleştirmeye gerek kalmaz.

IP Limitini Sıfırla

İstemcinin birikmiş IP günlüğünü temizler (`POST /clearIps/:email`), böylece eş zamanlı bağlantı limiti aşımından kaynaklanan geçici engeli kaldırır. Bildirim: "Günlük temizlendi".

Sil

Sil (`POST /del/:email`) — istemcinin tamamen silinmesi. Onay iletişim kutusu: başlık "İstemci {email} silinsin mi?", metin "İstemci tüm bağlı girişlerden kaldırılacak ve trafik kaydı yok edilecek. Bu işlem geri alınamaz.". Silme işlemi, istemciyi **tüm** inbound'lardan kaldırır ve trafik kaydını yok eder. Bildirim: "İstemci silindi".

8.4. Toplu İşlemler

İstemci listesinde birden fazla kayıt işaretlenebilir (**Tümünü Seç, Tümünü Temizle**); sayaç — "{count} seçildi". Seçilenlere uygulanabilecek işlemler:

- **Sil ({count})** (`POST /bulkDel`) — toplu silme. Onay: "{count} istemci silinsin mi?", "Seçilen her istemci tüm bağlı girişlerden kaldırılır, trafik kaydı yok edilir. Bu işlem geri alınamaz.". Bildirim: "{count} istemci silindi", kısmi başarısızlık durumunda — "Silindi: {ok}, başarısız: {failed}".
- **Düzenle ({count}) / Düzeltme** (`POST /bulkAdjust`) — süre ve/veya kotanın toplu değiştirilmesi. İletişim kutusu "{count} istemciyi düzenle" ipucuyla: "Pozitif değerler ekler, negatif değerler azaltır. Sınırsız süre veya trafiğe sahip istemciler ilgili alan için atlanır.". Alanlar: **Gün Ekle, Trafik Ekle (GB)** ve **Set flow**. Mantık:
 - **Süre:** Süresiz istemciler (`expiryTime == 0`) atlanır ("unlimited expiry"); tarihi olan istemciler için süre belirtilen gün kadar kaydırılır; "ilk kullanımdan sonra" modundaki istemciler (negatif süre) için bekleme süresi ayarlanır. Kalanı aşan azaltma atlanır ("reduction exceeds remaining time/delay window").
 - **Trafik:** Limitsiz istemciler (`totalGB == 0`) atlanır ("unlimited traffic"); aksi halde kota belirtilen miktarda değiştirilir, sıfırın altına düşmez.
 - **Flow:** **Set flow** açılır listesi, seçili tüm istemciler için XTLS flow'u aynı anda ayarlamaya veya sıfırlamaya olanak tanır. Varsayılan olarak **No change** (değişiklik yok) seçilidir. **Disable (clear flow)** seçeneği flow'u sıfırlar; `xtls-rprx-vision` ve `xtls-rprx-vision-udp443` değerleri ilgili vision-flow'u ayarlar. Vision-flow ayarı yalnızca flow'u destekleyen inbound'lara uygulanır; uygun olmayan inbound'lar değiştirilmez ve atlandı olarak işaretlenir; flow sıfırlama ise her zaman geçerlidir.
 - **Otomatik etkinleştirme (3.4.2'den itibaren): yalnızca tükenme nedeniyle** (süresi dolmuş veya kotası aşılmış) devre dışı bırakılan bir istemci, uzatma sırasında düzeltme onu limitlere geri döndürürse otomatik olarak yeniden etkinleştirilir — hem yerel olarak hem de düğümünde. Elle devre dışı bırakılanlar veya hâlâ tükenmiş olanlar kapalı kalır (`enable` alanı artık açıkça yazıldığından durum inbound'u olmayan istemcilerde de korunur).
 - Gün, trafik veya flow belirtilmemişse: "Uygulamadan önce gün, trafik veya flow belirtin.". Bildirim: "Değiştirildi: {count}" / "Değiştirildi: {ok}, atlandı: {skipped}".

Örnek: seçili istemcileri 30 gün uzatmak ve 50 GB eklemek. **Düzenle** iletişim kutusunda **Gün Ekle** = 30 , **Trafik Ekle (GB)** = 50 girin. Tersine, bir hafta eksiltmek ve kotayı 10 GB azaltmak için negatif değerler girin: **Gün Ekle** = -7 , **Trafik Ekle (GB)** = -10 (ilgili alan için süresiz veya limitsiz istemciler atlanır).

- **Bağla ({count}) / Bağlantıyı Kes ({count})** (POST /bulkAttach / bulkDetach) – seçili istemcilerin seçili inbound'lara toplu bağlanması/bağlantısının kesilmesi. Hedefler yalnızca çok kullanıcı inbound'lardır. Bağlantı kesme sonucu: "{detached} bağlantı kesildi, {skipped} atlandı.". 3.5.0'dan itibaren istemci formundaki inbound seçicisi **devre dışı bırakılmış inbound'ları gizler** (istemcinin zaten bağlı olduğu inbound'lar hariç); «Bağlantıyı Kes» penceresinde aynı istemcinin kayıtlarının kopya olarak birikmesi giderildi (veriler panel başlangıcında otomatik olarak onarılır).
- **Abonelik bağlantıları ({count})** – seçili istemcilerin abonelik URL'lerini ve JSON abonelik URL'lerini **Tümünü Kopyala** düğmesiyle birlikte gösteren özet tablo. Hiç kimsenin subld'si yoksa: "Seçili istemcilerin hiçbirinin Abonelik ID'si yok".
- **Gruba Ekle ve Gruptan Çıkar** – grup etiketinin atanması ve kaldırılması.
- **Etkinleştir ({count}) / Devre Dışı Bırak ({count})** (POST /bulkEnable / bulkDisable) – seçili istemcilerin toplu etkinleştirilmesi ve devre dışı bırakılması. **Etkinleştir**, seçili her istemciyi tüm bağlı inbound'larda etkinleştirir; tükenmiş trafik kotasına veya süresi dolmuş tarihe sahip istemciler otomatik olarak yeniden devre dışı bırakılır. **Devre Dışı Bırak**, istemcilerin erişimini anında keser; ancak kayıtları ve birikmiş trafik bilgileri korunur. Panel işlem öncesinde onay ister ve işlemten sonra işlenen istemci sayısını ve varsa başarısız olanların sayısını içeren bir bildirim gösterir.

Duruma Göre Trafik Sıfırlama ve Silme

- **Tüm İstemcilerin Trafiğini Sıfırla** (POST /resetAllTraffics) – **tüm** panel istemcilerinin up / down sayaçlarını sıfırlar. Onay: "Tüm istemcilerin trafiği sıfırlansın mı?" ve "Tüm istemcilerin gönderme/alma sayaçları sıfıra indirilir. Kotalar ve geçerlilik süreleri etkilenmez. Bu işlem geri alınamaz.". Bildirim: "Tüm istemcilerin trafiği sıfırlandı".
- **Tükenmişleri Sil** (POST /delDepleted) – reset = 0 koşuluyla **kotası dolmuş** (total > 0 and up + down >= total) **veya süresi dolmuş** (expiry_time > 0 and expiry_time <= şimdi) tüm istemcileri siler (otomatik yenilemeli istemcilere dokunulmaz). Onay: "Tükenmiş istemciler silinsin mi?", "Trafik kotası dolmuş veya süresi dolmuş tüm istemciler silinir. Bu işlem geri alınamaz.". Bildirim: "{count} tükenmiş istemci silindi".

Dışa Aktarma, İçe Aktarma ve Bağısız İstemcileri Silme

Hiçbir şey seçili değilken, **İstemciler** sayfasındaki **Daha Fazla** menüsünde üç işlem kullanılabilir.

İstemcileri Dışa Aktar (GET /clients/export), tüm istemcilerin {client, inboundIds} biçimindeki JSON listesini kopyalama ve indirme düğmeleriyle (clients-export.json dosyası) birlikte gösteren bir görüntüleyici açar. **İstemcileri İçe Aktar** (POST /clients/import), böyle bir JSON'ın yapıştırıldığı ve **Import** düğmesine basıldığı bir düzenleyici açar: inboundIds 'li istemciler oluşturulur ve inbound'lara bağlanır; bağlantısız istemciler bağımsız "sade" kayıtlar olarak geri yüklenir; zaten mevcut email'ler **hiçbir zaman üzerine yazılmaz** – atlanmışlar listesine eklenir. Bildirimler: "{count} clients imported", "{ok} imported, {failed} skipped".

Bağısız İstemcileri Sil (`POST /clients/deleteOrphans`) – tehlikeli bir işlem: hiçbir inbound'a bağlı olmayan tüm istemcileri trafik kaydı, IP günlüğü ve harici bağlantılarıyla birlikte siler. Onay: "Delete clients without an inbound?", "Removes every client that is not attached to any inbound, along with its traffic record. This cannot be undone.". Bildirim: "{count} unattached clients deleted". İşlem geri alınamaz.

8.5. Arama, Filtreler ve Sıralama

Listenin üzerinde bir arama çubuğu ("Email, yorum, sub ID, UUID, parola, auth, Telegram ID ara...") bulunur – email, yorum, subId, UUID, parola, auth ve (3.5.0 ile yeniden) **Telegram ID** üzerinde arama yapar. Sonuç sayacı: "{total} içinden {shown} gösteriliyor".

İstemci listesi otomatik olarak güncellenir: panel her birkaç saniyede bir geçerli sayfayı yeniler; bu nedenle yeni bağlanan istemciler ve değişen sıralama düzeni manuel yenileme gerektirmeden görünür (arka plan yoklaması sırasında yükleme göstergesi yanıp sönmez).

3.5.0'dan itibaren tabloda («Trafik» ile «Kalan» arasında) bir «**Hız**» sütunu vardır: istemcinin canlı hızı – mavi bir **↑ yükleme / ↓ indirme** etiketi (~5 saniyelik kayan ortalama, her 5 saniyede bir güncellenir); trafik yoksa – gri bir çizgi. Mobil cihazlarda hız, istemci kartında bir satır olarak gösterilir.

İstemci Filtresi paneli, duruma (kategoriye), protokole, bağlı inbound'a, geçerlilik tarihi aralığına, kullanılan trafik aralığına, otomatik yenileme varlığına (**Var/Yok**), Telegram ID ve yorum varlığına ve gruba göre filtrelemeye olanak tanır. Düğümlü panellerde **Düğüm**ler çok seçimi belirir: liste seçili düğümlerin istemcileriyle sınırlandırılabilir; ayrı **Yerel Panel** seçeneği, düğüme bağlı olmayan inbound'ların istemcilerini filtreler (filtre yalnızca düğümler mevcut olduğunda görünür). Sıralama: **Önce Eskiler/Yeniler, Son Güncellenenler, Son Çevrimiçi, Email A→Z / Z→A, Daha Fazla Trafik, Daha Fazla Kalan, Yakında Sona Erecekler**.

8.6. Simgeler ve Durumlar

Durum önceliği: tükenmiş/süresi dolmuş → devre dışı → yakında sona erecek → etkin.

- **Çevrimiçi / Çevrimdışı** – aktif bağlantıya sahip (mevcut çevrimiçi listesinde yer alan) ve **etkin** istemci. Çevrimiçi listesi ayrı isteklerle güncellenir (`/onlines` , `/onlinesByGuid`).
- **Tükenmiş** (depleted) – kota doldu (`up + down >= totalGB`) **veya** süre doldu (`expiryTime <= şimdi`). Bu tür istemciler otomatik olarak devre dışı bırakılır ve **Tükenmişleri Sil** işlemi kapsamına girer.
- **Yakında Sona Erececek / Bitecek** (expiring) – etkin bir istemcide sona erme süresine eşik aralığından az kalmış **veya** kota tükenmesine eşik miktarından az kalmış (eşikler panel ayarlarında belirlenir). İstemci zaten tükenmiş/devre dışıysa sayılmaz.
- **Devre Dışı** (deactive) – `enable = false` olan istemci (manuel olarak veya arka plan görevi tarafından devre dışı bırakılmış).
- **Etkin** (active) – etkin, tükenmemiş, süresi dolmamış ve eşiklere henüz ulaşılmamış.

9. İstemci Grupları

Bu, 3X-UI'nin bu çatalına özgü bir özelliktir. Orijinal 3x-ui (MHSanaei) projesinde "istemci grubu" kavramı yoktur — burada ayrı bir gruplar tablosu, panel menüsünde **Gruplar** sayfası ve ilgili API yöntemleri eklenmiştir. Yapılandırmayı orijinal 3x-ui'ye taşırsanız, grup etiketi hiçbir yerde dikkate alınmayacaktır.

9.1. İstemci Grubu Nedir ve Ne İşe Yarar

Grup, bir veya birden fazla istemciye atanabilen adlandırılmış bir mantıksal etikettir (label). Yeni bir bağlantı yöntemi oluşturmaz; bir inbound ya da düğüm değildir — yalnızca istemcileri filtrelemek ve toplu olarak işlemek için kullanılan organizasyonel bir etikettir.

Bu çatalın istemci modelindeki temel fikir: **istemci, email ile tanımlanan üst düzey bir varlıktır** (`clients` tablosundaki `email` alanının benzersiz bir dizini vardır). Aynı istemci (aynı email ve aynı kimlik bilgileriyle), farklı protokoller dahil olmak üzere aynı anda birden fazla inbound'da ve hatta birden fazla düğümde yer alabilir. Grup etiketi **istemci başına bir kez** saklanır, bu nedenle otomatik olarak istemcinin tüm inbound bağlantılarına yayılır.

Grup etiketi, gruplama için kullanılan mantıksal bir etikettir:

Katman	Nerede saklanır	Alan
İstemci kaydı (VT)	<code>clients</code> tablosu	<code>group_name</code> (varsayılan olarak boş dize <code>' '</code>)
Grup rehberi (VT)	<code>client_groups</code> tablosu	<code>name</code> (benzersiz dizin, boş olamaz)
inbound ayarları (Xray)	JSON <code>settings.clients[].group</code>	istemcinin üye olduğu her inbound'daki her istemci nesnesine kopyalanır

Pratikte bunun önemi:

- **Birden fazla inbound/düğümde tek istemci.** Bir istemci birden fazla inbound'a (örneğin farklı protokoller veya farklı düğümler) erişim olarak "satılıyorsa", grup onu tek bir bütün olarak yönetmeyi sağlar: trafiği sıfırlamak, silmek, etiketi yeniden adlandırmak — tüm inbound'larında tek bir işlemle.
- **Toplu işlemler ve filtreleme.** **İstemciler** sayfasında liste gruba göre filtrelenebilir; **Gruplar** sayfasında gruptaki tüm üyeler üzerinde toplu işlemler yapılabilir.
- **Büyük istemci parkını organize etme.** `vip`, `trial`, `team-A` gibi etiketler, ayrı inbound'lar oluşturmadan binlerce istemciyi mantıksal kategorilere ayırmaya yardımcı olur.

9.2. Grubun İstemciler, inbound'lar, Düğümler ve Protokollerle İlişkisi

Etiket senkronizasyonu basit olmadığından bu alt bölüm anlaşılması en kritik olanıdır.

Grup, inbound'u değil istemciyi tanımlar. Etiket, istemci kaydında (`clients.group_name`) yaşar. Bir istemci birden fazla inbound'a bağlı olduğunda, herhangi bir grup değişikliğinde panel, bu istemcinin üye olduğu **tüm** inbound'ları gezinir ve Xray ayarlarındaki (`settings.clients[]`) `group` alanını günceller/kaldırır. Teknik

olarak şöyle çalışır: istemcinin email'ine göre üye olduğu tüm inbound'lar bulunur, ardından bu email'e sahip istemci nesnesi her inbound'un JSON ayarlarında düzenlenir. Bu nedenle:

- Grup **protokolden bağımsızdır**. Aynı email, bir inbound'da VLESS istemcisi, başka bir inbound'da Hysteria istemcisi olabilir – grup etiketi yine de tek bir tanedir ve ikisine de uygulanır (her protokolün kimlik bilgileri farklıdır ve ayrı ayrı saklanır).
- Grup **düğümüleri kapsar**. Düğümlere ait inbound'lar, ana panel inbound'larından `nodeId` alanıyla ayırt edilir (ana panel inbound'larında `null / 0` olur). Grup etiketi, istemci nesnelerinin hangi inbound'da – ana panelde mi yoksa düğüm inbound'unda mı – olduğundan bağımsız olarak yayılır; yeter ki o email'e sahip istemci orada bulunsun.

Grup etiketi, düğümlerden gelen senkronizasyona ve ayarların yeniden oluşturulmasına karşı dayanıklıdır. Bu davranış özel olarak tasarlanmıştır:

- Bir düğüm trafik anlık görüntüsü gönderdiğinde, gelen veriler panel VT'sindeki istemcinin yerel `group_name` ve `comment` alanlarını **üzerine yazmaz**. Grup ve yorum, "yalnızca panel" alanları olarak kabul edilir – düğüm bunları yönetmez.
- inbound ayarları yeniden oluşturulurken gelen verilerdeki boş `group` değeri, zaten kaydedilmiş etiketi **sıfırlamaz**. Grup, inbound Xray ayarlarını düzenlemek yerine özellikle **Gruplar** sayfasından yönetilir; bu nedenle normal ayar yeniden oluşturma sırasında "boş grup" "silme" değil, "dokunmama" olarak yorumlanır.

Pratik sonuç: etiketi **kasıtlı olarak temizleyen** tek işlemler, grubu silmek ve istemciyi gruptan açıkça kaldırmaktır (aşağıya bakın). Normal inbound düzenlemesi veya arka planda düğümle senkronizasyon grubu kaybettirmez.

9.3. Grup Rehberi ve "Boş" Gruplar

Sayfadaki grup listesi iki kaynağın birleştirilmesiyle oluşturulur:

- Türetilmiş gruplar (derived)** – istemcilerde gerçekten bulunan tüm boş olmayan `group_name` değerleri, istemci sayısı ile birlikte.
- Kaydedilmiş gruplar (stored)** – `client_groups` tablosundaki kayıtlar.

Bu birleştirme önemli bir etki yaratır: bir grup **hiç istemcisi olmadan** var olabilir. Böyle bir grup, "Grup Ekle" düğmesiyle açıkça oluşturulur (`client_groups` 'a kayıt) ve listede `0` sayacıyla görüntülenir. Bu kayıtlar **boş gruplar** olarak adlandırılır. Liste her zaman büyük/küçük harf duyarsız ada göre sıralanır.

Sayfadaki özet sayacılar:

Alan (RU)	Neyi gösterir
Toplam grup	Toplam grup sayısı (kaydedilmiş ve türetilmiş birlikte).
Gruplu istemciler	Boş olmayan grup etiketine sahip istemci sayısı.
Boş gruplar	İstemcisi olmayan grup sayısı (<code>0</code> sayacı).
Gruptaki istemciler	Belirli bir gruptaki istemci sayısı (tablo sütunu).

9.4. Grup Alanları ve Sütunları

`client_groups` tablosundaki grup kaydı şunları içerir:

Alan	Tür	Varsayılan	Açıklama
Id	int	otomatik artan	Grup kaydının birincil anahtarı.
Name	string	– (zorunlu)	Grup adı. Benzersiz dizin, boş olamaz. Arayüzde – Ad sütunu.
CreatedAt	int64 (ms)	oluşturulma zamanı	Grup kaydının oluşturulma anı.
UpdatedAt	int64 (ms)	değiştirilme zamanı	Son değiştirilme anı.

Sayfadaki tabloda en azından **Ad** ve **Gruptaki istemciler** sütunları ile eylem düğmeleri (aşağıya bakın) görüntülenir.

9.5. Grup Oluşturma

Grup Ekle düğmesi.

Adımlar:

1. **Grup Ekle**'ye tıklayın.
2. Grup adını girin.
3. Onaylayın.

Arka uç davranışı (`POST /panel/api/clients/groups/create` , gövde `{"name": "..."} :`

- Ad baştaki ve sondaki boşluklardan arındırılır. Boş ad, "group name is required" hatasıyla reddedilir.
- Bu adda bir grup zaten varsa – "group already exists" hatası.
- Başarı durumunda `client_groups` 'ta bir kayıt oluşturulur (başlangıçta istemcisiz – boş grup).

Başarı mesajı: «**{name}** grubu oluşturuldu.»

Örnek: API üzerinden boş grup oluşturma. İstemcilerle doldurmadan önce etiket kümesini hazırlayın:

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/create' \
-H 'Content-Type: application/json' \
-b cookie.txt \
-d '{"name": "vip"}'
```

Başarı durumunda yanıt:

```
{ "success": true, "msg": "«vip» grubu oluşturuldu.", "obj": null }
```

Aynı adla tekrarlanan çağrı `"success": false` ve `group already exists` mesajı döndürür.

Boş grup önceden oluşturmak, bir etiket kümesi hazırlayıp ardından "İstemci Ekle..." aracılığıyla istemcilerle doldurmak istediğinizde kullanışlıdır.

9.6. Grubu Yeniden Adlandırma

Yeniden Adlandır düğmesi, iletişim kutusu başlığı – «**{name}** grubunu yeniden adlandır».

Davranış (`POST /panel/api/clients/groups/rename` ,gövde `{"oldName": "...", "newName": "..."}:`

- Her iki ad da baştaki ve sondaki boşluklardan arındırılır. Boş eski ad – "old group name is required" hatası, boş yeni ad – "new group name is required" hatası.
- Yeni ad eskiyle aynıysa – hiçbir şey yapılmaz (`0` istemci etkilenir).
- Aksi takdirde yeniden adlandırma atomik olarak gerçekleştirilir:
 - `client_groups` 'taki kayıt yeniden adlandırılır;
 - `group_name = oldName` olan tüm istemcilerin alanı `newName` olarak güncellenir;
 - etkilenen istemcilerin üye olduğu **tüm inbound'larda** (düğüm inbound'ları dahil) Xray ayarlarındaki `group` değeri eskiden yeniye güncellenir.
- Yeniden adlandırmadan sonra panel Xray'i yeniden başlatma gerektiriyor olarak işaretler ve istemci değişiklik bildirimi gönderir.

Mesajlar:

- Başarı: «**Grup {count} istemci için yeniden adlandırıldı.**»
- Arayüzde ad çakışması: «**{name} adında bir grup zaten mevcut.**»

9.7. Gruba İstemci Ekleme

İstemci Ekle... düğmesi, başlık – «**"{name}" grubuna istemci ekle**».

İletişim kutusundaki ipucu metni:

«Bu gruba eklenecek istemcileri seçin. Mevcut inbound bağlantıları korunur; yalnızca grup etiketi değişir. Bu grupta zaten olan istemciler gösterilmez.»

Eklenecek kimse yoksa «**Eklenecek başka istemci yok.**» görüntülenir.

Davranış (`POST /panel/api/clients/groups/bulkAdd` ,gövde `{"emails": [...], "group": "..."}:`

- Grup adı zorunludur (aksi halde "group name is required" hatası); boş email listesi – işlem hiçbir şey yapmaz.
- Böyle bir grup ne `client_groups` 'ta ne de türetilmiş gruplar arasında yoksa – otomatik olarak oluşturulur.
- Seçilen email'ler için istemcilerin `group_name = group` alanı güncellenir; **istemcilerin inbound bağlantıları değişmez** – yalnızca etiket etkilenir. Ardından bu istemcilerin tüm inbound'larında `group` alanı güncellenir.
- Etkilenen istemci kayıtlarının sayısı döndürülür; Xray yeniden başlatma gerektiriyor olarak işaretlenir.

Başarı mesajı: «**{name} grubuna {count} istemci eklendi.**»

Örnek: tek istekle birden fazla istemciyi grupla etiketleme. İstemciler email ile belirtilir, inbound bağlantıları değişmez:


```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/bulkAdd' \
-H 'Content-Type: application/json' \
-b cookie.txt \
-d '{"emails": ["alice@example.com", "bob@example.com"], "group": "vip"}'
```

`vip` grubu henüz yoksa otomatik olarak oluşturulur. İstekten sonra bu istemcilerin kaydında `group_name = "vip"` ayarlanır ve her inbound'larının Xray ayarlarındaki istemci nesnesi `"group": "vip"` alanını alır:

```
{ "id": "6f1b...", "email": "alice@example.com", "group": "vip", "enable": true }
```

9.8. Gruptan İstemci Kaldırma (İstemcileri Silmeden)

İstemci Kaldır... düğmesi, başlık – «**"{name}" grubundan istemci kaldır**».

İpucu metni:

«Bu gruptan kaldırılacak üyeleri seçin. İstemcilerin kendisi korunur (tam silme için "Gruptaki istemcileri sil" seçeneğini kullanın).»

Davranış (`POST /panel/api/clients/groups/bulkRemove` , gövde `{"emails": [...]}`): teknik olarak bu, boş grupta "Gruba Ekle" ile aynıdır. Seçilen istemcilerin `group_name` alanı temizlenir ve Xray ayarlarından `group` alanı kaldırılır. İstemcilerin kendisi ve inbound bağlantıları korunur.

Başarı mesajı: «**{name} grubundan {count} istemci kaldırıldı.**»

9.9. Grup Trafikini Sıfırlama

Trafik Sıfırla düğmesi.

Onay iletişim kutusu:

- Başlık: «**{name} grubunun trafiği sıfırlansın mı?**»
- Metin: «**Yalnızca grubun trafik sayacı sıfırlanır. Tek tek istemcilerin sayaçları etkilenmez.**»

Davranış: **yalnızca grubun kendi görüntülenene sayacı** sıfırlanır – panel grubun mevcut `up/down` toplamını temel işaret olarak hatırlar ve sonrasında grup listesinde `max(0, toplam - temel işaret)` gösterir. Tek tek istemcilerin `up / down` sayaçları, kotaları ve `enable` alanı **değişmez**, Xray'i yeniden başlatma gerekmez. Bu, sıfırlamanın grubun tüm istemcilerinin trafiğini sıfırladığı önceki sürümlere göre bir davranış değişikliğidir.

İstek: `POST /panel/api/clients/groups/resetTraffic` , gövde `{"name": "<grup adı>"}` .

Başarı mesajı: «**{name} grubunun trafiği sıfırlandı.**»

9.10. Grubu Silme ve Gruptaki İstemcileri Silme

Sayfada **temelden farklı iki silme işlemi** vardır – bunları birbirine karıştırmak kolaydır, bu nedenle aradaki fark kritiktir.

9.10.1. Grubu Sil (istemcileri koru)

«Grubu Sil (istemcileri koru)» düğmesi.

İletişim kutusu:

- Başlık: «**{name}** grubu silinsin mi?»
- Metin: «**Bu, grubu siler ve {count} istemcideki etiketini temizler. İstemcilerin kendisi silinmez.**»

Davranış (POST /panel/api/clients/groups/delete , gövde { "name": "..."}): grup kaydı client_groups 'tan silinir, tüm istemcilerinin group_name alanı temizlenir ve inbound'larından group alanı kaldırılır. **İstemciler, bağlantıları ve trafiği korunur.** Xray yeniden başlatma gerektiriyor olarak işaretlenir.

Başarı mesajı: «**{count} istemcideki grup etiketi temizlendi.**»

9.10.2. Gruptaki İstemcileri Sil (tam silme)

«Gruptaki istemcileri sil» düğmesi.

İletişim kutusu:

- Başlık: «**{name}** grubundaki tüm istemciler silinsin mi?»
- Metin: «**Bu, {count} istemciyi trafik kayıtlarıyla birlikte kalıcı olarak siler. Grup etiketi de temizlenir. Bu işlem geri alınamaz.**»

Bu yıkıcı bir işlemdir: istemcilerin kendisini (email bazında toplu silme, POST /panel/api/clients/bulkDelete uç noktası aracılığıyla), trafik kayıtları dahil siler ve böylece onları tüm inbound'lardan kaldırır.

Mesajlar:

- Başarı: «**{count} istemci silindi.**»
- Kısmi sonuç: «**{ok} silindi, {failed} atlandı**»

Grup boşsa, üyeleri üzerindeki eylemler kullanılamaz – «**Bu grupta henüz istemci yok.**» görüntülenir.

9.11. "İstemciler" Sayfasıyla İlişki

Grup etiketi, **Gruplar** sayfasının dışında da görüntülenir ve kullanılır:

- Kompakt istemci kaydında group alanı bulunur, bu nedenle istemci listesinde grup üyeliği görüntülenir.
- İstemci listesi (/panel/api/clients/list/paged), group filtre parametresini kabul eder: virgülle ayrılmış bir veya birden fazla ad geçirilebilir. Eşleştirme alan içinde büyük/küçük harf duyarlı "VEYA" mantığıyla çalışır. Özel durum: filtre grupları listesindeki boş öge, "grupsuz istemciler"i (group alanı boş olanları) temsil eder.
- İstemci sayfası yanıtında, kullanıcı arayüzünün filtre açılır listesini oluşturabilmesi için mevcut tüm grup adlarını içeren groups dizisi döndürülür.

Örnek: istemcileri gruplara göre filtreleme. İstek yalnızca `vip` veya `trial` etiketli istemcileri döndürür (birden fazla ad virgülle ayrılır, "VEYA" mantığı):

```
GET /panel/api/clients/list/paged?group=vip,trial
```

Grupsuz istemcileri almak için listeye boş öge geçirin – örneğin `group=` (boş dize) veya `group=vip,` (`vip` etiketi artı grupsuz istemciler) filtre değeri.

9.12. API Uç Noktaları Özeti

Tüm grup rotaları `/panel/api/clients` altına bağlanmıştır:

Yöntem ve yol	Amaç	İstek gövdesi
GET <code>/panel/api/clients/groups</code>	İstemci sayacıyla grupların listesi	–
GET <code>/panel/api/clients/groups/:name/emails</code>	Gruptaki tüm üyelerin email'leri (email'e göre sıralı)	–
POST <code>/panel/api/clients/groups/create</code>	Boş grup oluştur	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/rename</code>	Grubu yeniden adlandır	<code>{"oldName", "newName"}</code>
POST <code>/panel/api/clients/groups/delete</code>	Grubu sil, istemcileri koru (etiket temizleme)	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/bulkAdd</code>	Gruba istemci ekle (email bazında)	<code>{"emails": [...], "group"}</code>
POST <code>/panel/api/clients/groups/bulkRemove</code>	Gruptan istemci kaldır (etiket temizleme)	<code>{"emails": [...]}</code>
POST <code>/panel/api/clients/bulkDel</code>	İstemcileri tam sil ("Gruptaki istemcileri sil" tarafından kullanılır)	<code>{"emails": [...], "keepTraffic"}</code>

Örnek: API üzerinden tipik grup yaşam döngüsü senaryosu.

```
# 1. trial etiketini oluştur
curl -s ../panel/api/clients/groups/create -d '{"name":"trial"}'

# 2. İki istemciye etiket ata
curl -s ../panel/api/clients/groups/bulkAdd -d '{"emails":
["u1@example.com","u2@example.com"],"group":"trial"}'

# 3. Tüm üyelerin trafiğini sıfırla (/groups/trial/emails adresinden email al)
curl -s ../panel/api/clients/groups/bulkRemove -d '{"emails":["u2@example.com"]}'

# 4. Grubu sil, istemcileri koru (yalnızca etiket temizleme)
curl -s ../panel/api/clients/groups/delete -d '{"name":"trial"}'
```

1. adım, grup kaydını siler ve istemcilerinin `group_name` alanını temizler; ancak istemcilerin kendisi, bağlantıları ve trafiği korunur. İstemcilerin kendisini kalıcı olarak silmek için bunun yerine `bulkDel` kullanılır.

İstemcilerdeki etiketi değiştiren işlemler (`rename` , `delete` , `bulkAdd` , `bulkRemove`), Xray'i yeniden başlatma gerektiriyor olarak işaretler ve istemci değişiklik bildirimi gönderir.

9.13. Gruba Göre Trafik

3.3.0 sürümünün yeniliği: **Gruplar** bölümündeki (istemci yönetim sekmesindeki "İstemciler" sayfası) grup tablosu artık yalnızca her gruptaki istemci sayısını değil, grubun toplam tüketilen trafiğini de göstermektedir. Sütun «**Kullanılan Trafik**» olarak etiketlenmiştir.

Sütunun Gösterdikleri

Her grup satırı için, gruptaki tüm istemcilerin trafiklerinin toplamı – yani tüm üyelerin `up` + `down` (gönderilen + alınan trafik) değerlerinin toplamı – görüntülenir. Bu, "tüm grup toplamda ne kadar indirdi/gönderdi" sorusuna istemcileri tek tek açıp manuel olarak toplamak zorunda kalmadan hızlıca yanıt verir.

Grup tablosunda yanı sıra şunlar da görüntülenir:

Sütun	Anlamı
Ad	Grup adı
İstemciler	Bu grupta etiketlenmiş istemci sayısı (sütun daha önce "Gruptaki istemciler" olarak adlandırılıyordu)
Gönderilen	Gruptaki tüm istemcilerin toplam <code>up</code> değeri (gönderilen trafik)
Alınan	Gruptaki tüm istemcilerin toplam <code>down</code> değeri (alınan trafik)
Kullanılan Trafik	Gruptaki tüm istemcilerin toplam <code>up</code> + <code>down</code> değeri

Gönderilen ve alınan trafik ayrı **Gönderilen** ve **Alınan** sütunlarında görüntülenirken **Kullanılan Trafik** sütunu bunların toplamını gösterir. İstemci sayısı sütunu yalnızca **İstemciler** olarak adlandırılır.

Tablonun üzerindeki özet, tüm gruptaki toplamı da gösterir – «**Toplam grup**» ve «**Gruplu istemciler**»; toplam trafik iki karta bölünmüştür: «**Toplam gönderilen / alınan**» (yukarı/aşağı oklar – tüm grupların gönderilen ve alınan trafiği ayrı ayrı) ve «**Toplam trafik**» (grafik simgesi – bunların genel toplamı).

Hesaplama Yöntemi

Hesaplama, istemciler tablosuna trafik muhasebe tablosunun birleştirildiği (`LEFT JOIN`) tek bir SQL sorgusuyla gerçekleştirilir:

- grup etiketi alanına (`group_name`) göre istemciler gruplanır, sayıları hesaplanır – bu "Gruptaki istemciler" değeridir;
- trafik, birleştirilmiş `client_traffics` tablosundan `up + down` toplamı olarak alınır; yani her istemcinin hem gönderilen (`up`) hem de alınan (`down`) baytları toplanır;
- email hem istemciler tablosunda hem de trafik tablosunda benzersiz olduğundan, birleştirme tek bir istemcinin trafiğini iki kez saymaz.

Değerlere ilişkin özellikler:

- **Trafik kaydı olmayan istemciler** üye sayacına dahil edilir ancak toplama 0 ekler, dolayısıyla yeni oluşturulan bir grup 0 trafik gösterir.
- **Boş gruplar** (oluşturulmuş ancak istemcisiz) de sıfır sayaç ve sıfır trafikle listede yer alır: istemci etiketlerinden "türetilen" grupların yanı sıra açıkça kaydedilmiş gruplar da sonuca eklenir ve ardından liste büyük/küçük harf duyarsız ada göre sıralanır.
- Grup etiketi olmayan istemciler (`group_name` boş) hesaba dahil edilmez.

İlgili İşlemler

Grup tablosundan tüm grup üzerindeki işlemler hâlâ kullanılabilir; bunlar arasında «**Trafiği Sıfırla**» da yer alır – **yalnızca grubun kendi sayacını** (temel işareti) sıfırlar, tek tek istemcilerin trafiğine dokunmaz (bkz. 9.9). Bu sıfırlama işleminden sonra o grup için "Kullanılan Trafik" sütunu 0 gösterir.

10. Abonelikler (Subscription)

Abonelik (subscription), istemciye tek bir kalıcı bağlantı (URL) vermeyi sağlayan bir mekanizmadır; bu bağlantı aracılığıyla VPN istemcisi tam yapılandırma setini kendisi indirir ve periyodik olarak günceller. Her inbound için kullanıcıya ayrı ayrı bağlantı göndermek yerine, `https://alan-adı:port/sub/<subId>` biçiminde tek bir adres iletilir. Panel bu adres üzerinden söz konusu istemciye bağlı tüm yapılandırmaları anında bir araya getirir ve istemcinin istediği formatta teslim eder. Sunucu ayarları değiştiğinde (yeni adres, Reality anahtarlarının rotasyonu, inbound ekleme) istemci, kullanıcıdan herhangi bir işlem gerektirmeksizin bir sonraki otomatik güncellemede güncel yapılandırmayı alır.

Aboneliği, panelin içindeki bağımsız bir HTTP/HTTPS sunucusu yönetir; bu sunucu web panelinden bağımsız olarak başlatılır ve kendi portunda dinler. Bu, güvenlik amacıyla yapılmıştır: abonelik portu dışarıya açılabilirken panelin kendi portu açık bırakılmak zorunda değildir.

10.1. subId nedir ve bağlantı nasıl oluşturulur

Bir inbound'daki her istemcinin `subId` alanı vardır (arayüzde «Abonelik ID»). Bu değer abonelik anahtarıdır: panel tüm inbound'larda `subId` değeri istekle eşleşen istemcileri arar ve yapılandırmalarını tek bir yanıtta birleştirir.

- Birden fazla istemcide (aynı ya da farklı inbound'larda) aynı `subId` tanımlanmışsa, yapılandırmaları tek bir abonelikte yer alır. Bu, bir kullanıcıya tek bir bağlantı üzerinden birden fazla sunucu/protokol sunmanın standart yöntemidir.

Örnek: tek kullanıcı – tek bağlantıyla iki sunucu. Diyelim ki iki inbound var (A sunucusunda VLESS ve B sunucusunda Trojan). Kullanıcıya her iki yapılandırmayı tek bir bağlantıyla vermek için her iki istemcisine de aynı `subId` 'yi atayın:

```
Inbound 1 (VLESS): email = ivan@vpn, subId = ivan2025
Inbound 2 (Trojan): email = ivan@vpn, subId = ivan2025
```

Bu durumda `https://sub.example.com:2096/sub/ivan2025` adresinden panel her iki yapılandırmayı birden teslim eder. Daha sonra aynı `subId` ile üçüncü bir inbound eklerseniz, yeni bağlantı göndermeksizin bir sonraki otomatik abonelik güncellemesinde kullanıcıya görünür.

- İstemcinin `subId` alanı boşsa, genel erişim bağlantısı paylaşılamaz. Arayüz bunu şu ipucuyla belirtir: «Bu istemcinin subId'si yok, genel erişim bağlantısı kullanılamaz.»

İstemcinin harici bağlantıları ve abonelikleri («Links» sekmesi)

İstemci formunda yalnızca o istemcinin aboneliğine karıştırılan ek yapılandırma kaynaklarını ekleyebileceğiniz «Links» sekmesi bulunur (RAW, JSON ve Clash formatları):

- Add External Link** – harici bir paylaşım bağlantısı (`vless://`, `trojan://`, `ss://` vb.). Çıktıya olduğu gibi eklenir; JSON/Clash için ayrıca yapılandırmaya ayrıştırılır.
- Add External Subscription** – harici abonelik adresi. Panel onu kendisi indirir (önbellekleme ve kısa zaman aşımıyla) ve elde edilen yapılandırmaları istemcinin genel listesiyle birleştirir.

Bu, istemciye inbound'larınıza ek olarak aynı tek bağlantı üzerinden ek sunucular sunmak için kullanışlıdır. Uzak abonelik yanıtı çok büyük olduğunda artık sessizce kesilmez: panel hata döndürür ve en son başarıyla ön belleğe alınan değeri kullanmaya devam eder.

- `subId` değeri isteğe bağlı ayarlanamaz: kaydetme sırasında boşluk, `/`, `\` ve kontrol karakteri içermediği doğrulanır. İlgili doğrulama ipucu: «Abonelik ID'si boşluk, `/`, `"` veya kontrol karakteri içeremez».

Elde edilen bağlantı `<taban>/<subPath>/<subId>` biçiminde oluşturulur (abonelik sunucusu ayarları ve «Ters proxy URI» alanı ile ilgili bölüme bakın). `subId` 'ye karşılık gelen istemci bulunamazsa (istemci silinmişse, `subId` mevcut değilse) sunucu gövdesiz HTTP 404 döndürür. Dahili hata durumunda ise HTTP 500 döndürülür. VPN istemcileri yalnızca yanıt koduna göre hareket eder, bu yüzden hata gövdesi kasıtlı olarak boş bırakılır.

Abonelikteki inbound bağlantılarının sırası

Her inbound'un, abonelik çıktısındaki o inbound bağlantılarının konumunu belirleyen 1'den başlayan bir sayı olan «**Abonelikteki sıra**» (`subSortIndex`) alanı vardır. Küçük değerler önce gelir; eşit değerlerde orijinal oluşturulma sırası (id'ye göre) korunur. Sıra, tüm çıktı formatlarına uygulanır: ham metin, abonelik sayfası, JSON ve Clash. Bu alan, panelin kendi inbound listesinin sıralamasını etkilemez.

Alan, inbound formunda paylaşım adresi (share address) ayarlarının yanında düzenlenir ve normal kurallara göre düğümlere senkronize edilir. En az bir inbound'un sırası 1'den farklıysa Inbounds listesinde kompakt bir «**Sıra**» sütunu görünür.

10.2. Abonelik sunucusu ayarları

Tüm abonelik parametreleri, panel ayarlarının «**Abonelik**» sekmesinde yer alır. Aşağıda her parametre açıklanmış; parantez içinde ayarın dahili anahtarı ve varsayılan değeri belirtilmiştir.

Bölümün kendisi «**Panel Ayarları**», «**Bilgi**», «**Profil**», «**Sertifikalar**», «**Happ**» ve «**Clash / Mihomo**» sekmelerine ayrılmıştır. Abonelik başlığı, destek URL'si, profil sayfası, duyuru ve tema kataloğu alanları «**Profil**» sekmesinde; Happ ve Clash/Mihomo yönlendirme kuralları ilgili sekmelerde; abonelik güncelleme aralığı ise «**Bilgi**» sekmesindedir.

Temel parametreler

Alan (UI)	Anahtar	Varsayılan değer	Açıklama
Aboneliği etkinleştir	<code>subEnable</code>	<code>true</code> (etkin)	Ayrı bir abonelik sunucusu başlatır. İpucu: «Ayrı yapılandırılmalı abonelik özelliği». Devre dışıysa abonelik sunucusu başlamaz ve bağlantıların hiçbirisi çalışmaz.
Dinleme IP'si	<code>subListen</code>	boş	Abonelik sunucusunun bağlantıları kabul ettiği IP adresi. İpucu: «Tüm IP adreslerini izlemek için varsayılan olarak boş bırakın».
Abonelik portu	<code>subPort</code>	<code>2096</code>	Abonelik sunucusunun TCP portu. İpucu: «Abonelik hizmetine hizmet edecek port numarası, sunucuda kullanılmamalıdır» — port serbest olmalı ve panel veya Xray ile çakışmamalıdır.

Alan (UI)	Anahtar	Varsayılan değer	Açıklama
URI yolu	subPath	/sub/	Normal aboneliklerin sunulduğu yol. İpucu: «/' ile başlamalı ve '/' ile bitmelidir».
Dinleme alanı	subDomain	boş	Aboneliğe erişime izin verilen alan adı (Host doğrulaması). İpucu: «Tüm alan adlarını ve IP adreslerini dinlemek için varsayılan olarak boş bırakın». Tanımlanmışsa, farklı Host'lu istekler reddedilir.

Güvenlik notu: Varsayılan /sub/ yolu (ve JSON için /json/) yaygın olarak bilinir ve kolayca tahmin edilebilir. Panel şu uyarıyı gösterir: «Varsayılan "/sub/" abonelik yolu çok iyi bilinmektedir – değiştirin.» ve JSON için de benzer bir uyarı. Kendi tahmin edilemez yolunuzu belirlemeniz önerilir.

TLS / sertifika

Alan (UI)	Anahtar	Varsayılan	Açıklama
Abonelik sertifikasının genel anahtar dosyası yolu	subCertFile	boş	Sertifika dosyasının tam yolu (.crt / fullchain). İpucu: «/' ile başlayan tam yolu girin».
Abonelik sertifikasının özel anahtar dosyası yolu	subKeyFile	boş	Özel anahtar dosyasının tam yolu. İpucu: «/' ile başlayan tam yolu girin».

Her iki yol da tanımlanmış ve sertifika başarıyla yüklenebiliyorsa, abonelik sunucusu **HTTPS** üzerinden çalışır. Alanlar boşsa ya da sertifika okunamazsa sunucu **HTTP**'ye geri düşer (hata günlüğe yazılır). Geçerli TLS'in varlığı, taban URL'nin oluşturulmasını da etkiler: TLS ile port 443 ve TLS olmadan port 80 kullanıldığında bağlantıdaki port numarası atlanır.

Güncelleme aralığı

Alan (UI)	Anahtar	Varsayılan	Açıklama
Abonelik güncelleme aralıkları	subUpdates	12	İstemci uygulamasının aboneliği ne sıklıkla (saat cinsinden) yeniden sorgulaması gerektiği. İpucu: «İstemci uygulamasındaki güncellemeler arasındaki aralık (saat)». 3.5.0'dan itibaren alan 0-525600 kabul eder ve aralığı gösterir (önceki 168 arayüz sınırı sunucuyla uyumuyor ve 2.x'ten yükseltme sonrasında hiçbir ayarın kaydedilememesine yol açıyordu).

Değer, istemciye `Profile-Update-Interval` HTTP başlığı ile iletilir; modern istemciler bunu yapılandırmanın otomatik güncelleme periyodu olarak kullanır.

Yanıt formatı ve bilgisi

Alan (UI)	Anahtar	Varsayılan	Açıklama
Kodla	subEncrypt	true	İpucu: «Abonelikteki döndürülen yapılandırmaları şifrele». Teknik olarak bu şifreleme değil, tüm normal abonelik gövdesinin Base64 kodlamasıdır (çoğu istemcinin beklediği format). Devre dışı bırakıldığında bağlantılar düz metin olarak, her satırda bir tane gelir.

Alan (UI)	Anahtar	Varsayılan	Açıklama
Kullanım bilgisini göster	subShowInfo	true	İpucu: «Yapılandırma adından sonra kalan trafiği ve bitiş tarihini görüntüle». Etkinleştirildiğinde, her yapılandırmanın adına (remark) kalan trafik işaretçileri () ve geçerlilik süresi (ör. 5D, 3H) eklenir; süresi dolmuş/erişilemeyen istemci için N/A gösterilir.
Ada E-postayı dahil et	subEmailInRemark	true	İpucu: «İstemci e-postasını abonelik profilinin adına dahil et.». İstemcinin e-postasını profil adına ekler.

Açıklama şablonu (Remark Template)

Abonelikteki her yapılandırmanın görünen adı (remark), «**Açıklama şablonu**» alanı (remarkTemplate) aracılığıyla – abonelik ayarlarının «**Bilgi**» sekmesinde – **açıklama şablonu** olarak belirlenir. Önceki açıklama modeli yapıcısı (inbound/e-posta/harici proxy parçaları ve ayırıcı sembol için ayrı seçimler) arayüzden kaldırılmıştır; artık istediğiniz ad formatını yazarak içine değişkenler ekleyebilirsiniz. Varsayılan değer `{{INBOUND}}-{{EMAIL}}|{{TRAFFIC_LEFT}}|{{DAYS_LEFT}}D` şeklindedir (yani varsayılan olarak profil adı istemcinin e-postasını içerir). Alan boş bırakılırsa eski (arayüz üzerinden yapılandırılmayan) açıklama modeli kullanılır.

Değişkenler **Client**, **Traffic** ve **Time & status** bölümlerine göre gruplandırılmış ve alanın yanında fareyle üzerine gelindiğinde ipucu gösteren tıklanabilir `{{VAR}}` çipleri olarak sunulur; tıklamak simgeyi şablona ekler, canlı önizleme mevcuttur. Her değişken, abonelik oluşturulurken belirli bir istemci için ayrı ayrı yerleştirilir. Tek parantez biçimi de desteklenir (`{DATA_LEFT}` , `{EXPIRE_DATE}` , `{PROTOCOL}` , `{TRANSPORT}` vb.) – panel bunu dahili `{{...}}` biçimine kendiliğinden dönüştürür.

Kullanılabilir değişkenler:

- **İstemci tanımlaması:** `{{EMAIL}}` (eş anlamlısı – `{{USERNAME}}`), `{{INBOUND}}` (inbound'un kendi açıklaması), `{{HOST}}` (host açıklaması), `{{ID}}` (UUID), `{{SHORT_ID}}` (UUID'nin ilk 8 karakteri), `{{SUB_ID}}` , `{{COMMENT}}` , `{{TELEGRAM_ID}}` , `{{PROTOCOL}}` , `{{TRANSPORT}}` } .
- **Trafik:** `{{TRAFFIC_USED}}` , `{{TRAFFIC_LEFT}}` , `{{TRAFFIC_TOTAL}}` (ve tam bayt cinsinden `*_BYTES` varyantları), `{{UP}}` , `{{DOWN}}` , `{{USAGE_PERCENTAGE}}` } .
- **Süre ve durum:** `{{DAYS_LEFT}}` , `{{TIME_LEFT}}` , `{{EXPIRE_DATE}}` (YYYY-AA-GG), `{{JALALI_EXPIRE_DATE}}` (Jalali takvimine göre tarih), `{{EXPIRE_UNIX}}` , `{{CREATED_UNIX}}` , `{{RESET_DAYS}}` , `{{STATUS}}` (active / expired / disabled / depleted), `{{STATUS_EMOJI}}` } .
- **Bağlantı (Connection):** `{{PROTOCOL}}` – protokol (VLESS, VMess, Trojan vb.), `{{TRANSPORT}}` – taşıma ağı (tcp, ws, grpc vb.), `{{SECURITY}}` – taşıma güvenliği (TLS, REALITY, NONE; büyük harfle gösterilir). Trafik tüketimi ve süre değişkenleri gibi bu üç değişken de yalnızca abonelik gövdesinde çalışır ve paneldeki görüntülenen bağlantılardaki (QR/«Bilgi») ve abonelik bilgi sayfasındaki açıklamalardan otomatik olarak çıkarılır.

Şablon, | karakteriyle bölümlere ayrılabilir. Bir değişkenin «sınırsız» değer ürettiği bölüm (ör. sınırsız istemcide `{{TRAFFIC_LEFT}}` veya `{{DAYS_LEFT}}`) – ∞ döndürdüğünde – otomatik olarak gizlenir. Ayrıca trafik tüketimi ve süre bloğu her yapılandırmada tekrarlanmaması için istemcinin yalnızca ilk bağlantısında bir kez gösterilir.

Örnek. `{{EMAIL}} | {{TRAFFIC_LEFT}} | {{DAYS_LEFT}}D` şablonu, 42 GB kalan ve 7 günü olan bir istemcide `ivan@vpn 42.00GB 7D` adını üretirken, sınırsız istemcide yalnızca `ivan@vpn` görünür (∞ döndüren bölümler atlanır).

3.4.2 sürümünden itibaren `{{EMAIL}}` değişkeni (ve eş anlamlısı `{{USERNAME}}`) abonelik gövdesinde istemcinin yalnızca **ilk** bağlantısında gösterilir – tıpkı kalan trafik/süre bloğu gibi; aynı istemcinin diğer bağlantılarında atlanır.

Panelde görüntülenen bağlantılarda (QR kodu ve «İstemciler» sayfasındaki «Bilgi» pencereleri) ve abonelik bilgi sayfasında, istemcinin e-postası profil adında bulunur: host tanımlanmışsa «inbound-host-e-posta» biçiminde, host yoksa «inbound-e-posta» biçiminde. Trafik ve süre bilgisi (ve «Bağlantı» grubunun değişkenleri) bu görüntülenen adlara eklenmez – bunlar yalnızca VPN istemcisinin aldığı abonelik gövdesinde çalışır.

İstemcinin trafik istatistiği satırı, inbound'un silinip yeniden oluşturulmasından sonra «yetim» kaldıysa, `{{TRAFFIC_USED}}` değişkeni (ve diğer tüketim göstergeleri) artık `0.00B` göstermez: panel istemcinin e-postasına göre istatistiği ek olarak arar ve doğru kullanılan trafiği yerleştirir. | Açıklama şablonu | `remarkTemplate | {{INBOUND}} | {{TRAFFIC_LEFT}} | {{DAYS_LEFT}}D` | Her yapılandırmanın görünen adının (remark) `{{VAR}}` değişken yerleştirmeli serbest şablonu. Abonelik oluşturulurken her istemci için ayrı ayrı uygulanır. Önceki «açıklama modeli» yapıcısı (inbound/e-posta/harici proxy ve ayırıcı seçimi) arayüzden kaldırılmış olup yalnızca alan boş bırakılırsa yedek olarak kullanılır. Ayrıntılar için aşağıdaki «Açıklama şablonu (Remark Template)» bölümüne bakın. |

Profil meta verileri (yanıt başlıkları)

Bu dizeler, istemciye HTTP yanıt başlıklarıyla iletilir ve VPN istemcisinde profil meta verisi olarak görüntülenir. Hepsi varsayılan olarak boştur.

Alan (UI)	Anahtar	Başlık	Açıklama
Abonelik başlığı	<code>subTitle</code>	<code>Profile-Title</code> (Base64'te)	«VPN istemcisinde istemcinin gördüğü abonelik adı». Clash için ayrıca <code>Content-Disposition</code> aracılığıyla içe aktarılan profil adı olarak kullanılır.
Destek URL'si	<code>subSupportUrl</code>	<code>Support-Url</code>	«VPN istemcisinde görüntülenen teknik destek bağlantısı».
Profil URL'si	<code>subProfileUrl</code>	<code>Profile-Web-Page-Url</code>	«VPN istemcisinde görüntülenen web sitenizin bağlantısı». Tanımlanmamışsa gerçek abonelik isteği URL'si kullanılır.
Duyuru	<code>subAnnounce</code>	<code>Announce</code> (Base64'te)	«VPN istemcisinde görüntülenen duyuru metni». 3.5.0'dan itibaren metin (boş değilse) abonelik bilgi sayfasının üstünde bir bilgi afişi olarak da gösterilir; özel şablonlarda <code>announce</code> değişkeni kullanılabilir.

Buna ek olarak her yanıtta, istemcinin toplu trafik verileri olan `upload`, `download`, `total` ve `expire` (saniye cinsinden bitiş anı) bilgilerini içeren `Subscription-Userinfo` başlığı iletilir. İstemci bunu kalan trafik ve geçerlilik süresini göstermek için kullanır.

Yönlendirme (yalnızca Happ istemcisi için)

Alan (UI)	Anahtar	Varsayılan	Açıklama
Yönlendirmeyi etkinleştir	<code>subEnableRouting</code>	<code>false</code>	«VPN istemcisinde yönlendirmeyi etkinleştirmek için genel ayar. (Yalnızca Happ için)». <code>Routing-Enable</code> başlığı ile iletilir.
Yönlendirme kuralları	<code>subRoutingRules</code>	boş	«VPN istemcisi için genel yönlendirme kuralları. (Yalnızca Happ için)». <code>Routing</code> başlığı ile iletilir.

| Sunucu ayarlarını gizle | `subHideSettings` | `false` | «Abonelikteki sunucu ayarlarını gizle (yalnızca Happ için)». Etkinleştirildiğinde Happ istemcisinde sunucu parametrelerini görüntüleme ve değiştirme özelliği gizlenir. Seçenek yalnızca Happ istemcisi için geçerlidir. |

Incy yönlendirmesi (yalnızca Incy istemcisi için)

Incy VPN istemcisi için abonelik ayarlarında ayrı bir «**Incy**» sekmesi bulunur; burada iki alan vardır: «**Yönlendirmeyi etkinleştir**» geçiş düğmesi (`subIncyEnableRouting` , varsayılan olarak devre dışı) ve `incy://routing/onadd/<base64>` formatında «**Yönlendirme kuralları**» metin alanı (`subIncyRoutingRules`). Yönlendirme etkin ve alan doluysa bu dize, abonelik gövdesine (ham format) ayrı bir satır olarak eklenir – böylece yönlendirme profili Happ istemcisinin `Routing` başlığıyla çakışmadan Incy istemcisine iletilir. Ayarlar yalnızca Incy istemcisi için geçerlidir.

Ters proxy URI

Alan (UI)	Anahtar	Varsayılan	Açıklama
Ters proxy URI	<code>subURI</code>	boş	«Proxy sunucularının arkasında kullanmak için abonelik URL adresinin taban URI'sini değiştir».

Alan boşsa, taban adresini panel abonelik alanından ve portundan (TLS dikkate alınarak) kendisi oluşturur. Abonelik farklı bir alan adı veya yolda harici bir ters proxy/CDN üzerinden sunuluyorsa bu alana son taban URI girilir ve tüm bağlantılar bu adresten oluşturulur. JSON (`subJsonURI`) ve Clash (`subClashURI`) için de benzer ayrı alanlar mevcuttur.

Yalnızca genel `subURI` tanımlanmış, JSON ve Clash için ayrı alanlar boş bırakılmışsa, bu formatlara ait bağlantılar abonelik sayfasında port ve `http` yerine `subURI` 'den şema ve host devralır – böylece ters proxy adresiyle eşleşir.

Örnek: ters proxy arkasındaki abonelik. Abonelik `2096` portunda dinliyor ancak dışarıdan nginx/CDN aracılığıyla `https://cfg.example.com/u/` üzerinden erişilebilir durumda. Yanıttaki bağlantıların iç `alan-adı:2096` yerine dış adresten oluşturulması için «Reverse proxy URI» alanına son taban URI girilir:

Reverse proxy URI: `https://cfg.example.com/u`

Bu durumda son bağlantı `https://cfg.example.com/u/ivan2025` biçiminde olur. JSON ve Clash formatları için gerektiğinde `subJsonURI` ve `subClashURI` ayrı alanları aynı şekilde doldurulur.

10.3. Çıktı formatları

Abonelik, her biri ayrı ayrı etkinleştirip devre dışı bırakılabilen kendi endpoint'ine sahip üç bağımsız formatta sunulabilir.

Çıktıdaki sunucu adresi ve düğümler

Abonelik bağlantılarındaki sunucu adresi, paneldeki normal bağlantılar ve QR kodlarıyla aynı bağlantı adresi stratejisine göre yerleştirilir: «listen» – yönlendirilebilir bağlanma adresi, «custom» – kullanıcı tanımlı adres (`shareAddr`), «node» (varsayılan) – düğüm adresi. Açıkça belirtilmiş bir stratejisi olmayan inbound'lar için abonelik çıktısı değişmez. Bu, belirli bir genel IP'ye bağlı bir düğüm inbound'unun istemcilere erişilebilir bir adres sunmasına olanak tanır. Strateji ham, JSON ve Clash formatlarına uygulanır.

Düğüm adı (Node) abonelikteki profil adına (remark) eklenmez: istemci uygulamasında yalnızca yönetici tarafından belirlenen inbound açıklaması gösterilir, @düğüm-adı gibi bir dahili sonek olmadan. Çok düğümlü bir abonelikte aynı adlı girişleri birbirinden ayırt etmek için farklı açıklamalar atayın ya da kendi Remark'larına sahip yönetilen hostları (Hosts) kullanın.

Düğümler arasındaki senkronizasyon bozukluğu nedeniyle aynı istemci dahili JSON inbound'da iki kez yer alıyorsa, abonelik çıktısı üç formatta da e-postaya göre bu kopyaları otomatik olarak kaldırır; bu nedenle çıktıda tekrarlanan profiller görünmez.

Yönetilen hostlar (Hosts)

Hosts bölümü (kenar çubuğu menüsü; Toplam/Etkin/Devre Dışı sayısı ve listeye özet sayfa), abonelik bağlantıları için adres geçersiz kılmaları tanımlar. Her inbound için bir veya daha fazla **host** – istemciye iletilen abonelik bağlantılarında inbound'un kendi adresi, portu ve TLS parametrelerinin **yerine geçen** endpoint'ler – eklenebilir. Bu, inbound'u değiştirmeksizin trafiği CDN veya röle üzerinden dağıtmak için kullanışlıdır.

3.5.0 sürümünden itibaren host bir «grup»tur: tek kayıt aynı anda **birden fazla inbound** ve **birden fazla adres** kapsar. Liste işlemleri (etkinleştirme/devre dışı bırakma/silme/yer değiştirme) ve API gruplarla çalışır (dize türünde `groupId`); toplu `POST /panel/api/hosts/bulk/add` uç noktası eklendi (`{ "inboundIds": [...], "hosts": [...], ... }`); `list / get / update / del / setEnable` ise grup nesneleri üzerinde işlem yapar.

Her host (grup) için şunlar belirlenir:

- **Remark** ve açıklama (Description), **Inbounds'a** bağlama (aramalı çoklu seçim; en az bir), **Enable** geçiş düğmesi ve düğümlere atama (**Nodes**).
- **Address** – 3.5.0'dan itibaren bu bir **adres listesidir** (etiket olarak giriş; ayırıcılar – virgül, noktalı virgül, boşluk; yer tutucu `cdn.example.com, cdn2.example.com:443`). Her kaydın kendi gömülü `:port` 'u olabilir (köşeli parantezli IPv6 dahil, `[:1]:443`); açılır öneri, diğer hostların zaten kullandığı adresleri sunar. Boş liste – inbound'un kendi adresi devralınır (listede bu, turuncu **Inherits** etiketidir). **Port** (`0` – inbound portu devralınır), gömülü portu olmayan kayıtlar için varsayılan port görevi görür; **Tags** (yalnızca RAW aboneliğinde dikkate alınır).
- **Security** sekmesi – SNI, parmak izi (fingerprint), ALPN, sertifika sabitleme (pinned-cert), `allowInsecure` ve ECH ile birlikte `same / tls / none / reality` .
- **Advanced** sekmesi – Host başlığı, Yol, VLESS yolu, Mux, Sockopt, Final Mask ve hostu ayrı abonelik formatlarından (raw / json / clash) dışlama. 3.5.0'dan itibaren **hostun Final Mask'ı raw bağlantılara da girer**

(`fm=` ; önceden yalnızca JSON/Clash'e): hostun TCP/UDP maskeleri inbound'un maskelerine eklenir, hostun QUIC parametreleri yalnızca inbound'da yoksa alınır. **Allow insecure** anahtarı artık **Hysteria/Hysteria2** için geçerlidir (bağlantıda `insecure=1` , Clash'te `skip-cert-verify: true`).

- **Clash (mihomo)** sekmesi – IP sürümü, Mihomo X25519, host karıştırma (Shuffle host).

Listede **Endpoint** sütunu adresleri çipler halinde gösterir (ilki görünür, geri kalanlar «+N» açılır kutusundadır); **Inbounds** sütunu ise protokole göre renklendirilmiş inbound çiplerini gösterir. 3.5.0'dan itibaren hostlar inbound kapsamında değil, **genel olarak** sıralanır (sıralama düzenine, ardından açıklamaya göre); toplu etkinleştirme/devre dışı bırakma/silme korunmuştur. Yönetilen hostlar eski External Proxy dizisinin yerini almaktadır.

VLESS rotası (VLESS route). 3.4.2 sürümünden itibaren bu tek bir `0-65535` sayıdır (port listesi değil; ipucu – «UUID'ye gömülen tek bir VLESS route değeri (0-65535), örneğin 443; boş – yok», yer tutucu `443`). Belirtilen değer, oluşturulan her aboneliğin UUID'sine gerçekten «gömülür» (raw / JSON / Clash): Xray, UUID'nin 6-7. baytlarını okur ve kimlik doğrulamadan önce maskeler, bu nedenle istemci yine de eşleşir. Boş veya geçersiz değer UUID'yi değiştirmez.

Normal bağlantılar (SUB) – Base64 / düz metin

Temel format, endpoint `subPath` (varsayılan `/sub/`). Her zaman etkin (abonelik genel olarak etkinleştirildiğinde). Xray bağlantılarının listesini döndürür (`vless://` , `vmess://` , `trojan://` , `ss://` vb.) – her satırda bir tane. «Kodla» (`subEncrypt`) seçeneği etkinleştirildiğinde tüm liste Base64'e kodlanır; devre dışıysa düz metin olarak döndürülür. Bu format neredeyse tüm istemciler tarafından anlaşılır (v2rayNG, V2RayTun, Sing-box, NekoBox, Streisand, Shadowrocket, Happ vb.).

Örnek: «Kodla» devre dışıyken yanıt gövdesi. `subEncrypt = false` iken `/sub/` endpoint'i düz metin döndürür – her satırda bir bağlantı:

```
vless://3c8f...@a.example.com:443?security=reality&...#srvA-ivan
trojan://p4ss@b.example.com:443?security=tls&...#srvB-ivan
```

`subEncrypt = true` (varsayılan) olduğunda aynı liste tamamen Base64'e kodlanır ve tek bir dize olarak döndürülür – çoğu istemci tam olarak bu biçimi bekler.

JSON aboneliği (sing-box ve uyumlu istemciler)

Endpoint `subJsonPath` (varsayılan `/json/`), ayrı bir onay kutusuyla etkinleştirilir.

Alan (UI)	Anahtar	Varsayılan	Açıklama
JSON aboneliği	<code>subJsonEnable</code>	<code>false</code>	«JSON abonelik endpoint'ini bağımsız olarak etkinleştir/devre dışı bırak.».

Tam JSON yapılandırması döndürür (sing-box ve türev istemcilerin anlayabileceği format – Podkop, OpenWRT sing-box, Karing, NekoBox). 3.5.0'dan itibaren **yerel WireGuard inbound'larının** istemcileri de JSON aboneliğine girer (`secretKey` , `address` , `publicKey` / `endpoint` / `preSharedKey` / `keepAlive` / `allowedIPs` içeren

peers[] , mtu) – önceden sessizce atlanıyorlardı. Bu format için ek parametreler mevcuttur (subFormats sekmesi):

- **Mux** (subJsonMux , varsayılan boş) – JSON aboneliğinin her akışına eklenen çoğullama (Mux) JSON ayarları. «Tek bir bağlantıda birden fazla bağımsız veri akışının iletimi.».
- **Final Mask** (subJsonFinalMask , varsayılan boş) – «Her JSON abonelik akışına eklenen xray finalmask maskeleri (TCP/UDP) ve QUIC ayarları. İstemcide güncel bir xray sürümü gerektirir.» Alt alanlar aracılığıyla yapılandırılır: «Paketler» (packets), «Uzunluk» (length), «Aralık» (interval), «Maks. bölünme» (maxSplit), «Gürültüler» (noises : «Tür»/ type , «Paket»/ packet , «Gecikme (ms)»/ delayMs , «Uygula»/ applyTo , «+ Gürültü» düğmesi), ayrıca «Eşzamanlılık» (concurrency), «xudp eşzamanlılığı» (xudpConcurrency) ve «xudp UDP 443» (xudpUdp443).
- **Yönlendirme kuralları** (subJsonRules , varsayılan boş) – JSON yapılandırmasına eklenen genel kurallar.

Clash / Mihomo aboneliği (YAML)

Endpoint subClashPath (varsayılan /clash/), ayrı bir onay kutusuyla etkinleştirilir.

Alan (UI)	Anahtar	Varsayılan	Açıklama
Clash / Mihomo aboneliği	subClashEnable	false	Clash ve Mihomo istemcileri için YAML yapılandırması oluşturmayı etkinleştirir.
Yönlendirmeyi etkinleştir	subClashEnableRouting	false	«Oluşturulan YAML aboneliklerine genel Clash/ Mihomo yönlendirme kuralları ekle.».
Genel yönlendirme kuralları	subClashRules	boş	«Her YAML aboneliğinin başında MATCH,PROXY'den önce eklenen Clash/Mihomo kuralları.».

Yanıt application/yaml; charset=utf-8 türüyle döndürülür. «Abonelik başlığı» (subTitle) tanımlanmışsa aynı zamanda Content-Disposition başlığında (attachment; filename*=UTF-8'<başlık>) iletilir; böylece Clash istemcisi içe aktarılan profili bu isimle adlandırır.

Oluşturulan bağlantıların ve YAML'ın formatı, modern istemciler için güncel tutulur: Shadowsocks-2022 (SS2022) artık userinfo'yu Base64 ile kodlamaz; http gizleme ile Shadowsocks bağlantıları obfs-local eklentisiyle SIP002 formatında döndürülür; Clash/Mihomo abonelikleri için XHTTP alanlarının tam seti uygulanır. 3.5.0'dan itibaren **yerel WireGuard inbound'larının** istemcileri de Clash/Mihomo aboneliğine girer (private-key , public-key , pre-shared-key , persistent-keepalive , ip / ipv6 , mtu , dns alanları) – önceden sessizce atlanıyorlardı. Bunlar için ayrı bir ayar gerekmez – bağlantılar yalnızca istemciler tarafından daha doğru tanınır.

Not: Bu derleme yalnızca üç formatı destekler – normal bağlantılar (Base64/metin), JSON (sing-box uyumlu) ve Clash/Mihomo (YAML). Abonelik sunucusunda ayrı bir Outline formatı yoktur.

10.4. Abonelik bilgi sayfası ve QR kodları

Üç abonelik bağlantısından herhangi biri – raw,JSON veya Clash – tarayıcıda açılırsa (veya URL'ye açıkça ?html=1 ya da ?view=html parametresi eklenirse ya da Accept: text/html başlığı gönderilirse), sunucu «ham» yanıt yerine görsel bir **abonelik bilgi sayfası** («Abonelik Bilgisi») döndürür. 3.5.0'dan önce bu yalnızca

ana `/sub/` bağlantısında çalışıyor, `/json/` ve `/clash/` ise tarayıcıya ham JSON/YAML döndürüyordu. VPN istemcileri HTML talep etmediğinden makine tarafından okunabilir yanıt almaya devam eder.

Sayfa (Vite ile derlenen tek sayfalık uygulama) şunları gösterir:

- **Abonelik bilgisi** (Descriptions bloğu):
 - «Abonelik ID» – `subId` değeri;
 - «Durum» – «Aktif», «Aktif değil» veya «Sınırsız». İstemci devre dışıysa, trafik limitini aştıysa veya süresi dolduysa durum «aktif değil» olarak ayarlanır;
 - «İndirilen» ve «Yüklenen» – trafik hacimleri;
 - «Toplam limit» – trafik limiti veya sınırsızsa ∞ ;
 - «Geçerlilik süresi» – bitiş tarihi veya «Süresiz»;
 - kalan trafik ve son çevrimiçi zamanı.
 - Tarihler, panel ayarına göre («Calendar Type» / `datepicker`, varsayılan `gregorian`) Gregoryen veya Jalali takvimiyle görüntülenir.
- **Abonelik bağlantıları**: etkinleştirilen her format için – renkli etiket (yeşil **SUB**, mor **JSON**, altın **CLASH**), kopyalama düğmesi ve **QR kodu** düğmesi (açılır pencere, 240 piksel boyut) olan ayrı bir satır. JSON ve CLASH satırı yalnızca ilgili format ayarlarda etkinleştirilmişse görünür.
- **Bireysel bağlantılar** («Bağlantıyı kopyala»): aboneliğe dahil olan ayrı yapılandırmaların tam listesi; her biri protokol etiketi, kopyalama düğmesi ve QR koduyla (kuantum sonrası bağlantılar için QR oluşturulmaz).
- **«Tüm yapılandırmaları kopyala» düğmesi** (bireysel bağlantılar listesinin üzerinde): tek bir tıklamayla tüm yapılandırma bağlantılarını panoya kopyalar (her biri yeni satırda), tek tek kopyalama gereksinimi ortadan kalkar; işlem tamamlandığında «Tüm yapılandırmalar kopyalandı» bildirimi gösterilir.
- **Uygulamalara hızlı içe aktarma düğmeleri** (platforma göre açılır menüler): Android için – `v2box`, `v2rayNG` (derin bağlantı `v2rayng://install-config?url=...`), `Sing-box`, `V2RayTun`, `NPV Tunnel`, `Happ` (`happ://add/...`), `Incy` (`incy://add/...`); iOS için – `Shadowrocket` (`flag=shadowrocket` parametresiyle), `v2box` (`v2box://install-sub?url=...&name=...`), `Streisand` (`streisand://import/...`), `V2RayTun`, `NPV Tunnel`, `Happ`, `Incy`. Bu düğmeler ya abonelik adresi önceden doldurulmuş şekilde ilgili uygulamanın derin bağlantısını açar ya da bağlantıyı panoya kopyalar.

Bilgi sayfası, istemcinin trafik ve geçerlilik süresi verilerini her zaman güncel görmesi için önbellek engelleme başlıklarıyla (`Cache-Control: no-cache`) döndürülür.

10.5. Abonelik sayfası için özel şablonlar

3.3.0 sürümünden itibaren standart abonelik açılış sayfasını kendi HTML şablonunuzla değiştirebilirsiniz.

Varsayılan olarak abonelik adresinde yerleşik sayfa sunulur; ancak kendi şablonunuzu içeren bir dizin belirtirseniz panel bunu oluşturur ve içine istemcinin güncel verilerini (trafik, geçerlilik süresi, bağlantılar vb.) yerleştirir.

Önemli: panel hazır şablon **sunmaz** – kendi temanızı kendiniz oluşturmanız gerekir. Oluşturma talimatları ve kullanılabilir değişkenlerin listesi [docs/custom-subscription-templates.md](https://github.com/3X-UI/3X-UI/blob/main/docs/custom-subscription-templates.md) dosyasındadır.

Nereden etkinleştirilir

Tema dizini panel ayarlarından belirlenir:

Ayarlar → **Abonelik** → «**Abonelik bilgisi**» bölümü, «**Abonelik tema dizini**» alanı (`subThemeDir`).

Arayüzdeki alan açıklaması: «Abonelik sayfası için özel şablon (index.html/sub.html) içeren klasörün mutlak yolu (ör. /etc/3x-ui/sub_templates/my-theme/). Varsayılan sayfayı kullanmak için boş bırakın.»

Aynı bölümde, şablonda erişilebilen değerlere sahip ilgili ayarlar da bulunur:

«Abonelik tema dizini» alanının açıklamasında, kendi abonelik sayfası tasarım şablonlarını oluşturma belgelerine yönlendiren bir «**Şablon kılavuzu** ↗» bağlantısı vardır.

- «**Abonelik başlığı**» (`subTitle`) – istemcinin gördüğü ad;
- «**Destek URL'si**» (`subSupportUrl`) – teknik destek bağlantısı.

Ayar parametresi

Parametre	Varsayılan değer	Amaç
<code>subThemeDir</code>	"" (boş)	HTML şablonunuzu içeren dizinin mutlak yolu. Boş = yerleşik varsayılan sayfa.

Kendi şablonunuzu nasıl uygularsınız

1. Sunucuda tema için bir klasör oluşturun (herhangi bir yerde), ör. `/etc/3x-ui/sub_templates/my-theme/`.
2. İçine `index.html` veya `sub.html` adında bir HTML dosyası koyun.

Örnek: tema yolu. Sunucudaki son düzen ve ayarlardaki alan değeri:

```
/etc/3x-ui/sub_templates/my-theme/
└─ index.html      (veya sub.html – önceliği vardır)
```

```
Ayarlar → Abonelik → «Abonelik tema dizini»:
/etc/3x-ui/sub_templates/my-theme/
```

Yol **mutlak** olmalıdır (/ ile başlamalıdır). Klasörde ne `index.html` ne de `sub.html` yoksa panel yerleşik sayfayı döndürür. 3. Panelde **Ayarlar** → **Abonelik** kısmını açın ve bu klasörün **mutlak** yolunu «Abonelik tema dizini» alanına girin. 4. Ayarları kaydedin.

Dosya seçimi ve oluşturma davranışı:

- Dizinde `sub.html` varsa bu dosya kullanılır; yoksa `index.html` alınır. Yani `sub.html`, `index.html` 'ye göre önceliklidir.
- Şablon, Go'nun standart `html/template` motoru ile oluşturulur.
- Ayrıştırılan şablon **önbelleğe alınır** ve yalnızca dosyanın değiştirilme zamanı değiştiğinde diskten yeniden okunur. Bu nedenle şablon değişiklikleri panel yeniden başlatılmadan uygulanır, ancak her istekte okuma/ayrıştırma ek yükü olmaz.

- Yanıt tamamen arabelleğe alınır ve ancak sonra istemciye gönderilir: şablon yürütme sırasında başarısız olursa kısmen oluşturulmuş (bozuk) bir sayfa kullanıcıya ulaşmaz.

Varsayılan davranış ve geri dönüş (fallback)

- Alan boş → yerleşik SPA sayfası döndürülür (veriler `window.__SUB_PAGE_DATA__` 'ya eklenir).
- Yol mevcut değil veya izin değil → varsayılan sayfa kullanılır.
- Dizinde ne `index.html` ne de `sub.html` var → günlüğe «subThemeDir set but no usable template found» uyarısı yazılır, varsayılan sayfa döndürülür.
- Şablon dosyası var ancak ayrıştırılamıyor → günlüğe «custom template parse failed» hatası yazılır, varsayılan sayfa döndürülür.
- Şablon yürütme hatası → günlüğe «custom template execution failed» yazılır, varsayılan sayfa döndürülür.

Yani özel şablondaki herhangi bir sorun aboneliği «bozmaz» – panel her zaman yerleşik sayfaya geri düşer. Tüm abonelik sayfaları (hem özel hem de standart), istemcilerin her zaman güncel trafik ve süre verilerini alması için önbellek engelleme başlıklarıyla (`Cache-Control: no-cache, no-store, must-revalidate`) döndürülür.

Kullanılabilir şablon değişkenleri

Şablon bağlamına abonelik istemcisinin veri seti iletilir. Erişim `{{ .değişkenAdı }}` sözdizimi kullanılarak sağlanır:

Değişken	Tür	Açıklama
<code>{{ .sId }}</code>	string	Abonelik ID (UUID).
<code>{{ .enabled }}</code>	bool	İstemcinin/aboneliğin etkin olup olmadığı.
<code>{{ .download }}</code>	string	Biçimlendirilmiş indirme hacmi (ör. «2.5 GB»).
<code>{{ .upload }}</code>	string	Biçimlendirilmiş yükleme hacmi.
<code>{{ .total }}</code>	string	Biçimlendirilmiş toplam trafik limiti.
<code>{{ .used }}</code>	string	Biçimlendirilmiş kullanılan trafik (indirme + yükleme).
<code>{{ .remained }}</code>	string	Biçimlendirilmiş kalan trafik.
<code>{{ .expire }}</code>	int64	Geçerlilik süresi – saniye cinsinden Unix zamanı (<code>0</code> = süresiz). <code>JS Date</code> için 1000 ile çarpın.
<code>{{ .lastOnline }}</code>	int64	Son çevrimiçi zamanı – milisaniye cinsinden Unix zamanı (<code>0</code> = hiç bağlanmadı).
<code>{{ .downloadByte }}</code>	int64	Tam bayt cinsinden indirme.
<code>{{ .uploadByte }}</code>	int64	Tam bayt cinsinden yükleme.
<code>{{ .totalByte }}</code>	int64	Tam bayt cinsinden toplam limit.
<code>{{ .subUrl }}</code>	string	Abonelik sayfasının URL'si.
<code>{{ .subJsonUrl }}</code>	string	JSON abonelik yapılandırmasının URL'si.
<code>{{ .subClashUrl }}</code>	string	Clash/Mihomo yapılandırmasının URL'si.

Değişken	Tür	Açıklama
<code>{{ .subTitle }}</code>	string	Ayarlardan abonelik başlığı (boş olabilir).
<code>{{ .subSupportUrl }}</code>	string	Ayarlardan destek URL'si (boş olabilir).
<code>{{ .links }}</code>	[]string	Yapılandırma dizeleri listesi (VMess, VLESS vb.). Döngü: <code>{{ range .links }}</code> ... <code>{{ end }}</code> .
<code>{{ .emails }}</code>	[]string	Abonelik ilişkili e-posta listesi.
<code>{{ .datepicker }}</code>	string	Panelin geçerli takvim formatı: <code>gregorian</code> veya <code>jalali</code> («Takvim Türü» ayarından alınır; boşsa <code>gregorian</code>).

Değişkenlerin bir kısmını kullanan minimal şablon gövdesi örneği:

```
<h1>{{ .subTitle }}</h1>
<p>Kullanılan: {{ .used }} / {{ .total }} (kalan {{ .remained }})</p>
{{ range .links }}<div>{{ . }}</div>{{ end }}
```

Örnek: `expire` alanından **bitiş tarihi**. `{{ .expire }}` alanı **saniye** cinsinden Unix zamanıdır; bu nedenle JavaScript'te 1000 ile çarpılır; `0` değeri «süresiz» anlamına gelir:

```
<script>
  var exp = {{ .expire }};
  document.write(exp === 0
    ? 'Süresiz'
    : 'Bitiş: ' + new Date(exp * 1000).toLocaleDateString());
</script>
```

Not: `{{ .lastOnline }}` zaten **milisaniye** cinsinden verilir – 1000 ile çarpmaya gerek yoktur.

11. Xray: yönlendirme, outbounds, DNS ve uzantılar

«**Xray Ayarları**» bölümü, panelin çekirdek için nihai `config.json` 'ı oluşturduğu Xray-core yapılandırma şablonunun düzenleyicisidir. Şablon bölümünün açıklaması: «*Şablona dayanılarak Xray yapılandırma dosyası oluşturulur.*» inbounds'ların aksine (bunlar ayrı veritabanında saklanır ve yapılandırma derlenirken şablona eklenir), geri kalan her şey – günlükler, yönlendirme, outbounds, DNS, politika, istatistik – burada tanımlanır.

Önemli: şablon değeri veritabanında `xrayTemplateConfig` anahtarı altında saklanır. Kaydedildiğinde panel, çeşitli otomatik dönüşümlerin üzerinden geçer (bkz. 11.11). Sözdizimsel olarak hatalı JSON, «*xray template config invalid*» hatasıyla reddedilir.

Menüdeki konumu: «Outbounds» ve «Yönlendirme»

«**Outbounds**» ve «**Yönlendirme**» (Routing) – her birinin kendi adresi olan ayrı kenar çubuğu menü öğeleridir (/outbound ve /routing). Bu sayfalara doğrudan bağlantılar ve sayfa yenileme beklendiği gibi çalışır. «**Xray Yapılandırmaları**» alt menüsünde yalnızca şunlar kalır: Temel, Yük Dengeleyici, DNS ve Gelişmiş Şablon. Aşağıdaki açıklamada 11.3 ve 11.4 bölümleri sırasıyla «Yönlendirme» ve «Outbounds» sayfalarına karşılık gelir.

11.1. Düzenleyici yapısı: sekmeler/modlar

Düzenleyici, şablonun birkaç görüntüleme modunu sunar (JSON bölümlerine göre filtreler):

Mod	Neler gösterilir
Temel	Temel bölümler (Günlük, temel yönlendirme, ana ayarlar)
Gelişmiş Şablon	Tam Xray JSON şablonu
Tümü	Tüm bölümler aynı anda

Düzenleyicinin içindeki mantıksal ayar grupları:

- **Ana Ayarlar** (açıklama: «*Bu parametreler genel ayarları açıklar*»).
- **Günlük** (bkz. 11.10).
- **Temel Bağlantılar**: engellemeler ve doğrudan yollar.
- **Inbounds** (açıklama: «*Belirli istemcilerin bağlantısı için yapılandırma şablonunu değiştirme*»).
- **Outbounds** (bkz. 11.4).
- **Yük Dengeleyici** (bkz. 11.5).
- **Yönlendirme** (açıklama: «*Her kuralın önceliği önemlidir!*», bkz. 11.3).
- **DNS / Fake DNS** (bkz. 11.6).

11.2. Ana Ayarlar (General)

Freedom Protocol Strategy

Alan	Başlık	Açıklama	Varsayılan
FreedomStrategy	Freedom Protokol Strateji Ayarı	Doğrudan (freedom) outbound için ağ çıkış stratejisi. Açıklama: «Freedom protokolündeki ağ çıkış stratejisini ayarla». freedom protokolüne sahip outbound'un settings içindeki domainStrategy alanını kontrol eder.	Referans şablonda, direct freedom-outbound için domainStrategy AsIs değerindedir (adres çözülmez, orijinal haliyle iletilir).

Freedom için domainStrategy (Xray-core değerleri): **AsIs** (sunucu tarafında etki alanını çözme) ve ayrıca UseIP / UseIPv4 / UseIPv6 ailesi ile bunların «zorunlu» varyantları olan ForceIP* – çıkış sunucusunu etki alanını çözmeye ve alınan IP üzerinden bağlanmaya zorlar. Çıkış sunucusunda IPv6 yoksa veya yalnızca IPv4 üzerinden geçmeniz gerekiyorsa UseIPv4 olarak değiştirin.

Freedom Happy Eyeballs (IPv4/IPv6)

Alan	Başlık	Açıklama
FreedomHappyEyeballs	Freedom Happy Eyeballs (IPv4/IPv6)	Açıklama: «Doğrudan (freedom) giden için çift yığın bağlantı – IPv4 ve IPv6'ya sahip çıkış sunucularında kullanışlıdır.» Freedom-outbound için Happy Eyeballs algoritmasını etkinleştirir (her iki adres ailesinde eş zamanlı deneme).

Ayrı bir Happy Eyeballs, **belirli bir outbound'un dial (sockopt) ayarlarında** da vardır: 3.5.0'dan itibaren gerçekten etkinleşir (önceden yapılandırma kapalı olarak serileştiriliyor ve anahtar «geri dönüyordu»). «Try delay (ms)» varsayılanı artık **250**'dir; açıkça girilen **0** (= kapalı) korunur; ayrıca Prioritize IPv6, interleave (1) ve maxConcurrentTry (4) da mevcuttur. | try delay | (açıklama) | «Başka adres ailesini denemeden önce geçen milisaniye. 150–250 ms iyi bir başlangıç noktasıdır.» Alternatif adres ailesine geçmeden önceki gecikme. Önerilen aralık: 150–250 ms. |

Overall Routing Strategy

Alan	Başlık	Açıklama	Varsayılan
RoutingStrategy	Etki Alanı Yönlendirme Strateji Ayarı	Yönlendirme için genel DNS çözümleme stratejisi. Açıklama: «Genel DNS çözümleme yönlendirme stratejisini ayarla». routing.domainStrategy alanını kontrol eder.	Referans şablonda routing.domainStrategy = AsIs .

routing.domainStrategy, IP yönlendirme kurallarının etki alanı sorguları ile nasıl eşleştirileceğini belirler: **AsIs** (yalnızca etki alanı kuralları, çözümleme yok), **IPIfNonMatch** (etki alanı kurallarla eşleşmezse – çözümle ve IP kurallarını kontrol et), **IPOnDemand** (IP kuralıyla karşılaşıldığında hemen çözümle). Etki alanı sorguları için IP kurallarının (örn. geoip:*) çalışması genellikle **IPIfNonMatch** gerektirir.

Outbound Test URL

Alan	Başlık	Açıklama	Varsayılan
outboundTestUrl	Giden Test URL'si	outbound test edilirken bağlantıyı kontrol etmek için URL. Açıklama: «Giden bağlantısını kontrol etmek için URL». Şablondan ayrı olarak xrayOutboundTestUrl anahtarı altında saklanır.	https://www.google.com/generate_204

Değer sanitizasyondan geçer. outbound test edilirken, SSRF'ye karşı koruma olarak ek bir doğrulama genel URL'ye yapılır: kullanıcı istemci aracılığıyla rastgele (dahili dahil) bir URL giremez, test URL'si her zaman sunucu ayarından alınır. Kaydetme/test sırasında boş bir değer varsayılan generate_204 ile değiştirilir.

Default Outbound (3.5.0'dan itibaren)

«**Temel Yönlendirme**» sekmesinin ilk ögesi – **Default Outbound** seçicisi: «Hiçbir yönlendirme kuralıyla eşleşmeyen trafik bu outbound'u kullanır (Xray listedeki ilk outbound'u alır)». Listede direct , blocked ve mevcut tüm etiketler bulunur; varsayılan direct . Seçim, belirtilen outbound'u şablonun ilk konumuna taşır; eksik direct / blocked anında oluşturulur ve geçiş sırasında silinmez.

Block BitTorrent

Alan	Başlık	Açıklama
Torrent	BitTorrent'i Engelle	routing.rules 'a protocol: ["bittorrent"] içeren trafiği blocked outbound'una gönderen bir kural ekler. Referans şablonda bu kural varsayılan olarak mevcuttur.

Bağlantı Sınırları (Connection Limits)

Açıklama: «Seviye 0 kullanıcılar için bağlantı düzeyi politikaları. Xray'in varsayılan değerini kullanmak için alanı boş bırakın.» Bu parametreler policy.levels.0 'a yazılır.

Alan	Başlık	Açıklama	Varsayılan
connIdle	Boşta Kalma Zaman Aşımı (saniye)	«Belirtilen saniye boyunca boşta kaldıktan sonra bağlantıyı kapatır. Değeri azaltmak, yoğun sunucularda belleği ve dosya tanımlayıcılarını daha hızlı serbest bırakır (Xray varsayılanı: 300).»	boş → Xray varsayılanı 300
bufferSize	Arabellek Boyutu (KB)	«Bağlantı başına dahili arabellek boyutu KB cinsinden. Az RAM'li sunucularda bellek kullanımını en aza indirmek için 0 olarak ayarlayın (Xray varsayılan değeri platforma bağlıdır).» Yer tutucu: «otomatik».	boş → platforma bağlı; 0 – en aza indir

Örnek (policy.levels.0). Bu gruptaki alanlar 0. düzey politikasına yazılır. Az RAM'li yoğun bir sunucuda kaynak serbest bırakmayı şöyle hızlandırabilirsiniz:

```
"policy": {
  "levels": {
    "0": {
      "connIdle": 120,
      "bufferSize": 0
    }
  }
}
```

Burada bağlantı, varsayılan 300 yerine 120 saniye boşta kaldıktan sonra kapatılır ve `bufferSize: 0` arabelleklerdeki bellek tüketimini en aza indirir. Formda boş bırakılan alan JSON'a eklenmez ve Xray kendi varsayılan değerini uygular.

11.3. Yönlendirme kuralları (routing)

`routing.rules` kural listesi. **Sıra kritiktir** («Her kuralın önceliği önemlidir!»): kurallar yukarıdan aşağıya değerlendirilir, ilk eşleşen tetiklenir. Açıklama: «Sırayı değiştirmek için sürükleyin». Sıra kontrol düğmeleri: **İlk**, **Son**, **Yukarı Taşı**, **Aşağı Taşı**.

Her kuralın `type: "field"` değeri vardır. Düğmeler: **Kural Oluştur**, **Kuralı Düzenle**. Liste alanlarının açıklaması: «Virgülle ayrılmış öğeler».

«Yönlendirme» sayfasında «**Kuralları İçer Aktar**» ve «**Kuralları Dışa Aktar**» düğmeleri, «Outbounds» sayfasındaki gibi «**daha fazla**» (more) açılır menüsünde toplanmıştır. «**Kuralları Dışa Aktar**» düğmesi hemen dosya indirmeyi başlatmaz, JSON ön izlemeli ve «**Kopyala**» ile «**İndir**» düğmeli bir modal pencere açar: içerik kaydedilmeden önce görüntülenebilir. «Outbounds» sayfasındaki outbound dışa aktarımı da aynı şekilde çalışır.

Route Tester (yol test aracı)

Yönlendirme sekmesinde **Route Tester** alt sekmesi bulunur — çalışan Xray'e, gerçek trafik göndermeden belirli bir bağlantıyı hangi outbound'un işleyeceğini sorar. Bir etki alanı veya IP, port, ağ (TCP/UDP) ve gerekirse inbound ile yakalanmış protokol (`http / tls / quic / bittorrent`) belirtin, ardından **Test Route** düğmesine basın. Karar, doğrudan canlı yönlendirme motorundan alınır.

Yanıta seçilen outbound gösterilir ve yük dengeleyici kullanılıyorsa yük dengeleyici etiketi de eklenir. Hiçbir kural eşleşmezse test aracı, trafiğin varsayılan outbound'a (ilk `outbounds` listesindeki) gittiğini bildirir. Bu, kurallara güvenmeden önce sırayı doğrulamak için kullanışlıdır.

Tek bir kuralı etkinleştirme ve devre dışı bırakma

Tek bir yönlendirme kuralı, silinmeden geçici olarak anahtar ile **devre dışı bırakılabilir**. Kural tablosunda anahtar (Switch) içeren «**Etkinleştir**» sütunu ve kural formunda «**Etkinleştir**» alanı da bir anahtardır. Devre dışı bırakılan kural nihai Xray yapılandırmasına eklenmez ancak şablonda saklanır ve istediğiniz zaman yeniden etkinleştirilebilir.

İstatistik servis kuralı (`inboundTag: ["api"] → outboundTag: "api"`) devre dışı bırakılamaz — anahtarı kilittir, böylece panel trafik sayımı bozulmaz (bkz. [11.11](#)).

Kural formu alanları

Form alanı	Başlık	JSON alanı	Açıklama
Kaynak	Kaynak	<code>source</code>	Kaynak IP adresleri/alt ağları. Virgülle ayrılmış liste.
Kaynak portu	Kaynak Portu	<code>sourcePort</code>	Kaynak port(lar)ı.
Hedef	Hedef	<code>domain + ip + port</code>	Hedef etki alanları, IP'ler ve portlar. Etki alanları <code>domain:</code> , <code>full:</code> , <code>regex:</code> , <code>keyword:</code> örneklerini ve <code>geosite:*</code> 'yi; IP'ler <code>geoip:*</code> ve CIDR'ı destekler.
Ağ	–	<code>network</code>	<code>tcp</code> , <code>udp</code> veya <code>tcp,udp</code> . 3.4.2'den itibaren yapılandırmaya küçük harfle (<code>tcp</code> / <code>udp</code>) yazılır, yönlendirme tablosunda ise büyük harfle (<code>TCP</code> / <code>UDP</code>) gösterilir; boş – herhangi bir ağ.
Protokol	–	<code>protocol</code>	<code>http</code> , <code>tls</code> , <code>bittorrent</code> (sniffing ile belirlenir).
Kullanıcı	Kullanıcı	<code>user</code>	Kullanıcı e-posta/tanımlayıcısına göre filtrele.
Özellikler / Değer	Özellikler / Değer	<code>attrs</code>	Eşleştirme için HTTP başlık öznitelikleri.
VLESS route	VLESS route	–	VLESS için route alanına göre yönlendirme.
Gelen etiketleri	Gelen Etiketleri	<code>inboundTag</code>	Kuralın uygulandığı bir veya daha fazla inbound etiketi (yerleşik <code>api</code> ve DNS ayarlarındaki DNS etiketi dahil). inbound listelerinde, inbound ayrı bir açıklamaya sahipse «etiket (açıklama)» olarak gösterilir, aksi takdirde yalnızca etiket; kaydedilen kurallarda yalnızca etiketler saklanır.
Giden etiketi	Giden Etiket / Giden Bağlantı	<code>outboundTag</code>	Eşleşen trafiğin nereye yönlendirileceği.
Dengeleyici etiketi	Dengeleyici Etiket / Yük Dengeleyici	<code>balancerTag</code>	Açıklama: « <i>Trafiği yapılandırılmış yük dengeleyicilerden biri üzerinden yönlendirir</i> ».

`outboundTag` ve `balancerTag` birbirini dışlar: «*balancerTag ve outboundTag'ı aynı anda kullanmak mümkün değildir. Aynı anda kullanılırsa yalnızca outboundTag çalışır.*» Tek bir kuralda ya giden etiket ya da dengeleyici etiketi belirtin.

Referans şablonunun yerleşik kuralları

Standart `config.json` 'da `routing` bölümü şu üç kuralı içerir (bu sırayla):

1. `inboundTag: ["api"]` → `outboundTag: "api"` – panel istatistik gRPC API'si için servis kuralı.
2. `ip: ["geoip:private"]` → `outboundTag: "blocked"` – özel aralıkları engelle.
3. `protocol: ["bittorrent"]` → `outboundTag: "blocked"` – BitTorrent'i engelle.

api → api kuralı, istatistik isteğinin üst sıradaki catch-all kural tarafından yakalanmaması için kaydetme sırasında her zaman otomatik olarak 0. konuma yükseltilir (bkz. 11.11).

Kural örneği. Rus siteleri ve özel ağlara giden tüm trafiği doğrudan (proxy'yi atlayarak) gönderin, geri kalanını dengeleyiciye yönlendirin. Sıra önemlidir: «doğrudan yönlendir» kuralı catch-all'ın üstünde olmalıdır.

routing.rules 'da:

```
{
  "type": "field",
  "domain": ["geosite:category-ru", "domain:example.ru"],
  "ip": ["geoip:ru", "geoip:private"],
  "outboundTag": "direct"
}
```

IP kurallarının (geoip:ru) etki alanı sorguları için de tetiklenmesi için genellikle üst düzey yönlendirmede routing.domainStrategy: "IPIfNonMatch" gerekir (bkz. 11.2).

Önceden yapılandırılmış yönlendirme grupları (Temel Bağlantılar)

«Temel Bağlantılar» modunda panel, hazır listelerden tipik kurallar oluşturmanıza yardımcı olur:

Grup	Alanlar	Açıklama
Protokol/site engellemesi	—	«İstemcilerin belirli protokollere erişememesi için yapılandırın»
Ülkeye göre engelleme	Engellenen IP Adresleri, Engellenen Etki Alanları	«Bu ayarlar trafiği hedef ülkeye göre engeller.»
Doğrudan bağlantılar	Doğrudan IP Adresleri, Doğrudan Etki Alanları	«Doğrudan bağlantı, belirli trafiğin başka bir sunucu üzerinden yönlendirilmeyeceği anlamına gelir.»
IPv4 kuralları	—	«Bu ayarlar, istemcilerin hedef etki alanlarına yalnızca IPv4 üzerinden yönlendirilmesini sağlar»
WARP kuralları	—	«Bu seçenekler trafiği WARP üzerinden belirli bir hedefe göre yönlendirir.»
NordVPN yönlendirmesi	—	«Bu seçenekler trafiği NordVPN üzerinden belirli bir hedefe göre yönlendirir.»

MTPROTO-inbound: Xray üzerinden Telegram trafiğini yönlendirme

MTPROTO-inbound'un «Route through Xray» anahtarı (varsayılan olarak kapalı) ve isteğe bağlı **Outbound** seçimi vardır. Etkinleştirildiğinde panel, Xray yapılandırmasına inbound'un kendi etiketiyle döngüsel bir SOCKS köprüsü ekler ve mtg Telegram trafiğini bunun üzerinden yönlendirir. Bundan sonra giden Telegram trafiği yönlendirici tarafından yönetilir: Yönlendirme sekmesinde inbound etiketine göre normal kurallarla eşleştirilebilir ya da **Outbound** alanı aracılığıyla seçilen bir outbound veya dengeleyiciye zorla yönlendirilebilir. Kararı yönlendirme kurallarına bırakmak için **Outbound**'u boş bırakın.

11.4. Outbounds (giden bağlantılar)

outbounds listesi. Düğmeler: **Giden Bağlantı Oluştur**, **Giden Bağlantıyı Düzenle**. Açıklama: «Bu sunucu için giden bağlantıları tanımlamak üzere yapılandırma şablonunu değiştirme».

Referans şablonda iki zorunlu outbound vardır:

- protocol: "freedom", tag: "direct" – internete doğrudan çıkış (domainStrategy: "AsIs" ve finalRules: [{action: "allow"}] ile);
- protocol: "blackhole", tag: "blocked" – engellenen trafik için «kara delik».

Outbound formu genel alanları

Alan	Başlık	Açıklama
Etiket	Etiket (açıklama: «Benzersiz etiket»)	outbound'un benzersiz tanımlayıcısı. Yer tutucu: «benzersiz-etiket». Doğrulama: «Etiket zorunludur», «Etiket başka bir giden tarafından kullanılıyor».
Protokol	—	Giden türü (aşağıya bakın).
Adres / Port	Adres / Port	Bağlantı hedefi. Adres ve port zorunludur.
Üzerinden Gönder	Üzerinden Gönder	Giden arayüzünün yerel IP adresi (sendThrough). Yer tutucu: «yerel IP».
Target Strategy	Target Strategy	3.5.0'da yeni. Hedef alan adının bağlanmadan önce nasıl çözümleneceği. 11 değer: AsIs (varsayılan, çözümlenmez), UseIP , UseIPv4 , UseIPv6 , UseIPv6v4 , UseIPv4v6 (fallback'li çözümlenme), ForceIP , ForceIPv6v4 , ForceIPv6 , ForceIPv4v6 , ForceIPv4 (başarılı çözümlenme gerektirir). Boş = AsIs (yapılandırmaya yazılmaz); freedom için değer, önceki domainStrategy 'ye geri düşülerek okunur (eski çekirdekler için legacy anahtar yazılmaya devam eder).
Dialer proxy (zincir)	—	Açıklama: «Bir proxy zinciri oluşturmak için bu outbound'u başka bir outbound üzerinden (etikete göre) bağlayın. Doğrudan bağlantı için boş bırakın.» Yer tutucu: «Zincir için outbound seçin». streamSettings.sockopt.dialerProxy aracılığıyla uygulanır.

Dialer Proxy açılır listesi yalnızca yerel outbound'ları değil, aboneliklerden alınan outbound etiketlerini de gösterir – böylece zincir bir abonelik üzerinden alınan çıkış noktası üzerinden de kurulabilir. Listedeki blackhole-outbound ve düzenlenen outbound hariç tutulmaya devam eder. Doğrudan bağlantı için alanı boş bırakın.

Desteklenen outbound protokolleri

Form tarafından desteklenen protokoller:

- **freedom** – doğrudan çıkış. settings.domainStrategy , finalRules (aşağıya bakın), Happy Eyeballs alanları. Test edilemez («Outbound has no testable endpoint»).
- **blackhole** – trafiği atar. **Yanıt türü** alanı. Test edilemez.

- **socks** , **http** – `address / port` içeren `settings.servers[]` listesi; **Yetkilendirme parolası** alanı. **http** protokolü için **Username/Password** alanlarının altında, yukarı akış HTTP proxy'sine gönderilen CONNECT başlıkları için anahtar/değer çifti içeren **Headers** (Başlıklar) düzenleyicisi bulunur. Bu başlıklar, outbound yeniden açıldığında ve kaydedildiğinde korunur (daha önce kayboluyordu). Not: yalnızca ayar düzeyindeki başlıklar (`settings.headers`) uygulanır; xray-core bireysel sunucu düzeyindeki başlıkları yoksayar.
- **vmess** – `settings.vnext[]` (`address / port`).
- **vless** – `settings.address / settings.port` .
- **trojan** , **shadowsocks** – `settings.servers[]` .
- **wireguard** – `endpoint` içeren `settings.peers[]` artı anahtarlar (bkz. 11.8).
- **hysteria** – `settings.address / settings.port` (UDP aktarımı).

loopback türünde outbound için, inbound'daki ile aynı parametreleri içeren **Sniffing** bloğu mevcuttur: etkinleştirme, **destOverride**, **Metadata Only**, **Route Only** ve **dışlanan etki alanları** listesi.

UDP maskesinde (FinalMask), **Hysteria2** için ek modlar mevcuttur. **Salamander** maskesinin **Mode** seçicisi **Salamander** ve **Gecko** değerlerine sahiptir: Gecko modu, **Min/Max** boyut alanlarıyla (`packetSize` , 1–2048 aralığı, varsayılan 512–1200) rasgele paket dolgusu ekler – bu, paket uzunluğuna göre parmak izine karşı koruma sağlar. **Realm** maskesinde (UDP hole-punching), **Server Name** (SNI), **ALPN** (`h3 / h2 / http/1.1`), **Fingerprint** (uTLS) alanları ve **Allow Insecure** anahtarı içeren isteğe bağlı **TLS Config** bloğu eklendi.

3.5.0 ile yeni bir **TCP maskesi** türü eklendi – **XMC (Minecraft)** (`xmc`): akış Minecraft protokolü gibi maskelenir. Alanlar: **Hostname** (isteğe bağlı; «el sıkışmada taklit edilen sunucu adresi»), **Usernames** (isteğe bağlı liste; «prob yapanlara sunulan oyuncu adları; çekirdek varsayılan olarak Dream kullanır») ve zorunlu **Password** (16 karakter, ☺ düğmesiyle otomatik oluşturma; «gizleme parolası»). Önemli: **TCP Finalmask ile REALITY kombinasyonu kaydetme sırasında reddedilir** – ilk bağlantıda Xray-core'u çökertiyordu (hata: «Finalmask is not supported with REALITY security...»; bkz. Xray-core#6453). REALITY ile UDP maskesi kullanılabilir; daha önce kaydedilmiş hatalı kombinasyonlar yapılandırma oluşturulurken otomatik olarak «iyileştirilir» (maske atılır).

Örnek: üst akış SOCKS üzerinden zincir. `upstream` outbound'u harici bir SOCKS5 proxy'ye bağlanır, `chained` ise trafiğini onun üzerinden (`dialerProxy`) gönderir ve böylece bir zincir oluşturur. `outbounds` 'da:

```
[
  {
    "tag": "upstream",
    "protocol": "socks",
    "settings": {
      "servers": [{ "address": "203.0.113.10", "port": 1080 }]
    }
  },
  {
    "tag": "chained",
    "protocol": "freedom",
    "streamSettings": {
      "sockopt": { "dialerProxy": "upstream" }
    }
  }
]
```

Artık `outboundTag: "chained"` içeren bir yönlendirme kuralı, trafiği `upstream` üzerinden internete iletir.

Paylaşım bağlantısından outbound içe aktarma

outbound, paylaşım bağlantısından (`vless://` , `vmess://` vb.) içe aktarılabilir. İçe aktarma sırasında bağlantının `extra=` bloğunda iletilen **xmux** (XHTTP) çoklayıcı ayarları da korunur: içe aktarma sonrasında değerleri oluşturulan outbound'un **XMUX** alt formuna eklenir.

Mux (çoğullama) alanları

Maks. Paralellik, Maks. Bağlantı, Maks. Yeniden Kullanım, Maks. İstek, Maks. Yeniden Kullanım Süresi, Keep Alive Periyodu. Bu parametreler outbound'un mux/XUDP davranışını yapılandırır.

Sockopts (soket ayarları)

Sockopts grubu: **Keep Alive Aralığı, Mark (fwmark), Arayüz, Yalnızca IPv6, Proxy Protokolü Kabul Et, Proxy Protokolü, TCP user timeout (ms), TCP keep-alive idle (s).** Zincir dialer-proxy'si de burada ayarlanır.

Freedom finalRules (özel IP engelinin geçersiz kılınması)

Freedom-outbound için **Son Kurallar** grubu mevcuttur:

Alan	Başlık	Açıklama
<code>overrideXrayPrivateIp</code>	Xray'deki Varsayılan Özel IP Engelinin Geçersiz Kıl	Xray'in özel IP'lere giden bağlantılara yönelik yerleşik yasağını kaldırır.
<code>action</code>	Eylem	<code>allow</code> (referans şablondaki gibi: <code>finalRules: [{action: "allow"}]</code>), <code>redirect</code> (Redirect) veya diğerleri.
<code>blockDelay</code>	Blok Gecikmesi (ms)	Bağlantıyı atmadan önce gecikme.
<code>redirect / fragment</code>	Redirect / Fragment	Trafik yeniden yönlendirme ve parçalama eylemleri.

fragment maskesi: parça başına Lengths ve Delays

fragment maskesinde (FinalMask'ta TCP için fragment türü) tekil Length ve Delay alanları, **Lengths** ve **Delays** listeleriyle değiştirildi: her segment için ayrı uzunluk aralığı (örn. `100-200`) ve milisaniye cinsinden gecikme (örn. `10-20` veya `0`) belirtilebilir. Liste satırları eklenip kaldırılabilir; önceden kaydedilen tek değerler, otomatik olarak tek elemanlı bir diziye aktarılır.

Diğer form alanları

- **UDP over TCP** ve **UoT Sürümü** — shadowsocks benzeri protokoller için.
- **gRPC başlığı olmadan, Uplink Chunk Boyutu** — gRPC aktarım parametreleri.
- TLS/uTLS alanları: **Peer adını doğrula, Pinned SHA256, Short ID, Vision testpre**, «sunucu adı» yer tutucusu.

Giden bağlantıları test etme

Düğmeler: **Test**, **Tümünü Test Et**. Durumlar: **Bağlantı test ediliyor...**, **Test başarılı**, **Test başarısız**, **Giden bağlantı test edilemedi**. Sonuç: **Test Sonucu**, milisaniye cinsinden gecikme.

İki mod (açıklama: «TCP: hızlı yalnızca dial araştırması. HTTP: xray üzerinden tam istek.»):

- **TCP** (mode=tcp) — host:port 'a basit dial, tüm uç noktalarda paralel olarak gerçekleştirilir, ~5 s zaman aşımı. Yalnızca TCP erişilebilirliğini kontrol eder, proxy protokolünü doğrulamaz. freedom / blackhole / blocked etiketi için «Outbound has no testable endpoint» döner.
- **HTTP** (mode=http veya boş) — Xray'in geçici bir örneğini başlatır, gerçek bir HTTP isteğini (probe URL = sunucu outboundTestUrl) gönderir, gerçek gecikmeyi ölçer. Güvenilir ancak maliyetli mod: global kilit tarafından serileştirilir («Another outbound test is already running, please wait»). Tek deneme zaman aşımı — 10 s, sonuç bekleme penceresi — 15 s (yavaş veya tünelli kanallar üzerindeki sağlıklı outbound'ların «Başarısız» olarak işaretlenmemesi için artırıldı). Başarısız olduğunda gerçek neden (DNS hatası, bağlantı reddedildi, son tarih aşımı, TLS hatası vb.) panel/Xray günlüğüne yazılır ve genel zaman aşımı mesajları buna işaret eder.
- **Real delay (3.5.0'da yeni)** — anahtarın üçüncü modu. Aynı geçici Xray örneğini kullanır, ancak **tünel kurulumu dahil «soğuk» isteğin tam süresini** bildirir — istemci uygulamalarının gösterdiğine yakın bir rakam. Anahtarın açıklaması: «TCP: hızlı dial araştırması. HTTP: xray üzerinden tam istek. Real delay: bağlantı kurulumu dahil tam süre». Normal **HTTP** modu 3.5.0'dan itibaren «ısınmış» (keep-alive) bağlantı üzerinden ölçülür; bu nedenle rakamları daha düşüktür ve tek bir isteğin gecikmesine daha yakındır.

UDP protokolleri (wireguard , hysteria) ve UDP aktarımları (kcp , quic , hysteria) TCP istenmiş olsa bile **her zaman** HTTP modunda test edilir — saf UDP dial'i «canlı» uç noktayı «ölü»den ayırt edemez. wireguard için test yapılandırmasında noKernelTun: true zorunlu olarak ayarlanır.

Egress ve Country sütunları (3.5.0'dan itibaren)

Başarılı bir **HTTP** veya **Real delay** testinden sonra panel, aynı geçici rota üzerinden Cloudflare trace meta verilerini sorgular ve outbound listesinde iki sütun gösterir: **Egress** — çıkış IP'si (IPv4/IPv6, her biri «göz» arkasına gizlidir; henüz test yoksa — «Çıkış IP'sini ve ülkeyi göstermek için bir HTTP testi çalıştırın» ipuçlu çizgi) ve **Country** — çıkış ülkesinin bayrağı ve adı; çıkış Cloudflare WARP üzerinden gidiyorsa turuncu bir WARP etiketi eklenir. TCP testi bu sütunları doldurmaz; mobil cihazlarda veriler outbound kartında gösterilir.

Toplu kontrol ve aşama dökümü

HTTP modundaki **Test** ve **Tümünü Test Et**, outbound paketi için tek bir ortak geçici Xray örneği başlatır, her biri için bir döngüsel SOCKS-inbound ve kural oluşturur ve üzerinden paralel olarak gerçek bir HTTP isteği gönderir; **Tümünü Test Et** outbound'ları gruplar halinde kontrol eder. **Tümünü Test Et**, aboneliklerden gelen outbound'ları da kontrol eder (yalnızca okunabilir «aboneliklerden» tablosu) — bunların satırları da test sonucuyla vurgulanır. Bu durumda freedom («direct») ve dns outbound'ları hiçbir modda test edilmez (bunlar proxy değildir): test düğmeleri devre dışıdır, **Tümünü Test Et** bunları atlar ve sunucu koruması API'ye doğrudan çağrıldığında bile HTTP testlerini engeller. Başarı/hataların yanı sıra açılır sonuç, HTTP yanıt durumunu ve

aşama bazında zaman dökümünü gösterir: **Proxy connect** (proxy'ye bağlantı), **TLS via outbound** (outbound üzerinden TLS) ve **First byte** (ilk bayta kadar geçen süre) – bu, gecikme veya arızanın hangi adımda oluştuğunu anlamaya yardımcı olur.

Outbound trafik istatistikleri

Panel, etiket başına trafik sayaçlarını tutar (up / down / total). Sıfırlama düğmesi, belirli bir etiket veya tüm etiketler için sayaçları sıfırlar (tag = "-alltags-"). **Hesap Bilgisi** ve **Giden Bağlantı Durumu** alanları özet gösterir.

11.5. Yük Dengeleyiciler (Balancers)

`routing.balancers` listesi. Düğmeler: **Yük Dengeleyici Oluştur**, **Yük Dengeleyiciyi Düzenle**.

Yük Dengeleyiciler sekmesinde canlı durum sütunları bulunur: **Live Target**, çalışan Xray'deki yük dengeleyicinin mevcut aktif hedefini gösterir ve **Override**, hedef seçimini manuel olarak geçersiz kılmanıza olanak tanır (**Auto (strategy)** değeri seçimi stratejiye göre döndürür). Durum ayrı bir düğmeyle güncellenir. Yük dengeleyici çalışan Xray'de henüz aktif değilse panel önce değişiklikleri kaydetmenizi veya Xray'i başlatmanızı önerir.

Alan	Başlık	Açıklama
Etiket	Etiket (açıklama: «Benzersiz etiket»)	Benzersiz tanımlayıcı. Yer tutucu: «benzersiz yük dengeleyici etiketi». Doğrulama: «Etiket zorunludur», «Etiket başka bir yük dengeleyici tarafından kullanılıyor».
Seçiciler	Seçiciler	Yük dengeleyicinin çıkış seçtiği outbound etiketlerinin listesi (alt dizeye göre). En az biri seçilmelidir: «En az bir giden seçin».
Yedek	Yedek	Hiçbir seçici eşleşmezse yedek etiket. 3.5.0'dan itibaren başka bir yük dengeleyici de seçilebilir (listede bu seçenekler mavi «Balancer» etiketiyle işaretlenir, ipucu «Fallback olarak başka bir yük dengeleyici seçin»). Xray bunu doğal olarak yapamaz; bu nedenle panel gizli loopback outbound <code>_bl_<hedef></code> nesnesini ve yönlendirme kuralını kendisi oluşturur (yol: yük dengeleyici → loopback → sunucu → hedef yük dengeleyici → outbound; fazladan «sekme (hop)» uyarısı formda gösterilir). Döngülere karşı koruma («seçilemez – döngüsel bağımlılık oluşurdu» hatası, listedeki seçenek döngü yolu ipucuyla engellenir) ve kullanılan yük dengeleyicinin silinmesine karşı koruma («Silinemez – şuralarda fallback olarak kullanılıyor: ...») vardır. <code>_bl_</code> etiket öneki ayrılmıştır (etiketlerde yasaktır); hizmet amaçlı <code>_bl_</code> nesneleri Outbounds/Routing listelerinden gizlenir; Fallback/Live Target/Override sütunları loopback etiketini değil, hedef yük dengeleyicinin adını gösterir.
Strateji	Strateji	Seçim algoritması (aşağıya bakın).

Strateji ve gözlem parametreleri

Strateji (`strategy.type`), yük dengeleyicinin seçiciler arasından outbound'u nasıl seçeceğini belirler. Xray-core değerleri: `random` (rasgele), `roundRobin` (sırayla), `leastPing` (observatory sonuçlarına göre minimum gecikme), `leastLoad` (minimum yük). `leastLoad / leastPing` için `strategy.settings` parametreleri kullanılır:

Alan	Başlık	Açıklama
expected	Beklenen	Yer tutucu: « <i>optimum düğüm sayısı</i> » – hedef canlı düğüm sayısı.
maxRtt	Maks. RTT	Aday seçiminde izin verilen maksimum RTT üst sınırı.
tolerance	Tolerans	Gecikme/yük karşılaştırmasında tolerans.
baselines	Baselines	Düğümleri gruplamak için gecikme eşik değerleri.
costs	Costs	Bireysel etiketler için ağırlık katsayıları (cost).

Strateji örnekleri. strategy bloğu yük dengeleyicinin içinde yaşar (JSON'da tag ve selector ile yan yana):

```
"strategy": { "type": "random" }      // seçicilerden rasgele seçim
"strategy": { "type": "roundRobin" }  // sırayla, dönüşümlü
"strategy": { "type": "leastPing" }   // minimum gecikme (gözlemci gerektirir)
```

leastLoad için parametreler settings 'te belirtilir:

```
"strategy": {
  "type": "leastLoad",
  "settings": {
    "expected": 2,
    "maxRTT": "1s",
    "tolerance": 0.05,
    "baselines": ["500ms", "1s", "2s"],
    "costs": [
      { "regexp": false, "match": "proxy-premium", "value": 0.1 },
      { "regexp": true, "match": "^proxy-cheap-.*$", "value": 5 }
    ]
  }
}
```

Nasıl çalıştığı (örnek). Gözlemcinin çıkışlar için ölçtüğü gecikmeler: A = 250 ms , B = 280 ms , C = 700 ms , D = 1500 ms . Yukarıdaki ayarlarla seçim şu şekilde gerçekleşir:

- maxRTT: "1s"** – gecikmesi 1 s'nin üzerindeki çıkışlar elenir: D (1500 ms) çıkar. A , B , C kalır.
- baselines + expected** – çıkışlar gecikme eşiklerine göre gruplandırılır ve en az expected çıkış içeren **en küçük** eşik seçilir. 500ms eşiği zaten A ve B 'yi içerir – bu 2 (= expected) olduğundan { A , B } grubu seçilir. C (700 ms) hızlılar yeterli olduğu sürece seçime girmez (o «sıcak yedek»tir).
- tolerance: 0.05** – seçilen grup içinde gecikmeleri %5'ten fazla farklı olmayan çıkışlar eşdeğer kabul edilir ve yük aralarında eşit dağıtılır. A (250) ve B (280) ~%12 farklıdır (> %5), bu yüzden diğer koşullar eşitken daha hızlı olan A tercih edilir; fark %5 içinde olsaydı trafik hem A hem B üzerinden akardı.
- costs** – karşılaştırmadan önce bireysel çıkışların «maliyetini» düzenler: daha küçük value çıkışı daha cazip kılar, daha büyük ise tersine. Örnekte proxy-premium 0.1 alır (daha «ucuz» olur ve daha sık seçilir), tüm proxy-cheap-* ise (düzenli ifadeyle, regexp: true) 5 alır (daha «pahalı» olur ve son çare olarak kullanılır). Bu sayede çıkışları kesin olarak dışlamadan yumuşakça önceliklendirebilirsiniz.

Sonuç: trafik ağırlıklı olarak A üzerinden akar (gecikmeler yakınsa B ile eşit olarak), C yedek olarak kalır, D RTT'si maxRTT 'nin altına düşene kadar dışlanır.

Gözlemci: observatory ve burstObservatory (leastPing / leastLoad için ölçümler)

leastPing ve leastLoad stratejileri kendileri hiçbir şey ölçmez – her outbound için gecikme ve erişilebilirlik verilerine ihtiyaçları vardır. Bunları **gözlemci** (observatory) toplar: her izlenen outbound'u periyodik olarak «ping'ler» ve yanıt süresi ile erişilebilirliği kaydeder. Aynı veriler «**Gözlemevi**» sekmesinde de gösterilir (**Aktif / Erişilemiyor** durumları, «**Son Aktivite**», «**Son Deneme**»).

3.4.2 sürümünden itibaren «Yük Dengeleyiciler» sayfası «**Yük dengeleyici ayarları**» (tablo ve ekleme) ve «**Observatory**» sekmelerine ayrıldı ve gözlemci bir **yapılandırılmış formla** düzenleniyor (eski ham JSON düzenleyicisi kaldırıldı). Panel, gözlemcileri gerektiğinde kendisi oluşturup siler: sıradan **observatory** – **Least Ping** stratejisi için, gelişmiş **burstObservatory** – **Least Load** için ve ayrıca **belirli bir fallbackTag'e sahip Random/Round-robin** için (bunu xray-core gerektirir; son **fallbackTag** temizlendiğinde gözlemci kaldırılır). Bir gözlemci oluşturmak veya silmek Xray'ın tam olarak yeniden başlatılmasını gerektirir.

İki seçenek mevcuttur:

- **observatory** – basit: `subjectSelector` + `probeURL` + `probeInterval`.
- **burstObservatory** – `pingConfig` ile ince ping ayarına sahip gelişmiş; birden fazla çıkış için kullanışlı.

`burstObservatory` bloğu örneği:

```
{
  "subjectSelector": ["WS-SE", "WS-FR", "WS-PL"],
  "pingConfig": {
    "destination": "https://www.google.com/generate_204",
    "interval": "1m",
    "connectivity": "http://connectivitycheck.platform.hicloud.com/generate_204",
    "timeout": "5s",
    "sampling": 2
  }
}
```

Alan açıklamaları:

Alan	Ne ayarlar
<code>subjectSelector</code>	İzlenecek outbound için etiket örneklerinin listesi. Xray, etiketleri belirtilen dizelerle başlayan tüm outbound'ları alır. Örnekte <code>WS-SE...</code> , <code>WS-FR...</code> , <code>WS-PL...</code> çıkışları izlenir. Bu etiketler yük dengeleyicinin Seçicilerinde seçilenlerle eşleşmelidir.
<code>pingConfig.destination</code>	Gecikmeyi ölçmek için her outbound üzerinden istenen URL. Gövdesiz <code>204</code> yanıtı olan «hafif» bir sayfa kullanılır – örneğin <code>https://www.google.com/generate_204</code> . Yanıtı kadar geçen süre ölçülen gecikme değeridir.
<code>pingConfig.interval</code>	Her outbound'u ne sıklıkla ping'lemek için. Süre dizesi: <code>"1m"</code> – dakikada bir, ayrıca <code>"30s"</code> , <code>"5m"</code> vb. Daha sık → daha taze veriler, ancak daha fazla arka plan trafiği.
<code>pingConfig.connectivity</code>	(isteğe bağlı) Sunucunun temel bağlantısını kontrol etmek için URL. Erişilemiyorsa – sunucu ağında sorun var demektir ve gözlemci outbound'u erişilemiyor olarak işaretlemez (yerel arızalarda yanlış alarm koruması). Genellikle <code>204</code> yanıtı bir uç nokta.
<code>pingConfig.timeout</code>	Girişimi başarısız saymadan önce bir ping yanıtı için bekleme süresi (örn. <code>"5s"</code>).

Alan	Ne ayarlar
<code>pingConfig.sampling</code>	Her outbound için saklanacak ve ortalaması alınacak son ölçüm sayısı. 2 – son iki ping'i dikkate al (rasgele sıçramaları düzler).
<code>pingConfig.httpMethod</code>	3.4.2'de yeni. Yoklama HTTP yöntemi: HEAD (varsayılan) veya GET .

«**Observatory**» sekmesinde aynı parametreler JSON yerine bir formla ayarlanır. Sıradan `observatory` için bunlar: **Watched Outbounds** (`subjectSelector` , salt okunur), **Probe URL** (`probeURL` , varsayılan `https://www.google.com/generate_204`), **Probe Interval** (`probeInterval` , `1m`) ve **Enable Concurrency** (`enableConcurrency` , varsayılan açık). `burstObservatory` için – yukarıda sayılan alanların yanı sıra yeni «**HTTP yöntemi**». Yük dengeleyicinin her iki gözlemcisi de varsa Observatory ve Burst formları arasında bir segment anahtarı geçiş yapar.

Her şeyi nasıl bağlarsınız:

1. «**Observatory**» sekmesinde gözlemci parametrelerini yapılandırın (seçilen stratejiye göre otomatik olarak oluşturulur; `subjectSelector` yük dengeleyicinin seçicilerini izler).
2. Bir yük dengeleyici oluşturun: **Strateji** = `leastPing` , **Seçicilerde** aynı outbound etiketlerini belirtin (`WS-SE` , `WS-FR` , `WS-PL`).
3. Trafiği yönlendirme kuralıyla ona yönlendirin (**Dengeleyici Etiketi** alanı, bkz. [11.3](#)).
4. Xray'i yeniden başlatın. «**Gözlemevi**» sekmesinde çıkış durumları görünür ve yük dengeleyici canlı olanlar arasından en hızlısını seçmeye başlar.

Tek bir kuralda aynı anda `balancerTag` ve `outboundTag` belirtilemez – yalnızca `outboundTag` çalışır.

Bir outbound veya yük dengeleyici silindiğinde ne olur (3.4.2'den itibaren)

Bir outbound veya yük dengeleyici silinirken panel, aynı işlemde **yönlendirmedeki referansları temizler** ve onay diyalogunda bir **sonuç önizlemesi** gösterir (başlık «Silme işlemiyle yönlendirmeniz de güncellenecek:», ardından her kural için – «silindi (hedef kalmadı)» veya «korundu (artık ... kullanıyor)» ve «Yük dengeleyici ... silindi (hedef kalmadı)»). Böylece, önceden bir sonraki yeniden başlatmada xray-core'un başlamamasına neden olabilen «sarkan» referans (`balancerTag` / `outboundTag` / `dialerProxy`) önlenir. Yük dengeleyici silme: kurallar kendi `outboundTag` 'ini korur, aksi takdirde silinir. Outbound silme: etiketi yük dengeleyicilerin seçicilerinden ve `fallbackTag` 'inden kaldırılır, diğer outbound'larda `dialerProxy` temizlenir, boşalan yük dengeleyiciler artık konusuz kalan kurallarla birlikte silinir, gözlemciler yeniden derlenir.

11.6. DNS

`dns` bölümü. Etkinleştirme: **DNS'yi Etkinleştir** (açıklama: «Yerleşik DNS sunucusunu etkinleştir»).

3.5.0'dan itibaren **özel IP'lerdeki** DNS sunucuları (örneğin aynı docker ağındaki kendi AdGuard/Pi-hole'unuz) «kutudan çıktığı gibi» çalışır: panel, yönetilen bir izin kuralını (DNS sunucusunun portuyla sınırlı direct) engelleyici `geoip:private` kuralının **önüne** otomatik olarak ekler, sürdürür ve `dns.servers` değiştiğinde senkronize eder. Önceden Xray'in kendi DNS sorguları bu kuralla sessizce

engelleniyor ve her yeni alan adına ~4 s gecikme ekliyordu; artık elle kural gerekmez (elle oluşturulanlara dokunulmaz).

Genel DNS parametreleri

Alan	Başlık	JSON	Açıklama / ipucu
<code>tag</code>	DNS Etiket Adı	<code>dns.tag</code>	«Bu etiket, yönlendirme kurallarında gelen etiket olarak kullanılabilir.» DNS isteklerinin kendisini <code>inboundTag</code> aracılığıyla yönlendirmeye olanak tanır.
<code>clientIp</code>	İstemci IP'si	<code>dns.clientIp</code>	«DNS istekleri sırasında sunucuya belirtilen IP konumunu bildirmek için kullanılır» (EDNS Client Subnet).
<code>strategy</code>	İstek Stratejisi	<code>dns.queryStrategy</code>	«Genel etki alanı adı çözümleme stratejisi». Değerler: <code>UseIP</code> , <code>UseIPv4</code> , <code>UseIPv6</code> .
<code>disableCache</code>	Önbelleği Devre Dışı Bırak	<code>dns.disableCache</code>	«DNS önbelleklemeyi devre dışı bırakır».
<code>disableFallback</code>	Yedek DNS'yi Devre Dışı Bırak	<code>dns.disableFallback</code>	«Yedek DNS sorgularını devre dışı bırakır».
<code>disableFallbackIfMatch</code>	Eşleşmede Yedek DNS'yi Devre Dışı Bırak	<code>dns.disableFallbackIfMatch</code>	«DNS sunucusu etki alanı listesi eşleştiğinde yedek DNS sorgularını devre dışı bırakır».
<code>enableParallelQuery</code>	Paralel Sorguları Etkinleştir	—	«Daha hızlı çözümleme için birden fazla sunucuya paralel DNS sorgularını etkinleştir».
<code>useSystemHosts</code>	Sistem Hosts'u Kullan	<code>dns.useSystemHosts</code>	«Yükli sistemden hosts dosyasını kullan».

dns bloğu örneği. Google etki alanı sorguları Cloudflare DoH sunucusu üzerinden çözülür, geri kalanı

1.1.1.1 üzerinden; Google yanıtları için yalnızca özel olmayan IP'ler beklenir. Yapılandırmanın üst düzeyinde:

```
"dns": {
  "tag": "dns-inbound",
  "queryStrategy": "UseIPv4",
  "servers": [
    {
      "address": "https://cloudflare-dns.com/dns-query",
      "domains": ["geosite:google"],
      "expectIPs": ["geoip:!private"]
    },
    "1.1.1.1"
  ]
}
```

Alansız sunucu dizesi ("1.1.1.1") diğer tüm etki alanları için varsayılan sunucudur. `dns-inbound` etiketi daha sonra DNS isteklerini gerekli outbound üzerinden yönlendirmek için yönlendirme kurallarında `inboundTag` olarak kullanılabilir.

Süresi dolmuş kayıt önbellesi

Alan	Başlık	Açıklama
<code>serveStale</code>	Eskiye Kullan	«Arka planda güncellenirken önbelleden süresi dolmuş sonuçlar döndürür».
<code>serveExpiredTTL</code>	Eskinin TTL'si	«Süresi dolmuş önbellek kayıtlarının geçerlilik süresi (saniye); 0 = süresiz».

DNS sunucuları (`dns.servers` listesi)

Düğmeler: **DNS Oluştur**, **DNS'yi Düzenle**, **Tümünü Sil** (onay: «Tüm DNS sunucuları listeden kaldırılacak. Bu işlem geri alınamaz.»). Şablonlar: **Şablon Kullan**, **DNS Şablonları** penceresi, **Aile** ön ayarı dahil.

Bir DNS sunucu kaydında **DNS'yi Düzenle** tıklandığında (Fake DNS kayıtlarında olduğu gibi), düzenleme penceresi varsayılan değerler yerine sunucunun kaydedilmiş değerlerini doldurur.

DNS sunucu alanları:

Alan	Başlık	Açıklama
<code>address</code>	—	DNS adresi (IP, DoH-URL, <code>localhost</code> , <code>fakedns</code> vb.).
<code>domains</code>	Etki Alanları	Bu sunucunun kullanıldığı etki alanları listesi.
<code>expectIPs</code>	Beklenen IP'ler	Yalnızca IP listedeyse yanıtı kabul et.
<code>unexpectIPs</code>	Beklenmeyen IP'ler	Belirtilen IP'lere sahip yanıtları at.
<code>skipFallback</code>	Yedekten Atla	Bu sunucuyu yedek olarak kullanma.
<code>finalQuery</code>	Son Sorgu	Sunucuyu zincirde son olarak işaretler.
<code>timeoutMs</code>	Zaman Aşımı (ms)	Sunucuya istek zaman aşımı.

Hosts (statik kayıtlar)

Hosts grubu (`dns.hosts`). **Host Ekle** düğmesi; boş durum **Host tanımlı değil**. Alanlar: etki alanı (yer tutucu: «Etki alanı (ör. domain:example.com)») ve değerler (yer tutucu: «IP veya etki alanı – girin ve Enter'a basın»).

DNS günlükleri

Bkz. 11.10: günlükleme bölümündeki **DNS Günlükleri** (`dnsLog`) bayrağı.

11.7. Fake DNS

`fakedns` bölümü. Düğmeler: **Fake DNS Oluştur**, **Fake DNS'yi Düzenle**.

Alan	Başlık	Açıklama
<code>ipPool</code>	IP Havuzu Alt Ağı	Sahte IP'lerin dağıtıldığı CIDR aralığı (örn. <code>198.18.0.0/15</code>).
<code>poolSize</code>	Havuz Boyutu	Döngüsel havuzda tutulacak adres sayısı.

Fake DNS, inbound üzerinde sniffing ile birlikte kullanılır: çekirdek istemciye sahte bir IP verir, etki alanı[?] IP eşleşmesini hatırlar ve yönlendirme sırasında etki alanını geri yükler. Fake DNS'nin çalışması için `fakedns` adresli DNS sunucusu, DNS sunucu listesine eklenmelidir.

Örnek: Fake DNS + DNS sunucusu kombinasyonu. Önce sahte adres havuzunu tanımlarız, ardından etki alanı sorgularının bu havuzdan IP alması için `fakedns` DNS sunucusunu ekleriz:

```
"fakedns": [
  { "ipPool": "198.18.0.0/15", "poolSize": 65535 }
],
"dns": {
  "servers": [
    { "address": "fakedns", "domains": ["geosite:geolocation-!cn"] },
    "1.1.1.1"
  ]
}
```

Ek olarak inbound'da `destOverride: ["fakedns"]` ile sniffing etkinleştirilmelidir; aksi takdirde çekirdek geri yüklemek için gerçek etki alanını nereden alacağını bilemez.

11.8. WireGuard / WARP / NordVPN

WireGuard alanları (`wireguard`)

Alan	Başlık	Açıklama
<code>secretKey</code>	Gizli Anahtar	Yerel arayüzün özel anahtarı.
<code>publicKey</code>	Genel Anahtar	Peer'in genel anahtarı.
<code>psk</code>	Paylaşılan Anahtar	PreShared Key (isteğe bağlı).

Alan	Başlık	Açıklama
allowedIPs	İzin Verilen IP Adresleri	Tünele yönlendirilen aralıklar.
endpoint	Uç Nokta	Peer'in host:port 'u.
domainStrategy	Etki Alanı Stratejisi	WireGuard-outbound için çözümleme stratejisi.

Cloudflare WARP (warp)

Entegrasyon <https://api.cloudflareclient.com/v0a4005> API'sini kullanır (client-version a-6.30-3596). Kontrolcü eylemleri (/xray/warp/:action): config , reg , license , data , del .

Adım adım:

1. **WARP hesabı oluştur** → reg : panel özel (privateKey) ve genel (publicKey) anahtarları oluşturur/ kabul eder, Cloudflare'e bir cihaz kaydeder ve access_token , device_id , license_key , private_key (ve client_id) değerlerini warp ayarına kaydeder.
2. **WARP / WARP+ lisans anahtarı** → license : 26 karakterlik WARP+ anahtarının ayarlanması (yer tutucu: «26 karakterlik WARP+ anahtarı»). Hata durumunda: «WARP lisansı ayarlanamadı.» Yapılandırma henüz alınmadıysa: «Önce WARP yapılandırmasını alın.»
3. **Hesap bilgileri: Cihaz Adı, Cihaz Modeli, Cihaz Etkin, Hesap Türü, Rol, WARP+ data, Kota, Kullanım.**
4. **Giden ekle** — alınan anahtarlar ve Cloudflare uç noktasıyla bir WireGuard-outbound oluşturur.
5. **Hesabı sil** → del : kaydedilmiş WARP verilerini temizler.

NordVPN (nord / nordvpn)

Entegrasyon NordLynx (= WireGuard) kullanır. Kontrolcü eylemleri (/xray/nord/:action): countries , servers , reg , setKey , data , del .

Adım adım:

1. **Erişim tokeni** → reg : panel api.nordvpn.com 'dan NordLynx kimlik bilgilerini ister ve nordlynx_private_key 'i çıkarır. private_key ve token değerlerini nord ayarına kaydeder. Alternatif — setKey : **Özel Anahtarı** doğrudan girin (boş olamaz).
2. **Ülke** → countries ülke listesini yükler; **Şehir** (ya da **Tüm Şehirler**).
3. **Sunucu** → servers seçilen ülkenin sunucularını yükler (countryId enjeksiyon koruması için sayı olarak doğrulanır). Filtre: yalnızca **Yük**'ü > %7 olan sunucular gösterilir. Sunucu yoksa: «Seçilen ülke için sunucu bulunamadı». Sunucunun NordLynx genel anahtarı yoksa: «Seçilen sunucu NordLynx genel anahtarını bildirmiyor.»
4. Giden oluşturma/güncelleme: «NordVPN gideni eklendi» / «NordVPN gideni güncellendi» bildirim mesajları.

IPv4 önceliği ve kullanıcı alanı TUN

WARP ve NordVPN sihirbazları tarafından oluşturulan WireGuard-outbound'lar, ForceIP yerine domainStrategy: "ForceIPv4v6" kullanır (IPv4 önceliği, yalnızca v6 sunucularda IPv6'ya yedek) — bu, Cloudflare uç noktasının AAAA kaydının seçildiği durumlarda, yarı yapılandırılmış IPv6 olan sunuculardaki el sıkışma «donması»nı giderir. Ayrıca bunlar için kernel TUN yerine kullanıcı alanı TUN etkinleştirilir

(noKernelTun: true): kernel TUN yetki ve fwmark yönlendirmesi gerektirir ve birçok VPS'de sessizce çöker; oysa panelin yerleşik bağlantı kontrolü her zaman kullanıcı alanı TUN üzerinden test eder – artık gerçek trafik ve kontrol aynı yolu izler. Değişiklik yalnızca yeni eklenen veya sıfırlanan outbound'lar için geçerlidir; zaten kaydedilmiş şablonlar kendi ayarlarını korur.

11.9. Reverse-proxy ve TUN

Reverse (ters proxy)

Xray yapılandırmasının reverse bölümü. outbound formunda **Ters Proxy** türüne geçiş vardır. Düğmeler: **Ters Proxy Oluştur**, **Ters Proxy'yi Düzenle**.

Alan	Başlık	Açıklama
Tür	Tür	Bridge veya Portal – Xray ters proxy'nin iki rolü.
Etki alanı	Etki Alanı	bridge[?]portal çifti için hizmet etki alanı etiketi.
Etiket / Bağlantı	Etiket / Bağlantı	bridge ve portal'ı bağlamak için etiketler.
Reverse Tag	Ters Proxy Etiketi	Açıklama: «Basit VLESS ters proxy için giden bağlantı etiketi. Devre dışı bırakmak için boş bırakın.» Yer tutucu: «giden etiketi (boş = devre dışı)». Basitleştirilmiş VLESS reverse'i uygular.

outbound formunda ayrıca ters akış alanları bulunur: **Ters Sniffing**, **Çalışanlar**, **Ayrılmış**, **Min. Yükleme Aralığı (ms)**, **Maks. Yükleme Boyutu (bayt)**.

TUN (tun)

Alan	Başlık	Açıklama	Varsayılan
name	–	«TUN arayüzünün adı.»	xray0
mtu	–	«Maksimum aktarım birimi. Maksimum veri paketi boyutu.»	1500
userLevel	Kullanıcı Düzeyi	«Bu gelen akış üzerinden kurulan tüm bağlantılar bu kullanıcı düzeyini kullanır.»	0

11.10. Günlükler ve istatistik (Stats, metrics)

Günlük (log)

Açıklama: «Günlükler sunucunun yavaşlamasına neden olabilir. Yalnızca gerektiğinde ihtiyaç duyduğunuz günlük türlerini etkinleştirin!» Referans şablonun log bölümü: access: "none", error: "", loglevel: "warning", dnsLog: false, maskAddress: "".

Alan	Başlık	JSON	Açıklama	Varsayılan
logLevel	Günlük Düzeyi	loglevel	«Hata günlükleri için günlük düzeyi...» Değerler: debug, info, warning, error, none.	warning

Alan	Başlık	JSON	Açıklama	Varsayılan
accessLog	Erişim Günlükleri	access	«Erişim günlüğü dosyasının yolu. «none» özel değeri erişim günlüklerini devre dışı bırakır.»	none
errorLog	Hata Günlükleri	error	«Hata günlüğü dosyasının yolu. «none» özel değeri hata günlüklerini devre dışı bırakır.»	"" (varsayılan)
dnsLog	DNS Günlükleri	dnsLog	«DNS sorgu günlüklerini etkinleştirir»	false
maskAddress	Adres Maskeleye	maskAddress	«Etkinleştirildiğinde gerçek IP adresi günlüklerde maskeleye adresiyle değiştirilir.»	"" (kapalı)

İstatistik (stats / policy)

İstatistik grubu. `policy.system` ve `policy.levels` 'ta sayaçları etkinleştirir. Referans şablonda: `statsInboundUplink: true, statsInboundDownlink: true, statsOutboundUplink: false, statsOutboundDownlink: false; 0.düzy için – statsUserUplink: true, statsUserDownlink: true.`

Alan	Başlık	Açıklama	Varsayılan
statsInboundUplink	Gelen Uplink İstatistiği	«Tüm gelen proxy'lerin giden trafiği için istatistik toplamalarını etkinleştirir.»	true
statsInboundDownlink	Gelen Downlink İstatistiği	«Tüm gelen proxy'lerin gelen trafiği için istatistik toplamalarını etkinleştirir.»	true
statsOutboundUplink	Giden Uplink İstatistiği	«Tüm giden proxy'lerin giden trafiği için istatistik toplamalarını etkinleştirir.»	false
statsOutboundDownlink	Giden Downlink İstatistiği	«Tüm giden proxy'lerin gelen trafiği için istatistik toplamalarını etkinleştirir.»	false

İstemci ve inbound'ların trafik istatistikleri (uplink/downlink) – panodaki ve istemcilerdeki trafik görüntülemesinin temelidir; devre dışı bırakılması önerilmez. outbound istatistiği varsayılan olarak kapalıdır ve yalnızca giden etiketlerine göre trafik izliyorsanız gereklidir.

Metrics

Referans şablonda `metrics` bölümü (`listen: "127.0.0.1:1111"` , `tag: "metrics_out"`) ve karşılık gelen `metrics_out` API'si bulunur. Panel bu listener'ı metrik ve observatory anlık görüntülerini toplamak için kullanır; şablondan `metrics.listen` 'ı ayırıştırır, `/debug/vars` 'ı sorgular ve etikete göre gecikme geçmişini toplar. `metrics.listen` adresini/portunu değiştirirseniz panel yeni adrese başvurur; `metrics` bölümünü silmek observatory grafik toplamayı devre dışı bırakır.

HTTP modundaki outbound testi, ana yapılandırmadakiyle aynı listener olmayan, rasgele bir portta kendi `metrics` -listener'ına sahip ayrı bir geçici Xray örneği başlatır.

11.11. Kaydetme, yeniden başlatma ve otomatik dönüşümler

Düğmeler

Düğme	Eylem
Kaydet	POST /xray/update : şablonu ve outboundTestUrl 'yi doğrular ve kaydeder.
Xray'i Yeniden Başlat	Hizmeti kaydedilmiş yapılandırma ile yeniden yükler. Onay: «Xray yeniden başlatılsın mı?» / «Xray hizmetini kaydedilmiş yapılandırma ile yeniden yükler.»

Bildirim mesajları: başarı – «Xray başarıyla yeniden başlatıldı», «Xray başarıyla durduruldu»; hatalar – «Xray yeniden başlatılırken hata oluştu.», «Xray durdurulurken hata oluştu.» **Xray Yeniden Başlatma Çıktısı** penceresi çekirdeğin tanılama çıktısını gösterir.

Değişiklikleri sıcak uygulama (tam yeniden başlatma olmadan)

inbounds, outbounds ve yönlendirme kurallarındaki değişiklikler «canlı» uygulanır: **Kaydet** düğmesine basıldığında panel, eski ve yeni yapılandırma arasındaki farkı hesaplar ve yalnızca değişen parçaları gRPC API Xray (HandlerService/RoutingService) aracılığıyla işlemi yeniden başlatmadan uygular. Tam yeniden başlatma yalnızca sıcak yeniden yükleme API'si olmayan bölümler değiştiğinde otomatik olarak gerçekleştirilir (log , dns , policy , observatory vb.). Bu nedenle Xray sayfasında ayrı bir «Yeniden Başlat» düğmesi gerekmez – **Kaydet** değişiklikleri kendisi uygular. Gerektiğinde çekirdeğin yeniden başlatılması otomatik olarak gerçekleştirilmeye devam eder (aynı zamanda abonelik güncellemeleri ve WARP rotasyonu sırasında otomatik yeniden yüklemeye bakın).

Varsayılan şablonu geri yükleme

GET /xray/getDefaultJsonConfig uç noktası referans şablonu (config.json , ikili dosyaya gömülü) döndürür. Bu, yapılandırmayı fabrika varsayılanına sıfırlamak için kullanılabilir.

Kaydetme sırasında otomatik dönüşümler

Xray ayarları kaydedilirken panel şu işlemleri gerçekleştirir (bu sırayla):

- Sarmalayıcıları kaldırma** – { "xraySetting": <yapılandırma>, "inboundTags": ..., "outboundTestUrl": ... } gibi sarmalayıcıları kaldırır, eğer bunlar yanlışlıkla değere girmişse (aksi takdirde her kaydetmede katmanlar birikir). 8 katmana kadar kaldırılır.
- Yapılandırma kontrolü** – JSON, Xray yapılandırma yapısına ayrıştırılır; hata durumunda – «xray template config invalid» ile reddedilir.
- İstatistik kuralının garantisi** – inboundTag: ["api"] → outboundTag: "api" kuralı, routing.rules 'da zorunlu olarak 0. konuma yükseltir (veya yoksa eklenir). Bu, panel gRPC istatistik

isteğinin üst sıradaki catch-all kuralı tarafından yakalanmamasını garanti eder (aksi takdirde proxy çalışırken istemciler sıfır trafik ile çevrimdışı görünebilir).

1. madde nedeniyle `api` → `api` kuralını kaldırmaya veya taşımaya çalışmayın – panel her kaydetmede onu yerine geri getirecektir. Bu, istatistik servis altyapısıdır, kullanıcı tanımlı bir yol değildir.

11.12. Abonelikten outbound (otomatik güncelleme ile)

3.3.0 sürümünden itibaren panel, VPN sağlayıcılarının istemci uygulamaları için dağıttığı biçimle aynı formattaki abonelik URL'sinden doğrudan `outbound` 'ları içe aktarabilir. Abonelikler arka planda düzenli olarak yeniden okunur, böylece sunucudaki `outbound` 'lar yapılandırma şablonu manuel olarak düzenlenmeden güncel tutulur. 3.5.0'dan itibaren query parametrelili (`?plugin=` , `?type=`) veya sonda `/` bulunan `ss://` bağlantılarının (SIP002) portu doğru ayrıştırılır (önceden sessizce `0` oluyordu); portu gerçekten geçersiz olan bağlantı ise bozuk şekilde içe aktarılmak yerine atlanır.

Bölüm panelde «**Giden Abonelikleri**» olarak adlandırılır, açıklaması: «Uzak abonelik URL'lerinden (vmess/vless/trojan/ss/...) gidenleri içe aktarın. Etiketler yük dengeleyicilerde ve yönlendirme kurallarında kullanım için değişmeden kalır. Güncelleme otomatik olarak yapılır.» Bölüm, Xray sayfasında `outbound` ayar panelinin üzerinde yer alır.

Nasıl çalışır

Abonelikler, Xray yapılandırma şablonundan ayrı saklanır. Şablon **hiçbir zaman üzerine yazılmaz**: aboneliklerden gelen `outbound` 'lar, Xray yapılandırması oluşturulurken her seferinde anında nihai yapılandırmaya eklenir.

Abonelik ekleme

«Abonelik Ekle» formunda şu alanlar mevcuttur:

Alan	Anahtar	Varsayılan	Amaç
Abonelik URL'si	<code>url</code>	– (zorunlu)	Abonelik adresi. Yer tutucu: «https://... (base64 ile kodlanmış bağlantı listesi)». Yalnızca HTTP(S) kabul edilir; adres güvenlik açısından kontrol edilir.
Açıklama	<code>remark</code>	boş	Serbest etiket (yer tutucu «örn. HK düğümleri»).
Etiket öneki	<code>tagPrefix</code>	<code>subN–</code>	İçe aktarılan <code>outbound</code> etiketlerinin başladığı önek. Boş bırakılırsa panel <code>sub1–</code> , <code>sub2–</code> gibi en küçük boş numarayı otomatik seçer.
Güncelleme aralığı	<code>updateInterval</code>	600 saniye (10 dakika)	Aboneliğin ne sıklıkla yeniden okunacağı. UI'da saat/dakika cinsinden ayarlanır.
Etkin	<code>enabled</code>	evet (<code>true</code>)	Yalnızca etkin abonelikler yapılandırmaya dahil edilir ve otomatik güncellenir.
Özel adreslere izin ver	<code>allowPrivate</code>	hayır (<code>false</code>)	localhost, LAN ve özel IP'lerdeki URL'lere izin verir. SSRF koruması için varsayılan olarak kapalıdır – yalnızca güvenilir yerel kaynak için etkinleştirin.

Alan	Anahtar	Varsayılan	Amaç
Manuel gidenlerin önünde	prepend	hayır (false)	Etkinleştirilirse bu aboneliğin outbound 'ları şablondaki manuel outbound 'ların önüne konur ve biri varsayılan outbound olabilir. Aksi takdirde sonraya eklenir.

«Önizleme» düğmesi (POST /outbound-subs/parse), kaydetmeden önce URL'yi indirip ayrıştırmanıza ve hangi outbound 'ların ve etiketlerin elde edileceğini görmenize olanak tanır; bu sırada veritabanına hiçbir şey yazılmaz. URL'den hiçbir şey tanınmazsa «Bu URL'den giden bulunamadı.» mesajı görüntülenir.

Genel outbound listesindeki birden fazla aboneliğin sırası öncelik (priority) ile ayarlanır ve yukarı/aşağı okları ile değiştirilir (POST /outbound-subs/:id/move).

Hangi abonelik biçimleri kabul edilir

URL'deki yanıt gövdesi şu şekilde işlenir:

- İçerik önce **base64** olarak denir (standart ve URL-safe varyantlar, otomatik dolgu ve boşluk/satır sonu kaldırma ile). base64 ise çözülür; değilse olduğu gibi alınır.
- Ardından gövde satırlara bölünür. # ile başlamayan her boş olmayan satır bağlantı olarak ayrıştırılır. Tanınmayan satırlar (yorumlar, desteklenmeyen protokoller) sessizce atlanır.
- Desteklenen bağlantı şemaları: vmess:// , vless:// , trojan:// , ss:// (Shadowsocks), hysteria2:// / hy2:// , wireguard:// / wg:// .

Yani çoğu sağlayıcıda olduğu gibi «base64 kodlanmış bağlantı listesi» formatındaki normal abonelikler uygundur.

Kararlı etiketler

Her bağlantı için kararlı bir «kimlik» hesaplanır (parça-açıklama olmadan URI çekirdeği; vmess için ps alanı olmayan dahili JSON). «Kimlik → etiket» eşleşmesi korunur ve sonraki güncelleme sırasında aynı sunucu, açıklama veya ikincil parametreler değişmiş olsa bile aynı etiketi alır. Bu, güncellemelerden sonra yük dengeleyicilerin ve yönlendirme kurallarının çalışmaya devam etmesi için özel olarak yapılmıştır:

- Yük dengeleyici/kuraldaki tam etiket, aynı sunucuyu işaret etmeye devam eder.
- Önek/joker karakter seçici (örn. hk-*), aboneliğin daha sonra döndüreceği yeni sunucuları otomatik olarak yakalar – bu, bir «havuza abone olmak» için önerilen yöntemdir.
- Sunucu abonelikten kaybolursa etiketi nihai outbound dizisinden yalnızca düşer; yük dengeleyicide fallbackTag varsa Xray onu kullanır.
- Sağlayıcı sunucunun UUID/host/kimlik bilgilerini değiştirirse kimlik değişir – bu, yeni etiketli yeni bir outbound olarak kabul edilir.

Tek bir çıkış içinde etiketler -N son ekiyle tekilleştirilir. Abonelik etiketleri ASCII olmayan karakterleri (örn. Kiril) korur ve okunabilir kalır: Unicode harf ve rakamlar slug'da korunur, noktalama işaretleri ise tire ile değiştirilir – Kiril adlarından gelen etiketler artık yalnızca rakamlara indirgenmez.

Otomatik güncelleme nasıl çalışır

- Abonelik güncelleme arka plan görevi **her 5 dakikada bir** çalışır.
- Her çalışmada etkin abonelikleri dolaşır ve yalnızca kendi aralığı dolmuş olanları günceller: bir abonelik henüz hiç güncellenmemişse ya da son güncellemeden bu yana en az `updateInterval` kadar süre geçmişse güncellenir. Bu şekilde görev abonelikleri sık kontrol eder, ancak her belirli abonelik kendi `updateInterval` 'inden (varsayılan 10 dakika) daha sık okunmaz. UI'da buna karşılık gelen bir açıklama ile yansıtılır.
- Güncelleme: URL güvenlik açısından yeniden genel olarak kontrol edilir (özel adresler, abonelikte `allowPrivate` ayarlı değilse engellenir), istek panel proxy istemcisi üzerinden `User-Agent: 3x-ui-outbound-sub/1.0` başlığıyla gönderilir. Yönlendirme zinciri 10 atlamayla sınırlıdır ve her atlama da özellik açısından kontrol edilir (SSRF koruması). HTTP 200 beklenir; aksi takdirde hata kaydedilir.
- Başarılı ayrıştırma sonrasında sonuç kaydedilir, son güncelleme zamanı ayarlanır, hata temizlenir. Hata durumunda metni UI'da «Son Hata» olarak görünür ve önceden alınan `outbound` 'lar geçerliliğini korur.
- En az bir abonelik gerçekten güncellendiyse görev Xray'i yeniden başlatılması gereken olarak işaretler ve arayüzün yeni `outbound` 'ları çekmesi için UI geçersizleştirme gönderir. Xray'in gerçek yeniden yüklenmesi yönetici tarafındaki en yakın 30 saniyelik döngüde gerçekleşir.

Tek bir aboneliğin manuel güncellenmesi – «**Şimdi Güncelle**» düğmesi (`POST /outbound-subs/:id/refresh`); bu da Xray'i yeniden başlatılacak olarak işaretler. Abonelik ekleme, değiştirme, silme işlemleri de Xray yeniden başlatma bayrağını kaldırır (silme işleminde `outbound` 'ları sonraki yeniden yüklemeye yapılandırmadan düşer). UI şunu önerir: «Ekleme veya güncelleme sonrasında Xray'i yeniden başlatın (ya da bir sonraki otomatik yeniden yüklemeyi bekleyin) ve gidenlerin etkin olmasını sağlayın.»

Xray yapılandırmasına nasıl dahil edilir

Xray yapılandırması her oluşturulduğunda, etkin abonelik `outbound` 'ları iki gruba ayrılır – `prepend` («Manuel gidenlerin önünde» bayrağı) ve geri kalanlar – ve şablonla birleştirilir:

`[abonelik prepend'leri] + [şablon outbound'ları] + [diğer abonelikler]` . Her grup içinde abonelikler önceliğe göre sıralanır. Şablondaki manuel `outbound` 'lar bu işlemde etkilenmez; şablonun `outbound` 'lar dizisi herhangi bir nedenle ayrıştırılmazsa abonelik `outbound` 'ları karıştırılmaz (manuel olanları kaybetmemek için).

İçe aktarılan `outbound` 'lar ayrıca panelin `outbound` bölümünde ayrı bir «**Giden Aboneliklerinden (yalnızca okunabilir)**» bloğu olarak gösterilir – orada düzenlenemezler, yönetim yalnızca «Giden Abonelikleri» bölümü üzerinden yapılır.

11.13. WARP'ta IP rotasyonu

3X-UI'de bir WARP-outbound kurulabilir – Cloudflare WARP'a bir WireGuard giden bağlantısı (Xray yapılandırmasında `warp` etiketi). Panel, Cloudflare sunucularında bir cihaz hesabı kaydettirir, WireGuard anahtarlarını ve adreslerini alır ve bunları `warp` etiketli outbound'a yerleştirir. Bu outbound aracılığıyla trafik, Cloudflare WARP IP adresi altında internete çıkar. 3.3.0 sürümünün yeniliği – WARP hesabını manuel olarak yeniden oluşturmadan bu giden IP'yi manuel olarak veya zamanlamaya göre değiştirebilme imkânı.

Yönetim, **Xray** bölümündeki WARP kartında bulunur («WARP Hesabı Oluştur» ve yapılandırmayı aldıktan sonra; öncesinde eylemler kullanılamaz – panel «Önce WARP yapılandırmasını alın» mesajı gösterir).

IP değiştirildiğinde ne olur

«IP Değiştir» düğmesi IP değişimini başlatır. Mantık:

1. Yeni bir WireGuard anahtar çifti oluşturulur.
2. Yeni anahtarla Cloudflare sunucularına yeni bir WARP cihazı kaydedilir (yeni `device_id` , `access_token` , adresler ve peer verileri).
3. Yeni veriler, Xray yapılandırmasının WARP-outbound'una yazılır: `secretKey` , `address` (v4 /32 ve v6 /128) , `reserved` (`client_id` 'den) ve peer'in `publicKey` ve `endpoint` 'i güncellenir.
4. Daha önce bir WARP+ lisans anahtarı ayarlandıysa (en az 26 karakter uzunluğunda), yeni hesaba otomatik olarak yeniden uygulanır. Başarısızlık durumunda bu yalnızca günlüklerde uyarıdır – IP değişimi iptal edilmez.
5. Başarılı değişim sonrasında Xray, yeni outbound'un geçerli olması için yeniden başlatma gerektiren olarak işaretlenir.

Başarı durumunda arayüz «WARP IP adresi başarıyla değiştirildi!» mesajı gösterir.

Zamanlamaya göre otomatik rotasyon

WARP kartında «IP Adresini Otomatik Güncelle» anahtarı ve «Aralık (gün)» alanı bulunur. Açıklama: «0 – devre dışı. IP adresini otomatik olarak değiştirir.»

Parametre	Değer
Veritabanındaki ayar	<code>warpUpdateInterval</code> (tam sayı, ≥ 0)
Varsayılan değer	0 (otomatik rotasyon kapalı)
Ölçü birimi	gün
0	otomatik değişimi devre dışı bırakır
> 0	IP'yi her N günde bir değiştir

Aralığı kaydetmek `warpUpdateInterval` 'i kaydeder ve 0'dan büyük bir değerle «son güncelleme zamanı»nı sıfır an olarak ayarlar – aksi takdirde zamanlayıcı IP'yi bir sonraki tik'te değiştirirdi.

Zamanlamayı, saatte bir başlatılan bir arka plan görevi yürütür – yani panel saatte bir rotasyon zamanının gelip gelmediğini kontrol eder. Kontrol algoritması:

- aralık ≤ 0 ise – hiçbir şey yapmaz;
- «son güncelleme zamanı» 0'a eşitse (örn. aralık veritabanında doğrudan düzenlenerek ayarlandıysa) – bu ilk çalıştırmadır: görev yalnızca temel zaman damgasını kaydeder ve IP'yi hemen **değiştirmez**;
- son güncellemeden bu yana `aralık  $\times 24 \times 3600$` saniye veya daha fazla geçmişse – aynı IP değişimi gerçekleştirilir, zaman damgası güncellenir ve Xray yeniden başlatması planlanır.

Önemli bir ayrıntı: «IP Değiştir» düğmesiyle manuel değişim de son güncelleme zaman damgasını sıfırlar. Bu nedenle manuel rotasyondan sonra otomatik aralığın geri sayımı yeniden başlar ve planlanan değişim hemen arkasından tetiklenmez.

«Panel proxy'si üzerinden»

3.3.1'de değiştirildi. Ayrı «Panel Ağ Proxy'si» (`panelProxy`) ayarı kaldırıldı. Panelin kendi giden trafiği (WARP API istekleri dahil) artık seçilen **panel trafiği için outbound** – Xray-outbound veya yük dengeleyici – üzerinden yönlendirilir (bkz. bölüm 13). Aşağıdaki açıklama 3.3.1 öncesi sürümlere aittir.

Cloudflare WARP API'sine yapılan tüm istekler (kayıt, yapılandırma alma, lisans ayarlama, IP değiştirme) doğrudan değil, 15 saniyelik zaman aşımli panel HTTP istemcisi üzerinden yapılır. Bu istemci, panel ayarlarından «**Panel Ağ Proxy'si**» (`panelProxy`) ayarına saygı gösterir.

Ayar açıklamasından: proxy, panelin kendi giden isteklerini yönlendirir (geo-veritabanı güncellemeleri, Xray/panel sürüm kontrolleri, Telegram ve artık WARP aramaları) – sunucu filtrelemesini aşmak için. `socks5://` veya `http(s)://` şeklinde adresler kabul edilir, örneğin Xray'in kendi yerel SOCKS gelen bağlantısı. Alan boşsa veya proxy yanlış ayarlanmışsa – doğrudan bağlantı kullanılır (davranış bozulmaz).

WARP için faydası: sunucu `api.cloudflareclient.com` 'a doğrudan ulaşamıyorsa kayıt ve rotasyon daha önce başarısız oluyordu. Artık `panelProxy` 'ye çalışan bir proxy (kendi Xray inbound dahil) belirterek WARP API'sinin erişilebilirliğini ve hem manuel düğmenin hem de planlı rotasyonun çalışabilirliğini garanti edebilirsiniz.

Ne zaman işe yarar

- outbound için giden IP'yi düzenli olarak değiştirme (WARP üzerinden giden bağlantı için) – tek bir adrese göre engelleme ve izleme riskini azaltır.
- Mevcut Cloudflare adresi kara listeye girmişse veya yavaş çalışıyorsa IP'yi manuel olarak «tazeleyin».
- Cloudflare WARP API'sine doğrudan erişimi olmayan sunucular: isteklerin `panelProxy` üzerinden yönlendirilmesi, kaydı ve rotasyonu işlevsel kılar.

12. Düğümler (çoklu panel, master/slave)

Düğümler bölümü, sıradan bir 3X-UI kurulumunu, diğer 3X-UI (alt) panellerini uzaktan izleyen ve yöneten bir **merkezi (master) panele** dönüştürür. Her düğüm, kendi sunucusunda ayrı bir 3X-UI kurulumudur; master, kendi HTTP API'si üzerinden bu kurulumla bağlanır, durumunu sorgular ve kendisine atanmış inbound'ları ve istemcileri senkronize eder. Bu özellik **çoklu panel** imkânını sunar: her panele ayrı ayrı giriş yapmak yerine tüm sunucuları tek bir listede görür ve merkezi olarak yönetirsiniz.

Önemli ilke: **Düğüm bir ajan değil, tam teşekküllü bir 3X-UI panelidir.** Master, üzerine hiçbir şey "yüklemeyi" — yalnızca token aracılığıyla API'sine bağlanır. Düğümü listeden silmek yalnızca izlemeyi durdurur; uzaktaki panelin kendisi bundan etkilenmez (ipucu: «Bu işlem düğüm izlemeyi durduracak. Uzaktaki panelin kendisi etkilenmeyecek»).

12.1. Liste Başındaki Özet

Düğümler tablosunun üstünde toplu sayaçlar görüntülenir:

Alan	Açıklama
Toplam düğüm	Listedeki toplam düğüm sayısı.
Çevrimiçi	online durumundaki düğüm sayısı.
Çevrimdışı	offline durumundaki düğüm sayısı.
Ortalama gecikme	Düğümlere olan ortalama gecikme (ping), milisaniye cinsinden.

12.2. Düğüm Ekleme ve Düzenleme

Düğüm Ekle ve **Düğümü Düzenle** düğmeleri, düğüm alanlarını içeren bir form açar.

Ad, Adres, Port ve **API Token** alanları zorunludur (ipucu: «Ad, adres, port ve API token zorunludur»).

«Kaydet» düğmesine basıldığında (hem eklemede hem düzenlemede) panel önce 6 saniyelik zaman aşımıyla **düğümün erişilebilirliğini kontrol eder**. Düğüm yanıt vermezse kayıt yapılmaz ve hata gösterilir. Yani erişilemeyen bir düğüm eklenemez.

Form Alanları

Alan	Varsayılan	Geçerli Değerler	Açıklama
Ad	— (zorunlu)	boş olmayan, benzersiz dize	Düğümün dahili adı. Ad sütununa benzersizlik kısıtı uygulanır — aynı adla iki düğüm oluşturulamaz. Yer tutucu ipucu: örn. de-frankfurt-1 . Kaydedilirken baştaki ve sondaki boşluklar kırpılır.
Not	boş	herhangi bir dize	Düğüme ait isteğe bağlı not/açıklama. Çalışmayı etkilemez.

Alan	Varsayılan	Geçerli Değerler	Açıklama
Şema	<code>https</code>	<code>http / https</code>	Uzak panele bağlantı protokolü. Boş bırakılırsa veya geçersiz bir değer girilirse normalleştirme <code>https</code> olarak ayarlar. Düğüm düz HTTP ile yanıt verirken şema <code>https</code> olarak ayarlıysa panel anlaşılır bir ipucu döndürür: «the server speaks HTTP, not HTTPS; set the node scheme to http».
Adres	– (zorunlu)	ana bilgisayar veya IP	Uzak panelin adresi. Yer tutucu: <code>panel.example.com</code> veya <code>1.2.3.4</code> . Adres normalleştirilir; SSRF koruması amacıyla varsayılan olarak özel/yerel adresler engellenir – «Özel adrese izin ver» bölümüne bakın.
Port	– (zorunlu)	1-65535 tam sayı	Uzak düğümün web paneli portu. Aralık dışındaki değerler reddedilir («node port must be 1-65535»).
Temel Yol	<code>/</code>	yol dizesi	Uzak panelin web temel yolu (web base path), eğer ayarlanmışsa. Normalleştirilir: başında ve sonunda <code>/</code> olması garanti edilir (boş değer → <code>/</code>). Panel bu yola sorgulama sırasında <code>panel/api/server/status</code> ekler.
API Token	– (zorunlu)	uzak panelin token'ı	Düğümün API'sine erişim için Bearer token. Authorization: Bearer <token> başlığıyla iletilir. Yer tutucu: «Uzak panelin Ayarlar sayfasındaki Token». İpucu: «Uzak panel, API token'ını Ayarlar → API Token bölümünde gösterir». Yani token düğümün kendisinde (Ayarlar → API Token) oluşturulup buraya yapııştırılmalıdır.
Etkin	<code>true</code>	evet/hayır	Düğüm izleme ve senkronizasyonunu etkinleştirir. Devre dışı düğümler arka plan görevleri (heartbeat ve trafik senkronizasyonu) tarafından sorgulanmaz ve toplu panel güncellemesine dahil edilmez.
Özel adrese izin ver	<code>false</code>	evet/hayır	SSRF korumasını kaldırır ve özel/yerel adreslerle düğüme bağlanmaya izin verir. İpucu: «Yalnızca özel ağdaki veya VPN üzerindeki düğümler için etkinleştirin». Yalnızca düğüm gerçekten özel bir ağda veya VPN üzerinden erişilebilir durumdaysa etkinleştirin.

Düğüm Tarafında Token Alma ve Yenileme

Token, uzak panelin **Ayarlar** → **API Token** bölümünden alınır. Aynı yerden yenilenebilir: **Token'ı Yeniden Oluştur** düğmesi uyarıyla birlikte çalışır: «Yeniden oluşturma mevcut token'ı geçersiz kılar. Onu kullanan tüm merkezi paneller, güncelleme yapılarına kadar erişimi kaybeder. Devam edilsin mi?». Yenilemeden sonra master paneldeki eski token çalışmayı durdurur – düğüm formunda güncellenmesi gerekir.

Bağlantı outbound (Connection outbound)

Connection outbound (`outboundTag`) alanı, master'ın bu düğümün API'sine yönelik trafiğinin sunucuyu nasıl terk edeceğini belirler. Bir Xray outbound etiketi seçilirse panelin düğüme yönelik istekleri doğrudan değil, belirtilen outbound üzerinden yönlendirilir; panel çalışan yapılandırmaya otomatik olarak loopback üzerinde bir köprü inbound ekler ve bunu yeniden başlatma gerektirmeden canlı olarak uygular. İpucu: «Route this node's panel API traffic through the selected Xray outbound. A loopback bridge inbound is added to the running config automatically and applied live. Leave empty for a direct connection».

Seçici, panel outbound seçimine benzer şekilde çalışır: etiketler **Outbounds** (normal giden bağlantılar) ve **Balancers** (dengeleyiciler) olarak gruplandırılır; blackhole outbound'lar listeden gizlenir. Boş değer (yer tutucu «Direct connection») = düğüme doğrudan bağlantı anlamına gelir.

Inbound İçe Aktarma (senkronize edilecek inbound'ların seçimi)

Düğüm formunda, iki modlu **Inbound İçe Aktar** (`inboundSyncMode`) ayarı bulunur: **Tüm inbound'lar** (`all` , varsayılan) ve **Seçilenler** (`selected`). Varsayılan olarak master, bu düğümü seçmiş tüm inbound'ları düğüme senkronize eder; mevcut düğümler «Tüm inbound'lar» modunda çalışmaya devam eder.

Seçilenler modunda alanın altında bir inbound etiketi çoklu seçimi belirir. **Inbound'ları Yükle** düğmesine basın – master, henüz kaydedilmemiş bağlantı parametrelerini kullanarak düğümden inbound listesini (`POST /panel/api/nodes/inbounds` uç noktası) alır ve etiketlerini gösterir; istediklerinizi işaretleyin. Panel yalnızca işaretli etiketleri düğüme senkronize edip dağıtır; doğrudan düğümde bulunan diğer inbound'lar dokunulmadan kalır – master onları silmez ve yönetmez.

Örnek: Seçici içe aktarma için düğümden inbound listesi alma. Gövdede henüz kaydedilmemiş bağlantı parametreleri iletilir; yanıtta düğümde mevcut inbound'ların etiketleri döner:

```
POST /panel/api/nodes/inbounds
Content-Type: application/json

{ "name": "de-fra-1", "scheme": "https", "address": "node1.example.com",
  "port": 2053, "basePath": "/", "apiToken": "abcdef..." }
```

12.3. TLS Doğrulama (https düğümleri için)

Bu alan grubu, master'ın düğümün HTTPS sertifikasını nasıl doğrulayacağını belirler. Bu ayarlar **yalnızca https şeması için geçerlidir**; `http` düğümlerinde yok sayılır.

TLS Doğrulama – açılır liste, ipucu: «Panelin düğümün HTTPS sertifikasını nasıl doğruladığı. Sabitleme veya Atla – öz imzalı sertifikalar için (yalnızca https düğümleri)».

Mod	Değer	Varsayılan	Açıklama
Doğrula (standart CA)	<code>verify</code>	evet (varsayılan)	Güvenilir CA ile standart sertifika zinciri doğrulaması. Genel/Let's Encrypt sertifikalı düğümler için uygundur. Tüm <code>http</code> düğümleri için de kullanılır.
Sertifikayı Sabitle (SHA-256)	<code>pin</code>	–	CA zinciri doğrulanmaz ancak düğümün yaprak sertifikasının SHA-256 değeri kaydedilmiş parmak iziyle karşılaştırılır (constant-time karşılaştırma). Öz imzalı sertifikalar için MITM korumasını korur. Parmak izi alanının doldurulması gerekir.
Doğrulamayı Atla	<code>skip</code>	–	Sertifika doğrulaması tamamen devre dışı bırakılır. Uyarı: «Doğrulamayı atlamak, ortadaki adam saldırılarına karşı korumayı kaldırır – API token ele geçirilebilir. Sertifikayı sabitlemeniz önerilir».

3.4.0'da yukarıdaki üç moda dördüncü bir mod eklendi – **Mutual TLS (istemci sertifikası)** (`mtls`), diğerleri gibi yalnızca `https` şeması için kullanılabilir.

Mod	Değer	Varsayılan	Açıklama
Mutual TLS (istemci sertifikası)	mtls	—	Düğümün sertifikasını doğrulamanın yanı sıra master, kendisini düğüme kendi CA'sı tarafından verilen bir istemci sertifikasıyla da kanıtlar. Bu modda düğüm için API token isteğe bağlı hale gelir – düğüm, master'ı sertifikasıyla tanır. Mod seçildiğinde şu ipucu gösterilir: «This node authenticates the panel with a client certificate. Copy this panel's CA from the Node mTLS section onto the node, set its Trusted parent CA, then restart it».

Düğüm için mutual TLS etkinleştirmek için: düğüm tarafında **Mutual TLS** modunu ayarlayın, **Node mTLS** bölümünden (aşağıya bakın) yönetim panelinin CA'sını kopyalayın, bunu düğümde **güvenilir üst CA** olarak tanımlayın ve düğümü yeniden başlatın.

skip, pin veya mtls dışında herhangi bir değer seçilirse normalleştirme zorla verify olarak ayarlar.

Sertifika Sabitleme

Sertifika Sabitle seçildiğinde şunlar görünür:

- **Sabitlenmiş sertifikanın SHA-256 değeri** – giriş alanı. **base64** (Xray'den pinnedPeerCertSha256 biçimi) veya iki nokta üst üste ile ya da onsuz **hex** biçiminde (openssl -fingerprint stili) parmak izi kabul edilir. İpucu: «SHA-256 in base64 or hex. Click «Fetch» to read it from the node now». Yer tutucu: «SHA-256 base64 veya hex biçiminde». pin seçildiğinde boş veya hatalı parmak izi, kayıt sırasında doğrulama hatası verir.

Örnek: aynı parmak izinin iki biçimi. Alan her iki varyantı da kabul eder – ikisi de aynı sertifikayı temsil eder:

```
# base64 (Xray'den pinnedPeerCertSha256 biçimi)
607TNg3l2k0pq8R1sT2uV3wX4yZ5a6B7c8D9e0F1g2=

# iki nokta üst üste ile hex (openssl x509 -fingerprint -sha256 stili)
E8:E2:D3:60:DE:5D:9A:4D:29:AB:CF:11:B2:7C:34:...
```

Parmak izi henüz bilinmiyorsa **AI** düğmesine basın – master, sertifika doğrulaması yapmadan HTTPS üzerinden düğüme bağlanır ve mevcut yaprak sertifikasının SHA-256 değerini okuyarak alana yapıştırır.

- **AI** düğmesi – sertifika doğrulaması yapmadan HTTPS üzerinden düğüme bağlanır ve mevcut yaprak sertifikasının SHA-256 değerini okur (POST /certFingerprint uç noktası), alana yapıştırır. Başarı durumunda – «Düğümün mevcut sertifikası alındı»; başarısızlık durumunda – «Sertifika alınamadı». Yalnızca https düğümleri için kullanılabilir.

Node mTLS (paneller arası karşılıklı TLS kimlik doğrulaması)

Düğümler sayfasında ayrı bir **Node mTLS** bölümü bulunur – «panel → düğüm» çağrılarında API token'ına ikinci faktör (istemci sertifikası) ekleyen karşılıklı TLS kimlik doğrulaması ayarı. Mutual TLS isteğe bağlıdır; bölüm alanları boş bırakılırsa düğümler önceki şemayla çalışmaya devam eder – **yalnızca API token'ı** (ipucu: «Mutual

TLS adds a client-certificate factor on top of the API token for node-to-node calls. It is opt-in: leave it empty to keep token-only auth»). Bölümde iki işlem bulunur:

- **Bu panelin CA'sını Kopyala** (`POST /panel/api/nodes/mtls/ca`) – bu panelin kök sertifikasını (CA) panoya kopyalar. Bu CA'nın yönetilen düğümlere iletilmesi gerekir; böylece düğümler panelin istemci sertifikasına güvenir; düğümlerde bunun ardından TLS doğrulama modu **Mutual TLS** olarak ayarlanır (ipucu: «Hand this CA to the nodes this panel manages, then set their TLS verification to Mutual TLS»). Kopyalama sonrası – «CA certificate copied to clipboard».
- **Güvenilir Üst CA** (`Trusted parent CA` , `POST /panel/api/nodes/mtls/trustCA`) – bu panelin kendisi bir üst (yönetim) panel için düğüm olarak çalıştığında kullanılan alan. Yönetim panelinin CA'sını buraya yapıştırın, böylece panelden istemci sertifikası talep edilsin ve **CA'yı Kaydet** düğmesine basın. Değişiklik **panelin yeniden başlatılmasını gerektirir** (ipucu: «When this panel is itself a node, paste the managing panel's CA here to require its client certificate. Restart the panel to apply»).

12.4. Her Düğüm İçin Gösterilenler

Tablo sütunları ve düğüm kartı alanları (her heartbeat sorgulamasında güncellenen gözlemsel durum):

Alan	Açıklama
Durum	<code>online</code> / <code>offline</code> / <code>unknown</code> – aşağıya bakın.
CPU	Uzak sunucunun işlemci kullanımı, yüzde olarak.
Bellek	RAM kullanımı, yüzde olarak ($\text{current}/\text{total} \times 100$ hesabıyla).
Çalışma Süresi	Sunucunun kesintisiz çalışma süresi (saniye cinsinden).
Gecikme	Düğümün son sorgusuna yanıt süresi (ms).
Son Ping	Son başarılı heartbeat zamanı (unix saniye cinsinden; <code>0</code> = «hiç»; yakın değer «az önce» olarak gösterilir).
Xray Sürümü	Düğümde çalışan Xray-core sürümü.
Panel Sürümü	Düğümdeki 3X-UI sürümü – güncelleme göstergesi için güncel sürümle karşılaştırılır.
(inbound'lar)	Bu düğümde fiziksel olarak barındırılan inbound sayısı.
(istemciler)	Düğümün inbound'larındaki istemci sayısı.
(çevrimiçi)	Şu an çevrimiçi olan düğüm istemcisi sayısı.
(tükenenler)	Süresi dolmuş veya trafik limitini aşmış düğüm istemcisi sayısı. Elle devre dışı bırakılan istemciler bu sayaca dahil edilmez.
(hız)	Düğümde barındırılan inbound'lardaki anlık (canlı) transfer hızı.

Inbound/istemci/çevrimiçi sayaçları, düğüme yerel id yerine kararlı GUID'si (`panelGuid`) üzerinden bağlanır – böylece alt düğümdeki istemci, senkronize edildiği ara düğüme değil, düzgün biçimde alt düğüme atanır.

Düğümde barındırılan inbound'lar için sayfa, çevrimiçi istemcileri, sayaçları ve **anlık transfer hızını** gösterir. Kararlı GUID'e göre bağlama, aynı `panelGuid` 'e sahip «klonlanmış» düğümleri de doğru biçimde ayırt eder.

Düğüm Durumları

Durum	Açıklama	Koşul
online	Çevrimiçi	Düğüm, <code>panel/api/server/status</code> sorgusuna <code>success=true</code> ile yanıt verdi.
offline	Çevrimdışı	Düğüm yanıt vermedi, HTTP hatası, <code>success=false</code> ya da tanınamayan yanıt döndürdü.
unknown	Bilinmiyor	Başlangıç değeri; düğüm henüz hiç sorgulanmadı.

Başarısız sorgularda hata metni kaydedilir ve anlaşılır bir biçimde gösterilir; bu, «offline» nedeninin teşhisine yardımcı olur.

12.5. Düğüm Üzerindeki İşlemler

- **Bağlantıyı Test Et** (`POST /test`) – düğüm formunda, henüz kaydedilmemiş parametrelerle 6 saniyelik zaman aşımıyla bağlantıyı test eder. Sonuç: «Bağlantı başarılı ({ms} ms)» veya «Bağlantı kurulamadı». Kaydetmeden önce adres/port/token/TLS sorunlarını ayıklamak için kullanışlıdır.
- **Şimdi Kontrol Et** («Şimdi Kontrol Et» düğmesi, `POST /probe/:id`) – önceden kaydedilmiş bir düğümü plansız olarak sorgular; durumu ve metrikleri (CPU/bellek/çalışma süresi/gecikme/sürümler) anında günceller ve heartbeat kaydeder. Başarısızlık durumunda – «Kontrol başarısız».

Örnek: Master API'si üzerinden düğümü test etme ve sorgulama. «Bağlantıyı Test Et» formdaki henüz kaydedilmemiş parametreleri dener:

```
POST /panel/api/nodes/test
Content-Type: application/json

{ "scheme": "https", "address": "de-frankfurt-1.example.com", "port": 2053,
  "basePath": "/", "apiToken": "eyJhbGciOi...", "tlsMode": "verify" }
```

id 7 olan önceden kaydedilmiş düğümü plansız sorgulama:

```
POST /panel/api/nodes/probe/7
```

- **Paneli Güncelle** (`POST /updatePanel`, gövde: `{ids: [...]}`) – düğümde kendi standart güncelleyicisini çalıştırır: düğüm 3X-UI'nin son sürümünü indirip yeniden başlatır. **Seçilenleri Güncelle ({count})** düğmesi bunu işaretlenmiş birden fazla düğüm için aynı anda yapar. Düğümün yanında bir gösterge belirir: **Güncelleme Mevcut** veya **Güncel**, düğümün panel sürümü en son sürümle karşılaştırılarak belirlenir.

Örnek: Tek istekle birden fazla düğümü güncelleme. Gövdede işaretlenen düğümlerin id'leri iletilir; yalnızca etkin ve `online` olanlar güncellenir, geri kalanlar atlandı olarak döner.

```
POST /panel/api/nodes/updatePanel
Content-Type: application/json

{ "ids": [3, 7, 12] }
```

«Güncelleme 2 düğümde başlatıldı, 1 başarısız oldu» biçiminde yanıt: örneğin 12 numaralı düğüm offline olduğu için atlanmış olabilir.

- Onay: «{count} düğümü en son sürümüne güncellensin mi? Seçilen her düğüm son sürümünü indirip yeniden başlatacak. Yalnızca çevrimiçi etkin düğümler güncellenir».
- **Yalnızca onLine durumundaki etkin düğümler güncellenir.** Devre dışı düğüm sonuçlarda «node is disabled», offline ise «node is offline» olarak işaretlenir. Sonuç: «Güncelleme {ok} düğümde başlatıldı, {failed} başarısız oldu». Uygun düğüm seçilmemişse – «En az bir çevrimiçi etkin düğüm seçin».

Güncelleme onay iletişim kutusunda (hem tek düğüm hem toplu güncelleme için) **Geliştirme kanalına güncelle (son commit)** onay kutusu bulunur. İşaretlenirse seçilen düğümler stabil sürüm yerine dev-latest (main dalının son commit'i) rolling derlemesini yükler; işaretlenmemişse düğüm kendi normal kanalından güncellenir. Kutu işaretlendiğinde altında şu uyarı gösterilir: «Geliştirme derlemeleri main dalındaki her commit'i takip eder ve stabil sürüm değildir – otomatik geri alma yoktur». Dev bayrağı, `POST /panel/api/nodes/updatePanel` üzerinden düğümüne iletilir ve düğüm güncellemeyi tam olarak dev kanalından başlatır.

- **Set Cert from Panel** (yardımcı, `GET /webCert/:id`) – düğümde inbound oluştururken, dosyaların düğümde mevcut olması için merkezi panelin değil, **düğümün kendi** web-TLS sertifikasının yollarını doldurmaya yarar. Düğümün etkin ve erişilebilir olmasını gerektirir.
- **Düğümü Sil** (`POST /del/:id`) – onay: «"{name}" düğümü silinsin mi? Bu işlem düğüm izlemeyi durduracak. Uzaktaki panelin kendisi etkilenmeyecek». Düğüm kaydını ve birikmiş trafik istatistiklerini siler; uzaktaki panel normal çalışmaya devam eder. **Düğüm yalnızca kendisinden tüm inbound'lar kaldırıldıktan sonra silinebilir.** Düğüm hâlâ en az bir inbound bağlıysa (`node_id` üzerinden), panel «cannot delete node: N inbound(s) still attached to it; detach or delete them first» hatası vererek silme işlemi reddeder – önce bu inbound'ları ayırın veya silin, ardından düğümü silin. Bu, silinmiş bir düğümüne ait sarkan referanslarla «sahipsiz» inbound oluşmasını önler.

12.6. Metrik Geçmişi

Geçmiş düğmesi/grafiği `GET /history/:id/:metric/:bucket` uç noktasına başvurur. Kullanılabilir metrikler: **cpu** ve **mem** – her başarılı heartbeat'te biriktirilir. Toplama aralığı boyutu (`bucket`, saniye cinsinden) beyaz listeye sınırlıdır:

Örnek: Geçmiş sorgusu. 7 numaralı düğümün CPU yük geçmişi, 60 saniyelik aralıklarla toplanmış (en fazla 60 nokta döner):

```
GET /panel/api/nodes/history/7/cpu/60
```

Bellek ve «gerçek zamanlı» mod (2 s) için sırasıyla `.../7/mem/60` ve `.../7/cpu/2`. Beyaz liste dışındaki değerler reddedilir («invalid metric» / «invalid bucket»).

Bucket (s)	Kullanım Amacı
2	Gerçek zamanlı mod
30	30 saniyelik aralıklar
60	1 dakikalık aralıklar
120	2 dakikalık aralıklar

Bucket (s)	Kullanım Amacı
180	3 dakikalık aralıklar
300	5 dakikalık aralıklar

En fazla 60 nokta döner. Geçersiz metrik veya bucket reddedilir («invalid metric» / «invalid bucket»).

12.7. Inbound'lar ve İstemciler Nasıl Senkronize Edilir

Bir inbound, `node_id` alanı aracılığıyla bir düğüme «aittir» (inbound düzenleyicide düğüm seçilir):

Örnek: Düğüm formundaki token. Token, alt panelden (Ayarlar → API Token) alınıp master'ın **API Token** alanına yapıştırılır. Her sorguda master bunu başlık olarak gönderir:

```
GET https://panel.example.com:2053/panel/api/server/status
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.abc123...
```

Alt panelde bir **temel yol** (web base path) tanımlıysa, örneğin `/secret/`, master bunu `panel/api/server/status` öncesine otomatik ekler → `https://panel.example.com:2053/secret/panel/api/server/status`.

- 1. Yapılandırma dağıtımı (reconcile).** Bir düğüme bağlı inbound/istemci her değiştiğinde, düğüm «kirli» olarak işaretlenir. Arka plan görevi, **online** durumundaki her etkin düğüm için değişiklik varsa inbound'ları (`node_id` ile) düğüme dağıtır ve ardından «kirli» bayrağını sıfırlar. Devre dışı, offline veya «kirli» bir düğüm «beklemede» sayılır – bağlantı yenilenene kadar dağıtım ertelenir.
- 2. Trafik toplama.** Aynı görev, düğümde trafik anlık görüntüsü ister ve yerel istatistiklere birleştirir. Birleştirilen trafik üzerinden limit/süre aşımı kontrolü yapılır ve gerekirse istemciler devre dışı bırakılır; düğüme ait «tükenler» sayacı bunu yansıtır. Düğüm erişilemezse çevrimiçi istemcileri temizlenir.

Birden fazla panele bağlı bir istemci için master, aynı görevde her düğüme istemcinin **tüm paneller genelindeki toplam** trafik tüketimini ayrıca gönderir (düğümde master'ın GUID'ini anahtar olarak kullanan ayrı bir tabloya; her gönderimde üzerine yazılır, dolayısıyla master tarafında sıfırlama da yayılır). Düğümde istemcinin trafiğinde iki değerin büyüğü – yerel veya gönderilen – kullanılır; toplam kota aşıldığında istemci **düğümün kendisinde yerel olarak** devre dışı bırakılır (Xray'i yeniden başlatarak mevcut bağlantıları kesen otomatik devre dışı mekanizmasıyla). Bu, düğümün yalnızca kendi trafik payını görerek eksik hesaplaması ve genel limiti aşmış bir istemciye hizmet vermeye devam etmesi sorununu ortadan kaldırır. Trafik sıfırlama, otomatik yenileme veya istemci silme durumunda gönderilen sayaçlar temizlenir.

Bir düğüme yerleştirilen inbound'un **ilk** senkronizasyonunda (yeni düğüm ekleme veya inbound'u yeniden içe aktarma), master istemci trafik sayaçlarını düğümde gelen gerçek değerlerle başlatır. Önceden bu durumda inbound'un toplam sayacı doğru aktarılırken istemcilerin bireysel sayaçları sıfırlanıyor, master düğüme bağlanmadan önce biriken tüm geçmiş tüketimi eksik hesaplıyordu. Artık, inbound aynı senkronizasyonda oluşturuluyorsa yeni `client_traffics` satırı düğümde alınan sayaç değerini temel (baseline) alır; dolayısıyla bir sonraki delta sıfırdır ve trafik iki kez sayılmaz. Tohum değer yalnızca aynı pasajda oluşturulan inbound için uygulanır: halihazırda var olan bir inbound altında yeniden görünen istemci sıfırdan başlar (hayalet trafik koruması); az önce silinen ve inbound'u yeniden oluşturulan istemci «diriltilmez».

3. **Heartbeat.** Ayrı bir arka plan görevi, tüm **etkin** düğümleri periyodik olarak (paralellik sınırıyla) `panel/api/server/status` üzerinden sorgular, durum/metrik/sürümleri günceller ve web istemcileri varsa güncel düğüm ağacını WebSocket üzerinden yayınlar.

12.8. Düğüm Zincirleri (alt düğümler / geçişli düğümler)

Topoloji düz olmayabilir: bir düğüm kendisi de kendi düğümleri için master olabilir. Bu tür aşağı yöndeki paneller size **Alt Düğümler** olarak gösterilir – bunlar doğrudan düğümden alınan **yalnızca okunur** **projeksiyonlardır**.

- İpucu: «Yalnızca okunur: {üst} üzerinden erişilebilen bağlı düğüm. Onu {üst}'in kendi panelinden yönetin». Yani alt düğüm burada düzenlenemez, silinemez veya güncellenemez – tüm işlemler doğrudan üst panelden yapılır.
- Alt düğümün kimliği GUID'yle belirlenir; bu sayede çevrimiçi istemciler ve inbound'lar, `Node1` → `Node2` → `Node3` zincirinde bile fiziksel olarak barındıran düğüme atanır (master, her doğrudan düğüm üzerinden bir seviye derine «geçer»).
- Doğrudan düğüm erişilemez hale gelirse alt düğüm ön belleği temizlenir ve bağlantı yenilenene kadar alt düğümler ağaçtan kaybolur.

12.9. 3.5.0 Düzeltmeleri (düğüm kararlılığı)

3.5.0'daki, düğüm operatörünün fark edeceği düzeltme paketi:

- **İstemciyi değişiklik yapmadan kaydetmek artık düğüm trafiğini bozmuyor.** Önceden istemcinin herhangi bir şekilde kaydedilmesi (hatta «açtım – kaydettim») düğüme, Xray işleyicisinin yeniden oluşturulmasını içeren tam bir inbound güncellemesi gönderiyordu – düğüm çevrimiçi görünüyordu ama elle yeniden başlatılana kadar trafik geçirmiyordu. Artık no-op kaydetmeler hiçbir şey yapmaz; gerçek düzenlemeler ise yeniden oluşturma olmadan tek bir hafif güncellemeyle gider.
- **Düğümün Host geçersiz kılmaları master'a alınır:** düğümün inbound'u ilk kez kabul edildiğinde Hosts satırları (SNI, parmak izi, ALPN vb.) içe aktarılır ve abonelikler doğru TLS parametrelerini taşır.
- **Düğümdeki otomatik yenileme yeni bir kota penceresi açar** («30 günde bir 100 GB» türü paket yeniden tam hacmi verir).
- **İstemcinin master'da silinmesi onu düğümlerde tamamen siler** (kayıt + trafik), yalnızca bağlantısını kesmez; önceki toplu silmelerden kalanlar, düğümdeki «Bağız İstemcileri Sil» işlemiyle temizlenebilir.
- **Düğümün inbound'ları ilk kabulden önce süpürülmez:** yeni eklenmiş bir düğümü kaydetmek artık onun canlı inbound'larını silemez.
- **Tek bir hatalı inbound, düğümün trafik senkronizasyonunu durdurmaz** (ayrıca düğümlerdeki eski `socks` inbound'ları `mixed` olarak yeniden adlandırılır).
- **Port çakışması denetimi kendi düğümüyle sınırlandırıldı** – düğümdeki inbound'un dinleme adresini düzenlemek, başka bir düğümdeki aynı port nedeniyle artık reddedilmez («port ... already used by inbound ...»).

12.10. Düğümler: 3.3.0'daki Yenilikler

3.3.0 sürümünde **Düğümler** bölümü üç önemli iyileştirme aldı: çok atlamalı (multi-hop) topolojilerde doğru trafik ve çevrimiçi istemci ataması, düğümler arasında istemci IP senkronizasyonu ve düğümün paneli çevrimiçiyken Xray çekirdeğinin çökmesi durumu için ayrı bir durum göstergesi.

1. Multi-hop: Alt Düğüm Zincirinde Doğru Trafik Ataması

Önceden sayaçlar (inbound sayısı, çevrimiçi istemciler, tükenenler) «doğrudan» düğüm düzeyinde hesaplanıyordu. Master → Düğüm1 → Düğüm2 → Düğüm3 gibi bir zincirde, fiziksel olarak Düğüm2 / Düğüm3 üzerinde yaşayan her şey, master'a ulaşmayı sağlayan Düğüm1'e hatalı biçimde atanıyordu. 3.3.0'da atama gerçek kaynağa göre yapılır.

Nasıl çalışır:

- **Alt düğümler ayrı satırlar olarak görünür.** Her panel kendi doğrudan düğümlerinin listesini yayınlar; yalnızca bilinen Guid'e sahip düğümler dahil edilir – kararlı kimlik, düğümü bir «atlama» yukarıya atamak için gereklidir. Master, heartbeat görevinden periyodik olarak bu listeleri çeker, önbellege alır ve ardından doğrudan düğümlere «geçişli» alt düğümler ekler.
- **Geçişli düğümler yalnızca okunurdur.** Arayüzde «Alt Düğüm» olarak etiketlenirler; ipucu: «Yalnızca okunur: {üst} üzerinden erişilebilen bağlı düğüm. Onu {üst}'in kendi panelinden yönetin.» Bu satırda yönetim düğmeleri yoktur – düğüm, doğrudan üst panelden yönetilir.
- **GUID üzerinden hiyerarşi.** Doğrudan bir düğümün ParentGuid'i master'ın GUID'idir; geçişli bir düğümünki ise kendi üst düğümünün GUID'idir. Bu sayede ağaç oluşturulur.
- **Sayaçlar için gerçeklik kaynağı – inbound üzerindeki origin_node_guid'dir.** Bu, inbound'u fiziksel olarak barındıran düğümün panelGuid'idir. Düğümünden inbound senkronize edilirken atanır ve **sonraki atlamalar sırasında olduğu gibi korunur**; bu nedenle derin iç içe geçmiş bir inbound, ara düğüme değil gerçek düğüme atanır. Bu GUID'e göre inbound sayısı, çevrimiçi istemci ve tükenmiş istemci sayaçları yeniden hesaplanır. Anahtar seçim mantığı:

Inbound durumu	Kime atanır
origin_node_guid tanımlı	bu GUID'e (gerçek kaynak düğüm)
boş ama node_id tanımlı	düğümün yapay GUID'i (henüz panelGuid'ini bildirmemiş eski derleme)
boş ve node_id de boş	master'ın kendi GUID'i (yerel Xray'deki inbound)

Çevrimiçi istemciler de GUID'e göre gruplanır; bu nedenle her düğüm satırı yalnızca gerçekten kendisine bağlı olanları gösterir.

Kullanıcı ne görür: Düz topolojide (düğümler doğrudan master'ın altında) hiçbir şey değişmez – GUID ve id'ye göre sayaçlar aynıdır. Ancak düğüm zinciri olduğu anda listede «Alt Düğüm» satırları belirir ve her düğümün inbound/çevrimiçi/tükenmiş sayıları artık transit olarak geçen her şeyin toplamını değil, yalnızca o düğümün kendi yükünü yansıtır.

2. Düğümler Arasında access.log'dan İstemci IP Senkronizasyonu

IP başına istemci limiti (limitIp), Xray'in access.log dosyasına yazdığı adreslere dayanır. Önceden her düğüm yalnızca kendi bağlantılarını görüyordu; bu nedenle «istemci başına en fazla N IP» kısıtlaması kümede çalışmıyordu: bir istemci farklı düğümlere bağlanarak limiti aşabiliyordu. 3.3.0'da gözlemlenen IP'ler tüm küme genelinde senkronize edilir.

Nasıl çalışır:

- Her düğümde bir arka plan görevi `access.log`'u ayırıştırır; her satırdan IP, istemci e-postası ve zaman damgasını çıkararak yerel tabloya depolar (e-posta başına bir kayıt, IP'ler JSON dizisi olarak `{ip, timestamp}` biçiminde tutulur). `127.0.0.1` ve `::1` yerel adresleri atılır.
- 10 saniyede bir** senkronizasyon, çevrimiçi her etkin düğüm için iki yönlü alışveriş yapar: IP'leri düğümden çeker ve yerel tabloya birleştirir, ardından master'ın genel resmini düğüme gönderir.
- Birleştirme, birden fazla düğümde görülen aynı IP'yi **çift saymadan** ve eski kayıtları **diriltmeden** mevcut ile gelen gözlemleri bir araya getirir: yerel görevdekiyle aynı eskime eşiği uygulanır – **30 dakika**. Her IP için en yeni zaman damgası saklanır. Başka düğümlerden gelen kayıtlar yeni bir yerel id alır (düğüm id alanları bağımsızdır); aynı e-postanın eş zamanlı eklenmesi, tekrarlara karşı korunur.
- Limit hesaplanırken, ya mevcut yerel taramada görülen ya da senkronize edilmiş veritabanında çok taze bir zaman damgasına sahip (**2 dakika içinde**) IP «canlı» sayılır. Bu, limitin tüm küme genelinde çalışmasını sağlar; adres başka bir düğümde görülmüş olsa bile. Limit aşıldığında en eski «canlı» IP'ler fail2ban günlüğüne gönderilir ve bağlantılar zorla sonlandırılır (Xray API üzerinden istemci kaldırma/yeniden ekleme).

Kullanıcı ne görür: IP sayısı kısıtlaması artık her düğüm için ayrı ayrı değil, tüm küme genelinde geçerlidir; panelde bir istemci için herhangi bir düğümde (30 dakikalık pencere içinde) görülen IP'ler gösterilir. Bu için ayrı bir düğme veya ayar yoktur – senkronizasyon arka planda otomatik çalışır; düğümde `access.log` etkin ve erişilebilir olduğu sürece (limitin kendisi için düğümde Fail2Ban da gereklidir).

3. Ayrı Durum Göstergesi: Düğüm Paneli Çevrimiçi ama Xray Çökmüş

Önceden düğüm durumu özünde «çevrimiçi / çevrimdışı» şeklindeydi. Düğüm paneli yanıt veriyorsa, üzerindeki Xray çekirdeği çalışmıyor ve istemciler gerçekte bağlanamıyor olsa bile düğüm çevrimiçi sayılıyordu. 3.3.0'da panelin sağlığı ve Xray çekirdeğinin sağlığı birbirinden ayrıldı.

Nasıl çalışır:

- Düğüm sorgulanırken master, uzak `/panel/api/server/status` yanıtından `xray.state` ve `xray.errorMessage` alanlarını alır ve düğüme kaydeder. Bu alanlar, çekirdek sağlıklı olduğunda bile başarılı panel ping'inde doldurulur – tam olarak panelin erişilebilirliğini Xray durumundan ayırt etmek için.
- `xray.state` değerleri: `"running"` (çalışıyor), `"stop"` (durduruldu), `"error"` (hata).
- Bu değerler düğüm durumlarına dönüştürülür. Tanıdık olanlara yenileri eklendi:

Durum Anahtarı	Etiket	Koşul
<code>online</code>	«Çevrimiçi»	panel yanıt veriyor, Xray çalışıyor (<code>running</code>)
<code>offline</code>	«Çevrimdışı»	panel erişilemez / ping başarısız
<code>unknown</code>	«Bilinmiyor»	durum henüz belirlenmedi
<code>xrayError</code>	«Xray Hatası»	panel çevrimiçi ama Xray <code>error</code> durumunda (var olan <code>errorMessage</code> ile)
<code>xrayStopped</code>	«Durduruldu»	panel çevrimiçi ama Xray durdurulmuş (<code>stop</code>)

- Bu tür durum için arayüzde **ayrı bir mor gösterge** kullanılır (çevrimiçi için yeşil ve çevrimdışı için kırmızıdan farklı bir renk). Mor, düğüme ulaşılabildiğini ancak sorunun ağda veya panelin kendisinde değil, Xray çekirdeğinde olduğunu doğrudan işaret eder.

Kullanıcı ne görür: Çökmüş çekirdeğe rağmen yanıltıcı «yeşil» renk yerine düğüm **mor** olarak «**Xray Hatası**» veya «**Durduruldu**» durumuyla vurgulanır. Bu, düğümün erişilebilirliğini araştırmak yerine düğüm üzerindeki Xray'i düzeltmek (çekirdeği yeniden başlatmak, `errorMsg` 'e bakmak) gerektiğini hemen ortaya koyar. Aynı `xrayState / xrayError` geçişli alt düğümlere de (bkz. madde 1) yansıtılır; böylece çekirdekteki hatalı durum tüm zincir boyunca görülür.

13. Panel Ayarları

«Ayarlar» bölümü (sayfa başlığı – **Ayarlar**, İng. *Panel Settings*), 3X-UI web panelinin kendi davranışını yönetir: hangi adres ve portu dinlediğini, nasıl korunduğunu, Telegram botu ve harici servislerle nasıl etkileşime girdiğini ve zamanlanmış görevleri hangi saat diliminde yürüttüğünü. Her parametre, veritabanının `settings` tablosunda «anahtar – değer» çifti olarak saklanır; eğer değer veritabanında yoksa, varsayılan değer uygulanır.

Önemli – değişikliklerin uygulanması. Bu sayfadaki herhangi bir değişiklik **Kaydet** (*Save*) düğmesiyle kaydedilmeli, ardından değişikliklerin geçerli olması için panel yeniden başlatılmalıdır. Tam ipucu: «Değişiklikleri kaydedin ve uygulamak için paneli yeniden başlatın.» Kaydedildiğinde «Ayarlar değiştirildi» bildirimi görüntülenir.

13.1. Panelin Kaydedilmesi ve Yeniden Başlatılması

Öge	Amaç
Kaydet (<i>Save</i>)	Form alanlarının tümünü veritabanına yazar (<code>POST /panel/setting/update</code>). Yazılmadan önce değerler doğrulanır – geçersiz adresler, portlar veya yollar reddedilir ve panel hata döndürür.
Paneli Yeniden Başlat (<i>Restart Panel</i>)	Panel web sunucusunu yeniden başlatır (<code>POST /panel/setting/restartPanel</code>). Yeniden başlatma 3 saniyelik gecikmeyle gerçekleşir. İpucu: «Paneli yeniden başlatmak istediğinizden emin misiniz? Onaylayın; yeniden başlatma 3 saniye içinde gerçekleşecektir. Panel erişilemez hale gelirse sunucu günlüğünü kontrol edin». Başarı durumunda – «Panel başarıyla yeniden başlatıldı».
Varsayılanlara Sıfırla (<i>Reset to Default</i>)	Veritabanında kaydedilen tüm ayarları siler; bunun ardından panel varsayılan değerleri kullanır. Yönetici kimlik bilgileri bu işlemle sıfırlanmaz.

Yeniden başlatma, panel sürecine `SIGHUP` sinyali gönderilerek (veya kayıtlı yeniden başlatma kancası aracılığıyla) gerçekleştirilir. Windows'ta sinyal aracılığıyla otomatik yeniden başlatma desteklenmez. **Dinleme parametrelerindeki değişiklikler (IP, port, yol, alan adı, sertifikalar, saat dilimi) yalnızca panel yeniden başlatıldıktan sonra uygulanır.**

13.2. Genel Ayarlar (sekme «Panel» / *General*)

Arayüz Dili (*Language*)

Panel web arayüzünün dili. Kullanılabilir diller: `en-US` (İngilizce), `ru-RU` (Rusça), `zh-CN`, `zh-TW`, `fa-IR`, `ar-EG`, `es-ES`, `id-ID`, `ja-JP`, `pt-BR`, `tr-TR`, `uk-UA`, `vi-VN`. Bu bir görüntüleme ayarıdır ve Xray'in çalışmasını etkilemez.

Takvim Türü (*Calendar Type*)

- **Anahtar:** `datepicker`
- **Varsayılan değer:** `gregorian` (Miladi).

- **Amaç:** tarih seçiminde kullanılan takvim türü (örneğin istemci geçerlilik süresi belirlenirken). İpucu: «Zamanlanmış görevler bu takvime göre yürütülecektir.» Alternatif değer – Farsça (Jalali) takvimi; bu, panelin İranlı kullanıcılar arasında yaygın olması nedeniyle talep görmektedir.

Sayfalama Boyutu (*Pagination Size*)

- **Anahtar:** `pageSize`
- **Varsayılan değer:** 25
- **Geçerli değerler:** 0 ile 1000 arasında tam sayı.
- **Amaç:** tablolardaki (bağlantı/inbound listeleri) sayfa başına satır sayısı. İpucu: «Bağlantı tablosu için sayfa boyutunu belirleyin. Devre dışı bırakmak için 0 ayarlayın» – 0 seçildiğinde sayfalama devre dışı kalır ve tüm kayıtlar tek liste olarak gösterilir.
- **Panel yeniden başlatması gerekmez** (görüntüleme ayarı).

Otomatik Devre Dışı Bırakma Sonrası Xray'i Yeniden Başlat (*Restart Xray After Auto Disable*)

- **Anahtar:** `restartXrayOnClientDisable`
- **Varsayılan değer:** `true`
- **Amaç:** bir istemci otomatik olarak devre dışı bırakıldığında (geçerlilik süresi dolduğunda veya trafik limitine ulaşıldığında) Xray yeniden başlatılarak ilgili istemcinin mevcut bağlantıları kesilir. İpucu: «Bir istemci, geçerlilik süresi veya trafik limiti nedeniyle otomatik devre dışı kaldığında Xray'i yeniden başlatın.» İşlevin kendisi değişmedi – geçiş düğmesi yalnızca «Panel» (*General*) sekmesinde diğer genel ayarlarla birlikte yer almaktadır.

Not Modeli ve Ayırma Karakteri (*Remark Model & Separation Character*)

- **Anahtar:** `remarkModel`
- **Varsayılan değer:** `-ieo`
- **Amaç:** abonelikteki yapılandırma adının (remark) nasıl oluşturulacağını belirler. Dize **ilk karakterden** (ayırıcı) ve ardından **sıra harflerinden** oluşur:
 - `i` – inbound notu (*inbound remark*);
 - `e` – istemci e-postası;
 - `o` – ek etiket (*extra*).

Varsayılan `-ieo` değerinde ayırıcı `-` olup parçaların sırası şöyledir: inbound → e-posta → extra (örneğin `MyInbound-user@mail-extra`). Boş parçalar atlanır. Arayüzdeki «Örnek Not» (*Sample Remark*) alanı, oluşturulan adın önizlemesini gösterir. E-postanın ada dahil edilmesi, abonelik ayarlarındaki «E-postayı ada dahil et» parametresine de bağlıdır (abonelik bölümüne bakın).

Örnek: `remarkModel` değerinin yapılandırma adına etkisi. inbound adının `VLESS-Reality`, istemci e-postasının `alex@vpn` ve ek etiketin `RU` olduğunu varsayalım:

Alan değeri	Sonuç adı (remark)
<code>-ieo</code> (varsayılan)	<code>VLESS-Reality-alex@vpn-RU</code>

Alan değeri	Sonuç adı (remark)
_ie	VLESS-Reality_alex@vpn
-ei	alex@vpn-VLESS-Reality
o (boşluk ayırıcı, yalnızca etiket)	RU

Dizenin ilk karakteri her zaman ayırıcıdır; geri kalan harfler hangi parçaların ve hangi sırada ada dahil edileceğini belirler.

13.3. Panele Erişim: IP, Port, Yol, Alan Adı, Sertifika

Bu grup, panelin ağ giriş noktasını tanımlar. **Buradaki tüm değişiklikler yalnızca panel yeniden başlatıldıktan sonra uygulanır.**

Alan	Anahtar	Varsayılan değer	Açıklama
Panel yönetimi için IP adresi (Listen IP)	webListen	"" (boş)	Web panelinin dinlediği IP. Boş = tüm IP'lerde dinle. İpucu: «Herhangi bir IP'den bağlanmak için boş bırakın». Belirtilmişse, geçerli bir IP adresi olmalıdır (aksi takdirde kaydetme reddedilir).
Panel alan adı (Listen Domain)	webDomain	"" (boş)	İsteği alan adına göre doğrulamak için panel alan adı. Boş = herhangi bir alan adı ve IP'den bağlantıları kabul et. İpucu: «Herhangi bir alan adı ve IP'den bağlanmak için boş bırakın.»
Panel portu (Listen Port)	webPort	2053	Panelin çalıştığı port. İpucu: «Panelin çalıştığı port». 1–65535 geçerlidir. Port serbest olmalıdır; panel ve abonelik servisi aynı IP:port çiftini eş zamanlı kullanamaz.
URI yolu (URI Path)	webBasePath	/	Panel URL'sinin temel yolu (basePath). İpucu: «/' ile başlamalı ve '/' ile bitmelidir». Kaydetme sırasında panel, baştaki ve sondaki / eksikse otomatik olarak ekler. Yolda yasak karakterler reddedilir.

Panel Sertifikası (TLS / HTTPS)

Alan	Anahtar	Varsayılan değer	Açıklama
Panel sertifikası ortak anahtar dosyası yolu (Public Key Path)	webCertFile	""	Sertifika (zinciri) dosyasının tam yolu. İpucu: «/' ile başlayan tam yolu girin».
Panel sertifikası özel anahtar dosyası yolu (Private Key Path)	webKeyFile	""	Özel anahtar dosyasının tam yolu. İpucu: «/' ile başlayan tam yolu girin».

Sertifika/anahtar yollarından **en az biri** belirtilmişse, panel kaydederken «sertifika + anahtar» çiftini yüklemeye çalışır; hata oluşursa (var olmayan dosya, anahtar-sertifika uyumsuzluğu) kaydetme reddedilir. Her iki yol doğru şekilde belirtilmişse panel HTTPS'ye geçer. Her iki alan boşsa panel normal HTTP ile çalışır.

Güvenlik uyarıları (*Security warnings*). Panel, güvensiz yapılandırma tespit ettiğinde «Paneliniz açık olabilir:» uyarı bloğunu gösterir:

- normal HTTP ile çalışma – «üretim ortamı için TLS yapılandırın»;
- standart port 2053 – «rastgele biriyle değiştirin»;
- varsayılan temel yol / – «rastgele biriyle değiştirin»;
- standart abonelik yolu /sub/ ve JSON abonelik yolu /json/ – «değiştirin». Bunlar engelleyici değil, tavsiye niteliğindedir.

13.4. Oturum, Panel Proxy'si ve Güvenilir Proxy'ler (sekme «Proxy ve Sunucu» / *Proxy and Server*)

Oturum Süresi (*Session Duration*)

- **Anahtar:** sessionMaxAge
- **Varsayılan değer:** 360 (dakika, yani 6 saat).
- **Geçerli değerler:** 1 ile 525600 dakika arasında (1 yıl).
- **Amaç:** yöneticinin yeniden giriş yapmadan oturumunun ne kadar süre açık kalacağı. Birim – **dakika**. İpucu: «Sistemdeki oturum süresi (değer: dakika)».

Panel Trafiği için Outbound (*Panel Traffic Outbound*)

- **Anahtar:** panelOutbound
- **Varsayılan değer:** "" (boş = doğrudan bağlantı).
- **Amaç:** panelin **kendi isteklerini** – sürüm kontrolleri ve panel/Xray indirmeleri, Telegram çağrıları, rutin geo-dosyası güncellemeleri – göndereceği Xray **outbound**'unu belirler; böylece sunucu tarafındaki GitHub/Telegram filtrelemesi aşılır. Bu alan bir **açılır liste** olarak sunulur: içinde Xray yapılandırma şablonundaki outbound'lar, outbound aboneliklerindeki outbound'lar ve ayrıca yönlendirme **dengeleyicileri** (ayrı bir grup olarak) listelenir. blackhole türündeki outbound'lar listeden çıkarılmıştır – indirme trafiğini «kara deliğe» yönlendirmenin anlamı yoktur. Tam ipucu: «Panelin kendi isteklerini – sürüm kontrolleri ve panel/Xray indirmeleri, Telegram ve rutin geo-dosyası güncellemeleri – GitHub/Telegram sunucu filtrelemesini atlamak için bu Xray outbound'u üzerinden yönlendirir. Yerel köprü-inbound, çalışan yapılandırmaya otomatik olarak eklenir ve anında uygulanır. Xray'de yerleşik Geodata otomatik güncellemesi etkilenmez; kendine ait bir indirme outbound'u vardır. Doğrudan bağlantı için boş bırakın.»

Nasıl çalışır. Bir outbound seçildiğinde panel, çalışan yapılandırmaya kendiliğinden bir loopback-inbound (panel-egress etiketiyle SOCKS köprüsü) ve panelin kendi HTTP trafiğini seçilen outbound'a yönlendiren bir yönlendirme kuralı ekler. Bir dengeleyici seçilmişse kurala balancerTag eklenir ve panel trafiği katılımcıları arasında dağıtılır. Köprü ve kural, panelin tam yeniden başlatmasına gerek

kalmadan **anında** uygulanır. Doğrudan bağlantı için alanı boş bırakın. Xray'e yerleşik geo-verisi otomatik güncellemesi bu ayardan **etkilenmez** – Xray yönlendirmesi içinde kendi outbound'u vardır.

- **Biçim:** socks5:// (veya socks5h://) ya da http(s):// , gerektiğinde socks5:// user:pass@host:port şeklinde kimlik doğrulamayla. Desteklenen şemalar kesin olarak: socks5 , socks5h , http , https – diğer şemalar geçersiz sayılır ve panel doğrudan bağlantıya geri döner. Tipik örnek – Xray'in kendine ait yerel SOCKS-inbound'u.
- Tam ipucu: «Panelin kendi giden isteklerini (geo güncellemeleri, Xray/panel sürüm kontrolleri, Telegram) GitHub/Telegram sunucu filtrelemesini atlamak için bu proxy üzerinden yönlendirir. socks5:// veya http(s):// kabul eder, örn. Xray'in yerel SOCKS-inbound'u. Doğrudan bağlantı için boş bırakın.»
- Geçersiz bir proxy, kaydetme hatasına neden olmaz – panel yalnızca doğrudan bağlantıyı kullanır ve günlüğe bir uyarı yazar.

Alan değerleri örneği. Sunucuda 10808 portunda yerel bir Xray SOCKS-inbound'u zaten çalışıyorsa, panel isteklerini buradan yönlendirin:

```
socks5://127.0.0.1:10808
```

Kimlik doğrulamalı harici HTTP proxy için:

```
http://user:pass@proxy.example.com:8080
```

Kaydedip yeniden başlattıktan sonra panel, geo veritabanı güncellemelerini, sürüm kontrollerini ve Telegram çağrılarını belirtilen proxy üzerinden gerçekleştirecektir.

Güvenilir Proxy CIDR'leri (*Trusted proxy CIDRs*)

- **Anahtar:** trustedProxyCIDRs
- **Varsayılan değer:** 127.0.0.1/32, ::1/128 (yalnızca yerel makine).
- **Biçim:** virgülle ayrılmış IP adresleri veya CIDR alt ağları listesi (örneğin 10.0.0.0/8, 192.168.1.5). Her öge IP veya CIDR olarak doğrulanır – geçersiz değer kaydedilirken reddedilir.
- **Amaç:** X-Forwarded-Host , X-Forwarded-Proto ve gerçek istemci IP başlığını ayarlamasına izin verilen kaynakları listeler. Tam ipucu: «İletilen host, proto ve istemci IP başlıklarını ayarlamasına izin verilen IP/ CIDR, virgülle ayrılmış.» Panel ters proxy'nin (nginx, Caddy vb.) arkasında çalışıyorsa istemci IP'lerini ve şemayı doğru belirlemek için yapılandırılması gerekir.

Örnek: ters proxy arkasında panel. Nginx aynı makinede bulunuyorsa ve istekleri panele proxy yapıyorsa, yalnızca yerel makineye güveni bırakın (varsayılan değer):

```
127.0.0.1/32, ::1/128
```

Proxy, 10.0.0.0/8 iç ağındaki ayrı bir sunucudaysa, alt ağını ekleyin; aksi takdirde panel iletilen başlıkları yok sayar ve gerçek istemci yerine proxy IP'sini görür:

```
127.0.0.1/32, ::1/128, 10.0.0.0/8
```

Gerçek IP ve şemayı ileten ilgili nginx bloğu örneği:

```
proxy_set_header X-Real-IP      $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header X-Forwarded-Host $host;
```

13.5. Telegram Botu (sekme «Telegram Botu» / *Telegram Bot*)

Telegram Botunu Etkinleştir (*Enable Telegram Bot*)

- **Anahtar:** `tgBotEnable`
- **Tür/varsayılan:** mantıksal, `false`.
- **Amaç:** Telegram botunun çalışmasını etkinleştirir. İpucu: «Telegram botu aracılığıyla panel özelliklerine erişim».

Telegram Token'ı (*Telegram Token*)

- **Anahtar:** `tgBotToken`
- **Varsayılan:** `""`.
- **Amaç:** bot token'ı. İpucu: «Token'ı Telegram bot yöneticisi @botfather'dan almanız gerekmektedir».
- **Güvenlik özelliği:** token gizli bir değerdir. Panel ayar okuma yanıtında döndürülmez (alan temizlenir, yalnızca «yapılandırıldı/yapılandırılmadı» bayrağı verilir). Kaydedilirken alan boş bırakılırsa, önceden kaydedilen token **korunur** (silinmez).

Telegram Botu Dili (*Telegram Bot Language*)

- **Anahtar:** `tgLang`
- **Varsayılan:** `en-US`.
- **Amaç:** bot mesajlarının dili (web arayüzü dilinden bağımsız). Kullanılabilir diller listesi, panel dillerinkiyle aynıdır.

Bot Yöneticisi Kullanıcı Kimliği (*Admin Chat ID*)

- **Anahtar:** `tgBotChatId`
- **Varsayılan:** `""`.
- **Biçim:** virgülle ayrılmış bir veya birden fazla sayısal Telegram User ID.
- **Amaç:** bildirim alıcıları ve bot aracılığıyla paneli yönetme izni olan yöneticiler. İpucu: «Bir veya birden fazla Telegram bot yöneticisi User ID'si. User ID'yi öğrenmek için @userinfobot kullanın veya botta '/id' komutunu çalıştırın.»

Bildirim Sıklığı (*Notification Time*)

- **Anahtar:** `tgRunTime`
- **Varsayılan:** `@daily` (günde bir kez).

- **Biçim:** **Crontab** biçiminde dize (hem standart cron ifadeleri hem de `@daily`, `@hourly`, `@every 1h` gibi kısaltmalar desteklenir). İpucu: «Bildirim aralığını Crontab biçiminde belirtin». Botun periyodik raporlarını denetler.

Alan değerleri örnekleri.

Değer	Botun rapor gönderme zamanı
<code>@daily</code>	günde bir kez gece yarısı (varsayılan)
<code>@hourly</code>	her saat
<code>@every 6h</code>	her 6 saatte bir
<code>0 9 * * *</code>	her gün 09:00'da
<code>30 8 * * 1</code>	her Pazartesi 08:30'da

Saat, 13.6. bölümündeki «Saat Dilimi» ayarına göre hesaplanır.

SOCKS Proxy (SOCKS Proxy)

- **Anahtar:** `tgBotProxy`
- **Varsayılan:** `""`.
- **Amaç:** botun Telegram'a bağlanması için ayrı SOCKS5 proxy'si. İpucu: «Telegram'a bağlanmak için Socks5 proxy'ye ihtiyacınız varsa, parametrelerini kılavuza göre yapılandırın.» Özellikle bot trafiğine uygulanır (13.4. bölümündeki genel «Panel Ağ Proxy'si»nden farklıdır).

Telegram API Sunucusu (Telegram API Server)

- **Anahtar:** `tgBotAPIServer`
- **Varsayılan:** `""` (standart `api.telegram.org` sunucusunu kullan).
- **Biçim:** `http(s)://...` URL'si; kaydedilirken URL doğruluğu kontrol edilir – geçersiz adres reddedilir. İpucu: «Kullanılacak Telegram API sunucusu. Varsayılan sunucuyu kullanmak için boş bırakın.» Kendi kendinize barındırdığınız Telegram Bot API sunucusu için gereklidir.

Bot Bildirimleri (grup «Bildirimler» / *Notifications*)

Alan	Anahtar	Varsayılan	Açıklama
Veritabanı Yedekleme (Database Backup)	<code>tgBotBackup</code>	<code>false</code>	Rapor ile birlikte Telegram'a veritabanı yedek dosyası gönderir. İpucu: «Veritabanı yedek dosyası ile birlikte bildirim gönder».
Giriş Bildirimi (Login Notification)	<code>tgBotLoginNotify</code>	<code>true</code>	Panele giriş denemesi olduğunda bildir. İpucu: «Birileri panelinize giriş yapmaya çalıştığında kullanıcı adını, IP adresini ve zamanı gösterir.»
Oturum Sona Erme Bildirimi Gecikmesi (Expiration Date Notification)	<code>expireDiff</code>	<code>0</code>	İstemcinin geçerlilik süresi dolmadan kaç gün önce bildirim gönderileceği. <code>0</code> – devre dışı. <code>>= 0</code> geçerlidir. İpucu: «Eşik değerine ulaşılmadan önce oturum sona erme bildirimi al (değer: gün)».

Alan	Anahtar	Varsayılan	Açıklama
Trafik Bildirimi Eşiği (<i>Traffic Cap Notification</i>)	<code>trafficDiff</code>	<code>0</code>	Bildirim için kalan trafik eşiği. İpucu: «Eşiğe ulaşılmadan önce trafik tükenmesi bildirimi al (değer: GB)». <code>0–100</code> geçerlidir.
CPU Yük Eşiği (<i>CPU Load Notification</i>)	<code>tgCpu</code>	<code>80</code>	CPU kullanımı eşiği aşarsa yöneticilere bildir (% cinsinden). <code>0–100</code> geçerlidir. İpucu: «CPU yükü bu eşiği aşarsa Telegram'da yöneticileri bildir (değer: %)».

13.6. Tarih ve Saat (sekme «Tarih ve Saat» / *Date and Time*)

Saat Dilimi (*Time Zone*)

- **Anahtar:** `timeLocation`
- **Varsayılan değer:** `Local` (sunucunun sistem saat dilimi).
- **Biçim:** IANA tz veritabanındaki bölge adı (örneğin `Europe/Moscow`, `UTC`, `Asia/Tehran`).
- **Amaç:** panelin zamanlanmış görevleri (bot raporları, trafik sınırlama/kontrolleri, süreler) yürüttüğü saat dilimi. İpucu: «Zamanlanmış görevler bu saat dilimindeki saate göre yürütülür».
- **Doğrulama:** kaydetme sırasında bölge kontrol edilir – mevcut olmayan bölge reddedilir. Veritabanında sonradan geçersiz bir değer bulunursa panel çalışma zamanında `Local` 'e döner; o da erişilemez durumdaysa `UTC` 'ye döner.

13.7. Harici Trafik ve Xray Davranışı (sekme «Harici Trafik» / *External Traffic*)

Alan	Anahtar	Varsayılan	Açıklama
Harici Trafik Bildirimi (<i>External Traffic Inform</i>)	<code>externalTrafficInformEnable</code>	<code>false</code>	Her trafik güncellemesinde harici API'yi bildir. İpucu: «Her trafik güncellemesinde harici API'yi bildir.»
Harici Trafik Bildirimi URI'si (<i>External Traffic Inform URI</i>)	<code>externalTrafficInformURI</code>	<code>""</code>	Panelin trafik güncellemelerini gönderdiği URL. Kaydedilirken URL doğruluğu kontrol edilir. İpucu: «Trafik güncellemeleri bu URI'ye gönderilir».
Otomatik Devre Dışı Bırakma Sonrası Xray'i Yeniden Başlat (<i>Restart Xray After Auto Disable</i>)	<code>restartXrayOnClientDisable</code>	<code>true</code>	İstemci, süre dolumu veya trafik limiti nedeniyle otomatik devre dışı kaldığında Xray'i yeniden başlat. İpucu: «Bir istemci, geçerlilik süresi veya trafik limiti nedeniyle otomatik devre dışı kaldığında Xray'i yeniden başlatın.» Geçiş düğmesi «Panel» (General) sekmesindedir – bkz. 13.2. bölüm; burada bütünlük amacıyla verilmektedir.

13.8. Diğer: Xray Yapılandırma Şablonu ve Test URL'si

Xray Yapılandırma Şablonu (*xrayTemplateConfig*)

- **Anahtar:** `xrayTemplateConfig`
- **Varsayılan:** derlemeyle birlikte gelen yerleşik (embedded) JSON şablonu.
- **Amaç:** panelin inbound/outbound üzerine inşa ettiği temel Xray-core yapılandırma JSON şablonu. Bu değer, tüm ayarların normal çıktısında **verilmez** ve panel ayarları genel listesinde değil, ayrı Xray yapılandırma sayfasında düzenlenir. Standart varsayılan şablona `GET /panel/setting/getDefaultJsonConfig` üzerinden erişilebilir.

Giden Bağlantı Test URL'si (*xrayOutboundTestUrl*)

- **Anahtar:** `xrayOutboundTestUrl`
- **Varsayılan:** `https://www.google.com/generate_204`
- **Amaç:** giden (outbound) bağlantıların çalışabilirliğini test ederken kullanılan URL. Ayarlanırken HTTP(S) URL'si olarak doğrulanır.

13.9. Yönetici Hesabı ve API Token'ları

Bu parametreler ilgili sekmede («Hesap» / *Authentication*) yer alır ve güvenlik bölümünde ayrıntılı incelenir; burada yalnızca anahtarların kısa özeti verilmektedir.

- **Kimlik bilgisi değiştirme** («Geçerli kullanıcı adı», «Geçerli şifre», «Yeni kullanıcı adı», «Yeni şifre» alanları) ayrı bir `POST /panel/setting/updateUser` isteğiyle kaydedilir. Doğru geçerli kullanıcı adı ve şifre gereklidir; yeni kullanıcı adı ve şifre boş olamaz. Mesajlar: «Yönetici kimlik bilgilerini başarıyla değiştirdiniz.» / «Geçersiz kullanıcı adı veya şifre».
- **İki Faktörlü Kimlik Doğrulama (2FA)** – `twoFactorEnable` (varsayılan `false`) ve gizli `twoFactorToken` anahtarları. Token bir sırdır: 2FA etkinken kaydedilirken boş alan mevcut token'ı silmez. 2FA **ilk kez** etkinleştirildiğinde panel geçerli oturumları geçersiz kılar («giriş çağı» artırılır).
- **API token'ları** ayrı uç noktalarla yönetilir (`/panel/setting/apiTokens...`): liste, oluşturma (`apiTokens/create`), silme, etkinleştirme/devre dışı bırakma. Token'ın kendisi **yalnızca oluşturma sırasında bir kez gösterilir** ve okunabilir biçimde saklanmaz: «Bu token'ı şimdi kopyalayın. Güvenlik nedeniyle okunabilir biçimde saklanmamaktadır ve bir daha gösterilmeyecektir.»

2FA, şifreler, LDAP senkronizasyonu ve abonelik biçimleri (JSON/Clash, fragmentation, noises, mux) ayrıntıları, kılavuzun ilgili ayrı bölümlerine taşınmıştır.

13.10. 3.3.0 Sürümündeki API Değişiklikleri (entegrasyonlar için önemli)

3.3.0 sürümünde sunucu API yollarının yapısı değişti. Panele HTTP üzerinden erişen harici entegrasyonlarınız (betikler, botlar, merkezi paneller, CI görevleri) varsa, bunları **düzeltilmeniz** gerekir, aksi takdirde çalışmayı durduracaklardır.

⚠ BREAKING CHANGE: `/panel/setting/*` ve `/panel/xray/*` uç noktaları `/panel/api` altına taşındı

Daha önce panel ayar yönetimi ve Xray yapılandırması `/panel/setting/*` ve `/panel/xray/*` yolları altında ayrı konumlarda bulunuyordu. Artık her iki küme de ortak `/panel/api` API grubu içinde kayıtlıdır. Eski yollar **tamamen kaldırılmıştır** – bu yollara yapılan istekler 404 döndürecektir.

Bu neden yapıldı: `/panel/api` grubunun tamamı tek bir erişim denetimine tabi olduğundan, bu uç noktalar artık diğer API ile aynı `Authorization: Bearer <token>` başlığını kabul etmektedir. API token'ı tam yönetici erişimi anlamına gelir ve böylece API yüzeyi tamamen tekdüze hale gelmiştir.

Değişmeyen şeyler: web arayüzü sayfaları (SPA rotaları) `/panel/settings` ve `/panel/xray` yerinde kalmıştır – söz konusu olan yalnızca sunucu tarafı API uç noktalarıdır.

Yol eşleştirme tablosu (eski → yeni)

Aşağıdaki tüm yolların ön eki – `/panel/` 'den sonra yalnızca `api/` eklendi.

Eski (≤ 3.2.x)	Yeni (3.3.0)	Yöntem
<code>/panel/setting/all</code>	<code>/panel/api/setting/all</code>	POST
<code>/panel/setting/defaultSettings</code>	<code>/panel/api/setting/defaultSettings</code>	POST
<code>/panel/setting/update</code>	<code>/panel/api/setting/update</code>	POST
<code>/panel/setting/updateUser</code>	<code>/panel/api/setting/updateUser</code>	POST
<code>/panel/setting/restartPanel</code>	<code>/panel/api/setting/restartPanel</code>	POST
<code>/panel/setting/getDefaultJsonConfig</code>	<code>/panel/api/setting/getDefaultJsonConfig</code>	GET
<code>/panel/setting/apiTokens</code>	<code>/panel/api/setting/apiTokens</code>	GET
<code>/panel/setting/apiTokens/create</code>	<code>/panel/api/setting/apiTokens/create</code>	POST
<code>/panel/setting/apiTokens/delete/:id</code>	<code>/panel/api/setting/apiTokens/delete/:id</code>	POST
<code>/panel/setting/apiTokens/setEnabled/:id</code>	<code>/panel/api/setting/apiTokens/setEnabled/:id</code>	POST
<code>/panel/xray/</code>	<code>/panel/api/xray/</code>	POST
<code>/panel/xray/update</code>	<code>/panel/api/xray/update</code>	POST
<code>/panel/xray/getDefaultJsonConfig</code>	<code>/panel/api/xray/getDefaultJsonConfig</code>	GET
<code>/panel/xray/getXrayResult</code>	<code>/panel/api/xray/getXrayResult</code>	GET
<code>/panel/xray/getOutboundsTraffic</code>	<code>/panel/api/xray/getOutboundsTraffic</code>	GET
<code>/panel/xray/resetOutboundsTraffic</code>	<code>/panel/api/xray/resetOutboundsTraffic</code>	POST
<code>/panel/xray/testOutbound</code>	<code>/panel/api/xray/testOutbound</code>	POST
<code>/panel/xray/warp/:action</code>	<code>/panel/api/xray/warp/:action</code>	POST

Eski (≤ 3.2.x)	Yeni (3.3.0)	Yöntem
/panel/xray/nord/:action	/panel/api/xray/nord/:action	POST
/panel/xray/outbound-subs (ve /outbound-subs/*)	/panel/api/xray/outbound-subs (ve /outbound-subs/*)	GET/POST/DELETE

Alt yolların adları, istek gövdeleri ve yanıt biçimleri değişmedi – yalnızca **ön ek** değişti.

Mevcut entegrasyonlar nasıl düzeltilir

1. Betiklerinizde/yapılandırmalarınızda /panel/setting/ ve /panel/xray/ içeren tüm girişleri bulun.
2. Ön eki değiştirin: /panel/ 'den hemen sonra api/ ekleyin (örneğin /panel/setting/all → /panel/api/setting/all).
3. İstek gövdeleri, parametreler ve yanıt biçimi düzenlenmesine gerek yoktur – yalnızca URL değişir.
4. Ayarlar ve Xray yapılandırması artık /panel/api altında olduğundan, bunlara /panel/api/inbounds/* ve diğer uç noktalarla aynı Authorization: Bearer <token> API token'ıyla erişebilirsiniz (ve erişmelisiniz). /panel/api grubunun tamamında etkin olan CSRF middleware'ini unutmayın.

Örnek: API üzerinden tüm ayarların okunması. Eski hali (≤ 3.2.x):

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/setting/all" \
-H "Authorization: Bearer <token>"
```

Yeni hali (3.3.0) – /panel/ 'den sonra api/ eklendi:

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/api/setting/all" \
-H "Authorization: Bearer <token>"
```

Benzer şekilde panel yeniden başlatma: POST /panel/api/setting/restartPanel. Eski yol /panel/setting/restartPanel artık 404 döndürecektir.

Tiplendirilmiş API: şemalar ve belgeler (Swagger / OpenAPI)

3.3.0'da OpenAPI spesifikasyonu tamamen tiplendirilmiş hale getirildi. Önceden tiplendirilmiş yanıtlar boş {} nesnesiyle tanımlanıyordu; artık bileşenler ve şemalar (components.schemas) doğrudan veri modellerinden üretilmektedir. Bunun sayesinde:

- Swagger UI gerçek veri modellerini gösterir, kimliksiz yer tutucuları değil.
- Harici üreticiler (openapi-generator vb.) spesifikasyondan istenen dilde hazır istemciler oluşturabilir.
- Her tiplendirilmiş yanıtta belirli bir modele \$ref eklendi ve yanıt örnekleri sunuldu.

API belgelerine nereden bakılır:

- **Yerleşik Swagger sayfası.** Panel menüsünde – «API Belgeleri» ögesi (SPA rotası /panel/api-docs). Burada tüm uç noktalar açıklamalar, istek gövdeleri ve yanıt örnekleriyle etkileşimli olarak listelenmektedir.
- **Ham OpenAPI 3.0 spesifikasyonu** /panel/api/openapi.json adresinden sunulmaktadır. Bu URL doğrudan Postman, Insomnia veya openapi-generator 'a beslenebilir. Spesifikasyon, derleme aşamasında

ikili dosyaya gömülmüştür; panel standart dışı bir `webBasePath` altında çalışırken spesifikasyondaki `servers` alanı geçerli temel yola göre otomatik olarak yeniden yazılır; böylece «Try it out» düğmesi ve harici üreticiler doğru ön eki hedefler.

14. Telegram Botu

3X-UI paneli, sunucu ve istemci durumuna ilişkin bildirimler almak ve belirli istemcileri doğrudan mesajlaşma uygulaması üzerinden yönetmek için kullanılabilecek yerleşik bir Telegram botu içerir. Bot, long polling teknolojisiyle (Telegram'ın sürekli sorgulanması) çalışır; bu nedenle harici bir alan adına veya açık bir porta gerek yoktur – yalnızca Telegram sunucularına giden bağlantı yeterlidir.

Bot iki tür kullanıcıyı ayırt eder:

- **Yönetici** – Telegram User ID'si bot ayarlarında belirtilmiş kullanıcı («Bot yöneticisi User ID'si» alanı). Tüm işlemlere erişim hakkına sahiptir: sunucu istatistikleri, yedekleme, istemci yönetimi, Xray yeniden başlatma.
- **İstemci** – Telegram User ID'si belirli bir inbound istemcisiyle ilişkilendirilmiş diğer kullanıcılar (istemcinin `tgId` alanı). Yalnızca kendi aboneliklerine ait bilgileri görür.

Örnek: istemciyi Telegram'a bağlama. Bir kullanıcının aboneliğine ait istatistikleri alabilmesi için sayısal Telegram User ID'si istemcinin `tgId` alanına yazılır. İstemcinin JSON ayarlarında bu şöyle görünür:

```
{
  "email": "ivan",
  "id": "6f1e6b1a-0c3d-4f2a-9b7e-1a2b3c4d5e6f",
  "tgId": "123456789",
  "enable": true,
  "limitIp": 2,
  "totalGB": 53687091200,
  "expiryTime": 0
}
```

Bu işlemin ardından `123456789` User ID'sine sahip kullanıcı bota `/usage ivan` komutunu göndererek istatistiklerini görüntüleyebilir. Aynı ID'yi yönetici, istemci kartındaki « Telegram Kullanıcısını Ayarla» düğmesiyle de ekleyebilir – JSON'u elle düzenlemeye gerek yoktur.

14.1. Botu Etkinleştirme ve Yapılandırma

Tüm bot parametreleri panelde **Ayarlar** → **Telegram Botu** bölümünden ayarlanır. Ayarlar değiştirildikten sonra kaydetmek yeterlidir – panel bunları hemen uygular, paneli yeniden başlatmaya gerek yoktur. Etkinleştirme bayrağı (`tgBotEnable`), token, yönetici User ID'leri veya API sunucu adresi değiştirilirse panel, botu otomatik olarak durdurup yeni parametrelerle yeniden başlatır. Token değiştirildikten sonra panelin yeniden başlatılması gerektiğine dair eski kural artık geçerli değildir.

Alan (UI)	Ayar anahtarı	Varsayılan değer	Açıklama
Telegram Botunu Etkinleştir	<code>tgBotEnable</code>	<code>false</code>	Ana anahtar. İpucu: «Telegram botu aracılığıyla panel işlevlerine erişim». Devre dışıyken bot başlamaz ve bildirim görevleri planlanmaz.
Telegram Token	<code>tgBotToken</code>	(boş)	Bot token'ı. İpucu: «Token'ı Telegram bot yöneticisi @botfather'dan almanız gerekir». Boş olmayan bir token olmadan bot başlamaz.

Alan (UI)	Ayar anahtarı	Varsayılan değer	Açıklama
SOCKS Proxy	tgBotProxy	(boş)	Telegram bağlantısı için proxy. İpucu: «Telegram'a bağlanmak için Socks5 proxy'ye ihtiyaç duyuyorsanız, kılavuza göre parametrelerini ayarlayın».
Telegram API Server	tgBotAPIServer	(boş)	Alternatif Telegram API sunucusu. İpucu: «Kullanılacak Telegram API sunucusu. Varsayılan sunucuyu kullanmak için boş bırakın».
Bot yöneticisi User ID'si	tgBotChatId	(boş)	Virgülle ayrılmış bir veya birden fazla yönetici Telegram User ID'si. İpucu: «User ID almak için @userinfobot kullanın veya botta /id komutunu gönderin».
Yöneticiler için bot bildirim sıklığı	tgRunTime	@daily	Crontab biçiminde periyodik rapor takvimi. İpucu: «Bildirim aralığını Crontab biçiminde girin».
Veritabanı yedekleme	tgBotBackup	false	İpucu: «Veritabanı yedek dosyasıyla bildirim gönder». Yedeği periyodik rapora ekler.
Giriş bildirimi	tgBotLoginNotify	true	İpucu: «Biri panelinize giriş yapmaya çalıştığında kullanıcı adını, IP adresini ve zamanı gösterir».
Bildirim için CPU yük eşiği	tgCpu	80	Yüzde cinsinden CPU yük eşiği (0–100 doğrulaması). İpucu: «CPU yükü bu eşiği aşarsa Telegram'daki yöneticilere bildir (değer: %). 0 değerinde CPU kontrolü devre dışıdır».
Telegram Botu Dili	—	—	Botun tüm mesajları oluşturduğu dil.

@BotFather Üzerinden Token Alma

1. Telegram'da @BotFather ile sohbet açın.
2. /newbot komutunu gönderin ve talimatları izleyin (botun adı ve bot ile biten benzersiz bir username).
3. BotFather, 123456789:AA... biçiminde bir token verecektir. Bunu **Telegram Token** alanına kopyalayın.

Yönetici User ID'si Alma

User ID, hesabın sayısal tanımlayıcısıdır (username değil). İki yöntemle öğrenilebilir:

- @userinfobot botuna mesaj gönderin.
- Halihazırda yapılandırılmış botu başlatın ve /id komutunu gönderin – bot ID'nizi döndürür.

Aldığınız sayıyı **Bot yöneticisi User ID'si** alanına girin. Birden fazla yönetici atamak için ID'leri virgülle ayırın (örneğin 11111111,22222222). Her ID tam sayı olarak doğrulanır; geçersiz bir değer bot başlatma hatasına yol açar.

Örnek: «Bot yöneticisi User ID'si» alan değeri. Tek yönetici – yalnızca sayı:

123456789

Virgülle ayrılmış iki yönetici (boşluk bırakmaya gerek yoktur):

123456789,987654321

Her değer tam sayı olmalıdır. @username veya 123 456 (sayı içinde boşluk) biçimindeki girişler kabul edilmez – bot başlamaz.

Proxy

socks5://, http:// ve https:// şemaları desteklenir. Proxy alanı boş bırakılırsa bot, panelin genel proxy'sini kullanmaya çalışır (ayarlıysa ve şeması destekleniyorsa). Desteklenmeyen şemalı veya geçersiz sözdizimli URL'ler yok sayılır – bot doğrudan bağlanır. Proxy, Telegram API'ye sunucudan doğrudan erişim engellendiğinde faydalıdır.

E-posta Bildirimleri (SMTP)

Telegram'ın yanı sıra aynı olaylar e-posta yoluyla da alınabilir. Kanal **Ayarlar** → **Email** bölümündeki **SMTP Settings** sekmesinden yapılandırılır:

Alan (UI)	Ayar anahtarı	Varsayılan değer	Açıklama
Enable Email Notifications	smtpEnable	false	SMTP üzerinden e-posta bildirimlerinin ana anahtarı.
SMTP Host	smtpHost	(boş)	SMTP sunucu adresi (örneğin smtp.gmail.com).
SMTP Port	smtpPort	587	SMTP sunucu portu.
SMTP Username	smtpUsername	(boş)	SMTP kimlik doğrulaması için kullanıcı adı. Aynı zamanda gönderen adresi (From) olarak kullanılır.
SMTP Password	smtpPassword	(boş)	SMTP kimlik doğrulaması için parola. Gizli olarak saklanır; parola zaten ayarlanmışsa alan «yapılandırıldı» göstergesi görüntüler ve mevcut parolayı korumak için boş bırakılabilir.
Recipients	smtpTo	(boş)	Virgülle ayrılmış alıcı listesi (örneğin admin@example.com, ops@example.com).
Encryption	smtpEncryptionType	starttls	Bağlantı şifreleme türü: none (şifreleme yok), starttls (STARTTLS) veya tls (örtük TLS).

Send Test Email düğmesi deneme e-postası gönderir ve sonucu aşamalı olarak gösterir: **Connection** (bağlantı), **Authentication** (kimlik doğrulama) ve **Send** (gönderme). Bir sorun varsa tanılama, hatanın hangi aşamada oluştuğunu belirtir (örneğin «Authentication failed – check username and password» veya «Server requires STARTTLS – change encryption type»), bu da parametrelerin ayarlanmasını kolaylaştırır.

İkinci sekmede (**Notifications**) hangi olaylar için e-posta gönderileceği seçilir – Telegram ile aynı kart grupları kullanılır (bkz. 14.5. bölümündeki «Olay veri yolu ve bildirim seçimi»).

Telegram API Sunucusu

Bot varsayılan olarak resmi Telegram API'sine bağlanır. **Telegram API Server** alanına kendi Bot API sunucunuzun (telegram-bot-api) adresi girilebilir. URL güvenlik açısından doğrulanır; engellenmiş veya geçersiz bir adres reddedilerek varsayılan sunucu kullanılır.

14.2. Ana Menü ve Düğmeler

Menü **/start** komutuyla açılır. Düğmeler, mesaja eklenmiş inline klavyedir; düğme seti, yönetici mi yoksa istemci mi olduğunuza göre değişir.

Yönetici Menüsü

Düğme	İşlev
 Sıralı trafik kullanım raporu	Trafiğe göre sıralanmış tüm istemcileri ve her birinin kullanımını listeler; verisi olmayan fazlalık e-postalar « Sonuç yok» olarak işaretlenir.
 Sunucu durumu	Sunucu özeti (bkz. 14.5. bölümü). « Yenile» düğmesi verileri günceller.
 Tüm trafiği sıfırla	Tüm istemcilerin trafik sayaçlarını sıfırlar. «Emin misiniz?» onayı ister, ardından her istemci için « Başarılı» veya « Başarısız» gösterir, sonunda « Tüm istemciler için trafik sıfırlama tamamlandı» mesajı çıkar.
 Veritabanı yedeği	Veritabanı dosyasını ve config.json dosyasını gönderir (bkz. 14.6. bölümü).
 Ban günlüğü	IP limiti aşımı nedeniyle yasaklanan adreslerin günlük dosyalarını gönderir.
 Inbound bağlantılar	Tüm inbound'ların özeti: Remark, port, trafik, istemci sayısı, bitiş tarihi.
 Yakında bitiyor	Trafiği veya süresi yakında tükenecek inbound'ların ve istemcilerin listesi (bkz. 14.5. bölümü).
 Komutlar	Yönetici komutları yardımını gösterir.
 Çevrimiçi	Çevrimiçi istemcilerin sayısı ve listesi; e-postaya tıklamak istemci kartını açar. « Yenile» düğmesi.
 Tüm istemciler	inbound seçimini, ardından incelemek/yönetmek için istemci listesini açar.
 Yeni istemci	İstemci ekleme sihirbazını başlatır (inbound seçimi → taslak → onay).
Abonelik ayarları / bireysel bağlantılar / QR kodu	Abonelik bağlantısı, bireysel bağlantılar veya QR kodları almak için inbound ve istemci seçimi.

İstemci Menüsü

İstemciye sınırlı sayıda düğme sunulur:

Düğme	İşlev
 İstemci istatistikleri	İstemcinin Telegram User ID'siyle ilişkili tüm aboneliklere ait verileri gösterir.
 Komutlar	İstemci komutları yardımını gösterir.
Abonelik ayarları	Kendi istemcisini seçer → abonelik bağlantısı.

Düğme	İşlev
Bireysel bağlantılar	Kendi istemcisini seçer → bireysel bağlantılar.
QR kodu	Kendi istemcisini seçer → QR kodları.

Kullanıcının Telegram User ID'siyle eşleşen hiçbir istemci yoksa bot şu mesajı verir: «  Yapılandırmanız bulunamadı!  Lütfen yöneticiden Telegram User ID'nizi yapılandırmada kullanmasını isteyin.  User ID'niz: ...». Bu ID yöneticiye iletmeli ve istemci alanına girilmelidir.

14.3. Bot Komutları

3.5.0 sürümünden itibaren botun Telegram'daki «/» menüsünde **sekiz** kayıtlı komutu vardır:

Komut	Açıklama (menüden)	Erişim	Ne yapar
/start	Ana menüyü göster	herkes	Karşılama mesajı; yöneticiye ek olarak «  yönetim botuna hoş geldiniz!» ve ana menü gösterilir.
/help	Bot yardımı	herkes	Genel karşılama ve menü seçimi önerisi gösterir.
/status	Bot durumunu kontrol et	herkes	 « Bot normal çalışıyor» yanıtını verir.
/id	Telegram ID'nizi göster	herkes	 « User ID'niz: ... » döndürür. Kendi User ID'nizi öğrenmek için kullanışlıdır.
/usage	İstemci kullanımını göster: /usage email	herkes (aşağıya bakın)	3.5.0'da menüye eklendi.
/inbound	inbound ara: /inbound remark (admin)	yönetici	3.5.0'da menüye eklendi.
/restart	Xray çekirdeğini yeniden başlat (admin)	yönetici	3.5.0'da menüye eklendi.
/clearall	Tüm istemcilerin trafiğini sıfırla (admin)	yönetici	3.5.0'da yeni: onay ister («İptal» / «Trafik sınırlamayı onayla» düğmeleri) ve tüm istemcilerin trafiğini sıfırlar.

Bağımsız değişkenli komutların ayrıntıları:

- **/usage [Email]** – e-postaya göre istemci arama.
 - **Yönetici** için tam istemci kartını (yönetim düğmeleriyle birlikte) gösterir.
 - **İstemci** için yalnızca belirtilen e-postaya ait kendi aboneliğini gösterir (Telegram User ID eşleşmesine göre). Bağımsız değişken olmadan bot e-posta belirtmesini ister: «  Lütfen arama için e-posta belirtin».
- **/inbound [bağlantı adı]** – yalnızca yönetici için. Remark'a göre inbound arar ve tüm istemcilerin istatistikleriyle birlikte parametrelerini gösterir. Bağımsız değişken olmadan (veya istemci için) – «  Bilinmeyen komut».
- **/restart** – yalnızca yönetici için. Xray Core'u yeniden başlatır. Olası yanıtlar:  Xray çekirdeği başarıyla yeniden başlatıldı», «  Xray Core çalışmıyor» (çekirdek çalışmıyorsa), «  Xray core yeniden başlatılırken hata. ». /restart 'tan sonra herhangi bir bağımsız değişken, /restart ipucuyla birlikte bilinmeyen komut mesajı üretir.

Grup sohbetlerinde `/komut@botusername` biçimindeki komutlar yalnızca kullanıcı adı mevcut botun adıyla eşleşiyorsa işlenir.

3.5.0'daki üç değişiklik daha: **çevrimiçi istemciler** listesindeki düğmeler `email - inbound` açıklaması olarak etiketlenir (farklı inbound'lardaki aynı adlar artık karışmaz); **yedek** ve **ban günlüğü** mesajları `Hostname==...` satırıyla başlar – tek bot birden fazla panele hizmet verirken kullanışlıdır; istemci kartının **Telegram ID** ile aranması, ayarların JSON biçimlendirmesinden bağımsız çalışır (önceden düğümlerden veya içe aktarmadan gelen «kompakt» kayıtlar bulunamıyordu).

Yönetici yardımı («Komutlar» düğmesi):

 Xray Core'u yeniden başlatmak için: `/restart`
 E-postaya göre istemci aramak için: `/usage [Email]`
 Inbound bağlantıları aramak için (istemci istatistikleriyle): `/inbound [bağlantı adı]`
 Telegram User ID'niz: `/id`

İstemci yardımı:

 Abonelik bilgilerinizi görüntülemek için: `/usage [Email]`
 Telegram User ID'niz: `/id`

14.4. İstemci Yönetimi (Yalnızca Yönetici)

İstemci kartını açan yönetici («Tüm istemciler», «Çevrimiçi», «Yakında bitiyor» veya `/usage` aracılığıyla) istemci bilgilerini (e-posta, ilişkili inbound'lar, «Aktif» durumu, bağlantı durumu, bitiş tarihi, trafik kullanımı) ve inline yönetim düğmelerini görür:

Düğme	Amaç
 Yenile	İstemci kartını yeniden yükle.
 Trafiği sıfırla	İstemcinin trafik sayacını sıfırla. «  Trafik sıfırlamayı onaylayın?» onayı gerektirir.
 Trafik limiti	Trafik limiti belirle. Hazır değerler: ☹ Limitsiz (0), 1/5/10/20/30/40/50/60/80/100/150/200 GB veya «  Özel» – yerleşik sayısal klavyeyle (0–9 düğmeleri, «  » – 0'a sıfırla, «  » – son rakamı sil, «  Onayla: N») sayı girişi. Değer gigabayt cinsinden girilir.
 Bitiş tarihini değiştir	Hazır seçenekler: ☹ Limitsiz, «  Özel», 7/10/14/20 gün ekle, 1/3/6/12 ay ekle. Pozitif sayı süreyi uzatır (mevcut bitiş tarihine veya süre dolmuşsa «şu an»a gün ekler); 0 süre kısıtlamasını kaldırır.
 IP günlüğü	İstemcinin kayıtlı IP adreslerini gösterir (varsa zaman damgalarıyla birlikte). Günlükten «  Yenile» ve «  IP'yi temizle» (onaylı «  IP temizlemeyi onaylayın?») erişilebilir.
 IP limiti	Eş zamanlı IP kısıtlaması. Seçenekler: ☹ Limitsiz (0), 1–10 veya «  Özel» (sayısal klavye).
 Telegram Kullanıcısını Ayarla	İstemcinin mevcut ilişkili Telegram User ID'sini gösterir; ilişkilendirmeyi temizlemeye izin verir («  Telegram Kullanıcısını Kaldır» onaylı). Yeni kullanıcı ilişkilendirmesi sistem Telegram kişi seçiciyle yapılır.
 Aç/Kapat	İstemciyi etkinleştirir veya devre dışı bırakır. «  Kullanıcı aç/kapat işlemini onaylayın?» onayı gerektirir.

Yapılandırmayı değiştiren tüm işlemler (trafik/IP limiti, bitiş tarihi, Telegram kullanıcısı ilişkilendirme/kaldırma, aç/kapat) gerektiğinde Xray'i yeniden başlatılmak üzere işaretler, böylece değişiklikler yürürlüğe girer. Başarılı işlem sonrasında bot « : ...» biçiminde onay mesajı gösterir ve kartı yeniden görüntüler.

Sihirbazlardaki tüm sayısal girişler < 9999999 değeriyle sınırlıdır.

14.5. Bildirimler ve Raporlar

Bildirimler tüm yöneticilere (`tgBotChatId` 'deki tüm User ID'lere) gönderilir.

Olay Veri Yolu ve Bildirim Seçimi

Bildirimler tek bir olay veri yolu üzerine kuruludur ve iki iletim kanalı vardır: **Telegram** ve **e-posta (SMTP)**. Her kanal için hangi olayların bildirileceği ayrı ayrı seçilir. **Ayarlar** → **Telegram** bölümünde bu **Notifications** sekmesinden yapılır; **Ayarlar** → **Email** bölümünde ise aynı adlı sekmeden.

Olaylar kartlar halinde gruplandırılmıştır; her grubun, etkinleştirilen olay sayısını (n/toplam) ve yalnızca bir kısmı seçildiğinde ara durumu gösteren bir ana anahtarı vardır. Mevcut gruplar:

- **Outbound** – «Down» (`outbound.down`) ve «Up» (`outbound.up`): outbound'un düşmesi ve yeniden çalışmaya başlaması.
- **Xray Core** – «Crash» (`xray.crash`): Xray çekirdeğinin beklenmedik şekilde sonlanması.
- **Nodes** – «Down» (`node.down`) ve «Up» (`node.up`): düğümün erişilemez hale gelmesi veya yeniden çalışmaya başlaması.
- **System** – «CPU high (%)» (`cpu.high`) ve «Memory high (%)» (`memory.high`): yüksek işlemci ve RAM kullanımı. Her iki olayın yanında yüzde cinsinden eşik için bir inline alan bulunur.
- **Security** – «Login attempt» (`login.attempt`): panele giriş denemesi.

Etkinleştirilen olaylar ayrı ayrı saklanır: Telegram için `tgEnabledEvents`, e-posta için `smtpEnabledEvents`. Varsayılan olarak her iki kanalda da «Login attempt» ve «CPU high» etkindir (`login.attempt,cpu.high` değeri).

Panel Giriş Bildirimi

Giriş bildirimi (`tgBotLoginNotify`, varsayılan olarak etkin) ayarıyla yönetilir. Web paneline her giriş denemesinde yöneticilere şu mesaj gönderilir:

- Başarı durumunda: « Panele başarılı giriş.» + sunucu adı, kullanıcı adı, IP, zaman.
- Başarısızlık durumunda: « Panel giriş hatası.» + sunucu adı, **neden** (örneğin yanlış ikinci faktörde «2FA Hatası»), kullanıcı adı, IP, zaman.

CPU ve RAM Yük Aşımı

Panel dakikada bir işlemci ve RAM kullanımını kontrol eder. **tgCpu** eşiği > 0 ise ve dakikalık ortalama CPU yükü bu eşiği aşıyorsa yöneticilere şu mesaj gönderilir: « İşlemci yükü %N, %M eşik değerini aşıyor». Benzer şekilde RAM kullanımı **tgMemory** eşğine (varsayılan %80) göre kontrol edilir – «Memory high (%)» olayı.

Her iki eşik de Notifications sekmesindeki **System** grubundaki «CPU high (%)» ve «Memory high (%)» olaylarının yanındaki inline alanlarda ayarlanır (bkz. «Olay veri yolu ve bildirim seçimi»). E-posta kanalı için ayrı `smtpCpu` ve `smtpMemory` anahtarları geçerlidir. Eşik değeri 0 olduğunda ilgili kontrol devre dışı kalır.

Periyodik Rapor (Zamanlama)

Bildirim Sıklığı (`tgRunTime` , varsayılan `@daily`) alanındaki cron ifadesine göre planlanır. Değer boş veya geçersizse `@daily` kullanılır. Rapor şunları içerir:

Zamanlama Oluşturucu

Yöneticiler için bot bildirim sıklığı alanı, elle dize girişiyle değil, bir zamanlama oluşturucu aracılığıyla ayarlanır. Önce açılır listeden bir mod seçilir:

- **@every** – aralıkla tekrarlar – sayı alanı ve birim seçimi (**Saniye / Dakika / Saat**) görünür; sonuç `@every 6h` gibi bir ifade olarak oluşturulur.
- **@hourly** – her saat, **@daily** – her gün 00:00'da, **@weekly** – her hafta, **@monthly** – her ay – ilgili makro olarak kaydedilen hazır ön ayarlar (`@hourly` , `@daily` , `@weekly` , `@monthly`).
- **Özel (crontab)** – kendi crontab ifadeniz için alan. Panel zamanlayıcısı saniyeler dahil çalışır; bu nedenle özel ifade **6 alandan** oluşur: saniye, dakika, saat, ayın günü, ay, haftanın günü (örneğin `0 30 8 * * *` – her gün 08:30:00'da). Bu moda geçildiğinde alan, mevcut seçimin crontab eşdeğeriyle doldurulur.

Örnek: «Bildirim Sıklığı» (`tgRunTime`) alan değerleri. Hem hazır kısaltmalar hem de tam crontab biçimi desteklenir:

Değer	Ne zaman tetiklenir
<code>@daily</code>	Her gece yarısı günde bir kez (varsayılan değer)
<code>@hourly</code>	Her saat
<code>@every 6h</code>	Her 6 saatte bir
<code>0 9 * * *</code>	Her gün 09:00'da
<code>0 9 * * 1</code>	Her Pazartesi 09:00'da
<code>0 */12 * * *</code>	Her 12 saatte bir (00:00 ve 12:00'da)

Crontab alan sırası: dakika, saat, ayın günü, ay, haftanın günü.

1. « Zamanlanmış raporlar: » satırı ve mevcut tarih/saat.
2. **Sunucu durumu** (aşağıya bakın).
3. inbound'lar ve istemciler için «Yakında bitiyor» bloğu.
4. İlişkili Telegram User ID'sine sahip istemcilere kişisel bildirimler – yönetici olmayan her istemciye, trafiği veya süresi yakında tükenecek aboneliklerinin listesi gönderilir (devre dışı olanlar dahil).
5. **Veritabanı yedekleme** (`tgBotBackup`) etkinse – yöneticilere veritabanı yedeği.

Sunucu durumu şunları içerir: sunucu adı, 3X-UI ve Xray sürümleri, IPv4/IPv6, çalışma süresi (gün olarak), ortalama yük (Load1/2/3), RAM (kullanılan/toplam), çevrimiçi istemci sayısı, TCP/UDP bağlantı sayaçları, toplam ağ trafiği (↑/↓) ve Xray durumu.

«Yakında bitiyor» şunları gösterir:

- inbound'lar için: devre dışı sayısı ve «yakında tükenecek» sayısı, ardından bu inbound'ların listesi (Remark, port, trafik, bitiş tarihi);
- istemciler için: aynı bilgiler, artı istemci kartları ve e-posta düğmeleri (tıklama istemci kartını açar).

«Yakında tükenecek» eşikleri genel panel ayarlarından alınır: trafik rezervi (GB cinsinden) ve süre rezervi (gün cinsinden). Trafik limitine kalan miktarı eşikten az VEYA bitiş tarihine kalan gün sayısı eşikten az olan bir inbound/istemci «tükenmekte olan» olarak sayılır.

14.6. Yedekleme ve Günlükler

- **Veritabanı yedekleme** (« Veritabanı yedeği» düğmesi veya periyodik rapordaki onay kutusu): bot yedekleme zamanını, veritabanı dosyasını (`x-ui.db` veya PostgreSQL için `x-ui.dump`) ve Xray yapılandırma dosyasını `config.json` gönderir.

Botun gönderdiği yedek dosyasının adı sunucu adresiyle oluşturulur: **webDomain** değeri kullanılır; ayarlanmamışsa sunucunun genel IP'si kullanılır. Bu, yedekler birden fazla panelden toplanırken dosyanın hangi sunucudan geldiğini anlamaya yardımcı olur. Adres belirlenemezse genel bir ad uygulanır.

- **Ban günlüğü** (« Ban günlüğü» düğmesi): IP limiti aşımı nedeniyle yasaklanan IP adreslerinin mevcut ve önceki günlük dosyalarını gönderir. Boş dosyalar gönderilmez.

14.7. Çalışma Özellikleri

- **Uzun mesajlar** parçalara bölünür (~2000 karakter eşiği), inline klavye son parçaya eklenir.
- **Paralellik**: komutlar ve düğme tıklamaları eş zamanlı olarak işlenir (en fazla 10 eş zamanlı işleyici havuzu).
- **Gönderim güvenilirliği**: bağlantı hatalarında mesajlar üstel gecikmeyle yeniden gönderilir (1sn/2sn/4sn, en fazla 3 deneme).
- **Önbellekleme**: «Sunucu durumu» verileri önbelleğe alınır; böylece sık «Yenile» tıklamaları sistemi yormaz.
- **Bot yeniden başlatma**: botu etkileyen ayarlar (etkinleştirme bayrağı, token, yönetici User ID'leri veya API sunucu adresi) kaydedildiğinde panel önceki sorgulama döngüsünü kendisi durdurup güncel parametrelerle yenisini başlatır — bunun için panel yeniden başlatılmasına gerek yoktur. Aynı anda yalnızca bir güncelleme alma örneği çalışır.

15. Coğrafi Veritabanları (geoip / geosite ve Özel Kaynaklar)

Coğrafi veritabanları, Xray-core'un trafiği ülkeye göre (IP aralıkları) veya alan adı kategorisine göre yönlendirmek ve filtrelemek için kullandığı ikili `.dat` dosyalarıdır. Panel, hem standart geo-dosya setini hem de URL ile belirtilen rastgele kullanıcı tanımlı kaynakları indirip güncelleyebilir. Tüm dosyalar, Xray ikili dosyasının yanındaki `bin` dizininde saklanır (varsayılan yol `bin` ; `XUI_BIN_FOLDER` ortam değişkeniyle değiştirilebilir).

15.1. geoip.dat ve geosite.dat Nedir?

- **geoip.dat** – «IP adresi → ülke/bölge kodu» eşleme veritabanıdır. Yönlendirme kurallarında `geoip:<kod>` biçiminde kullanılır; örneğin `geoip:ru` , `geoip:cn` ve özel etiket `geoip:private` (özel/yerel ağlar). Kısaca «bu IP hangi ülkede?» sorusunu yanıtlar.
- **geosite.dat** – «alan adı → kategori/liste» eşleme veritabanıdır. `geosite:<kategori>` biçiminde kullanılır; örneğin `geosite:category-ads-all` (reklam alan adları), `geosite:google` , `geosite:ru` . Kısaca gruplandırılmış alan adı listeleridir.

Bu dosyalar, «Rus IP'lerine/alan adlarına giden tüm trafik doğrudan gitsin, geri kalanı outbound üzerinden geçsin» gibi kurallar oluşturmak için gereklidir. Kuralların kendisi Xray yönlendirme bölümünde tanımlanır; coğrafi veritabanları yalnızca bu kurallara veri sağlar. Güncel geo-dosyaları olmadan `geoip: / geosite:` referanslı kurallar çalışmaz ya da güncel olmayan listelere dayanır.

Örnek: «Rus alan adları ve IP'leri doğrudan» kuralı. Yönlendirme bölümündeki bu kural, Rus kaynaklarına giden tüm trafiği `direct` etiketli outbound'a yönlendirir:

```
{
  "type": "field",
  "domain": ["geosite:category-ru"],
  "ip": ["geoip:ru"],
  "outboundTag": "direct"
}
```

15.2. Standart Geo-Dosyalar ve Güncelleme

Panel, sabit kodlanmış indirme kaynaklarıyla birlikte altı standart dosyadan oluşan sabit bir «izin listesi» (allowlist) içerir. Güncelleme işlemi `POST /panel/api/server/updateGeofile/:fileName` (veya dosya adı belirtilmeden – tümünü aynı anda güncellemek için) aracılığıyla gerçekleştirilir.

Örnek: API üzerinden tek dosya ve tüm dosyaların güncellenmesi. Yalnızca `geoip_RU.dat` dosyasını güncellemek için:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile/
geoip_RU.dat' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

Tek bir istekle altı standart dosyanın tamamını güncellemek için (dosya adı belirtilmez):

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

Başarılı yanıt:

```
{ "success": true, "msg": "Geofile updated successfully", "obj": null }
```

Dosya adı	Kaynak (sürüm deposu)
geoip.dat	github.com/Loyalsoldier/v2ray-rules-dat (geoip.dat)
geosite.dat	github.com/Loyalsoldier/v2ray-rules-dat (geosite.dat)
geoip_IR.dat	github.com/chocolate4u/iran-v2ray-rules (geoip.dat)
geosite_IR.dat	github.com/chocolate4u/iran-v2ray-rules (geosite.dat)
geoip_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geoip.dat)
geosite_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geosite.dat)

Standart dosyaların güncellenmesine ilişkin özellikler:

- **Tek dosya güncelleme düğmesi.** İndirmeden önce onay iletişim kutusu gösterilir: «Geo-dosyayı gerçekten güncellemek istiyor musunuz?» ile «Bu işlem #filename# dosyasını güncelleyecek.» açıklaması (*Do you really want to update the geofile? This will update the #filename# file.*). Başarı durumunda «Geo-dosyalar başarıyla güncellendi» (*Geofile updated successfully*) bildirimi görünür.
- **«Tümünü Güncelle»** (*Update all*) düğmesi altı dosyanın tamamını indirir. Onay: «Bu işlem tüm geo-dosyaları güncelleyecek.» (*This will update all geofiles.*).
- **Koşullu indirme.** Yerel dosya zaten mevcutsa, isteğe dosyanın değiştirilme zamanını içeren `If-Modified-Since` başlığı eklenir. Sunucudan `304 Not Modified` yanıtı gelmesi dosyanın değişmediği anlamına gelir – dosya yeniden indirilmez, yalnızca dosya zaman damgası güncellenir.
- **Dosya adı güvenliği.** Yalnızca allowlist'teki adlar kabul edilir; ad `..`, yol ayırıcıları `/` ve `\`, mutlak yollar içermemeli ve `^[a-zA-Z0-9._-]+\\.dat$` şablonuyla eşleşmelidir. Liste dışındaki her ad «Invalid geofile name» hatasıyla reddedilir.
- **Xray yeniden başlatma.** Geo-dosyaları indirildikten sonra Xray-core, güncellenmiş veritabanlarını yeniden okumak üzere yeniden başlatılır. Yeniden başlatma başarısız olursa hata iletisine ilgili satır eklenir.

Geo-Veritabanlarını Komut Satırından Güncelleme (x-ui)

Geo-veritabanları panel olmadan da güncellenebilir – `x-ui` etkileşimli menüsünden (geo-dosya güncelleme seçeneği) veya etkileşimsiz `x-ui update-all-geofiles` komutuyla. Her dosya seti (geoip/geosite, IR ve RU setleri dahil) için ayrı bir durum görüntülenir: «güncellendi», «zaten güncel» veya «indirme hatası». İndirme başarısız olduğunda yanlış bir başarı iletisi yazdırılmaz. Xray yeniden başlatma (dolayısıyla etkin bağlantıların kesilmesi) yalnızca en az bir dosya gerçekten güncellendiğinde gerçekleşir; hiçbir dosya değişmediyse (tamamı `304 Not Modified` döndürdüyse) panel ve Xray yeniden başlatılmaz.

15.3. Xray Aracılığıyla Geo-Verilerinin Otomatik Güncellenmesi (Geodata Auto-Update)

Rastgele URL'lerden ek `.dat` kaynakları, panel araçlarıyla değil Xray-core'un yerel `geodata` bölümü aracılığıyla eklenir. İlgili bölüm, Xray güncellemeleri modal penceresinde yer alır (Kontrol Paneli → Xray güncellemeleri, `xrayUpdates`) – bu «Geodata Otomatik Güncelleme» (*Geodata Auto-Update*) sekmesidir. Panel burada yalnızca Xray yapılandırma şablonundaki `geodata` anahtarını düzenler; dosyaların indirilmesi, doğrulanması ve canlı yeniden yüklenmesi Xray çekirdeğinin kendisi tarafından yapılır.

Bölümün üst kısmında ipucu gösterilir: «Xray bu dosyaları zamanlamaya göre indirir ve yeniden başlatmadan sıcak olarak yeniden yükler. URL'ler HTTPS olmalıdır. Xray dosyayı güncelleyebilmeden önce dosyanın bin klasöründe zaten mevcut olması gerekir.» (*Xray downloads these files on schedule and hot-reloads them without a restart. URLs must be HTTPS. Each file must already exist in the bin folder once before Xray can update it.*).

Bölüm Alanları

- **Zamanlama (cron)** (*Schedule (cron)*) – 5 alanlı bir cron dizesi; varsayılan değer `0 4 * * *` (her gün 04:00'da). Kaydedilirken dizinin tam olarak 5 alan içerdiği doğrulanır; aksi takdirde «Cron 5 alan içermelidir, örn. `0 4 * * *`» hatası görüntülenir.
- **Outbound üzerinden indir (isteğe bağlı)** (*Download through outbound (optional)*) – Xray'in dosyaları indireceği mevcut outbound etiketlerinin (abonelik outbound'ları dahil) listesini gösteren açılır menü; `blackhole` protokollü outbound'lar listeye dahil edilmez. Alan boş bırakılabilir – bu durumda doğrudan bağlantı kullanılır. Bu seçim, panelin kendi istekleri için kullanılan outbound'dan (bkz. §11) bağımsızdır: `geodata` otomatik güncellenmenin indirme için kendi ayrı outbound'u vardır.
- **Dosya listesi** – her satır bir «URL + Dosya adı» (*File name*) çifti tanımlar. URL `https://` ile başlamalıdır (aksi takdirde «Her dosya için HTTPS URL gereklidir.»). Dosya adı yol ve ayırıcı içermeyen sade biçimde belirtilmelidir – yalnızca `^[A-Za-z0-9._-]+$` karakterleri (aksi takdirde «Dosya adı sade olmalıdır, örneğin `geosite_custom.dat` (yol içermeyen).»). URL girildiğinde panel, yolun son segmentinden dosya adını otomatik olarak doldurmaya çalışır. «Dosya Ekle» (*Add file*) düğmesi yeni satır ekler, çöp kutusu düğmesi satırı siler.

Liste boşsa şu ipucu gösterilir: «Yapılandırılmış dosya yok. Yönlendirme kurallarında dosyalara `ext:geosite_custom.dat:category` biçiminde başvurun.» (*No files configured. Reference files in routing rules as ext:geosite_custom.dat:category.*).

Kaydetme

«Kaydet ve Xray'i Yeniden Başlat» (*Save & Restart Xray*) düğmesi «Geodata ayarları kaydedilsin mi?» onayıyla birlikte «Xray yapılandırma şablonu güncellenecek ve Xray yeniden başlatılacak.» (*Save geodata settings? This updates the Xray config template and restarts Xray.*) açıklamasını gösterir. Kaydedildikten sonra `geodata` anahtarı yapılandırma şablonuna yazılır (`POST /panel/api/xray/update`) ve Xray yeniden başlatılır (`POST /panel/api/server/restartXrayService`). Dosya listesi boşsa `geodata` anahtarı şablondan kaldırılır.

Önemli özellikler:

- **Dosya bin dizininde zaten mevcut olmalıdır.** Xray yalnızca başlatma sırasında `bin` klasöründe zaten bulunan `.dat` dosyalarını günceller. Bu nedenle yeni bir özel dosya önce `bin` dizinine elle yerleştirilmeli

(veya en azından gerekli adla boş/eski bir sürüm oluşturulmalı), ardından Xray dosyayı zamanlamaya göre güncel tutmaya başlar.

- **Sıcak yeniden yükleme.** Planlı indirmeden sonra Xray, işlemi tamamen yeniden başlatmadan güncellenmiş veritabanlarını yeniden okur.
- **Uyumluluk.** Önceden indirilen geo-dosyaları (hem standart hem de özel) değişiklik olmaksızın `ext:` söz dizimiyle yönlendirme kurallarında çalışmaya devam eder.

Liste boşsa şu ipucu görüntülenir: «Henüz özel geo kaynağı yok – oluşturmak için Ekle'ye tıklayın» (*No custom geo sources yet – click Add to create one*).

Tablo Sütunları ve Kaynak Alanları

Alan (UI)	JSON	Varsayılan değer	Açıklama
Tür (<i>Type</i>)	<code>type</code>	– (zorunlu)	Kaynak türü: yalnızca <code>geosite</code> veya <code>geoip</code> . Sonuç dosyasının adını belirler.
Takma ad (<i>Alias</i>)	<code>alias</code>	– (zorunlu)	Kaynağın kısa tanımlayıcısı. Dosya adı, tür ve takma addan oluşturulur.
URL (<i>URL</i>)	<code>url</code>	– (zorunlu)	<code>.dat</code> dosyasına doğrudan bağlantı (http / https).
Etkin (<i>Enabled</i>)	–	–	Listedeki kaynağın etkinlik durumu.
Güncellenme zamanı (<i>Last updated</i>)	<code>lastUpdatedAt</code>	<code>0</code>	Son başarılı güncellemenin zamanı (Unix zaman damgası; <code>0</code> – henüz güncellenmedi).
Yönlendirme (ext:...) (<i>Routing (ext:...)</i>)	–	–	Yönlendirme kuralları için hazır dize: <code>ext:<dosya.dat>:tag</code> .
Eylemler (<i>Actions</i>)	–	–	«Düzenle», «Sil», «Şimdi Güncelle» düğmeleri.

Veritabanında ek dahili alanlar da saklanır: `localPath` (bin dizinindeki gerçek dosya yolu), `lastModified` (sunucudan gelen Last-Modified başlık değeri, koşullu indirme için kullanılır), `createdAt` ve `updatedAt`.

Dosya Adlandırma

Sonuç dosyasının adı, tür ve takma addan otomatik olarak oluşturulur:

- tür `geoip` → `geoip_<alias>.dat`;
- tür `geosite` → `geosite_<alias>.dat`.

Örneğin `geosite` türüyle `myads` takma adına sahip bir kaynak `geosite_myads.dat` dosyasını oluşturur.

Örnek: API üzerinden kaynak ekleme. `myads` takma adıyla özel bir reklam alan adı listesini `geosite` kaynağı olarak eklemek için:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/customGeo/add' \
-H 'Cookie: 3x-ui=<session-cookie>' \
-H 'Content-Type: application/json' \
-d '{
  "type": "geosite",
  "alias": "myads",
  "url": "https://example.com/lists/myads.dat"
}'
```

Panel, dosyayı `bin` dizinine `geosite_myads.dat` olarak indirecek, kaydı saklayacak ve Xray'i yeniden başlatacaktır.

Düğmeler ve Eylemler

- **Ekle (Add)** – «Kaynak Ekle» (*Add custom geo*) formunu açar. Kaydetme düğmesi – «Kaydet» (*Save*). API: `POST /add`.
- **Düzenle (Edit)** – «Kaynağı Düzenle» (*Edit custom geo*) formu. API: `POST /update/:id`. Tür veya takma ad değiştirildiğinde eski dosya silinir, yeni dosya yeniden indirilir.
- **Sil (Delete)** – «Bu özel geo kaynağını silmek istiyor musunuz?» (*Delete this custom geo source?*) onayı. Kaydı veritabanından ve `.dat` dosyasını siler. API: `POST /delete/:id`. Başarı durumunda: «Özel geo dosyası «» silindi».
- **Şimdi Güncelle (Update now)** – belirli bir kaynağı yeniden indirir ve zaman damgasını günceller. API: `POST /download/:id`. Başarı durumunda: «Geofile «» güncellendi».
- **Tümünü Güncelle** – tüm özel kaynakları aynı anda günceller. API: `POST /update-all`. Tamamen başarılı olduğunda: «Tüm özel geo kaynakları güncellendi» (*All custom geo sources updated*). En az bir kaynak güncellenemezse işlem kısmen başarısız sayılır ve «Bir veya daha fazla özel geo kaynağı güncellenemedi» (*One or more custom geo sources failed to update*) iletilisiyle başarılı ve başarısız kaynaklar yanıtta listelenir.

Eylemlerden herhangi birinin ardından (ekleme, düzenleme, silme, güncelleme, başarıların olduğu toplu güncelleme) Xray-core yeniden başlatılır.

Adım Adım: Kaynak Ekleme

1. «Ekle» düğmesine tıklayın.
2. «Tür» alanında `geosite` veya `geoip` seçin.
3. «Takma ad» alanına bir tanımlayıcı girin (yalnızca küçük Latin harfleri, rakamlar, `-` ve `_`; yer tutucu ipucu: `a-z 0-9 _ -`).
4. «URL» alanına `.dat` dosyasına doğrudan bağlantıyı girin (`http://` veya `https://` ile başlamalıdır).
5. «Kaydet» düğmesine tıklayın. Panel dosyayı hemen `bin` dizinine indirecek, kaydı saklayacak ve Xray'i yeniden başlatacaktır.

15.4. Doğrulama ve Kısıtlamalar

Kaynak oluşturma ve değiştirme sırasında katı kontroller uygulanır. Hata iletileri:

Koşul	İleti (TR)	İleti (EN)
Tür <code>geosite</code> / <code>geoip</code> değil	Tür <code>geosite</code> veya <code>geoip</code> olmalıdır	<i>Type must be geosite or geoip</i>

Koşul	İleti (TR)	İleti (EN)
Takma ad boş	Takma ad gereklidir	<i>Alias is required</i>
Takma adadaki geçersiz karakterler (<code>^[a-z0-9_-]+\$</code> eşleşmesi yok)	Takma ad geçersiz karakterler içeriyor	<i>Alias must match allowed characters</i>
Takma ad rezerve edilmiş	Bu takma ad rezerve edilmiş	<i>This alias is reserved</i>
URL boş	URL gereklidir	<i>URL is required</i>
URL ayrıştırılamıyor	Geçersiz URL	<i>URL is invalid</i>
Şema http/https değil	URL http veya https kullanmalıdır	<i>URL must use http or https</i>
Boş/geçersiz host veya SSRF koruması tarafından engellendi	Geçersiz URL hostu	<i>URL host is invalid</i>
«Tür + takma ad» tekrarı	Bu takma ad bu tür için zaten kullanımda	<i>This alias is already used for this type</i>
Kaynak bulunamadı	Kaynak bulunamadı	<i>Custom geo source not found</i>
İndirme hatası	İndirme başarısız	<i>Download failed</i>

Formdaki ipuçları (istemci tarafı doğrulama): «Takma ad: yalnızca a-z, rakamlar, - ve _» (*Alias may only contain lowercase letters, digits, - and _*) ve «URL http:// veya https:// ile başlamalıdır» (*URL must start with http:// or https://*).

Ek teknik kısıtlamalar:

- **Rezerve edilmiş takma adlar.** Standart dosyalarla çakışan takma adlar kullanılamaz. Rezerve edilenler (büyük/küçük harf duyarsız karşılaştırma, tire alt çizgiye eşdeğer sayılır): `geoip` , `geosite` , `geoip_ir` , `geosite_ir` , `geoip_ru` , `geosite_ru` .Örneğin `geosite-ru` , `geosite_ru` olarak reddedilir.
- **SSRF koruması.** URL hostu IP'ye çözümlenir; özel/iç bir adrese işaret ediyorsa indirme engellenir (kullanıcı «Geçersiz URL hostu» görür). Bu, panelin iç servislere erişim için kullanılmasını önler.
- **Yol geçişi koruması.** Dosyanın nihai yolu sembolik bağlantılar çözümlendikten sonra `bin` dizini içinde kalmalıdır; dışına çıkma girişimleri reddedilir.
- **Minimum dosya boyutu.** İndirilen dosya ancak 64 bayttan küçük değilse geçerli sayılır; çok küçük dosyalar indirme hatasıyla reddedilir.
- **Proxy ve koşullu indirme.** Panel ayarlarında proxy tanımlanmışsa indirme onun üzerinden gerçekleşir; diğer durumlarda SSRF güvenli aktarımla doğrudan bağlantı kullanılır. Standart dosyalarda olduğu gibi `If-Modified-Since / 304 Not Modified` uygulanır (değişmemiş dosyalar yeniden indirilmez). İndirme zaman aşımı 10 dakika, URL erişilebilirlik testi (HEAD, gerekirse kısmi GET) 12 saniyedir.

15.5. Panel Başlangıcında Otomatik Kontrol

Panel başlatıldığında tüm özel kaynakları tarar ve her biri için yerel dosyanın varlığını ve bütünlüğünü kontrol eder (dosya yok, izin veya 64 bayttan küçük). Dosya eksik veya bozuksa kaynak test edilir ve yeniden indirme denenir. Bu, `bin` dizininin yeniden kurulumdan veya kaybolmadan sonra özel geo-dosyalarının otomatik olarak geri yükleneceğini garanti eder.

15.6. Yönlendirme Kurallarında Geo-Veritabanlarının Kullanımı

Xray yönlendirme kurallarında geo-veritabanları `domain / ip` gibi alanlarda örnekler aracılığıyla kullanılır:

- **geoip:** IP veritabanları için – `geoip:<kod>`. Örnekler: `geoip:ru`, `geoip:cn`, `geoip:private`. `geoip.dat` dosyasından alınır (veya kural belirli bir dosyaya işaret ediyorsa `geoip_RU.dat` vb.).
- **geosite:** Alan adı veritabanları için – `geosite:<kategori>`. Örnekler: `geosite:category-ads-all`, `geosite:google`, `geosite:ru`. `geosite.dat` dosyasından alınır.

Örnek: geosite üzerinden reklam engelleme. Tüm reklam alan adlarını «kara deliğe» gönderen kural (`blocked` etiketli ve `blackhole` protokollü bir outbound varsayılır):

```
{
  "type": "field",
  "domain": ["geosite:category-ads-all"],
  "outboundTag": "blocked"
}
```

Özel dosyalar için `ext:` harici dosya söz dizimi kullanılır. UI'daki ipucu: «Yönlendirme kurallarında değer sütununu `ext:dosya.dat:etiket` biçiminde kullanın (etiketi değiştirin).» (*In routing rules use the value column as `ext:file.dat:tag` (replace tag).*). Biçim:

```
ext:<dosya_adı.dat>:<etiket>
```

burada `<dosya_adı.dat>`, `geoip_<alias>.dat` veya `geosite_<alias>.dat`; `<etiket>` ise dosya içindeki belirli liste/kategoridir. Panel «Yönlendirme (ext:...）」 sütununda `ext:geosite_myads.dat:tag` biçiminde hazır bir şablon gösterir – `tag` yerine gerekli etiketi yazmak yeterlidir. Bu tür bir dosyanın adı «Geodata Otomatik Güncelleme» bölümünde (bkz. §15.3) «Dosya adı» alanında belirlenir – örneğin `geosite_custom.dat`; kurallarda `ext:geosite_custom.dat:category` biçiminde başvurulur.

Örnek: özel dosyaya dayalı kural. `myads` takma adıyla `geosite` türünde bir kaynak eklenmiş ve `.dat` dosyası içindeki liste `ads` etiketiyle işaretlenmişse yönlendirme kuralı şöyle görünür:

```
{
  "type": "field",
  "domain": ["ext:geosite_myads.dat:ads"],
  "outboundTag": "blocked"
}
```

IP kaynağı için (tür `geoip`, takma ad `mycorp`, etiket `office`) alan `"ip": ["ext:geoip_mycorp.dat:office"]` olacaktır.

16. İşletim: Yedekler, Günlükler, Güncelleme, CLI

Bu bölüm panelin günlük bakımını kapsar: veritabanı yedeklerinin oluşturulması ve geri yüklenmesi, panel ve Xray günlüklerinin (loglarının) görüntülenmesi, servislerin yeniden başlatılması ve durdurulması, Xray ile panelin güncellenmesi, periyodik görevler (cron) ve panelin kaldırılması. İşlemlerin bir kısmı web arayüzünden («Dashboard» ve «Panel Ayarları» sayfasındaki sekmeler), bir kısmı ise sunucudaki `x-ui` konsol menüsünden gerçekleştirilir.

16.1. Veritabanı Yedekleme ve Geri Yükleme

Panelin tüm verileri (inbound'lar, istemciler, gruplar, düğümler, ayarlar) tek bir veritabanında saklanır. Yedek yönetimine «Dashboard» sayfasındaki «Yedek» sekmesinden, «Yedekleme ve Geri Yükleme» başlıklı blok üzerinden erişilir.

Panel iki farklı veritabanı motorunu destekler ve yedekleme davranışı buna göre değişir:

- **SQLite** (varsayılan seçenek) – veriler `x-ui.db` dosyasında saklanır.
- **PostgreSQL** – panel PostgreSQL kullanacak şekilde yapılandırılmışsa blokta şu bilgi görüntülenir:

«Bu panel PostgreSQL üzerinde çalışmaktadır. «Yedek» bir pg_dump arşivi (.dump) indirir, «Geri Yükle» ise bunu pg_restore aracılığıyla geri yükler. Geri yükleme ayrıca bir SQLite veritabanını (.db) veya SQLite geçiş dökümünü de kabul eder ve verilerini PostgreSQL'e aktarır. Sunucuda PostgreSQL istemci araçlarının (pg_dump ve pg_restore) kurulu olması gerekir.»

Dışa Aktarma (Yedek Oluşturma)

«Veritabanını Dışa Aktar» düğmesi (İng. Back Up) yedek dosyasını cihazınıza indirir.

Veritabanı Motoru	Dosya Adı	Sunucuda Gerçekleşen
SQLite	{host}_YYYY-AA-GG_SSDDSS.db	Önce WAL checkpoint gerçekleştirilir; böylece dosya en son kayıtları içerir, ardından dosya tamamı okunarak indirmeye sunulur
PostgreSQL	{host}_YYYY-AA-GG_SSDDSS.dump	pg_dump çalıştırılır, arşiv indirmeye sunulur

3.4.2 sürümünden itibaren indirilen yedek dosyasının adı, paneli açtığınız adresi ve **tarih-saati** içerir: `.db` (SQLite) veya `.dump` (PostgreSQL) uzantılı `{host}_YYYY-AA-GG_SSDDSS` – örneğin `panel.example.com_2026-06-27_000000.db`. Böylece dosyalar sunucuya göre gruplanır ve zamana göre sıralanır, aynı gün içindeki birden çok kopya birbirinin üzerine yazmaz. Aynı ad, Telegram botunun gönderdiği yedeklerde de kullanılır (`host` , panelin alan adından/IP'sinden alınır, yedek seçenek – `x-ui`).

Arayüz ipuçları:

- SQLite: «Mevcut veritabanınızın yedek kopyasını içeren .db dosyasını cihazınıza indirmek için tıklayın.»
- PostgreSQL: «Mevcut veritabanınızın PostgreSQL dökümünü (.dump) cihazınıza indirmek için tıklayın.»

Teknik olarak dışa aktarma `GET /panel/api/server/getDb` isteğiyle gerçekleşir. Ek dosyanın adı, sunucu tarafından motora bağlı olarak (`Content-Disposition` başlığıyla) belirlenir.

Yedek dosyasının adı sabit bir `x-ui.db` / `x-ui.dump` yerine sunucu adresinden türetilir. Tarayıcı üzerinden indirirken adres çubuğundaki panel adresinden (isteğin host adı) alınır; aksi takdirde yapılandırılmış web etki alanından, o da yoksa sunucunun genel IP adresinden (önce IPv4, sonra IPv6) alınır; bu da yoksa `x-ui` varsayılanına geri döner. Böylece farklı sunuculardaki yedekler kolayca ayırt edilebilir. Uzantı SQLite için `.db`, PostgreSQL için `.dump` olarak kalır; Telegram üzerinden gönderilen yedekler de aynı etki alanı/IP adlandırma kuralına göre isimlendirilir.

Örnek: API aracılığıyla yedek indirme. Aynı dışa aktarmayı konsoldan da yapabilirsiniz – örneğin otomatik yedekleme betiği için. Oturum açılmış bir oturum (cookie) gereklidir:

```
# 1) Giriş yapıp oturum cookie'sini kaydediyoruz
curl -s -c cookies.txt \
  -d 'username=admin&password=admin' \
  https://panel.example.com:2053/panel/login

# 2) Veritabanı dosyasını indiriyoruz (adı sunucu belirler: x-ui.db veya x-ui.dump)
curl -s -b cookies.txt -OJ \
  https://panel.example.com:2053/panel/api/server/getDb
```

Panel temel bir yol (Web Base Path) ile açılmışsa bunu URL'ye eklemeniz gerekir: `...:2053/<base_path>/panel/api/server/getDb`.

İçe Aktarma (Geri Yükleme)

«**Veritabanını İçe Aktar**» düğmesi (İng. `Restore`) dosya seçimi açar ve geri yükleme için dosyayı sunucuya yükler (`POST /panel/api/server/importDB`, form alanı `db`).

Arayüz ipuçları:

- SQLite: «Veritabanını geri yüklemek için cihazınızdan bir `.db` dosyası veya geçiş dökümü (`.dump`) seçip yüklemek için tıklayın.»
- PostgreSQL: «Veritabanını geri yüklemek için bir PostgreSQL yedeği (`.dump`), SQLite veritabanı (`.db`) veya SQLite geçiş dökümü seçip yüklemek için tıklayın. Bu işlem mevcut tüm verilerin yerini alır.»

3.5.0'dan itibaren dosya türü uzantıya göre değil içeriğe göre belirlenir (`PGDMP` – `pg_dump` arşivi; «SQLite format 3» başlığı – `.db` veritabanı; baştaki `PRAGMA / BEGIN TRANSACTION` – metin SQL dökümü); seçim iletişim kutusu her iki DBMS'te de `.dump`, `.db` kabul eder. PostgreSQL paneline yüklenen SQLite dosyaları PostgreSQL'e tek işlemle (transaction) aktarılır (bkz. 3.14); içe aktarmadan önce dosyanın gerçek bir panel veritabanı olduğu denetlenir, eski şemalar otomatik olarak güncel şemaya taşınır. Bir `pg_dump` arşivini geri yüklemekten önce panel (Xray durdurulmadan) okunabilirliğini önceden denetler: döküm daha yeni bir PostgreSQL ile oluşturulmuşsa, tam komutu içeren bir hata gösterilir – örneğin «...run 'x-ui pgclient 17'...».

SQLite için içe aktarma süreci (atomik ve geri alınabilir olduğunu anlamak önemlidir):

1. Yüklenen dosya format açısından doğrulanır – geçerli bir SQLite veritabanı olmalıdır; aksi takdirde «Invalid db file format» hatası döner.
2. Dosya geçici `x-ui.db.temp` olarak kaydedilir ve bütünlük denetiminden geçirilir.

3. Veritabanı değiştirme işleminden önce **Xray durdurulur**.
4. Mevcut veritabanı yedek olarak `x-ui.db.backup` adıyla yeniden adlandırılır (geri dönüş noktası).
5. Geçici dosya çalışma veritabanının yerine taşınır, şema başlatması ve göçleri çalıştırılır, ardından inbound göçü yapılır.
6. **Herhangi bir adım hata verirse** – geri alma işlemi gerçekleştirilir: eski veritabanı `x-ui.db.backup` 'tan geri yüklenir ve Xray eski verilerle yeniden başlatılır.
7. Başarı durumunda yedek dosyası silinir ve **Xray otomatik olarak yeniden başlatılır**, bu kez geri yüklenen verilerle.

İşlem sonucuna göre arayüz mesajları:

Sonuç	Metin
Başarılı	«Veritabanı başarıyla içe aktarıldı»
İçe aktarma hatası	«Veritabanı içe aktarılırken hata oluştu»
Dosya okuma hatası	«Veritabanı okunurken hata oluştu»

Geri yükleme mevcut verilerin tamamını değiştirir. İşlem sırasında Xray kısa süreliğine durduğundan, içe aktarma sırasında mevcut istemci bağlantıları kesilir.

Motorlar Arası Geçiş Dosyası (SQLite ⇌ PostgreSQL)

Normal yedekten ayrı olarak «**Geçiş Dosyasını İndir**» (Download Migration , GET /panel/api/server/getMigration isteği) işlevi de mevcuttur. Bu işlev, farklı bir veritabanı motoruna geçiş için taşınabilir bir dosya oluşturur:

3.5.0'dan itibaren düğme **yalnızca PostgreSQL panellerinde** gösterilir (SQLite'ta normal `.db` yedeği artık doğrudan PostgreSQL'e geri yüklenebiliyor – ayrı bir dışa aktarma gerekmez):

Mevcut Motor	İndirilen	Dosya Adı	Amaç
PostgreSQL	PostgreSQL verilerinden oluşturulan SQLite veritabanı	<code>x-ui.db</code>	Paneli tekrar SQLite'a geçirmek

İpucu: «PostgreSQL verilerinizden oluşturulan ve paneli SQLite üzerinde çalıştırmaya hazır SQLite veritabanını (.db) indirmek için tıklayın.»

SQLite için `.db` ⇌ `.dump` dönüştürmesi CLI'dan `x-ui migrateDB [file]` komutuyla da yapılabilir (bkz. bölüm 16.7). Ayrıca 3.5.0'dan itibaren motorlar arası geçiş **işlemsel (transactional) ve kayıpsız** hale geldi (istemci grupları ve genel trafik sayaçları da taşınır; başarısız içe aktarma hedef veritabanını değiştirmez); PostgreSQL istemci araçlarını güncellemek için `x-ui pgclient [sürüm]` komutu ve PostgreSQL menüsünde **10. Install/Upgrade client tools (pg_dump/pg_restore)** ögesi eklendi (gerekirse resmi PostgreSQL deposu bağlanır).

Telegram Botu Üzerinden Yedekleme

Telegram botu yapılandırılmışsa (bildirimlerle ilgili bölüme bakın) yönetici sohbetine doğrudan yedek gönderebilir. Telegram üzerinden gönderilen yedek **iki dosya** içerir: veritabanının kendisi (`x-ui.db` veya PostgreSQL'de `x-ui.dump`) ve Xray yapılandırması `config.json` . Mesajın önünde « Yedekleme zamanı: ...» satırı yer alır.

Telegram'da yedek almanın iki yolu vardır:

1. **İstek üzerine.** Bot menüsündeki « **Veritabanı Yedekle**» düğmesi – bot dosyaları mevcut sohbete hemen gönderir.
2. **Raporla birlikte otomatik olarak.** Bot ayarlarında «**Veritabanı Yedekleme**» (Database Backup) geçiş anahtarı bulunur; açıklaması «Veritabanı yedek dosyasıyla birlikte bildirim gönder» şeklindedir. Etkinleştirildiğinde, bot her periyodik rapor gönderiminde raporu tüm yöneticilere gönderdikten sonra yedek dosyasını da gönderir. Rapor gönderme sıklığı botun cron zamanlamasıyla belirlenir (bkz. bölüm 16.6). Bot, Telegram limitlerini aşmamak için dosyalar ve yöneticiler arasında kısa beklemler yapar.

Bot üzerinden yedekleme yalnızca bot çalışırken gönderilir; PostgreSQL'de sunucuda `pg_dump` 'ın bulunması da gereklidir.

16.2. Günlükleri Görüntüleme

Panelde birbirinden bağımsız iki günlük görüntüleyici bulunur; her ikisi de «Dashboard»'daki «**Günlükler**» sekmesinden açılır. Her pencere yenilenebilir (başlıktaki «yenile» simgesiyle) ve gösterilen içeriği `x-ui.log` dosyası olarak indirilebilir (indirme simgesi düğme).

Panel Günlükleri (Uygulama / Syslog)

Panel günlükleri penceresi (`POST /panel/api/server/logs/{count}`). Kontrol öğeleri:

Öge	Varsayılan Değer	Açıklama
Satır sayısı	20	Açılır liste: 20 / 50 / 100 / 500 / 1000
Seviye	Info	Minimum seviye: Debug / Info / Notice / Warning / Error
SysLog (onay kutusu)	kapalı	Günlüklerin alınacağı kaynak: uygulama arabelleği veya sistem günlüğü
Otomatik Güncelleme (onay kutusu)	kapalı	Günlüğü her 5 saniyede bir yeniden oku (aşağıya bakın)

Davranış **SysLog** onay kutusuna bağlıdır:

- **Kapalı (varsayılan):** Günlükler, seçilen seviyeye göre filtrelenmiş panelin dahili döngüsel arabelleğinden alınır. Girişler seviye (DEBUG / INFO / NOTICE / WARNING / ERROR) ve kaynakla birlikte görüntülenir: `X-UI:` – panelin kendi mesajları, `XRAY:` – Xray'den iletilen mesajlar.

Zaman damgası ve seviye içermeyen basit bildirimler (örneğin Windows'ta «Syslog is not supported» sistem mesajı) olduğu gibi tam olarak gösterilir. Yalnızca YYYY/MM/DD LEVEL – gövde biçimi tanınır; diğer her şey ayrıştırılmadan çıktılır, bu nedenle bu tür satırlar artık kırılmaz (daha önce ilk üç sözcük yanlışlıkla tarih/saat/seviye olarak yorumlanıyordu).

- **Açık:** Panel, sunucuda `journalctl -u x-ui --no-pager -n <count> -p <level>` komutunu çalıştırır; yani `x-ui` servisinin sistem günlüğünü gösterir. İzin verilen satır sayısı 1 ile 10000 arasındadır; seviye `syslog` değerlerini kabul eder (`emerg/0` , `alert/1` , `crit/2` , `err/3` , `warning/4` , `notice/5` , `info/6` , `debug/7`). Windows'ta SysLog modu desteklenmez – onay kutusunun kaldırılması ve uygulama günlüklerinin kullanılması gerektiğine dair uyarı gösterilir. `systemd /servis` kullanılamıyorsa `journalctl` başlatma hatası mesajı görüntülenir.

Örnek: Sunucu konsolundan aynı günlük. Panel kullanılamaz durumdayken (örneğin başlamıyorsa) sistem günlüğü doğrudan okunabilir – bu tam olarak panelin SysLog modunda çalıştırdığı komuttur:

```
# warning ve üzeri seviyede son 100 satır
journalctl -u x-ui --no-pager -n 100 -p warning

# günlüğü gerçek zamanlı izle
journalctl -u x-ui -f
```

Bu penceredeki seviye **çıktı** filtreler. Konsola/syslog'a en az hangi seviyenin yazılacağı panel günlükleme seviyesiyle belirlenir (ortam değişkeni, varsayılan `Info` ; dosyaya panel her zaman `DEBUG` seviyesinde yazar).

Xray Erişim Günlükleri (Erişim Defteri)

Xray access-log'u için ayrı bir pencere (`POST /panel/api/server/xraylogs/{count}`). Xray erişim günlüğündeki satırları ayrıştırır ve tablo olarak gösterir: **Date, From, To, Inbound, Outbound, Email**.

3.4.1 sürümünden itibaren bu pencere ve Xray durum kartındaki çağırma düğmesi **«Erişim Günlükleri»** (`Access Logs`) olarak etiketlenmiştir – önceden yalnızca «Günlükler» olarak adlandırılıyordu. Bu yeniden adlandırma, Xray access-log görüntüleyicisini aynı adı taşıyan panel günlüğü görüntüleyicisiyle karıştırmamak amacıyla yapılmıştır.

Öge	Varsayılan Değer	Açıklama
Satır sayısı	20	20 / 50 / 100 / 500 / 1000
Filtre	boş	Alt dize ile metin araması (Enter ile uygulanır)
Otomatik Güncelleme (onay kutusu)	kapalı	Günlüğü her 5 saniyede bir yeniden oku (aşağıya bakın)
Direct (onay kutusu)	açık	Doğrudan bağlantıları göster (freedom-outbound üzerinden trafik)
Blocked (onay kutusu)	açık	Engellenen bağlantıları göster (blackhole-outbound'a giden trafik)
Proxy (onay kutusu)	açık	Proxy'lenen trafiği göster

Olay türü, günlük satırındaki giden bağlantı etiketine göre otomatik olarak belirlenir: freedom etiketiyle eşleşenler → «DIRECT» (yeşil), blackhole → «BLOCKED» (kırmızı), diğerleri → «PROXY» (mavi). `api` → `api` satırları ve boş satırlar atlanır.

Otomatik Güncelleme. Her iki günlük penceresinde de («Günlükler» ve «Erişim Günlükleri») «**Otomatik Güncelleme**» (Auto Update) onay kutusu bulunur. Etkinleştirildiğinde, günlük içeriği her 5 saniyede bir tüm mevcut pencere ayarları (seçili satır sayısı, seviye/filtre ve Direct / Blocked / Proxy onay kutuları) korunarak otomatik olarak yeniden okunur. Pencere kapandığında veya onay kutusu kaldırıldığında sorgulama durur.

Bu pencerede kayıtların görünmesi için Xray'in **erişim günlüğünün** bir dosya yoluyla (none değil) etkinleştirilmiş olması gerekir – aşağıya bakın. Erişim günlüğü devre dışıysa veya dosyaya erişilemiyorsa pencere boş görünür («No Record...»).

16.3. Xray Günlük Seviyesi ve Yapılandırması

Xray'in günlükleme parametreleri «**Xray Yapılandırmaları**» sayfasındaki «**Log**» bloğunda ayarlanır; bu blokta şu uyarı yer alır:

«Günlükler sunucu performansını düşürebilir. Yalnızca gerekli olan günlük türlerini etkinleştirin!»

Alan	Çeviri	Varsayılan Değer	Açıklama
Günlük seviyesi (logLevel)	Log Level	warning	Xray hata günlüklerinin ayrıntı düzeyi. İzin verilen değerler: debug , info , notice , warning , error . İpucu: «Kaydedilmesi gereken bilgileri belirten hata günlükleri için günlük düzeyi.»
Erişim günlüğü (accessLog)	Access Log	none	Erişim günlüğü dosyasının yolu. none özel değeri erişim günlüklerini devre dışı bırakır. İpucu: «Erişim günlüğü dosyasının yolu. "none" özel değeri erişim günlüklerini devre dışı bırakır.»
Hata günlüğü (errorLog)	Error Log	boş (varsayılan yol)	Hata günlüğü dosyasının yolu; none devre dışı bırakır. İpucu: «Hata günlükleri dosyasının yolu. "none" özel değeri hata günlüklerini devre dışı bırakır.»
DNS günlüğü (dnsLog)	DNS Log	false (kapalı)	DNS sorgu günlüklemesini etkinleştir. İpucu: «DNS sorgu günlüklerini etkinleştir.»
Adres maskeleye (maskAddress)	Mask Address	boş (kapalı)	Etkinleştirildiğinde gerçek IP adresi günlüklerde otomatik olarak maskeli bir adresle değiştirilir. İpucu: «Etkinleştirildiğinde, gerçek IP adresi günlüklerde maskeli bir adresle değiştirilir.»

Varsayılan olarak «**Erişim günlüğü**» = none olduğundan «Xray Günlükleri» penceresi (bölüm 16.2) başlangıçta boştur. Çalışır hale getirmek için burada erişim günlüğü yolunu belirleyin ve Xray'i yeniden başlatın.

Boş erişim günlüğünün yalnızca bu pencereyi etkilediğine dikkat edin. «Dashboard»'daki çevrimiçi istemci listesi ve istemci formundaki IP sayısı limiti **erişim günlüğüne bağlı değildir** – panel, çevrimiçi istemcileri belirler ve IP adreslerini Xray çekirdeğinin online-stats API'si (bağlantı istatistikleri) aracılığıyla sayar. Bu API'nin bulunmadığı eski çekirdek sürümlerinde panel otomatik olarak önceki yönteme (erişim günlüğü okuma) geri döner; bu durumda IP limiti için burada erişim günlüğü yolu hâlâ gereklidir.

IP sayısı limiti ve fail2ban. İstemciye IP sayısı kısıtlamasının (istemci formunda ve toplu ekleme sırasında «IP Limit» alanı) sunucuda uygulanabilmesi için **fail2ban** kurulu olmalıdır – limiti aşan adresleri banlamak fail2ban'ın görevidir. Panel fail2ban'ın varlığını denetler (GET /panel/api/server/fail2banStatus); yoksa «IP Limit» alanı açıklayıcı bir ipucuyla (Windows'ta ayrı bir mesajla) devre dışı kalır ve daha önce ayarlanmış limitler bu sunucularda otomatik olarak sıfırlanır çünkü zaten geçerli değillerdi. fail2ban engellemesi hem TCP hem de UDP'ye uygulanır. 3.5.0'dan itibaren «ölü» bağlantı (istemcinin TCP'yi düzgün kapatmadan kaybolması) her 10 saniyelik taramada değil, **bir kez** kaydedilir: [LIMIT_IP] satırı ve bağlantı kesme döngüsü yalnızca gerçek bir yeniden bağlanmada tekrarlanır; bu nedenle fail2ban sayacı şişmez ve maxretry değerini yükseltmeye artık gerek yoktur (günlük biçimi ve failregex değişmedi). Normal sunucularda fail2ban artık panel kurulumu ve güncellemesi sırasında otomatik olarak yüklenir (bkz. bölüm 16.5).

Örnek: «Xray Günlükleri» penceresinin kayıt göstermesini sağlayan Log bloğu. Xray JSON yapılandırmasında şöyle görünür:

```
{
  "log": {
    "loglevel": "warning",
    "access": "./access.log",
    "error": "",
    "dnsLog": false,
    "maskAddress": ""
  }
}
```

Önemli olan "access": "none" değerini bir dosya yoluyla değiştirmektir (örneğin "./access.log"). Kaydedip Xray'i yeniden başlattıktan sonra «Xray Günlükleri» penceresindeki tablo satırlarla dolmaya başlar.

16.4. Xray Yönetimi: Durdurma ve Yeniden Başlatma

Xray'in durumu «Dashboard»'daki Xray kartından yönetilir. Mevcut durum şu değerlerden biriyle gösterilir: **Çalışıyor** (Running), **Durduruldu** (Stopped), **Bilinmiyor** (Unknown), **Hata** (Error). Hata durumunda «Xray başlatılırken hata» araç ipucu görüntülenir.

Düğme	Çeviri	Uç Nokta	Eylem
Durdur	Stop	POST /panel/api/server/stopXrayService	Xray sürecini durdurur. Başarı durumunda – «Xray service has been stopped» uyarı bildirimi.

Düğme	Çeviri	Uç Nokta	Eylem
Yeniden Başlat	Restart	POST /panel/api/server/restartXrayService	Xray'i mevcut yapılandırmayla yeniden başlatır (veya başlatır). Başarı durumunda – «Xray service has been restarted successfully» bildirimi.

Her iki işlemten sonra panel WebSocket üzerinden yeni durumu yayınlar; bu nedenle «Dashboard»'daki durum sayfa yenilenmeden güncellenir. İşlem hatayla biterse Xray durumu «Hata» olur ve hata metni bildirimine düşer.

Manuel yeniden başlatmanın yanı sıra panel, Xray'in yeniden başlatılmasının gerekip gerekmediğini (her 30 s'de bir arka plan görevi) ve sürecin çöküp çökmediğini (her saniye kontrol) kendisi de denetler – bkz. bölüm 16.6.

Tünel Sağlık Monitörü (Xray Otomatik Yeniden Başlatma)

3.4.1 sürümünde isteğe bağlı **tünel sağlık monitörü** eklendi. Etkinleştirildiğinde panel, belirtilen URL'nin erişilebilirliğini periyodik olarak denetler ve art arda birkaç başarısız kontrolün ardından Xray çekirdeğini otomatik olarak yeniden başlatır – bu, trafik geçirmeyi bırakan tünelin yeniden çalışır hale getirilmesine yardımcı olur. Monitör varsayılan olarak **devre dışıdır** ve **yalnızca servis ortam değişkenleriyle** yapılandırılır (web arayüzünde ayar yoktur – bu yazarlar tarafından kasıtlı olarak tasarlanmıştır).

Monitörü `XUI_TUNNEL_HEALTH_MONITOR=true` değişkeni etkinleştirir. `XUI_TUNNEL_HEALTH_PROXY` değişkeni yerel bir xray-inbound'a yönlendirilmelidir (örneğin `socks5://127.0.0.1:1080`) – bu sayede deneme Xray üzerinden gider ve tam olarak tüneli denetler; aksi takdirde yalnızca ana bilgisayar bağlantısı denetlenir ve sunucunun ağ bağlantısı sorununu yeniden başlatma düzeltemez. Diğer değişkenler kontrol parametrelerini belirler:

Değişken	Amaç	Varsayılan
<code>XUI_TUNNEL_HEALTH_MONITOR</code>	Monitörü etkinleştir (açık/kapalı)	<code>false</code>
<code>XUI_TUNNEL_HEALTH_PROXY</code>	Denemenin geçeceği proxy (yerel xray-inbound belirtin)	boş
<code>XUI_TUNNEL_HEALTH_URL</code>	Denetlenecek URL	<code>https://www.cloudflare.com/cdn-cgi/trace</code>
<code>XUI_TUNNEL_HEALTH_INTERVAL</code>	Kontroller arası aralık	<code>30s</code>
<code>XUI_TUNNEL_HEALTH_TIMEOUT</code>	Tek bir kontrolün zaman aşımı	<code>10s</code>
<code>XUI_TUNNEL_HEALTH_FAILURES</code>	Yeniden başlatmaya kadar art arda başarısızlık sayısı	<code>3</code>
<code>XUI_TUNNEL_HEALTH_COOLDOWN</code>	Yeniden başlatmalar arası minimum bekleme	<code>5m</code>

Xray'in yeniden başlatılması tüm bağlı istemcilerin bağlantısını keser; bu nedenle tek bir deneme başarısızlığının gereksiz yeniden başlatmalara yol açmaması için aralık ve başarısızlık eşliğini yeterince yüksek tutmak mantıklıdır.

16.5. Paneli Yeniden Başlatma ve Güncelleme

Paneli Yeniden Başlatma

«Panel Ayarları» sayfasında «**Paneli Yeniden Başlat**» (`Restart Panel` , `POST /panel/api/setting/restartPanel`) eylemi bulunur. Onaylandığında panel **3 saniye sonra** yeniden başlatılır.

Mesajlar:

- Onay: «Paneli yeniden başlatmak istediğinizden emin misiniz? Onaylarsanız yeniden başlatma 3 saniye içinde gerçekleşir. Panel erişilemez hale gelirse sunucu günlüğünü kontrol edin.»
- Başarı: «Panel başarıyla yeniden başlatıldı».

Teknik olarak Linux'ta yeniden başlatma, panel sürecine `SIGHUP` sinyali gönderilerek gerçekleştirilir (veya kayıtlı bir kanca aracılığıyla). Windows'ta `SIGHUP` gönderimi desteklenmez.

Panelin Kendi Kendini Güncellemesi (Update Panel)

«Dashboard»'da «**Paneli Güncelle**» (`Update Panel`) işlevi mevcuttur – 3X-UI'yi web arayüzünden doğrudan en son sürüme günceller.

Güncelleme öncesinde panel sürümleri karşılaştırır (`GET /panel/api/server/getPanelUpdateInfo`); GitHub'dan en son 3x-ui sürümü sorgulanır:

Alan	Çeviri
Mevcut panel sürümü	Current panel version
En son panel sürümü	Latest panel version
Panel güncel / «Güncel»	Panel is up to date / Up to date – yeni sürüm yoksa gösterilir

Güncelleme başlatma – `POST /panel/api/server/updatePanel` . Onay iletişim kutusu:

- «Paneli gerçekten güncellemek istiyor musunuz?»
- «Bu işlem 3X-UI'yi #version# sürümüne güncelleyecek ve panel servisini yeniden başlatacaktır.»

Başlatıldıktan sonra – «Panel update started» açılır mesajı; sürüm kontrolü başarısız olursa – «Panel update check failed».

Sunucuda gerçekleşen: kendi kendini güncelleme **yalnızca Linux'ta** desteklenir (diğer işletim sistemlerinde «panel web update is supported only on Linux installations» hatası döner). Panel, resmi `update.sh` betiğini GitHub'dan (`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh`) indirir ve ayrı bir süreçte çalıştırır: tercihen `systemd-run` aracılığıyla ayrı bir birimde (`x-ui-web-update-<timestamp>`), `systemd`

yoksa ayrı bir ayrılmış süreç olarak. Betik tamamlandığında bileşenleri günceller ve panel servisini yeniden başlatır. Çalıştırmak için `bash` gereklidir.

Güncelleme sırasında betik yeni rastgele bir web panel taban yolu (Web Base Path) oluşturduysa `x-ui` servisi otomatik olarak yeniden başlatılır; böylece yeni yol hemen çalışmaya başlar. (Yeniden başlatılmadan sunucu eski yolu sunmaya devam ederdi, arayüz yenisini gösterirdi ve yeni adres elle yeniden başlatılana kadar erişilemez olurdu.)

Dev kanalı (rolling), 3.4.2'den itibaren. «Dashboard»daki panel sürüm düğmesi artık her zaman güncelleme penceresini açar (önceden güncel kararlı derlemede GitHub sürümler sayfasına götürüyordu) ve «**Dev kanalı**» (`devChannelEnable`) geçişi her zaman – kararlı derlemede bile – gösterilir. Etkinleştirildiğinde, bir sonraki güncellemede «yuvarlanan» dev kanalına (`main` 'deki her commit'e göre derlemeler) geçilebilir. Etkinleştirilirken bir uyarı gösterilir: «Dev derlemeleri main'deki her commit'i izler ve kararlı sürümler değildir – otomatik geri alma yoktur». Kullanıcının açık bir eylemi olmadan hiçbir şey güncellenmez.

Dev Güncelleme Kanalı (Commit'e Dayalı Rolling Derlemeler)

Kararlı sürüme normal güncellemenin yanı sıra isteğe bağlı «**Geliştirici Kanalı**» (`Dev`) da mevcuttur. Geçiş anahtarı güncelleme penceresinde **yalnızca dev derlemelerinde** (ayrı bir commit'e göre oluşturulmuş CI derlemeleri) görünür; kararlı sürümlerde görünmez. Etkinleştirildiğinde panel, `main` dalının her commit'ini takip eden ve kararlı bir sürüm olmayan `dev-latest` rolling derlemesine güncellenir – dev derlemelerinin kararsız olduğu ve otomatik geri alma olmadığı uyarısı gösterilir. Dev modunda pencere sürüm numaraları yerine «Mevcut commit» / «En son commit» bilgilerini gösterir. Bu özellik yalnızca systemd'li Linux'ta kullanılabilir.

Dev derlemelerinde panel sürümünü kararlı bir sürüm numarası yerine `dev+<kısa-commit>` olarak gösterir – yan panel rozeti, «Dashboard» kartı, güncelleme penceresi, Telegram botu durum raporu ve `x-ui -v` komutunun çıktısında. Kararlı sürümlerde sürüm görünümü değişmez.

Düğümelerde (node'larda) aynı 3x-ui'nin paneli `POST /panel/api/nodes/updatePanel` aracılığıyla merkezi olarak güncellenir – düğümlerle ilgili bölüme bakın.

fail2ban'ın Otomatik Kurulumu

İstemcilerdeki IP sayısı limitinin (bölüm 16.3) kutudan çıktığı gibi çalışması için, normal bir sunucuda panel kurulumu ve güncellemesi sırasında `fail2ban` artık otomatik olarak kurulup yapılandırılır (önceden bu yalnızca Docker imajında gerçekleşiyordu). Davranışı `XUI_ENABLE_FAIL2BAN` ortam değişkeni kontrol eder: değişken tanımlı değilse veya `true` ise yapılandırma gerçekleştirilir. Manuel çalıştırma `x-ui setup-fail2ban` komutuyla mümkündür. `fail2ban` yapılandırmasındaki bir hata panel kurulumunu veya güncellemesini durdurmaz.

Yalnızca IPv6'ya Sahip Ana Bilgisayarlarda Kurulum ve Güncelleme

`install.sh` ve `update.sh` betikleri artık yalnızca IPv6'ya sahip sunucularda da düzgün çalışır: sürüm, `x-ui.sh` betiği ve servis dosyalarının indirilmesi artık zorunlu olarak IPv4'ü (`curl -4`) kullanmaz, mevcut protokolü alır. Bu nedenle panel IPv4 adresi olmayan bir ana bilgisayara da kurulabilir ve güncellenebilir.

3.5.0'daki Betik Düzeltmeleri

- **RHEL ailesi** (Rocky/Alma/RHEL/Oracle): menüden yerel PostgreSQL kurulumu artık çalışıyor (ident yerine parolayla kimlik doğrulama); IP limiti için fail2ban **EPEL**'den kuruluyor.
- **Arch/Manjaro**: kurulum ve güncelleme artık tam sistem yükseltmesi `pacman -Syu` çalıştırmıyor – yalnızca paket veritabanı güncelleniyor.
- **32 bit ARM**: indirilen Xray ikili dosyası `xray-linux-arm32` olarak adlandırılıyor – panelin onu çalıştırdığı adla birebir aynı (önceden güncelleme dosyayı farklı bir adla koyuyor ve panel eski çekirdeği çalıştırmaya devam ediyordu).
- **IP sertifikası**: otomatik algılanan genel IPv4, sertifika verilmeden önce onay için gösteriliyor (Enter – kabul); reddedilirse – doğrulanan elle giriş.
- **ACME portu seçimi**: Enter (varsayılan port 80'i kabul) artık yanlış «Your input is invalid» mesajı üretmiyor.

XUI_PORT Değişkeniyle Panel Portunu Geçersiz Kılma

Web panelinin dinleme portu `XUI_PORT` ortam değişkeniyle geçersiz kılınabilir – bu değişken yalnızca mevcut sürecin çalışma süresi boyunca geçerlidir ve veritabanındaki `webPort` değerini **değiştirmez**. İzin verilen değerler `1` ile `65535` arasındadır; boş, hatalı veya aralık dışı değerler yok sayılır (`webPort` kullanılır) ve günlüğe bir uyarı yazılır. Bu özellik dağıtımda, özellikle Docker'da kullanışlıdır: köprü ağı kullanırken yayınlanan konteyner portu `XUI_PORT` ile eşleşmelidir – örneğin `XUI_PORT=8080` ve `ports: "8080:8080"`.

Xray-core'u Güncelleme ve Sürüm Değiştirme

Aynı «Dashboard»'da panel sürümünden bağımsız olarak Xray-core sürümü de yönetilebilir.

- **Xray Güncellemeleri** (Xray Updates) / **Sürüm Seçimi** (Version) – mevcut sürümlerin açılır listesi. İpuçları: «İstediğiniz sürümü seçin» ve «Önemli: eski sürümler mevcut ayarları desteklemeyebilir» uyarısı.
- Sürüm kurma/değiştirme – `POST /panel/api/server/installXray/{version}`. İletişim kutusu: «Xray Sürümünü Değiştir» / «Xray sürümünü değiştirmek istediğinizden emin misiniz?». Başarı durumunda – «Xray başarıyla güncellendi».

Örnek: API isteğiyle Xray-core sürümü değiştirme. Sürüm, XTLS/Xray-core'dan sürüm etiketi olarak (`v` örneğiyle) belirtilir. Örneğin `v1.8.24` sürümüne geçiş:

```
curl -s -b cookies.txt -X POST \
  https://panel.example.com:2053/panel/api/server/installXray/v1.8.24
```

(`cookies.txt` – bölüm 16.1'deki örnekten cookie dosyası.) Kurulumdan sonra Xray seçilen sürümle otomatik olarak yeniden başlatılır.

Sunucuda sürüm değiştirilirken önce Xray durdurulur, GitHub'dan (XTLS/Xray-core) istenen sürümün arşivi indirilir, ikili dosya çıkarılıp değiştirilir; ardından arşiv/ikili dosya kontrol boyutları doğrulanarak Xray yeniden başlatılır.

16.6. Periyodik Görevler (Cron)

Panel başlangıçta bir dizi arka plan görevi kaydeder. Zamanlamaları sabittir (Telegram raporu ve LDAP senkronizasyonu zamanlaması dışında UI'da yapılandırılmaz). Aşağıda işletimle ilgili görevler verilmiştir.

Görev	Zamanlama	Amaç
Xray çalışma kontrolü	her 1 s	Xray sürecinin çalıştığını denetleme
Xray yeniden başlatma gerekliliği kontrolü	her 30 s	Yapılandırma değişmiş olarak işaretlendiyse yeniden başlatma
Xray trafiği toplama	her 5 s (başlangıçtan 5 s sonra başlar)	inbound/istemci trafik muhasebesi
İstemci IP kontrolü	her 10 s	Günlük aracılığıyla IP limitini denetleme
Düğüm heartbeat ve trafik senkronizasyonu	her 5 s	Düğümlemlerle (node'larla) veri alışverişi
Günlük temizleme	günlük (@daily)	IP-limit günlüklerini ve kalıcı erişim günlüğünü temizler; mevcut günlüğü <code>*.prev.log</code> olarak döndürür. 3.5.0'dan itibaren günlük temizleme Xray'in error günlüğünü de kapsar (önceden yalnızca access)
Xray günlük büyümesini sınırlama	her 10 dakikada (@every 10m)	3.5.0'da yeni. Xray'in access ve error günlüklerini, herhangi biri 64 MiB 'ı aştığında kırpar; devre dışı günlüklere (<code>none</code> /boş) dokunulmaz
Periyoda göre trafik sıfırlama	<code>@hourly</code> , <code>@daily</code> , <code>@weekly</code> , <code>@monthly</code>	İlgili otomatik sıfırlama periyodu ayarlanmış inbound'ların (ve istemcilerinin) trafik sayaçlarını sıfırlar
Telegram raporu	bot ayarlarında belirtilir (varsayılan <code>@daily</code>)	Yöneticilere rapor gönderimi; seçenek etkinse – ekli veritabanı yedeğiyle birlikte (bölüm 16.1)
Telegram hash deposu sıfırlama	her 2 m	Yalnızca bot etkinleştirilmişken
Telegram için CPU yükü denetimi	her 10 s	Yalnızca CPU eşiği > 0 olarak ayarlanmışsa

Ek olarak:

- **Periyodik trafik sıfırlama** yalnızca ilgili otomatik sıfırlama modu (saatlik/günlük/haftalık/aylık) seçili olan inbound'lar için tetiklenir. Görev hem inbound'un hem de tüm istemcilerinin trafiğini sıfırlar.
- **Süre sonu ve tükenme kontrolü.** İstemcilerin süre dolunca ve trafik limiti tükenince devre dışı bırakılması trafik muhasebesi kapsamında gerçekleştirilir: `expiry_time` dolmuş veya hacmi tükenmiş istemciler işaretlenip devre dışı bırakılır; gerekirse bir sonraki süre hesaplanır (döngüsel limitler ve «ilk kullanımda sayım başlat» modu için). «Dashboard»'da ve listelerde bu durum «Süresi Doldu»/«Tükendi»/«Yakında Bitiyor» durumlarıyla yansıtılır.
- **Telegram'da otomatik yedekleme** – rapor görevinin yan etkisidir; yalnızca yedekleme için ayrı bir cron zamanlaması yoktur. Bu nedenle otomatik yedeklemenin sıklığı bot raporunun sıklığına eşittir.

16.7. Konsol Menüsü ve CLI (`x-ui`)

Sunucuda panel `x-ui` komutuyla yönetilir. Bağımsız değişken olmadan «3X-UI Panel Management Script» etkileşimli menüsü açılır; bağımsız değişkenle belirli bir alt komut çalıştırılır. İşletimle ilgili menü öğeleri:

Menüde №	Öğe	Eylem
1	Install	Panel kurulumu (<code>install.sh</code> 'yi indirir ve çalıştırır)
2	Update	Veri kaybı olmadan tüm x-ui bileşenlerini en son sürümüne günceller; ardından – otomatik yeniden başlatma
3	Update to Dev Channel (latest commit)	Onay alınarak <code>dev-latest</code> rolling derlemesine güncelleme (son <code>main</code> dalı commit'i) (bkz. 16.5)
4	Update Menu	Yalnızca <code>x-ui</code> menü betiğini günceller
5	Legacy Version	Girilen sürüm numarasına göre belirtilen (eski) panel sürümünü kurar (örneğin <code>2.4.0</code>)
6	Uninstall	Paneli ve Xray'i tamamen kaldırır (bkz. 16.8)
7	Reset Username & Password	Yönetici kullanıcı adı/parolasını sıfırlar
8	Reset Web Base Path	Web paneli taban yolunu sıfırlar
9	Reset Settings	Ayarları varsayılan değerlere sıfırlar
10	Change Port	Panel portunu değiştirir
11	View Current Settings	Mevcut ayarları görüntüler
12–14	Start / Stop / Restart	Panel servisini başlatır, durdurur, yeniden başlatır
15	Restart Xray	Yalnızca Xray'i yeniden başlatır
16	Check Status	Servisin mevcut durumu
17	Logs Management	Günlükleri görüntüler ve temizler (aşağıya bakın)
18–19	Enable / Disable Autostart	İşletim sistemi başlangıcında servis otomatik başlatmayı etkinleştirir/devre dışı bırakır
27	Update Geo Files	Coğrafi dosyaları günceller (GeoIP/GeoSite)
25	PostgreSQL Management	PostgreSQL yönetimi

Menü öğelerinin numaralandırması 3.4.1 sürümünde değişti: 3. öğe «Update to Dev Channel»'in eklenmesiyle sonraki tüm öğeler bir birim kaydı. Toplam öğe sayısı 28 oldu, seçim `[0–28]` aralığında girilir.

CLI'da Günlük Yönetimi (16. Öğe)

«Logs Management» alt menüsü artık **17.** öğeyle açılır (önceden – 16.):

- **Debug Log** – servis günlüğünü akış olarak görüntüleme: `journalctl -u x-ui -e --no-pager -f -p debug` (Alpine'de – `/var/log/messages` üzerinde `grep`).
- **Clear All logs** – sistem günlüğünü temizleme: `journalctl --rotate + journalctl --vacuum-time=1s`, ardından servis yeniden başlatılır. (Alpine'de kullanılamaz.)

Doğrudan x-ui Alt Komutları

Tüm mevcut alt komutlar:

Komut	Açıklama
<code>x-ui</code>	Yönetim menüsünü aç
<code>x-ui start</code>	Paneli başlat
<code>x-ui stop</code>	Paneli durdur
<code>x-ui restart</code>	Paneli yeniden başlat
<code>x-ui restart-xray</code>	Xray'i yeniden başlat
<code>x-ui status</code>	Mevcut durum
<code>x-ui settings</code>	Mevcut ayarları göster
<code>x-ui enable</code>	İşletim sistemi başlangıcında otomatik başlatmayı etkinleştir
<code>x-ui disable</code>	Otomatik başlatmayı devre dışı bırak
<code>x-ui log</code>	Günlükleri görüntüle
<code>x-ui banlog</code>	Fail2ban ban günlüklerini görüntüle
<code>x-ui setup-fail2ban</code>	IP limiti için fail2ban'ı kur ve yapılandır (bkz. 16.5)
<code>x-ui update</code>	Paneli güncelle

| `x-ui update-dev` | Paneli geliştirici kanalına güncelle (rolling derleme `dev-latest`) ||
`x-ui update-all-geofiles` | Tüm coğrafi dosyaları güncelle (ardından yeniden başlatma) || `x-ui migrateDB [file]` | `.db` ➔ `.dump` veritabanı dönüştürme (SQLite) || `x-ui legacy` | Eski sürümü kur || `x-ui install` | Paneli kur || `x-ui uninstall` | Paneli kaldır |

`x-ui update` komutu resmi `update.sh` 'yi (bölüm 16.5'teki web güncellemesiyle aynı) indirir ve çalıştırır; onay ister: «This function will update all x-ui components to the latest version, and the data will not be lost.» Tamamlandığında panel otomatik olarak yeniden başlatılır.

setting alt komutundaki `-webCert` / `-webCertKey` bayrakları. Web paneli sertifikası ve özel anahtar yolları doğrudan `x-ui setting -webCert <yol> -webCertKey <yol>` alt komutuyla belirtilebilir – bu

bayraklardan herhangi birini belirtmek ilgili yolu kaydeder (ayrı `cert` alt komutu gibi) ve panel hemen HTTPS'e geçer.

CLI Aracılığıyla API Tokeni Alma

CLI aracılığıyla API tokeni alma komutu (menü öğesi/ `x-ui` komutu) önceden verilen tokenleri göstermez. API tokenleri yalnızca hash olarak saklanır, bu nedenle mevcut bir token düz metin olarak alınamaz. Tokenler zaten yapılandırılmışsa komut sayılarını bildirir, panelden (**Settings** → **API Tokens**, API tokenleriyle ilgili bölüme bakın) yönetilmelerini önerir ve arayüze girmeden CLI'ın kullanışlı kalması için `cli-fallback-<timestamp>` adında **yeni bir yedek token** oluşturup görüntüler.

16.8. Paneli Kaldırma

Kaldırma işlemi CLI'dan yapılır – menüde **5 (Uninstall)** veya `x-ui uninstall` komutu. Kaldırma öncesinde onay istenir (varsayılan «hayır»): «Are you sure you want to uninstall the panel? xray will also be uninstalled!».

Onaylanırsa betik:

1. Servisi durdurur ve otomatik başlatmasını devre dışı bırakır (`systemctl stop/disable x-ui` , veya Alpine'de – `rc-service / rc-update`), servis birim dosyasını siler ve systemd yapılandırmasını yeniden yükler.
2. Veri ve uygulama izinlerini (`/etc/x-ui/` , kurulum dizini) ve servis ortam dosyasını siler (`/etc/default/x-ui` , `/etc/conf.d/x-ui` veya `/etc/sysconfig/x-ui` – dağıtıma bağlı olarak).
3. `x-ui` betiğinin kendisini siler ve «Uninstalled Successfully.» mesajını ve yeniden kurulum komutunu çıktılar.

Panel PostgreSQL kullanıyorsa (ortam dosyasında `XUI_DB_TYPE=postgres`), panel dosyaları silindikten sonra betik ayrıca PostgreSQL sunucusunun tüm veritabanlarıyla birlikte silinmesi gerekip gerekmediğini sorar: «Also purge PostgreSQL and delete all of its data?». İstek açık onay gerektirir (varsayılan – `ret`) ve şu uyarıyla birlikte gelir: kaldırma işlemi makinedeki **TÜM** PostgreSQL veritabanlarını, diğer uygulamalara ait olanlar dahil, etkiler ve geri alınamaz. Reddetme durumunda PostgreSQL ve verileri dokunulmadan kalır.

Kaldırma işlemi geri alınamaz: panelle birlikte Xray ve tüm veriler (veritabanı dahil) silinir. Veriler gerekebilirse önceden veritabanını dışa aktarın (bölüm 16.1).

16.9. `x-ui migrateDB` Komutu

3.3.0 sürümünden itibaren `x-ui.sh` yönetim betiği, panel veritabanı SQLite'ı iki format arasında – ikili `.db` ve taşınabilir metin dökümü `.dump` (düz SQL metni) – dönüştürmek için yerleşik `x-ui` ikilisine (`x-ui migrate-db`) ait sarmalayıcı olan `migrateDB` alt komutunu aldı.

Komutun İşlevi

Komut iki yönde çalışır; yön **otomatik olarak** giriş dosyasına göre belirlenir:

Yön	Adı	Gerçekleşen
<code>.db → .dump</code>	döküm (dışa aktarma)	ikili SQLite veritabanı metin SQL dosyasına aktarılır
<code>.dump → .db</code>	geri yükleme	metin SQL dosyasından ikili SQLite veritabanı yeniden oluşturulur

Arka planda betik panel ikiliğini çağırır:

- dışa aktarma: `x-ui migrate-db --src <giriş> --dump <çıkış>`
- geri yükleme: `x-ui migrate-db --restore <giriş> --out <çıkış>`

Çağrı Sözdizimi

```
x-ui migrateDB [file.db|file.dump] [output]
```

- **[file.db|file.dump]** – giriş dosyası (birinci bağımsız değişken). Belirtilmezse varsayılan panel veritabanı kullanılır: `/etc/x-ui/x-ui.db`.
- **[output]** – çıkış dosyasının yolu (ikinci bağımsız değişken). İsteğe bağlıdır: yoksa ad, giriş dosyasının yanında otomatik olarak seçilir (aşağıya bakın).

Örnekler:

```
x-ui migrateDB                                # /etc/x-ui/x-ui.db -> /etc/x-ui/x-ui.dump
dışa aktarma
x-ui migrateDB /etc/x-ui/x-ui.db backup.dump
x-ui migrateDB backup.dump restored.db        # dökümden .db oluşturma
```

Yönün Belirlenmesi

Betik giriş dosyasının uzantısına bakar:

- `*.db`, `*.sqlite`, `*.sqlite3` → **döküm** modu (metne aktarma);
- `*.dump`, `*.sql` → **geri yükleme** modu (veritabanı oluşturma).

Uzantı tanınmazsa betik dosyanın ilk 16 baytını okur: `SQLite format 3` imzası ikili veritabanını (döküm modu) gösterir, aksi takdirde dosya döküm olarak kabul edilir (geri yükleme modu).

İkinci bağımsız değişken belirtilmemişse çıkış dosyasının adı:

- dışa aktarmada – giriş dosyasıyla aynı ad, `.dump` uzantısıyla;
- geri yüklemede – aynı ad, `.db` uzantısıyla.

Koruyucu Kontroller ve Davranış

- **İkili dosyanın varlığı.** `x-ui` ikilisi bulunamazsa veya çalıştırılamıyorsa – «x-ui binary not found ... Is the panel installed?» hatası görüntülenir.
- **Derlemede özellik desteği.** Betik, ikilinin `migrate-db --dump/--restore` işlevini desteklediğini doğrular (`x-ui migrate-db -h` aracılığıyla). Desteklemiyorsa – önce `x-ui update` komutuyla panel güncellenmesi önerilir.

- **Giriş dosyasının varlığı.** Giriş dosyası yoksa hata ve çağrı sözdizimi satırı yazdırılır.
- **Çıkışın üzerine yazma.** Çıkış dosyası zaten varsa onay istenir (varsayılan «hayır»); onaysız işlem iptal edilir. Geri yüklemede eski çıkış dosyası önceden silinir.
- **Canlı veritabanını koruma.** Varsayılan `/etc/x-ui/x-ui.db` veritabanına geri yükleme yapılırken panel çalışıyorsa, işlem reddedilir ve önce panelin durdurulması (`x-ui stop`) ya da farklı bir çıkış yolu seçilmesi istenir. Bu, çalışan servisin veritabanının üzerine yazılmasını önler.
- Veritabanı oluşturma başarısız olursa tamamlanmamış çıkış dosyası silinir.

Neden Kullanılır

- **Yedekleme.** Metin `.dump` dosyası insan tarafından okunabilir, sürüm kontrol sistemlerinde saklanmaya uygundur ve veritabanı içeriğinin farksal görüntülenmesi için kullanışlıdır.
- **Taşıma.** Döküm makineler arasında taşınabilir ve SQLite dosya biçimi sürümlerindeki farklılıklara karşı dayanıklıdır – yeni sunucuda çalışır bir `.db` oluşturulabilir.
- **Tanılama.** `.dump` dosyasından, SQLite araçları olmaksızın panel yapısı ve verileri gözle incelenebilir.

Etkileşimli Mod

Doğrudan çağrıya ek olarak, dönüştürme etkileşimli menüden de kullanılabilir. PostgreSQL alt menüsünde (`x-ui` → PostgreSQL çalışma bölümü) **9. Convert SQLite .db <--> .dump** ögesi bulunur: giriş dosyasının yolunu (varsayılan `/etc/x-ui/x-ui.db`) ve çıkış dosyasının yolunu (otomatik adlandırma için boş bırakılabilir) sorar; yön ise CLI modunda olduğu gibi otomatik olarak belirlenir.

Bu belge 3X-UI kaynak koduna dayanılarak hazırlanmıştır. Arayüzün herhangi bir ögesi sizin sürümünüzde farklıysa öncelik panelin davranışına ve UI'daki ipuçlarına aittir.